

药材的烘干问题：圆柱非稳态传热传质耦合模型的建立、达标时间反演与收缩效应分析

摘要

中药材干燥是产地加工的关键环节。本文建立一维轴对称圆柱非稳态传热传质耦合模型：控制方程取散度形式以兼容变物性，空间用元体平衡以严格守恒，时间推进分场选取；四问共用同一模型，仅物性、耦合方式、终止判据与几何假设不同。

针对问题一，题面给定常物性且两场解耦。温度方程用时间方向精确推进（矩阵指数与分段线性杜哈梅尔积分）使时间误差归零，含水率方程用后向欧拉全隐式配合追赶法，网格 0.25 mm、子步 0.1 s。1800 s 时中心温度 33.5753 °C、表面 36.7855 °C，含水率中心 2.5500 kg/kg 未变、表面降至 1.5104 kg/kg、降幅 40.8%；步长 1 s 与 60 s 相差 3.1×10^{-11} °C，印证误差归零。

针对问题二，物性升为含水率与温度的函数，两场强耦合。以交替推进求解，物性在每个内部子步按局部状态更新（子步 1/32 s），含水率侧配合欠松弛（0.7），界面变系数取含水率积分平均。3 h 时中心温度 49.8495 °C、表面 49.9664 °C，含水率中心 1.7662 kg/kg、表面 1.0081 kg/kg，较初值降约 30.7% 与 60.5%。题面未给汽化潜热，不含源项的代价为 0.07 至 0.10 °C 温降。

针对问题三，以全域含水率低于 0.15 kg/kg 为判据，事件驱动终止、子步级二分定位达标时刻，得烘干时长 57.53 h，带 ± 0.08 h 离散误差带（两类误差带取大者）。中心含水率前 24 h 降约九成，其后 33.5 h 仅由 0.2378 降至 0.1500 kg/kg。低含水率端外推贡献占时长 43% 至 66%，先验为主观设定。

针对问题四，引入物料坐标把收缩域固定为计算域，守恒形式写能量方程。烘干时长 50.6523 h，离散误差带 ± 0.0249 h，较固定半径加快约 61%；净效应为物性变化延长 71.9 h 与收缩缩短 78.8 h，合计较问题三缩短 6.9 h。模型还揭示压缩温升，最高温度 54.63 °C 高于环境上限。

以十类检验支撑结论：误差带、解析对拍、独立实现互验、守恒残差 10^{-4} 至 10^{-2} 、两族灵敏度与四组对照。

本文改进集中在数值格式与问题特化：时间方向精确推进取代加密步长逼近，使温度场时间误差归零；事件驱动终止配合子步级二分把达标时刻定位到 10^{-3} s；物料坐标与守恒形式配合，消除移动域插值误差与表面列收敛困难。本文为单一机理模型的四问递进求解，未做多模型融合。

关键词：非稳态传热传质；圆柱坐标系有限体积法；后向欧拉隐式格式；参数反演；移动边界

1 问题重述

中药材的干燥是决定药材品质的关键工序，热风烘干是产地常用的方式。就数学形式而言，题目要求求解的是一类以药材内部温度场与干基含水率场为未知函数的**传热传质耦合非稳态问题**：两场沿半径方向变化，并随烘房环境随时间演化；当物性随状态变化或几何域发生收缩时，控制方程表现为非线性。题目要求针对圆柱形药材建立描述烘干过程的数学模型，并依次解决以下四个问题。

问题一：给定药材初始温度 $28\text{ }^{\circ}\text{C}$ 、初始干基含水率 2.55 kg/kg ，以及烘房温度与水分浓度的实测时序，要求刻画预热平衡阶段 $0\text{--}1800\text{ s}$ 内药材内部温度场与水分浓度场的分布规律，并按题面规定的时刻与位置输出结果。

问题二：物性改为随含水率与温度变化（相关经验式按附录 3 取用），要求刻画整个烘干过程 $0\text{--}3\text{ h}$ 内两场的演化规律。与问题一的实质差别在于物性的状态依赖使两场由解耦转为耦合，输出要求与问题一相同。

问题三：在要求药材内部各处干基含水率均低于 0.15 kg/kg 的前提下，反演满足该要求的烘干时间（单位： h ），并给出指定时刻与位置的水分浓度。该问的实质是带终端判据的反问题。

问题四：药材在烘干中会因失水而收缩，要求依据附件 2 给出的半径时序（相关经验式按附录 4 取用）确定其烘干时长，并在输出中给出包括药材表面在内的水分浓度。与问题三的实质差别在于计算域随时间收缩。

1.1 建模对象与给定数据

本节把建模对象与给定数据一并交代（见表 1-1），后文各章不再重复。

表 1-1 建模对象与给定数据

项	内容
几何	圆柱形药材，长度 25 cm 、初始半径 2 cm ；长径比足够大，只考察半径方向的一维轴对称问题（见假设 1）
初值	温度 $28\text{ }^{\circ}\text{C}$ 、干基含水率 2.55 kg/kg ，全域均匀（题面仅给单一初值）
物性	题面分套给出：问题一按附录 2（常物性），问题二、三按附录 3，问题四按附录 4；表面换热系数 $25\text{ W}/(\text{m}^2 \cdot \text{K})$ 与对流传质系数 $8 \times 10^{-7}\text{ m/s}$ 由附录 2 给出并全程沿用
环境数据	附件 1： $0\text{--}14400\text{ s}$ 的烘房温度与水分浓度实测时序（241 点，间隔 60 s ）；其后以恒定值外推
收缩数据	附件 2：半径时序（145 点，间隔 1800 s ），由 2.000 cm 收缩至 1.198 cm （仅问题四使用）
达标判据	全域干基含水率 严格小于 0.15 kg/kg
输出契约	题面规定的六张结果表按题面行列给出（温度与水分浓度各三张；正式呈现见第五章相应各节），其中前两表以 s 计时、后四表以 h 计时，末表末列为药材表面；完整结果另存为四个结果文件，A 列一律为 s 、结果保留四位小数

题面所述“低于 0.15 kg/kg ”按**严格小于**执行，即在时间步级判定全域最大干基含水率小于该阈值，判定与终止方式见第五章问题三一节。

2 问题分析

中药材的干燥是决定药材品质的关键工序，热风烘干是产地常用的方式。就数学形式而言，题目要求求解的是一类以药材内部温度场与干基含水率场为状态变量的非稳态传热传质耦合问题，圆柱几何、时变环境边界与随含水率变化的物性共同决定了它的耦合结构。

就方程类型而言，这是抛物型偏微分方程组：时间上只含一阶导数、空间含二阶导数，故定解条件需且仅需一套初值与边界条件。就任务性质而言，四问分两类：问题一、问题二为正演，问题三、问题四为反演，由达标判据反求达标时刻；四问均无目标函数与决策变量，故不宜套用优化模型。全文难点在物性使两场牵制、低含水率端经验式大幅外推、界面系数取法决定反演答案三处。

四问共用同一个物理过程：圆柱形药材在热风环境中，内部以导热方式升温、以扩散方式失水，两者通过物性相互影响。因此全文只需建立一个模型，四问的区别仅在于四组旋钮的位置：物性体系、耦合方式、时程与判据、几何是否收缩。本文采用的技术路线是从守恒律建立控制方程，以元体平衡（单元中心有限体积）离散，以分场选取的时间格式推进，再用解析解对拍、独立实现互验与守恒核算三类手段验证，并以灵敏度分析界定结论的稳健范围。

本文输入全部来自题面与附件。环境温度与水分浓度取自附件 1（241 点、间隔 60 s），药材半径取自附件 2（145 点、间隔 1800 s），均无缺失与离群点，节点间线性插值、不作平滑；附件不覆盖全程，恒温段按末段实测均值延拓、自问题二起生效，物性与表面系数、达标阈值均由题面给定。

2.1 问题一的分析

问题的物理场景是预热平衡阶段：环境温度尚在上升，水分尚未大量脱除。关键判断是能否用集总参数法。传热毕奥数 $Bi_T = hR_0/k$ 约为 1.39、传质毕奥数 $Bi_m = k_m R_0/D$ 约为 3.24，二者均为一阶量级，说明内部导热阻与外部对流阻同量级，药材内部必然存在显著径向梯度，集总参数法不适用，必须做分布式求解^[1, 8]。按判据分档可写得更清晰：毕奥数小于 0.1 时内部阻力可略、可用集总参数法，介于 0.1 与 100 之间时须作分布式求解，大于 100 时控制权转入内部；本例两数均落中间档^[1, 8]。随干燥推进与半径收缩，传质毕奥数下降，控制权由外部对流向内部扩散迁移。

第二个判断是能否用半无限介质近似：由于扩散域径向厚度与半径之比不可忽略，忽略曲率的半无限近似会低估径向扩散路径。本文以对照计算验证：同一条件下半无限近似把表面含水率高估约 6.7%（1.6113 对 1.5104 kg/kg），故必须保留圆柱坐标下的曲率项。

第三个判断是两场能否解耦：问题一的物性为常值，扩散系数只依赖含水率而不依赖温度，含水率方程也不含温度，两场之间没有反向依赖，故可严格解耦、依次求解——这是本文在问题一能对温度场使用时间方向精确推进的前提。

2.2 问题二的分析

与问题一相比，变化集中在两处：物性从常值升为含水率与温度的函数（附录 3），两场由解耦转为双向强耦合^[2, 9]。因此必须回答为何必须联立迭代：若先算温度再算含水率而不回代，物性始终取旧状态，而干燥中段物性变化最快，误差会沿时间累积。本文采用交替推进加物性逐内部子步更新的方式化解，子步取 1/32 s 时物性冻结造成的滞后不进入四位小数量级。

2.3 问题三的分析

与问题二相比，仅动两处：时程由固定 3 h 改为事件驱动（达到判据即停），并新增终止判据（全域含水率严格小于 0.15 kg/kg）。分析要点有二：其一，中心恒为最慢点（径向扩散路径最长、边界通量最先影响表面），故“全域达标”与“中心达标”等价，判据可只在中心监测；其二，几何半径在本题设定下全程固定（收缩是问题四的设定），故模型与离散可完全复用问题二。这决定了本文对问题三的处理是“复用加终止机制”，而不是另建模型。

2.4 问题四的分析

在问题三基础上叠加尺寸收缩（移动边界）与附录 4 物性。分析要点是为何必须做坐标变换：若在固定坐标系中求解，计算域随半径收缩而变小，网格点与输出位置会逐渐错位，表面附近的控制体不断被压缩，既引入插值误差又恶化表面列的收敛性。本文引入物料坐标 $\xi = r/R(t)$ ，把收缩域映射为固定计算域 $[0, 1]$ ，使表面恒为节点、输出点恒为计算点，从源头消除上述两类困难。还需说明该变换的前提：附件 2 只给出半径的单一序列，药材内部各处的收缩分配未被规定，故本文按仿射收缩处理，并在第三章假设中一并声明其依据与适用前提。

四问的要素对照见下表。

表 2-1 四问要素对照（同一模型 × 四组旋钮）

要素	问题一	问题二	问题三	问题四
物性体系	附录 2（常物性）	附录 3（变物性）	附录 3（变物性）	附录 4（变物性）
耦合方式	严格解耦（依次求解）	双向强耦合（交替推进）	双向强耦合	双向强耦合
时程与终止	固定 0-1800 s	固定 0-3 h	事件驱动（达标即停）	事件驱动（达标即停）
几何	固定半径 R_0	固定半径 R_0	固定半径 R_0	收缩 $R(t)$ ，物料坐标
待求量	两场分布	两场分布	达标时间	烘干时长

四问由此形成“框架 → 耦合 → 反演 → 变域”的递进闭环：问题一建立解耦情形下的完整求解与验证框架；问题二在同一方程形式下引入变物性与双向耦合，并给出与问题一重叠段差异的归因；问题三复用问题二的求解核，加入反演机制得到达标时间；问题四再叠加移动边界，并用问题三的固定半径结果作为对照臂分解收缩效应。四问的旋钮分配与递进关系，也决定了后文各章的呈现次序与检验重点。

3 模型假设

本文按几何与介质、物性与耦合、表面与环境、几何演化与适用域四组给出八条假设。每条给出合理性依据（为什么可以这样假设）与适用前提（什么情况下不成立），后者同时说明假设失效时的后果。

第一组为几何与介质前提。

假设 1 药材视为无限长圆柱，温度场与干基含水率场只沿半径方向与时间变化。依据：长径比为 6.25，两端面面积仅占圆柱总表面积约 7.4%，端面贡献小^[14]；此处须一并声明所引判据的适用落差——文献^[14]对圆柱试样给出的长径比判据为 $L_0/R_0 > 15$

(此时一维近似对扩散系数的高估误差小于 5%), 本题 L_0/R_0 为 12.5, 略低于该判据, 故端面效应按已声明的方向处理, 其误差方向与量级见第七章不足 2。适用前提: 轴向近似均匀; 端面效应显著时须改用二维模型, 否则轴向中部的结果将偏低。

假设 2 药材均质且各向同性, 密度、比热容、导热系数与扩散系数均取标量。依据: 题面按标量给出四个物性经验式, 未给任何方向相关参数。适用前提: 视为连续均质介质; 强各向异性物料不适用, 须改写为张量物性形式。

第二组为物性与耦合口径。

假设 3 物性经验式按问题分套采用: 问题一用附录 2, 问题二、三全程用附录 3, 问题四用附录 4; 各套经验式在本题实际使用范围内视为有效。依据: 题面明确规定各问所用附录, 并说明经验式统一采用附录 3。适用前提: 干基含水率由 2.55 降至 0.15、温度约 28 至 50 °C; 低含水率端的外推代价已在第六章量化^[5, 6], 若经验式在该端失真, 尾段时长将出现同量级偏差。

假设 4 忽略蒸发潜热与热质交叉扩散, 能量方程不含源项; 固相与液相处于局部热力学平衡, 用单一温度描述。依据: 题面未给汽化潜热、交叉扩散系数与吸湿等温线, 引入将带来不可标定的参数^[3]。适用前提: 本文已对潜热作对照计算, 代价量化为温度系统性偏低约 0.07 至 0.10 °C, 属已知且有界; 若计入潜热, 脱湿变慢、烘干时长变长。

第三组为表面与环境。

假设 5 表面换热系数与对流传质系数全程沿用附录 2 取值, 即 $25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 与 $8 \times 10^{-7} \text{ m/s}$ 。依据: 题面仅在该附录给出这两个系数, 其余各问未另给取值。适用前提: 恒温干燥段真实系数若不同会产生偏差, 本文已在灵敏度分析中对两者作 $\pm 20\%$ 扰动。

假设 6 表面传质以干基浓度差为驱动力, 表面水分通量正比于表面含水率与烘房水分浓度之差, 不引入水分活度与平衡含水率。依据: 附录 2 直接以该形式给出表面边界式, 且未给吸湿等温线。适用前提: 低含水率段的真实驱动力若为活度差, 表面通量会出现偏差, 已在第七章列为局限。

假设 7 烘房环境单向给定: 环境温度与水分浓度由附件 1 的实测数据决定, 不因药材吸热放湿而改变; 14400 s 之后取恒定值 50.00 °C 与 0.04999 kg/kg, 该两值由附件 1 末段实测均值外推。依据: 附件 1 为固定时序数据, 题面未给药材对环境的反馈关系, 且烘房相对单根药材可视为大空间。适用前提: 环境作为给定边界; 若需考核双向耦合, 应另行联立求解。

第四组为几何演化与适用域。

假设 8 问题四的半径由附件 2 节点线性插值给出, 259200 s 之后取常值 1.198 cm; 并采用仿射(均匀)收缩, 即物质点半径按比例缩放, 以 ξ 为物质坐标。依据: 附件 2 只给表面半径时序, 题面未规定内部收缩分配方式; 仿射收缩是无需额外参数的最简分配, 等价于各向同性率为 1 的特例, 与整体失水导致均匀收缩的现象相符^[4, 7]。适用前提: 忽略表层先干先缩的非均匀效应; 若真实收缩非均匀, 问题四的内部水分场与烘干时长将出现偏差, 本文未做该项对照并已列为改进方向。

说明: 题面笔误与数据换算(如附录 3、附录 4 中扩散系数经验式的温度单位)属更正题目, 不计入本文假设, 正文在相应位置随文说明。

4 符号说明

本节给出全文主要符号一览(表 4-1), 参数的具体取值在各章首次使用处逐项给出。

注: 本表按论文呈现口径列出(长度 cm、温度 °C、时间 s), 模型内部计算一律采用国际单位制, 附录 3、附录 4 经验式内的温度须以热力学温度 K 代入(正文相应处已声明); 下标 i 为空间节点、 n 为时间层。两处易混: 符号 h 兼指表面换热系数与问题一温度侧精确推进的分段步长, 正文按上下文区分; 问题一温度场采用时间方向精确推进, 问题一含水率场与其余各问的场量采用后向欧拉全隐式, 两处不得混写。

表 4-1 主要符号一览

符号	含义	单位
自变量与几何		
t, r	时间；到药材中心的距离（径向坐标）	s、cm
R_0, L	药材初始半径、长度	cm
$R(t), \xi$	药材半径（随时间变化，仅问题四）；物料坐标 $r/R(t)$	cm、无量纲
场量与初值		
$T(r, t), C(r, t)$	药材温度；干基含水率（每千克干物质所含水分质量）	°C、kg/kg
T_0, C_0	药材初始温度、初始干基含水率	°C、kg/kg
$T_\infty(t), C_\infty(t)$	烘房温度、烘房水分浓度（给定边界）	°C、kg/kg
$C_{\text{crit}}, t_{\text{dry}}$	烘干达标阈值；达标所需时间	kg/kg、h
物性参数		
ρ, c_p, k	密度、比热容、导热系数	kg/m ³ 、 J/(kg · K)、 W/(m · K)
D, α	水分扩散系数；热扩散率 ($k/(\rho c_p)$)	m ² /s
h, k_m	表面换热系数、表面传质系数	W/(m ² · K)、 m/s
数值参数与无量纲数		
Bi_T, Bi_m, Fo	传热毕奥数、传质毕奥数、傅里叶数	无量纲
τ_T, τ_C	热扩散、质扩散的特征时间	s
$\Delta r, N, \Delta t$	空间步长、径向区间数、时间步长	mm、无量 纲、s
A, E, G_0, G_1	温度半离散系统的系数矩阵、其矩阵指数、分段线性杜哈梅尔积分系数	s ⁻¹ 、无量纲、 s、s ²
d^n, ω	隐式推进方程的右端向量、迭代欠松弛因子	与场量同、 无量纲

5 模型建立与求解

5.0 总体模型与求解框架

四问共用一个物理过程：圆柱形药材在热风环境中，内部以导热方式升温、以扩散方式失水，两者通过物性相互影响。本文据此建立一维轴对称圆柱非稳态传热传质耦合模型，其总体结构如图 5-1 所示。

模型由“题面与附件数据 → 守恒律建立控制方程 → 元体平衡离散 → 全隐式求解 → 检验与灵敏度”五段构成，四问沿同一主干分叉。由图 5-1 可见，四问的差别不体现在方程组形式上，而只体现在四组旋钮的位置：物性体系、耦合方式、时程与终止判据、几何是否收缩。本文是“一个模型、四组旋钮”的闭环结构，而非四个独立模型。

5.0.1 控制方程族（四问共用形式）

在圆柱坐标系下，令 $T(r, t)$ 为温度、 $C(r, t)$ 为干基含水率，取散度形式（保留曲率项 $1/r$ ）：

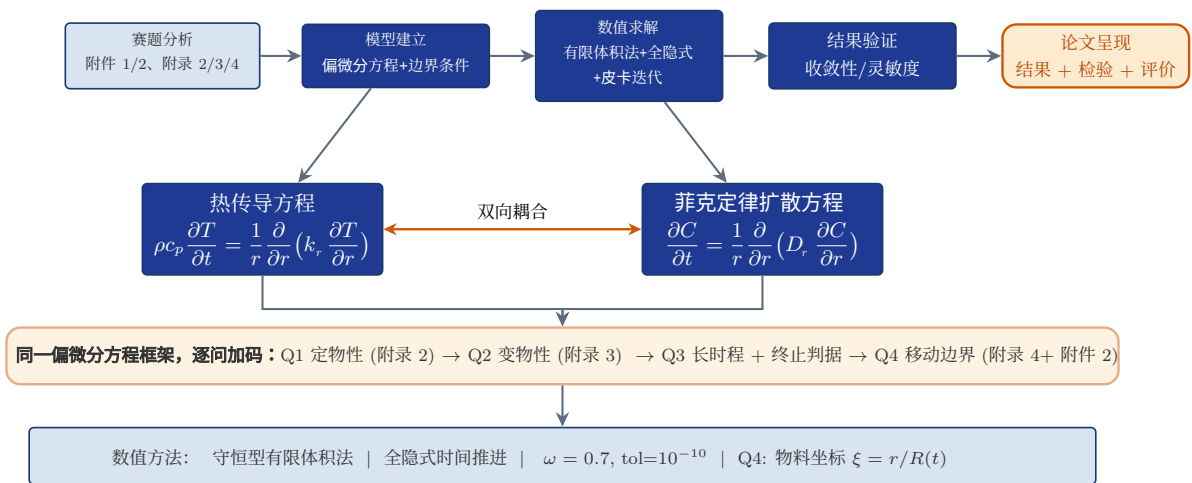


图 5-1 建模总体流程图

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(k r \frac{\partial T}{\partial r} \right), \quad \frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(D r \frac{\partial C}{\partial r} \right) \tag{5-1}$$

其中 ρ, c_p, k 与 D 的取值按问分套：问题一取常值，问题二、三取附录 3 的经验式，问题四取附录 4 的经验式并叠加几何收缩。需要强调的是，导热系数 k 必须位于散度算子之内；写成“ k 乘拉普拉斯算子”的形式在变物性下不再等价，会丢掉由物性梯度产生的附加通量。

5.0.2 求解与验证路线

空间上统一采用元体平衡（单元中心有限体积）离散，其选用依据有二：相邻控制体在公共界面使用同一数值通量，求和时内部通量两两抵消，区域总量的变化只取决于最外层边界通量，故守恒性在代数层面严格成立且与网格是否均匀无关；该格式导出的系数矩阵为 M 矩阵，离散极值原理随之成立，故固定计算域（问题一至问题三）下温度不越界、含水率沿径向单调属理论必然，而非经验观察（问题四的移动边界与压缩温升另见 5.4.5）。时间上问题一温度场采用时间方向精确推进，其余场量全部采用后向欧拉全隐式并配合追赶法，理由是隐式格式无条件稳定、长时程计算不受步长限制，而问题一温度场在半离散后恰为常系数线性系统、积分可闭式求得，可直接消除时间方向的离散误差。检验手段详见第六章；本文各环节的选型见表 5-1。

进一步说明排除口径。空间上不采用二阶中心差分：其中心点须作特殊处理，且守恒性只在代数上近似成立，而元体平衡在逐控制体意义下严格守恒、与网格是否均匀无关。时间上不采用显式格式：扩散算子的稳定性限幅会把含水率侧的子步压至 10^{-3} s 量级，长时程推进的计算量不可接受，而全隐式无条件稳定、长时程计算不受步长限制。温度场亦不采用“显式格式加大子步”的逼近路线：该路线的时间离散误差仅随步长线性下降，而问题一温度场在半离散后恰为常系数线性系统、积分可闭式求得，可直接把时间离散误差归零（步长 1 s 与 60 s 的结果相差 3.1×10^{-11} °C，即为其量化对照）。

表 5-1 求解方案选型对比

环节	本文选用（备选）	选用依据与量化对照
空间离散	元体平衡（差分 / 有限元）	逐控制体严格守恒；中心半格使中心奇点无须弱形式；中心系数两路推导差 2.8×10^{-14} K

(续)

环节	本文选用（备选）	选用依据与量化对照
温度场时间推进	精确推进（显式 / 后向欧拉）	常系数线性系统积分可闭式；步长 1 s 与 60 s 相差 $3.1 \times 10^{-11} \text{ }^{\circ}\text{C}$ ，时间离散误差归零
含水率场时间推进	后向欧拉全隐式（显式 / 梯形）	三对角严格对角占优，无条件稳定且满足离散极值原理；子步 0.1 s 下误差 10^{-5} kg/kg
子步与耦合	交替推进+固定子步（顺序单向 / 自适应）	物性双向依赖，单向求解误差沿时间累积；自适应轨与固定子步数值等价（不超过 $9.3 \times 10^{-6} \text{ kg/kg}$ ）
界面变系数	沿含水率的积分平均（算术 / 调和平均）	由界面通量守恒推出；尾部胞内 D 跨约两个数量级，调和平均使时长高估 52% 以上，仅作对照臂
移动边界	物料坐标固定域化（固定坐标+插值）	表面恒为节点、输出点恒为计算点，消除移动域插值误差与表面列收敛困难，并揭示压缩温升

5.1 问题一：预热平衡阶段（常物性·完全解耦）

5.1.1 分析与建模思路

药材在 0-1800 s 内处于预热平衡阶段：环境温度逐步上升，水分尚未大量脱除。首要判断是是否需要分布式求解，判据是两个毕奥数：

$$Bi_T = \frac{hR_0}{k} \approx 1.39, \quad Bi_m = \frac{k_m R_0}{D} \approx 3.24 \quad (5-2)$$

二者均为一阶量级，表明内部导热阻与外部对流阻同量级，药材内部必然存在显著的径向梯度，集总参数法不适用^[1, 8]。

其次是能否用半无限介质近似：扩散域厚度与半径之比不可忽略，忽略曲率会低估径向扩散路径。自研对照显示，同一条件下半无限近似把表面含水率高估约 6.7% (1.6113 对 1.5104 kg/kg)，故必须保留曲率项^[1]。

最后是两场能否解耦：问题一物性按附录 2 取常值，扩散系数只依赖含水率、不含温度，含水率方程也不含温度，两场无反向依赖，故可严格解耦依次求解——这也是后文对温度场使用时间方向精确推进的前提。

5.1.2 b 数据说明

环境温度与水分浓度取自附件 1，共 241 个实测点（0-14400 s，间隔 60 s），缺失点 0 个、三倍标准差离群点 0 个。节点间采用线性插值、不做平滑；对照计算表明平滑与否对结果的影响不超过 $1.08 \times 10^{-2} \text{ }^{\circ}\text{C}$ ，故保留原始数据不损失精度，也避免了人为的数据改动。初始温度 28 $^{\circ}\text{C}$ 、初始干基含水率 2.55 kg/kg 由题面给定。问题一的整个计算窗口 0-1800 s 完全落在升温段内，边界数据全部取自实测，不涉及任何外推。

5.1.3 控制方程与定解条件

问题一的物性取附录 2 常物性，控制方程为

$$\rho c_p \frac{\partial T}{\partial t} = \frac{k}{r} \frac{\partial}{\partial r} \left(r \frac{\partial T}{\partial r} \right), \quad \frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(D(C) r \frac{\partial C}{\partial r} \right) \quad (5-3)$$

其中扩散系数仍随含水率变化，这一依赖关系是问题一唯一的非线性来源，但它只出现在含水率方程内部，不与温度方程交叉。定解条件为

$$T(r, 0) = T_0 = 28^\circ\text{C}, \quad C(r, 0) = C_0 = 2.55 \text{ kg/kg} \quad (5-4)$$

$$\left. \frac{\partial T}{\partial r} \right|_{r=0} = 0, \quad \left. \frac{\partial C}{\partial r} \right|_{r=0} = 0 \quad (5-5)$$

$$-k \left. \frac{\partial T}{\partial r} \right|_{r=R_0} = h [T(R_0, t) - T_\infty(t)], \quad -D \left. \frac{\partial C}{\partial r} \right|_{r=R_0} = k_m [C(R_0, t) - C_\infty(t)] \quad (5-6)$$

其中 $h = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 、 $k_m = 8 \times 10^{-7} \text{ m/s}$ 由附录 2 给出并全程沿用； $T_\infty(t)$ 、 $C_\infty(t)$ 由附件 1 线性插值给出。中心条件 (5-4) 来自轴对称性，它使中心成为自然边界而非物理边界；表面条件 (5-5) 为第三类（对流换热与对流传质）条件，即表面通量由流体侧对流决定。两个条件的方向可作自检：预热阶段表面升温，故表面温度梯度为正向，与式 (5-6) 右端符号一致。

5.1.4 离散格式与求解算法

空间离散采用元体平衡（单元中心有限体积），取径向区间数 $N = 80$ （ Δr 取 0.25 mm、81 个节点），理由是使题面要求的 0.1 cm 输出点恰为计算点，从源头免除输出插值误差。中心控制体取半格，中心奇点由两条相互独立的路径装配（洛必达极限 / 半格通量平衡），系数完全一致、解级比对差异为 $2.8 \times 10^{-14} \text{ K}$ ；本文以半格通量平衡为交付路径，另一条留作交叉验证。

温度场时间推进采用时间方向精确推进。常物性使半离散后的温度方程成为常系数线性系统：

$$\frac{dT}{dt} = AT + w_0(t) + w_1(t) \quad (5-7)$$

其中 A 为半离散系数矩阵，为严格对角占优的非奇异 M 矩阵，故可逆；边界项由 w_0 、 w_1 承载。附件 1 为等步长 60 s 数据且节点间线性插值，故边界在每个 1 s 子区间内严格分段线性，积分可闭式求得：

$$T^{n+1} = E T^n + G_0 w_0^n + G_1 w_1^n, \quad E = e^{Ah}, \quad G_0 = A^{-1}(E - I), \quad G_1 = A^{-2}(E - I - Ah) \quad (5-8)$$

E 、 G_0 、 G_1 一经离线预分解即可全程复用。由于积分是闭式，该格式不引入任何时间离散误差；实测步长无关性为：分段步长 h 取 1 s 与 60 s 的结果相差 $3.1 \times 10^{-11}^\circ\text{C}$ 。

其适用边界为常系数前提：物性一旦随温度或含水率变化，闭式积分不再成立，故该推进只用于问题一，问题二至问题四的温度场改用后向欧拉全隐式。

含水率场时间推进采用后向欧拉全隐式。因扩散系数依赖含水率，方程为非线性，采用后向欧拉配合欠松弛迭代：

$$\frac{C^{n+1} - C^n}{\Delta t} = L_C(C^{(k)}) C^{(k+1)} + g^{n+1}, \quad \omega = 0.7 \quad (5-9)$$

其中上标 (k) 为迭代层而非时间层；每步的新解与旧解按松弛因子加权组合：

$$C^{(k+1)} = (1 - \omega) C^{(k)} + \omega \tilde{C}^{(k+1)}, \quad \omega = 0.7 \quad (5-10)$$

迭代矩阵为三对角；由对角元大于同行两个非对角元之和可知其严格对角占优，而严格对角占优正是追赶法可用与极值原理成立的代数前提，故追赶法无条件稳定并满足离散极值原理，含水率在计算中不会越出初值与边界值所夹的范围。含水率子步取 0.1 s。精度预算由输出要求反推：四位小数输出下，空间离散误差不超过 $1.7 \times 10^{-5} \text{ }^{\circ}\text{C}$ 、时间误差量级仅 10^{-5} kg/kg ，均在最低有效位之下。

问题一的求解原理可概括为三段式：空间上以元体平衡离散、温度场沿时间方向精确推进、含水率场以全隐式配合欠松弛迭代。其求解流程如图 5-2 所示。

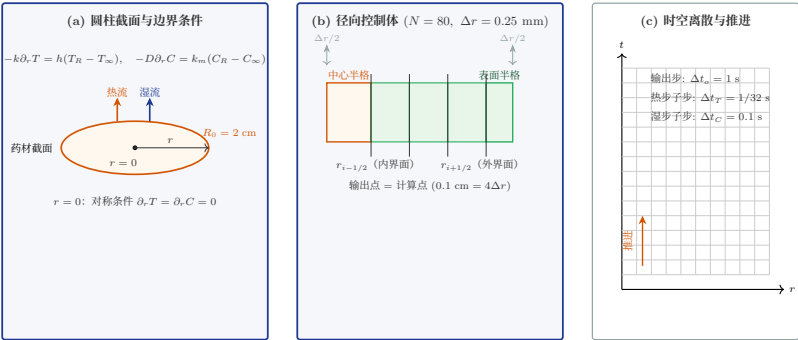


图 5-2 问题一求解原理示意图

由图 5-2 可见，三段之间的接口是”先空间、再按场分推”，温度场与含水率场只在物性求值处发生单向联系；这意味着问题一的解耦不是额外的近似假设，而是”物性只依赖含水率”这一事实的直接结果，故可依次求解而不引入附加误差。

5.1.5 结果与分析

按题面格式给出 0-1800 s 内若干时刻、若干位置的结果，见表 5-2 与表 5-3。

表 5-2 30 分钟内药材的温度（单位： $^{\circ}\text{C}$ ；各列为到药材中心的距离，单位：cm）

时间/s	0	0.5	1.0	1.5	2.0
100	28.0001	28.0003	28.0041	28.0327	28.1800
300	28.0408	28.0635	28.1514	28.3680	28.8487
600	28.4534	28.5360	28.8039	29.3159	30.1652
900	29.3243	29.4583	29.8754	30.6161	31.7304
1200	30.5427	30.7098	31.2223	32.1125	33.4276
1500	31.9957	32.1867	32.7660	33.7463	35.1203
1800	33.5753	33.7720	34.3642	35.3621	36.7855

同一组输出时刻与同一组半径位置上的水分浓度见表 5-3，两表共用同一套输出契约与有效数字口径。

表 5-3 30 分钟内药材的水分浓度（单位：kg/kg；各列为到药材中心的距离，单位：cm）

时间/s	0	0.5	1.0	1.5	2.0
100	2.5500	2.5500	2.5500	2.5500	2.2490
300	2.5500	2.5500	2.5500	2.5492	2.0517
600	2.5500	2.5500	2.5500	2.5352	1.8774
900	2.5500	2.5500	2.5497	2.5045	1.7550

(续)

时间/s	0	0.5	1.0	1.5	2.0
1200	2.5500	2.5500	2.5482	2.4646	1.6588
1500	2.5500	2.5499	2.5445	2.4206	1.5789
1800	2.5500	2.5497	2.5383	2.3755	1.5104

温度场与水分浓度场的时空演化如图 5-3 与图 5-4 所示。
该图横轴为到药材中心的距离、纵轴为时间、色标为温度；由图 5-3 可见，径向上始终维持由表及里的温度梯度，热量在 1800 s 内尚未渗透至中心。

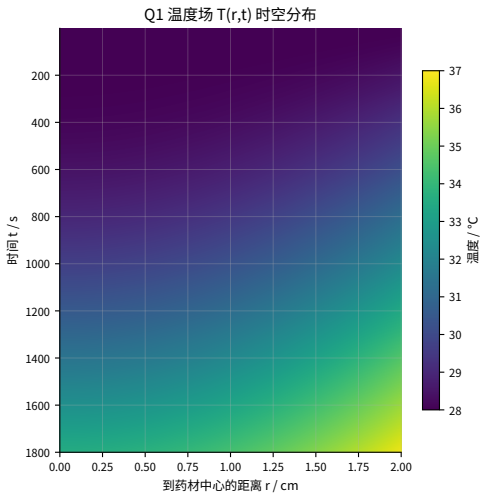


图 5-3 问题一温度场时空分布

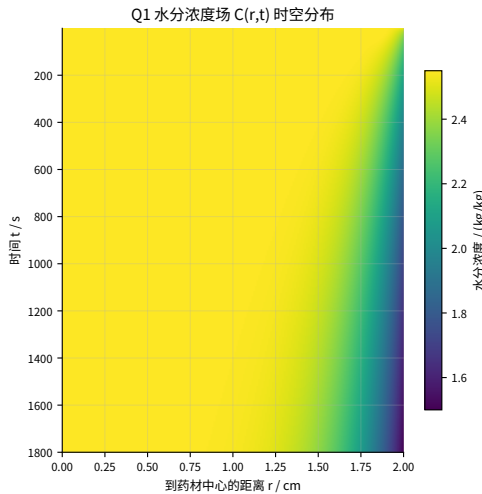


图 5-4 问题一水分浓度场时空分布

该图与图 5-3 同构，色标为干基含水率；由图 5-4 可见，含水率的变化带厚度约为 0.5-1.0 cm，其作用范围远小于同期的温度场。
由表 5-2、表 5-3 与图 5-3、图 5-4，可归纳出三条规律。
其一，温度外高内低，且 1800 s 内热量尚未渗透至中心。1800 s 时表面温度 36.7855 °C、中心温度 33.5753 °C。由图 5-3 与表 5-2 各行沿径向单调可见，径向上始终维持由表及里的温度梯度，这意味着预热阶段远未结束：1800 s 约为热扩散特征时间 $\tau_T \approx 2.37 \times 10^3$ s 的 0.76 倍。
其二，水分变化仅局限在表层。表 5-3 显示中心含水率在四位小数下始终保持 2.5500，即表层微降、内部尚未启动；而表面含水率由 2.55 降至 1.5104 kg/kg，降幅 40.8%，变化带厚度约为 0.5-1.0 cm。这意味着水分脱除过程尚处在最外层。
其三，两个过程的时间尺度相差约 34 倍，且温差仍在建立过程中。两场特征时间由各自扩散尺度定义：

$$\tau_T = \frac{R_0^2}{\alpha} = \frac{\rho c_p R_0^2}{k} \approx 2.37 \times 10^3 \text{ s}, \quad \tau_C = \frac{R_0^2}{D} \approx 8.1 \times 10^4 \text{ s} \tag{5-11}$$

二者之比约 34，而表面与中心的温差随时间持续增大。这意味着”温度平衡快、水分脱除慢”是本题的固有结构，直接决定后续长时程干燥的耗时量级，而预热段的温度梯度尚未进入准稳态。
上述规律的共同根源是热扩散与质扩散的时间尺度相差约 34 倍（即 τ_T 与 τ_C 之比），而水分的作用层厚度由扩散长度决定而非由药材半径决定：

$$\ell_D(t) = \sqrt{Dt} \quad (5-12)$$

故该时段内部未变化是物理必然，不是数值误差或模型缺陷；且在第三类对流边界下径向温差很快饱和、整体温度随环境平缓上移，故问题一全程处于预热而非平衡阶段。

问题一的参数灵敏度结构如图 5-5 所示。

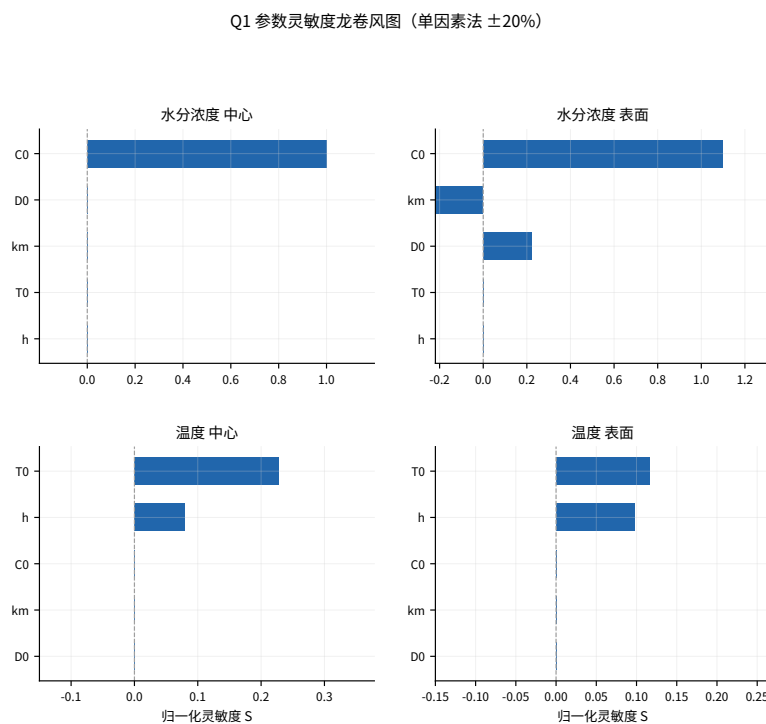


图 5-5 参数灵敏度龙卷风图

问题一参数灵敏度的龙卷风图给出各参数的归一化敏感度。由图 5-5 可见，环境温度与扩散系数为主控因素；全部算例中结果随参数单调变化，未出现临界敏感或反号现象。

5.1.6 模型检验（要点）

问题一的检验分四组：网格与时间收敛性给出离散误差带（空间二阶；温度场的网格收敛指数不超过 0.0001%，表面含水率不超过 0.011%）；与圆柱级数解析解对拍（含真实时变边界）；第二套独立实现互验；物理自检与守恒核算。另有两项专门检验（数据预处理对照与边界数据不确定度传播）见第六章 6.8 与 6.11。上述证据与图表统一见第六章，本节不重复其数值。

5.2 问题二：整个烘干过程（变物性·双向强耦合）

5.2.1 分析与建模思路

与问题一相比，变化集中在两处：物性体系由附录 2 换为附录 3，此时密度、比热容、导热系数与扩散系数均为含水率与温度的函数；耦合方式由严格解耦转为双向强耦合。

附录 3 的四项物性均为状态量的经验函数，扩散系数为两个独立负指数因子之积（含温度因子）；因该因子随温度升高显著提高扩散系数，这正是温度到水分这一耦合通道的物理来源。三套物性体系在初始状态处的取值对照见表 5-2。

表 5-4 三套物性参数在初始状态处的取值对照

物性（单位）	附录 2（问题一）	附录 3（问题二、三）	附录 4（问题四）
密度 ρ (kg/m ³)	820	976.4	989.5
比热容 c_p (J/(kg · K))	2600	3415.3	3394.4
导热系数 k (W/(m · K))	0.36	0.4830	0.2637
扩散系数 D (m ² /s)	4.9377×10^{-9}	5.6417×10^{-9}	1.0471×10^{-9}

注：均取含水率 2.55 kg/kg、温度 301.15 K 处；问题二、三、四的物性随含水率与温度变化，本表仅作初始状态对照，其演化见图 5-6。

耦合的机理是：水分扩散系数随温度升高而增大，而热物性随含水率下降而改变。若先算温度再算含水率而不回代，物性将始终取旧状态，误差会沿时间累积并以系统性偏差进入结果。本文采用交替推进并令物性在每个内部子步按局部状态更新，以化解这一矛盾^[9, 10]。物性随含水率的演化如图 5-6 所示。

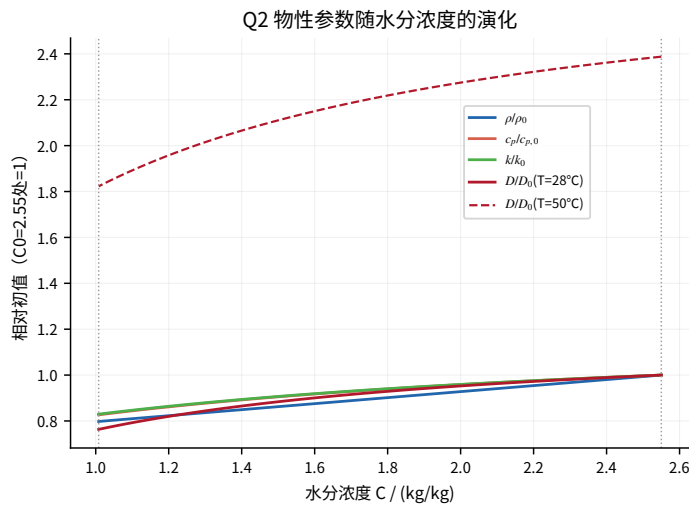


图 5-6 问题二变物性演化

给出密度、比热容、导热系数与扩散系数随含水率（及温度）的变化曲线。由图 5-6 可见，密度与比热容接近线性，而扩散系数呈强非线性，这正是后续耦合强度量化与灵敏度分析的焦点。

两场双向耦合的具体回路如图 5-7 所示。

由图 5-7 可见，温度经扩散系数影响含水率、含水率又经热物性反向影响温度，两条通路共同构成本问的双向依赖；这意味着”先温度后含水率”的单向顺序求解会在每个子步留下未被回代的状态偏差，故必须按子步更新物性并反复推进。

5.2.2 定解条件的变化

初始条件与问题一相同。环境数据在 14400 s 之后不再有实测覆盖，本文按附件 1 末段实测均值外推为恒定值：环境温度 50.00 °C、环境水分浓度 0.04999 kg/kg；该外推自问题二起生效。对恒温段实测点的线性趋势检验显示斜率不显著、与零无显著差异，故以段均值作常值延拓不引入趋势性偏差；其影响在第六章以灵敏度与扰动方式单独评估。

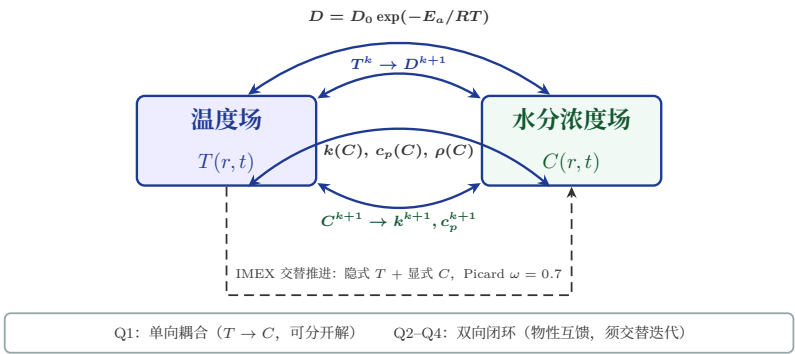


图 5-7 问题二双向耦合关系图

5.2.3 离散格式与求解算法

方程形式与问题一保持一致（仍取散度形式 (5-1)），变化在物性求值方式与耦合推进方式。

界面变系数的取法是本节的关键。控制体交界面上导热系数与扩散系数不再连续，本文取沿含水率的积分平均：

$$\langle D \rangle = \frac{1}{\Delta C} \int_{C_1}^{C_2} D \, dC$$

(5-13)

该式由界面通量守恒推出而非格式惯例。干燥尾部单个控制体内扩散系数跨越约两个数量级，取调和平均会严重低估有效扩散（问题三侧量化：取法错误可使烘干时长被高估 52% 以上）；故本文以积分平均为准，调和平均仅作对照臂。

该取法的必要性有其边界：界面两侧系数相差不大时两法在四位小数下不可辨识（问题一侧最大差 $8.5 \times 10^{-6} \text{ kg/kg}$ ），故其收益集中于干燥尾部的高湿差段。

时间推进采用交替推进：每个内部子步先解温度（全隐式，物性冻结于子步起点），再解含水率（欠松弛迭代， ω 取 0.7、容限取 10^{-10} ），物性在每个内部子步按局部状态更新，迭代收敛性的理论依据见文献^[11]。物性冻结造成的滞后正比于子步长度，子步取 1/32 s 使该滞后不进入四位小数量级。

上述交替推进的完整流程如图 5-8 所示。

由图 5-8 可见，每个内部子步按”更新物性、解温度、解含水率、判收敛”的顺序推进，物性在子步起点冻结、于下一步刷新；这意味着格式的显隐分离（两场全隐、物性显式求值）使误差来源可逐项归因，而不混入未离散的状态量。

5.2.4 结果与分析

按题面格式给出 0-3 h 内每隔 0.5 h、若干位置的结果，见表 5-5 与表 5-6。

表 5-5 3 小时内药材的温度（单位：℃；各列为到药材中心的距离，单位：cm）

时间/h	0	0.5	1.0	1.5	2.0
0.5	32.1893	32.3822	32.9660	33.9608	35.4131
1.0	40.3816	40.5536	41.0605	41.8783	42.9977
1.5	45.8468	45.9348	46.1930	46.6051	47.1401

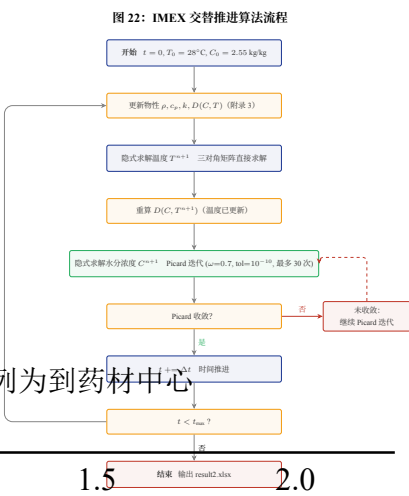


图 5-8 交替推进流程图

(续)

时间/h	0	0.5	1.0	1.5	2.0
2.0	48.4502	48.4881	48.5984	48.7736	49.0033
2.5	49.4670	49.4792	49.5136	49.5653	49.6609
3.0	49.8495	49.8553	49.8746	49.9101	49.9664

与表 5-5 同组时刻、同组位置上的水分浓度见表 5-6，其行次与题面模板一致。

表 5-6 3 小时内药材的水分浓度（单位：kg/kg；各列为到药材中心的距离，单位：cm）

时间/h	0	0.5	1.0	1.5	2.0
0.5	2.5499	2.5489	2.5255	2.3258	1.6486
1.0	2.5257	2.4947	2.3578	2.0230	1.4711
1.5	2.3860	2.3256	2.1344	1.8020	1.3475
2.0	2.1708	2.1084	1.9236	1.6259	1.2311
2.5	1.9567	1.9006	1.7360	1.4720	1.1166
3.0	1.7662	1.7165	1.5702	1.3333	1.0081

注：表 5-5、表 5-6 的行次按题面模板，自 0.5 h 起，故不列 0 h 行。

由表 5-5 与表 5-6，可归纳出两条结论。

其一，温度已基本趋于环境值，而含水率仍远高于达标阈值。3 h 时中心温度 49.8495 °C、表面温度 49.9664 °C，均已逼近环境温度 50.00 °C；而中心含水率 1.7662 kg/kg、表面含水率 1.0081 kg/kg，较初值的降幅分别为 30.7% 与 60.5%。这意味着干燥远未完成，与问题一揭示的两尺度分离相互印证。

其二，两层结果在重叠段的差异是模型必然，不是计算错误，而耦合强度可定量描述。变物性口径下的热扩散率较常物性低 14.2%（以问题一为基准），导致同一时刻的温度与含水率出现系统性差异，这意味着问题一与问题二的结果本就不应完全一致；同时两场之间的相互影响在干燥中段达到最强（耦合强度已按“温度 → 水分”与“物性演化”两条通道量化），这意味着交替迭代在该阶段的价值最大。

其后的机理需更正一处常见误解：在第三类传质边界下，表面通量由传质系数与表面同环境的浓度差决定，与扩散系数无关；扩散系数只决定内部含水率剖面的平坦程度。因此不能写成扩散系数越大传质越快、失水越多。同一时刻问题二的表面含水率高于问题一，原因正是其剖面更平坦，而非传质更快。

上述差异的来源可作分解如图 5-9 所示。

该图给出两问在 0-1800 s 重叠段内中心与表面量的差异及其成因分解；由图 5-9 可见，变物性口径下的热扩散率较常物性低 14.2%（以问题一为基准），与本节所报差异的量级一致。

5.2.5 模型检验（要点）

问题二的检验以退化对照为核心：把变物性退化为常物性后，与问题一的解析对拍口径比较，可同时验证求解框架与实现路径；此外还有强制解耦退化、独立实现互验、守恒核算、潜热对照与自适应步长对照。需要诚实声明的是，退化对拍证明的是求解框

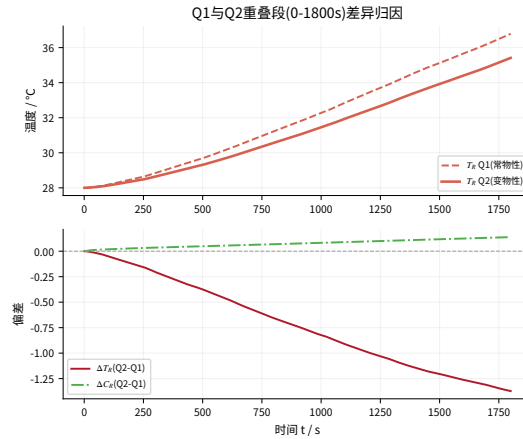


图 5-9 问题一与问题二重叠段差异归因

架与实现在常物性极限下正确，不能等同于变物性物理本身正确；后者由灵敏度分析与参数扰动共同支撑。证据与图表见第六章。

5.3 问题三：烘干所需时间（反演·事件驱动终止）

5.3.1 分析与建模思路

与问题二相比，本问仅动两处：时程由固定 3 h 改为事件驱动（达到判据即停），并新增判据要求全域干基含水率严格小于 0.15 kg/kg。

第一个关键判断是判据可否只在中心监测。由于中心到表面的扩散路径最长，且表面通量最先影响外层，中心恒为最慢点：任一时刻含水率沿半径自表面向中心单调递增，故“全域达标”与“中心达标”等价，判据只需在中心监测即可。

第二个关键判断是几何：本题设定下半径全程固定（收缩属问题四），故模型与离散可完全复用问题二，本问实质上是在既有求解器上加装终止机制。这一“复用加终止”的定位也决定了检验重点：既要验证终止机制本身，也要验证它与问题二在重叠段的逐位一致性。

5.3.2 定解条件的变化

终端判据取干基含水率严格小于 0.15 kg/kg，即等号处不触发；物性仍用附录 3，与问题二一致。需声明的是，本问把含水率区间由 1.0 kg/kg 以上扩展到 0.15 kg/kg，扩散系数经验式在低含水率端的外推幅度显著加大，外推段对烘干时长的贡献达 43%-66%；该代价已在第六章量化，并列为第七章的结论适用边界。另需界定本问所用“反演”一词的含义：此处指在严格判据下定位达标时刻，属单调、良态问题；它与扩散系数、辐射系数一类的病态参数反演性质不同，后者的统计反演方法与反问题正则化处理可参见文献^[19, 20]。

5.3.3 离散格式与求解算法

方程、边界与物性与问题二完全一致，沿用式 (5-1) 与式 (5-13)。本问新增两项机制：每个输出步检测全域含水率最大值是否小于 0.15 kg/kg；一旦触发，则在末两个输出步之间做子步级二分，定位达标时刻，精度取 10^{-3} s，即

$$\max_r C(\cdot, t_{\text{end}}) < 0.15, \quad |t_{\text{end}} - t^*| < 10^{-3} \text{ s} \quad (5-14)$$

内部步长不因输出变稀而放大：判据检测发生在输出步上，但时间推进仍保持 1/32 s 内部子步，以免牺牲精度、也免使二分区内的时间分辨率被输出间隔绑架。问题三的事件驱动求解流程如图 5-10 所示。

由图 5-10 可见，判据检测挂在输出步上、而二分定位发生在末两个输出步之间；这意味着达标时刻的定位精度由内部子步与二分容限共同决定（ 10^{-3} s），而不是由 6 h 的输出间隔决定，故表 5-7 的烘干结束行不因输出稀疏而变粗。

5.3.4 结果与分析

按题面格式给出每隔 6 h、若干位置的水分浓度，并给出烘干结束行，见表 5-7。

表 5-7 药材的水分浓度随时间和位置的变化（单位：kg/kg；各列为到药材中心的距离，单位：cm）

时间/h	0	0.5	1.0	1.5	2.0
6	1.0170	0.9881	0.9015	0.7550	0.5328
12	0.4562	0.4432	0.4031	0.3298	0.1642
18	0.2989	0.2916	0.2687	0.2253	0.0845
24	0.2378	0.2328	0.2167	0.1856	0.0665
30	0.2059	0.2019	0.1892	0.1641	0.0598
36	0.1859	0.1826	0.1718	0.1504	0.0566
42	0.1721	0.1692	0.1597	0.1407	0.0548
48	0.1619	0.1592	0.1507	0.1335	0.0537
54	0.1539	0.1515	0.1437	0.1278	0.0530
烘干结束时间	0.1500	0.1477	0.1402	0.1249	0.0526

注：行次按题面表 5-7 模板，自 6 h 起，末行为烘干结束时间；该行对应达标时刻 207113.094 s，即 57.5314 h，其时间列为精确值，已由结果文件逐格取回。

满足全域干基含水率严格低于 0.15 kg/kg 的烘干所需时间为

$$t_{\text{end}} = 57.53 \text{ h} \approx 2.40 \text{ 天}$$

经三档网格与三档时间步长核验，报告值带离散误差带 $\pm 0.08 \text{ h}$ ；合成口径为空间网格收敛指数估计与时间保守上界取较大者^[21]，两类不确定度分列报告、不相加。另以解析量级估计对该报告值作一次独立于数值实现的量级校验：下界为 25.8 h、上界为 97.7 h，本文报告值落在两界之内。长时程含水率演化如图 5-11 所示。

给出中心与表面含水率在全程 57.53 h 内的演化曲线。由图 5-11 可见，中心曲线在干燥后期明显变缓，而表面曲线早早进入平台，两条曲线的间距随时间不断拉大。

由表 5-7 与图 5-11，可归纳出两条规律。

其一，干燥过程呈明显的两段式：前 24 h 中心含水率相对初值下降约九成，其后约 33.5 h 仅由 0.2378 降至 0.1500 kg/kg，即最后少量水耗去了大半时间，烘干时长由末段的慢过程决定。

其二，中心恒为最慢点，而表面含水率早早趋于平衡。由表 5-7 可见每一时刻含水率沿半径单调递增、越靠近中心越高，且表面在 24 h 后基本稳定；这意味着判据只需在中心监测，与 5.3.1 的分析一致，同时干燥后期的控制环节趋势上转向药材内部扩散，表面侧参数（换热系数、传质系数、环境浓度）的影响均很小。

机理上，扩散系数经验式中含 $\exp(-0.45/C)$ 形式的因子，随含水率下降而急剧减小，导致干燥在后期自我减速。这意味着烘干时长的瓶颈在低含水率段；工程上应以最慢点即中心排产，并优先控制温度与改善物料特性，而非单纯加大风量。

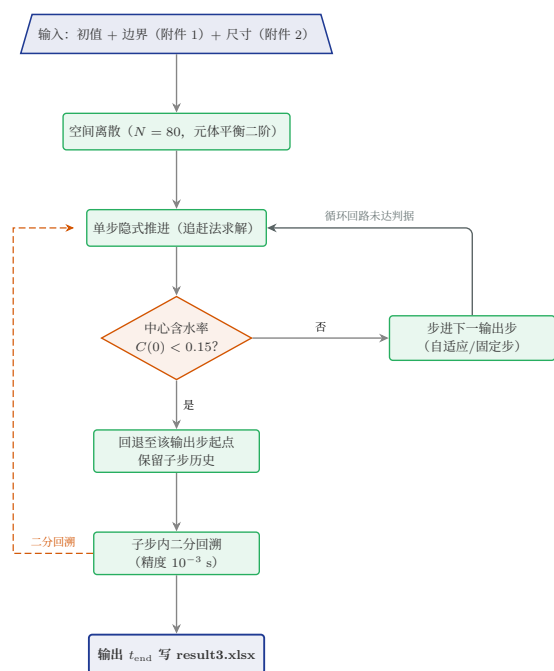


图 5-10 问题三求解流程图（事件驱动与子步级二分）

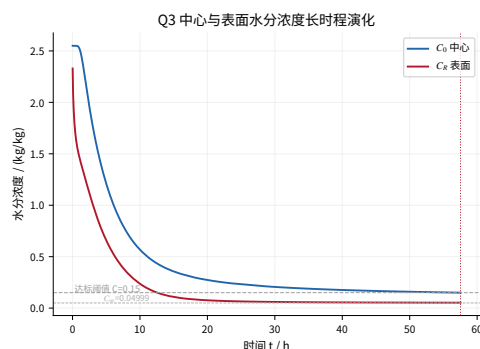


图 5-11 问题三长时程含水率演化

长时程还伴随一次主导机制转变：随含水率下降，扩散系数持续减小、质扩散特征时间不断延长，传质由外部对流控制转为内部扩散控制，故达标时间不可由单一时间常数线性外推——这正是采用事件驱动终止配合子步级二分、以严格判据直接锁定达标时刻的原因。

5.3.5 模型检验（要点）

问题三的检验以长时程重验与低含水率端外推评估为重点（后者对烘干时长的贡献达 43%-66%，为本问最大不确定性来源）；与问题二重叠段的逐位一致构成强交叉印证，其余检验项见第六章。

5.4 问题四：计入尺寸收缩的烘干时长（移动边界）

5.4.1 分析与建模思路

问题四在问题二、三的架构上引入两点实质变化：物性换用附录 4 体系^[5,6]；药材半径随时间收缩（由附件 2 给出 $R(t)$ ，2.000→1.198 cm），扩散域随之移动。

必须做坐标变换的理由来自两类困难：固定坐标系下计算域随半径收缩会使网格节点与输出位置错位、输出值须在移动网格上插值；表面附近控制体被不断压缩、体积趋零，表面列收敛性恶化（只能达到一阶），而表面恰是通量最大处。

本文引入物料坐标，把收缩域映射为固定计算域：

$$\xi = \frac{r}{R(t)}, \quad \xi \in [0, 1]$$

该变换带来两项收益：表面恒为节点，输出点恒为计算点^[15]；且物料坐标下同一物质点始终对应同一节点，质量方程无需引入对流项即可描述域的移动。其适用边界与仿射收缩假设绑定：若收缩分配随局部含水率变化，节点对应不再是一一对应，须改用加权

分配律——本文未做该对照，第七章不足 2 已列为改进方向。坐标变换的几何含义如图 5-12 所示。

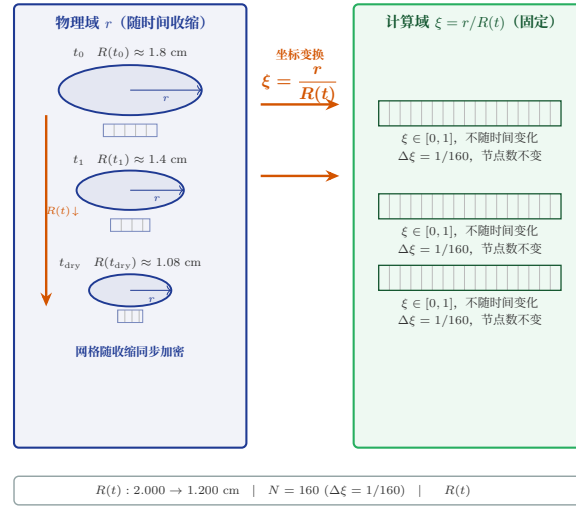


图 5-12 物质坐标变换示意图

给出物理域与计算域的对应关系；由图 5-12 可见，物料坐标下的节点数不随时间变化，这意味着网格与输出点全程保持同构，是本问消除上述两类数值困难的关键。

5.4.2 定解条件的变化

收缩律 $R(t)$ 由附件 2 的节点线性插值给出（145 个实测点、间隔 1800 s），259200 s 之后取常值 1.198 cm。本文采用仿射收缩假设（物质点半径按比例缩放）：附件 2 只给出半径的单一序列、药材内部各处的收缩分配未被规定，故按仿射处理，并在第三章假设中一并声明；含水率按干基定义，收缩不改变各处干物质的量，这一点在 5.4.3 的推导中自然体现。终端判据与问题三相同，仍为全域干基含水率严格小于 0.15 kg/kg。

本问输入、参数与输出均由题面或附录给定，不存在待优化决策变量，故不套用“目标函数加决策变量加约束”的优化模板，工作集中在离散格式与问题特化策略。

5.4.3 方程在物料坐标下的形式与算法

经坐标变换，质量方程在物料坐标下写为

$$\frac{\partial \tilde{C}}{\partial t} = \frac{1}{\xi R^2} \frac{\partial}{\partial \xi} \left(\xi D \partial_{\xi} \tilde{C} \right) \quad (5-15)$$

其中 $\tilde{C}(\xi, t) = C(r, t)$ 为物料坐标下的含水率。式 (5-15) 为干基形式，不含对流项与几何压缩项；所以能写成这一形式，是因为变量本身以干基计量，脱水不改变干物质的质量基准。能量方程则必须取守恒形式，把几何压缩项自动包含在时间导数之内：

$$\xi \frac{\partial}{\partial t} \left[\rho c_p T R^2 \right] = \frac{\partial}{\partial \xi} \left(k \xi \partial_{\xi} T \right) \quad (5-16)$$

上述处理属于移动边界问题的边界固定化方法族^[12, 15, 16, 17, 18]，其中收缩因子的数值路径见文献^[13]。表面条件经坐标换算后为

$$-\frac{k}{R} \partial_{\xi} T \Big|_{\xi=1} = h [T_R - T_{\infty}(t)], \quad -D \partial_{\xi} \tilde{C} \Big|_{\xi=1} = k_m R [\tilde{C}_R - C_{\infty}(t)] \quad (5-17)$$

两点值得说明。其一，守恒形式 (5-16) 中热容位于时间导数之内，与问题二、三的非守恒形式相差一项热容随时间的变化率与温度之积，即 $T \partial_t(\rho c_p)$ ；等价地，在固定物理位置上观察时，能量方程的时间导数会多出一项由半径变化率贡献的对流式作用，这就是几何压缩项进入方程的来源。该项是“热容随脱水衰减、同时又乘上体积压缩”这一耦合的物理来源，也是问题四出现压缩温升的机理；若沿用非守恒形式，这一项会被丢掉，压缩温升也随之消失。其二，界面系数仍取沿含水率的积分平均，其解析式与数值采样方式与问题二、三一致，以保证三个问题在同一离散口径下可比。

对报告值另作一次独立量级校验，即解析两界夹逼。下界由表面控制（集总极限）给出 8.36 h，该界严格成立；上界由固定半径数值解给出 129.48 h，它是同一问题在不收缩情形下的严格数值上界；另以渐近时标（半径平方与特征扩散系数之比，291.9 h）作量级校验。本文报告值 50.6523 h 落在两界之内，且与两界的相对位置符合“收缩加速干燥”的机理预期。

空间离散仍用元体平衡，径向区间数取 160（为问题二的 2 倍，因移动域的表面列收敛较慢）。时间上仍为交替推进，并对热容项采用预估-校正，使交付解中的热容项严格取更新层值，其依据是不做校正时物性冻结造成的滞后偏差达 $O(1\text{ K})$ ，已超出四位小数点的取值预算；事件判定与子步级二分（精度 10^{-3} s ）与问题三相同，并设 240 h 的熔断上界。

5.4.4 结果与分析

按题面格式给出每隔 6 h、若干位置（含药材表面）的水分浓度，见表 5-8。

表 5-8 计入收缩后药材的水分浓度随时间和位置的变化（单位：kg/kg；各列为到药材中心的距离，末列为药材表面，单位：cm）

时间/h	0	0.5	1.0	药材表面
6	1.6276	1.4621	0.9868	0.4173
12	0.7021	0.6248	0.3930	0.1598
18	0.3958	0.3576	0.2340	0.0869
24	0.2794	0.2561	0.1763	0.0665
30	0.2232	0.2066	0.1477	0.0591
36	0.1909	0.1779	0.1306	0.0558
42	0.1699	0.1592	0.1193	0.0540
48	0.1552	0.1460	0.1110	0.0529
烘干结束时间	0.1500	0.1413	0.1081	0.0526

注 1：因半径收缩，1.5 cm 与 2.0 cm 两处自烘干开始后即越出药材边界（由附件 2 复算得 6 h 时半径约为 1.374 cm 且半径单调不增），故表 5-8 不设该两列，末列改为药材表面。该列契约与题面表 5-8 的模板一致，也与结果文件的列一致。

注 2：行次按题面表 5-8 模板，自 6 h 起，故不列 0 h 行；烘干结束行对应达标时刻，此时半径约为 1.200 cm，该行已由结果文件逐格取回。

计入尺寸收缩后的烘干时长为

$$t_{\text{dry}} = 182348.109\text{ s} = 50.6523 \pm 0.0249\text{ h}$$

空间离散对烘干时长的贡献仅 0.0002 h，可以忽略；同一网格序列上表面含水率列仅为 0.99，即一阶，这正是本文对表面列额外加密的直接依据。时间三档呈非单调，故取保守上界 $\pm 0.0249\text{ h}$ ；合成口径为两者取较大者。

由表 5-8，可归纳出两条结论。

其一，收缩加快干燥。计入收缩后的 50.6523 h 短于问题三固定半径的 57.53 h。其净效应可分解为物性变化带来的延长与收缩带来的缩短两项，二者量级相当而方向相反（物性换套使时长延长约 71.9 h，收缩使其缩短约 78.8 h，净缩短 6.9 h；两项均为对照实验口径，不得作为独立结论引用）。这意味着几何收缩是本题中不可忽略的一阶效应，若沿用固定半径将明显高估烘干时长。

其二，守恒形式的能量方程揭示压缩温升，且中心仍恒为最慢点。最高温度达到 54.63 °C，高于环境温度上限；该升温的时间位置与量级与准稳态估计一致，并被绝热包络严格覆盖。这意味着该现象是移动边界模型的内秉效应，而非数值振荡；其物理来源是体积压缩对热容的影响，即式 (5-16) 中热容位于时间导数内所产生的附加升温。同时表 5-8 中每一时刻中心含水率均最高，48 h 时中心 0.1552 kg/kg、表面 0.0529 kg/kg，这意味着问题三的判据等价性结论在移动边界情形下依然成立。

压缩温升的机理与绝热包络如图 5-13 所示。

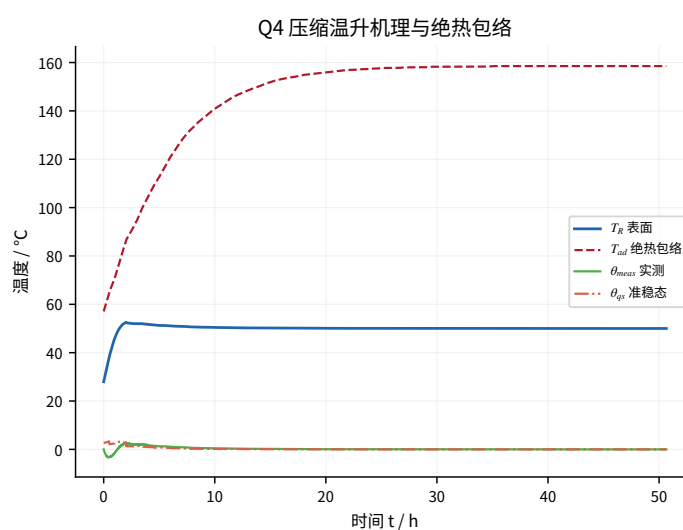


图 5-13 压缩温升机理与绝热包络

该图给出最高温度的时间位置、准稳态估计与绝热包络；由图 5-13 可见，该升温与准稳态估计在量级与时相上一致，并被绝热包络严格覆盖。

5.4.5 模型检验（要点）

问题四的检验包括网格收敛与时间误差带、常物性固定半径情形的级数解退化对拍、三项几何退化自检（绝热不变量、固定半径退化与问题二结果核一致、输出契约核对）、程序化断言、局部与全局灵敏度、守恒核算（质量与能量各一条）、潜热对照、判据敏感度、低含水率端外推不确定度，以及界面取法对照（调和平均仅作对照臂）。问题四的全局灵敏度结构按各因素的一阶敏感度指数排序，其中低含水率端截断因子为第一主导因子，其敏感度指数为 0.337，这意味着本问结论对低含水率段经验式的取法最为敏感，与 5.3.2 所声明的适用边界相互指向。证据与图表见第六章。

本问另有两处断言须声明：压缩温升使中心温度超过环境温度上限，故朴素极值原理在问题四不成立，看门狗断言相应改为温度不超压缩包络上界；且移动边界下交替推进的物性滞后为 $O(1\text{ K})$ 量级的系统偏差，问题四与固定域问题的差异属能量方程形式差异（模型级），不是实现缺陷。

5.5 四问结果的横向对照与讨论

前四节分别给出各问的建模、求解与结果。本节把四问的核心结果与可靠边界集中对照，不重复各问的结果呈现。为免歧义，本节所称含水率均指干基含水率。

四问核心结果对照见表 5-3。

表 5-9 四问核心结果对照

项	问题一	问题二	问题三	问题四
物性体系	附录 2（常物性）	附录 3	附录 3	附录 4
几何	固定，半径 2 cm	固定	固定	收缩，2.000→1.198 cm
时长口径	固定 1800 s	固定 3 h	反演（事件驱动）	反演（事件驱动）
时长结果	—	—	57.53±0.08 h	50.6523±0.0249 h
中心量（末端）	温度 33.5753 °C；含水率 2.5500 kg/kg	温度 49.8495 °C；含水率 1.7662 kg/kg	含水率 0.1500 kg/kg	含水率 0.1500 kg/kg
表面量（末端）	温度 36.7855 °C；含水率 1.5104 kg/kg	温度 49.9664 °C；含水率 1.0081 kg/kg	含水率 0.0526 kg/kg	含水率 0.0526 kg/kg；半径 1.200 cm
最高温度	36.7855 °C	49.9664 °C	—	54.63 °C

由表 5-3 可见，四问在物理上是同一过程的四个切片，结果的量级与趋势互为印证；“温度快、水分慢”这一时间尺度结构，也解释了为什么预热阶段的结果很早就接近环境条件，而干燥时长却由另一个更慢的过程决定。

跨问一致性共三条。其一，问题三与问题二在前 3 h 的逐时刻结果逐位一致，说明“复用加终止机制”未引入额外近似，长时程结果的偏差只能来自外推口径而非求解路径。其二，令问题四的半径取常值并使用附录 3 物性，其生产核与问题二结果一致至相对差 3.7×10^{-8} ，已属舍入噪声量级，说明移动边界是加装式而非替换式。其三，问题一与问题二在 1800 s 处的系统性差异（中心温度差 -1.39 °C、表面含水率差 $+0.138$ kg/kg）来自两问的物性换套，属模型必然而非计算错误。需强调的是，四问的衔接是“同量级且趋势连续”，而非“数值无任何跳变”。

从问题三的 57.53 h 到问题四的 50.6523 h，净缩短约 6.9 h，由两个方向相反的效应叠加而成：物性体系由附录 3 换为附录 4 使时长延长约 71.9 h，半径收缩（扩散路径变短）使其缩短约 78.8 h。两者量级相当而方向相反，故“收缩加快干燥”不能直接从结果表读出，须给出分解才能判断该结论对物性口径的敏感性；本文据此在第七章说明该结论的适用边界。

把四问结果连起来可得一条机理链：扩散系数随含水率下降而骤减，干燥因此在后期自我减速，烘干时长主要由低含水率段决定；收缩缩短了扩散路径，从而显著提前达标，而几何压缩在守恒形式下表现为压缩温升。工程上的启示是应以最慢点（中心）排产，并把控制重点放在温度与物料特性上。

本文结果的可靠边界见表 5-4（须与结论同页出现）。

表 5-10 结果的可靠边界

类别	量级	说明
离散误差带（问题三）	$\pm 0.08\text{ h}$	空间网格收敛指数估计与时间保守上界取较大者
离散误差带（问题四）	$\pm 0.0249\text{ h}$	同上；空间项可忽略，仅 0.0002 h
低含水率端经验式外推（问题三）	贡献 43%-66% 的时长	属最大不确定性来源
低含水率端经验式外推（问题四）	贡献 16.8%-55.1% 的时长	其概率区间的先验为主观设定，引用须声明
界面取法	取法错误可使时长高估 52% 以上	本文取沿含水率的积分平均
潜热忽略	温度系统性偏差 0.07-0.10 $^{\circ}\text{C}$	问题四的时长仅 +0.018%

口径纪律：上述两类不确定度分列报告、不得相加；报告值一律取”空间与时间误差带取较大者”这一合成口径。表中除前两类离散误差带外，其余各项均为模型设定或经验式取法带来的系统性影响：前者随计算资源增加而收敛，后者只能通过口径声明与灵敏度分析加以界定——两者分开报告，即是不把可收敛误差与不可收敛误差混为一谈。效应分解的两项（+71.9 h / -78.8 h）均为对照实验口径，不是相互独立的答案，其差值与净效应仅用于解释变快或变慢的原因，不得作为独立结论引用。

另按题面要求逐项核对交付物落位：表 5-2 / 表 5-3 见 5.1.4，表 5-5 / 表 5-6 见 5.2.4，表 5-7 及烘干结束行见 5.3.4，表 5-8（含药材表面）见 5.4.4，四个结果文件（四位小数）列入附录支撑材料，均已生成并逐格核对；问题三、问题四的核心答案（烘干时间）均已带离散误差带给出。

从上述结果可以引出三点工艺含义。其一是控温上限的余量：由于收缩与变物性共同产生压缩温升，物料内部最高温度（54.63 $^{\circ}\text{C}$ ）可高于环境温度上限，故工艺控温设定不宜直接按环境温度取上限，应留出该内秉温升的余量。其二是排程口径：收缩使烘干时长较固定半径情形缩短约 61%，若按常半径假设估算会系统性高估时长，故实际排程应以含收缩的模型结果为准。其三是停机判据：低含水率端经验式外推贡献了 43% 至 66% 的时长，末端时段的不确定性最大；本文认为，此时以含水率阈值判据停机、而非以定时停机，可把结论的不确定性限制在可控范围内。值得注意的是，这三点均来自本文已登记的对照实验口径，不构成新的独立结论。

6 模型检验

本章对全文模型作跨问系统检验。各问 5.x.5 只给出该问专属检验的要点，证据、图表与完整结论一律归本章，两处不复述同一结果。检验覆盖收敛性与误差带、解析对拍、独立实现互验、物理自检与断言、守恒核算、局部与全局灵敏度、预处理与简化代价、判据与经验式外推不确定度、输入不确定度与数据同化共十一类，且每一类都给出可核验的量值。为免歧义，本章所称含水率均指干基含水率。

6.1 网格与时间收敛性（含离散误差带）

空间取三档网格作加密序列，以广义理查森外推估计观测收敛阶与网格收敛指数；时间取三档步长作对照；对问题一的温度场另做步长无关性检验。为使结论可核，先给出两个定义式：报告值以区间给出，区间半幅取两类误差中的较大者，即

$$y \in [\hat{y} - e, \hat{y} + e], \quad e = \max(e_{\text{disc}}, e_{\text{extrap}}) \quad (6-1)$$

而观测收敛阶与网格收敛指数由三档序列估计：

$$p = \frac{\ln |(f_3 - f_2)/(f_2 - f_1)|}{\ln r} \quad (6-2)$$

三档网格与三档步长的收敛序列见图 6-1。

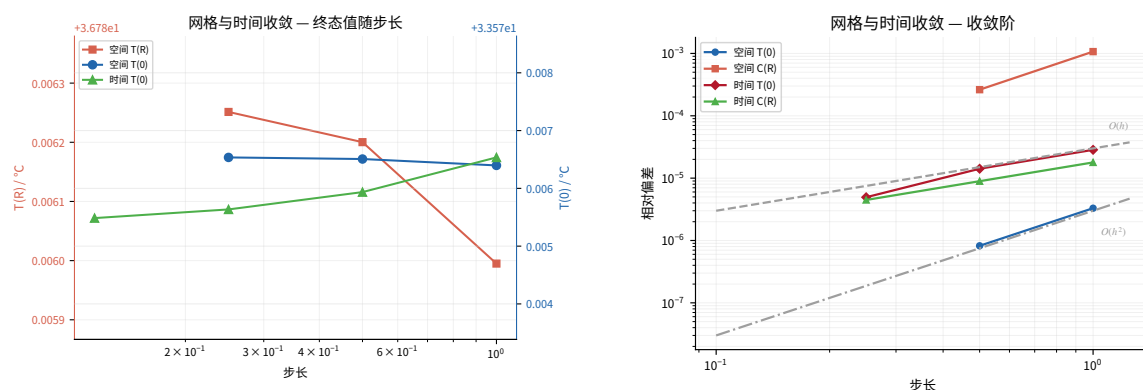


图 6-1 网格与时间收敛性

由图 6-1 可见，空间离散按问按场呈二阶（表面含水率列为一阶），且时间侧温度场步长无关——三档序列单调、沿半径与时间未见振荡或孤立异常点，故取 0.25 mm 为交付网格。

结果：三档序列一致显示”空间二阶、时间已消除离散误差”。分场看：问题一温度场（中心 / 表面 / 均方根）观测阶 2.002 / 2.006 / 2.003，网格收敛指数不超过 0.0001%；问题一含水率场为 2.014（均方根）与 2.028（表面列），表面列网格收敛指数 0.011%；问题三观测阶 1.650、网格收敛指数 0.0011%；问题四中心含水率在 24 h 上观测阶 1.98、烘干时长序列上 1.91，而表面含水率列为 0.99（一阶）。时间侧：问题三三档呈非单调，取保守上界 ±0.0802 h；问题四保守上界为 ±0.0249 h，空间项仅 0.0002 h 可以忽略；问题一温度场分段步长取 1 s 与 60 s 相差 3.1×10⁻¹¹ °C，即时间离散误差已归零，而含水率侧为后向欧拉一阶、子步 0.1 s 下误差量级仅 10⁻⁵ kg/kg。

结论：报告值已网格收敛，且其误差带是”可复算、可复现”的，而非经验估计——两类不确定度分列报告、不得相加，报告值一律采用”空间与时间误差带取较大者”的合成口径。这意味着四位小数输出的末位仍受误差支配，故正文对末位一律不作过强解读。

补充对照为三项排除性或等价性对照。其一为界面取法对照（问题一侧）：沿含水率的积分平均与沿扩散系数倒数的调和型积分给出含水率最大差 8.5×10⁻⁶ kg/kg，在四位小数下不可辨识；同一对照在问题四的干燥尾部进一步放大，这正是 5.2.3 采用积分平均的直接理由。界面取法与网格的二维对照见图 6-2。

由图 6-2 可见，错误取法下网格不收敛，这从反面确认了积分平均口径的必要性。

其二为离散分辨率扫描：步长与网格四种配置间的散布不超过 0.15%，均非结论的主导不确定度。其三为自适应时间步：按含水率变化率在 1/64 s 至 1/4 s 之间调整内部子步，其子步数降至固定子步的 18.75%、关键量偏差不超过 9.3×10⁻⁶ kg/kg，故报告值采用固定子步口径，以保证可复现。

6.2 解析解对拍

在常物性与分段常边界条件下，把问题退化为无限长圆柱的级数解问题（含真实时变边界的分段线性杜哈梅尔积分路径）；问题四另在常物性加固定半径的条件下作退化对拍。对拍目的在验证实现是否正确，而非验证模型是否完备。常物性与分段常边界下的级数解对拍见图 6-3。

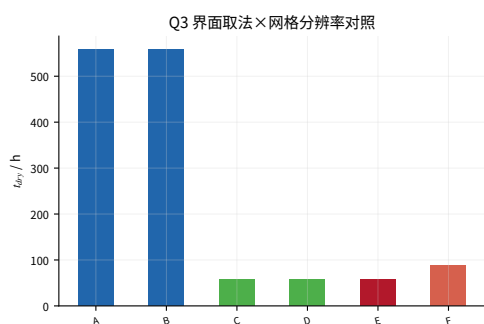


图 6-2 界面取法与网格对照

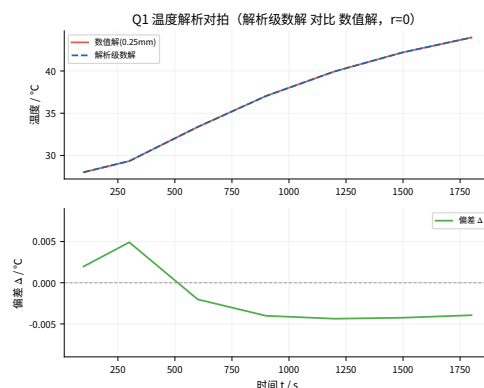


图 6-3 解析级数解对拍

由图 6-3 可见，改用精确推进后偏差转为随网格二阶下降。

结果：问题一的对拍偏差为 10^{-3} °C 量级；在改用时间方向精确推进后，该偏差转为随网格二阶下降（网格步长 1 / 0.5 / 0.25 mm 下分别为 $3.886 / 0.975 / 0.244 \times 10^{-4}$ °C），说明残差已归于空间离散；含水率侧对应为 $1.029 / 0.267 / 0.083 \times 10^{-3}$ 。问题四的退化对拍最大温差为 2.4×10^{-10} K。

结论：实现路径与解析解一致，对拍通过；其含义是本文的离散格式与代数装配未引入系统性偏差，残差已全部归于空间离散，可继续加密网格进一步压低。适用边界亦须声明：级数解在小的傅里叶数处不适用，且仅覆盖常物性与分段常边界情形。

6.3 独立实现互验

另写一套独立实现（显式格式，问题三侧为 Crank-Nicolson 与 Rannacher 启动轨），在相同输入下比较全场结果；两套实现的界面系数口径已预先对齐为沿含水率的积分平均，故其差值只反映时间格式差异，而不混入口径差异。该对照以逐点全场偏差的形式完成，差异集中在时间格式、量级低于四位小数，故本项由数值承载、不再单独制图。

结果：问题一的最大温差为 1.004×10^{-4} °C、含水率最大差为 1.710×10^{-5} kg/kg；问题三与主实现的相对差为 3×10^{-7} ；问题四的几何退化自检与主核一致至 3.7×10^{-8} 。

结论：两套实现给出同一答案，排除了单一代码路径的隐性缺陷与路径依赖；这意味着报告值不是某一份代码的产物，而是与实现无关的模型解。

6.4 物理自检与看门狗

设置方向性、单调性与上界控制三类程序化断言（共 6 项），并用人为主观注入错误的方式演示验证断言确实能捕获错误；其中极值类断言在问题四改为温度不超压缩包络上界，因压缩温升可使中心温度超过环境上限、朴素极值原理在该问不成立。问题四另设绝热不变量判决实验。

结果：全部断言通过；绝热条件下温度与半径平方之积的不变量偏差为 3.1×10^{-12} ；人为注错均被捕获。

结论：离散格式具备可自检的数学性质——三对角矩阵严格对角占优构成 M 矩阵，使离散极值原理先有理论、后设断言，故该组断言不是事后凑数。

6.5 守恒性核算

对质量与能量分别做区域积分闭合核算（问题四含时变环境通量），均以相对残差表述：

$$r_m = \frac{|m(t) - m(0) - \int_0^t \dot{m}_b dt|}{m(0)}, \quad r_e \text{ 同构} \quad (6-3)$$

其理论依据为元体平衡的代数严格守恒：相邻控制体的内部界面通量在求和时两两抵消，区域总量的变化只由最外层边界通量决定，因此残差只能来自时间积分与输出截断，而不可能来自内部装配。质量与能量守恒的总账见图 6-4。

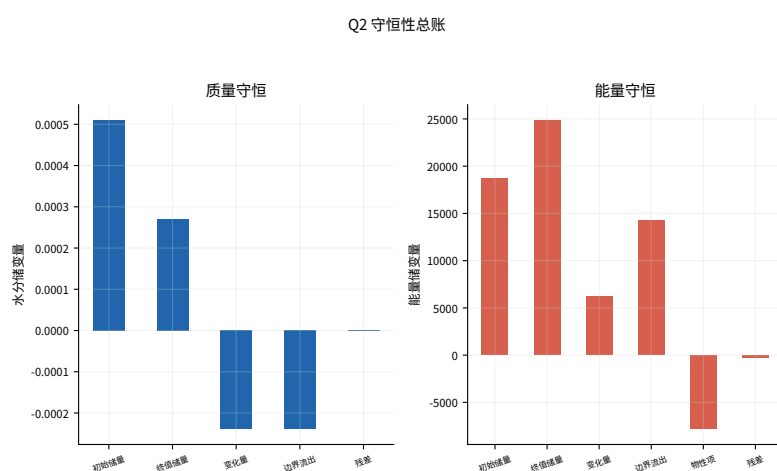


图 6-4 守恒性总账瀑布图

由图 6-4 可见，残差随问题推进未系统性增长，长时间积分的守恒性可控。

与之互为印证的是两条过程证据：温度与含水率的相轨迹形态与物性耦合一致、无越界点；全断面失水自表面向中心推进、无反向回流。

结果：问题一的质量与能量相对残差为 1.16×10^{-2} 与 6×10^{-4} （残差归因于 21 点输出加四位小数输出网格的积分截断）；问题二为 1.9×10^{-3} 与 1.8×10^{-2} ；问题三为 2.4×10^{-3} 与 2.2×10^{-3} ；问题四为 2.4×10^{-4} 与 2.0×10^{-4} 。

结论：守恒残差均在判据之内且未系统性增长，说明长时间积分的守恒性可控；问题四的残差最小，与其守恒形式方程的形式一致性相符。该组作用于全域积分量，能暴露逐点比对看不见的实现缺陷（如能量存储项一类的装配错误），与网格收敛指数互补而非重复。

6.6 参数灵敏度（局部）

对关键参数作单因素扰动：扩散系数、对流换热系数、对流传质系数各扰动 $\pm 20\%$ ，初始含水率扰动 $\pm 5\%$ ，初始温度扰动 $\pm 1^\circ\text{C}$ ；结果以龙卷风图呈现。单因素灵敏度以归一化形式报告，使不同量纲的参数可比：

$$S_i = \frac{\Delta Y / Y}{\Delta X_i / X_i} \quad (6-4)$$

其中 Y 为响应量、 X_i 为第 i 个参数。

结果：环境温度与扩散系数是烘干时程末端温度的最主要影响；对流传质系数的作用分问不同——对时程末端温度影响小，但对问题二的表面含水率仍较敏感（归一化灵敏度约 0.41 至 0.47，为该量的主控参数），而对问题三、问题四的烘干时长影响很小（约 0.15）。全部算例中结果随参数单调变化，未出现临界敏感或反号现象。

结论：结果对参数稳健，无临界敏感参数；这意味着报告值不会因参数的小幅估计误差而失去意义。按 $\pm 10\%$ 与 $\pm 20\%$ 两档幅度复核，上述参数影响排序与“无临界敏感参数”的判断不变，即结论不因参数估计误差的档位选择而改变。

6.7 参数灵敏度（全局）

采用两族独立方法（筛选法与方差分解法，后者含相关比）在参数空间内联合抽样，以检验单因素结论在参数联合变动下是否仍成立，并识别参数交互。

结果：全局方法与单因素方法给出一致的重要度排序；扩散系数的交互作用显著。问题四侧的联合抽样进一步显示，低含水率端截断因子为第一主导因子（敏感度指数 0.337，远高于扩散系数的 0.145、传质系数的 0.043、环境温度的 0.027、环境浓度的 0.021 与换热系数的 0.002；该因子与烘干时长的相关系数为 -0.833 ），与 6.10 的外推不确定度结论相互印证。

结论：重要度排序稳健，但参数并非彼此独立——这意味着单因素结论只能用于排序，不能直接外推为“调参收益”，本文据此不采用该外推路径。

6.8 数据预处理对照

对附件 1 的环境数据做平滑与不平滑两套前处理，其余设置不变。

结果：两者对温度结果的影响不超过 $1.08 \times 10^{-2} \text{ }^\circ\text{C}$ 。

结论：本文不做平滑是可接受的选择，环境数据中的实测波动不会传导为结论层面的偏差。

6.9 简化代价对照（潜热）

在能量方程的表面加入汽化潜热吸热项，重算并比较，以量化忽略蒸发潜热这一简化的代价。

结果：计入潜热后温度系统性地变化 0.07 至 0.10 $^\circ\text{C}$ ；问题四的烘干时长仅变化 0.018%。

结论：该简化的代价已知且有界。题面未给汽化潜热与吸湿等温线，不含源项属给定信息下的必要简化而非取舍失误，其代价已在第七章列为结论的适用边界。

6.10 判据敏感度与低含水率端外推不确定性

对达标阈值作非对称扰动（问题三、问题四均为收紧 13.3%，对应阈值为 0.13 kg/kg，与放宽 20%，对应阈值为 0.18 kg/kg，两档）；对低含水率端扩散系数经验式的适用上限作多档扫描，并给出相应的概率区间。低含水率端截断因子的扫描结果见图 6-5。

由图 6-5 可见，外推段贡献 43% 至 66% 的时长；干燥速率随含水率的演化显示低含水率段速率骤降，解释后期自我减速；问题四侧的外推不确定度为 16.8% 至 55.1%，其先验属主观设定。

结果：低含水率端外推对烘干时长的贡献，问题三为 43% 至 66%、问题四为 16.8% 至 55.1%，是全文最大的一项模型不确定性；概率区间所依据的先验分布属主观设定。

结论：核心答案对判据阈值敏感——收紧 13.3% 使烘干时长增加约 48%、放宽 20% 使其减少约 33%（问题三），问题四对应为增加 28.2% 与减少 23.4%；该敏感性属判据

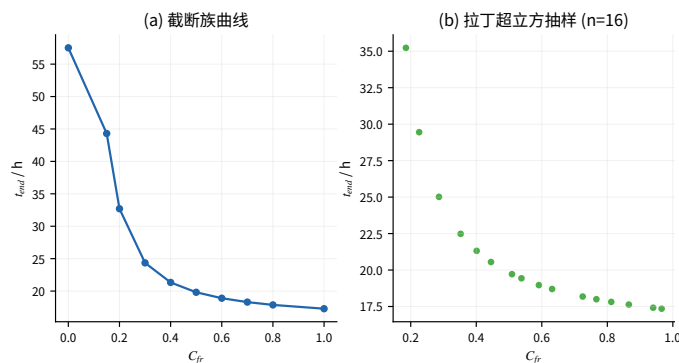


图 6-5 低含水率端外推不确定度

选择层面，不动摇题意裁定。故本文在给出烘干时间的同时同步声明该外推区间，引用概率区间时须注明其先验为主观设定，且不得与离散误差带相加。

6.11 输入不确定度与数据同化对照

对附件 1 的环境时序按实测散布叠加噪声（环境温度的标准差取 $0.1561\text{ }^{\circ}\text{C}$ ），取 40 个严格样本统计结果带宽；另行数据同化（状态空间滤波与平滑，状态取水平与斜率）给出后验带宽，作为与噪声抽样相互独立的第二条路线。

结果：噪声抽样下中心温度落在 33.5393 至 $33.6065\text{ }^{\circ}\text{C}$ 、表面温度落在 36.7148 至 $36.8560\text{ }^{\circ}\text{C}$ 、表面含水率落在 1.510344 至 1.510408 kg/kg ；数据同化的后验带半幅为中心温度 $5.2\times 10^{-3}\text{ }^{\circ}\text{C}$ 、表面温度 $1.0\times 10^{-2}\text{ }^{\circ}\text{C}$ ，方向与前者一致、带宽更窄。末位归属分场不同：温度由输入噪声支配，其量级约 $10^{-2}\text{ }^{\circ}\text{C}$ ，约为网格误差带的 10^2 倍；而表面含水率由网格离散支配，网格误差带 $\pm 2\times 10^{-4}$ 远大于噪声贡献 2.7×10^{-6} 。

结论：两条相互独立的路线给出方向一致的带宽，说明输入不确定度的量级可信；报告值的末位不应作为精确值解读，温度末位尤其如此。

7 模型评价与推广

7.1 模型优点

优点 1 离散格式严格守恒，且其数学性质可预先自检。空间上采用元体平衡（单元中心有限体积），保证逐控制体严格守恒；含水率方程的离散矩阵严格对角占优、非对角元非负，构成 M 矩阵，配合非负性与次随机性质，使离散极值原理先有理论、后设断言——本模型的物理自检不是事后凑数，而是格式性质的直接推论。其边界为固定计算域（问题一至问题三），问题四的移动边界另见 5.4.5。

优点 2 温度场时间方向精确推进，时间离散误差归零。针对常系数线性系统这一结构性质，采用矩阵指数与分段线性杜哈梅尔积分做解析推进，使时间维不再引入近似。效果可以量化：步长由 1 s 放宽到 60 s ，结果仅相差 $3.1\times 10^{-11}\text{ }^{\circ}\text{C}$ 。这意味着该场的精度不再受时间步长支配，而只由空间离散决定，从而把调参的自由度集中到真正决定精度的那一维上。

优点 3 中心奇点用两条独立推导互证，装配可靠。中心控制体为半格，其系数的两条推导路径，即洛必达极限与半格通量平衡，相互独立，所得系数一致，解级比对差异为 $2.8\times 10^{-14}\text{ K}$ 。这意味着最容易出错的一处几何奇异点由两条互不依赖的路径交叉确认，而不是依赖单一路径的自洽。

优点 4 误差带定量给出，验证体系可迁移。报告值均附离散误差带（问题三为 $\pm 0.08\text{ h}$ 、问题四为 $\pm 0.0249\text{ h}$ ），且两类不确定度分列报告、不得相加；所建立的“收敛性

检验、解析对拍、独立实现互验、守恒核算、灵敏度分析”五类手段，在变物性与移动边界情形下同样适用，无需重构验证方案。这一点使本文的验证工作不止服务于本题，而具有可复用的方法学价值。

定位说明：本文为单一机理模型的四问递进求解，未采用多模型融合——本题属完全自包含型（数据、参数与物性公式均由题面给出），引入数据驱动模型缺乏训练数据支撑，故改进集中于数值格式与问题特化策略。

7.2 模型不足与改进方向

不足 1 物理简化带来系统性但已量化的偏差。本文忽略蒸发潜热与热质交叉扩散（对应假设 4），并把表面换热与传质系数全程沿用附录 2 的取值（对应假设 5）、传质驱动力取干基浓度差（对应假设 6）。经对照计算，计入潜热后温度系统性变化 0.07–0.10 °C、问题四的烘干时长仅变化 0.018%，属已知且有界。

改进方向：若可获得汽化潜热与吸湿等温线，可把能量方程加回源项、把传质驱动力改为活度差，并用干燥实验标定表面换热与传质系数，从而把本节的三项简化一并解除。

不足 2 降维与几何演化假设限制了适用面。一维轴对称（对应假设 1）忽略了端面效应，两端面面积约占总表面积 7.4%，使轴中部结果的偏差方向略偏保守；问题四采用仿射收缩（对应假设 8），未考虑表层先干先缩的非均匀分配。

改进方向：对端面效应可用二维轴对称模型做一次对照，本文已由长径比判据说明其量级；对非均匀收缩可改用按局部含水率加权的收缩分配律，并以灵敏度分析检验其影响。需要说明的是，本文未做非均匀收缩的对照计算，此处只给出改进路径而不宣称其结果。

不足 3 低含水率端的外推构成主要不确定性。扩散系数经验式在低含水率端的外推对烘干时长的贡献，问题三达 43%–66%、问题四达 16.8%–55.1%，是全文最大的一项模型不确定性；其概率区间所依据的先验分布属主观设定，须在引用时声明，且不得与离散误差带相加。此外，表面含水率的网格分辨率存在限制，界面变系数的取法口径（沿含水率的积分平均）直接决定核心答案。

改进方向：可用低含水率段的独立实验数据标定扩散系数经验式，收窄外推区间；对表面附近做局部加密，进一步压低表面列的分辨率限制；把界面取法的选择作为显式口径随结论一并声明，使读者在引用核心答案时同时看到其口径前提。

7.3 模型的推广

本文模型与求解框架可推广到圆柱或球形物料在时变环境下的非稳态传热传质问题，如中药材与食品热风干燥、农产品冷藏温湿耦合、土壤热湿迁移等；其同构之处在于均以守恒律描述内部迁移、以第三类边界描述表面交换、且环境随时间变化。

迁移时需相应调整的内容为四项：物性经验式的形式与适用范围；边界条件的类型；是否计入几何收缩（若计入则沿用本文的物料坐标与守恒形式）；是否计入蒸发潜热与热质交叉扩散。四项均可在本文框架内替换或增补而无需改变离散与求解结构——因为空间离散、时间推进与验证体系均建立在守恒律之上、与具体物性形式解耦。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于文献检索、推导核对、代码调试与文字润色，详细使用情况见支撑材料。

参考文献

- [1] 陶文铨. 传热学 [M]. 5 版. 北京: 高等教育出版社, 2019.
- [2] 杨历, 陶斌斌. 多孔介质干燥过程传热传质研究 [J]. 农业工程学报, 2005, 21(1): 27-31.
- [3] 湖北工业大学工程技术学院. 化工原理: 第八章干燥 [Z]. 武汉: 湖北工业大学工程技术学院.
- [4] 徐妮, 丁静, 赵义, 等. 多孔物料干燥中物料体积的收缩特性 [J]. 华南理工大学学报 (自然科学版), 2006, 34(8): 61-65.
- [5] 种翠娟, 朱文学, 刘云宏, 等. 胡萝卜薄层干燥动力学模型研究 [J]. 食品科学, 2014, 35(9): 24.
- [6] 薛韩玲, 等. 基于湿度控制的大红袍花椒热风干燥动力学与品质研究 [J]. 食品与发酵工业, 2023, 49(22): 149-155.
- [7] 任广跃, 朱乐雯, 段续, 等. 苹果丁冷冻-热风联合干燥体积收缩机制 [J]. 农业工程学报, 2024, 40(2): 63-71.
- [8] 顾祥红. 无限长圆柱非稳态导热与集总参数法应用 [J]. 沈阳理工大学学报, 2009(2).
- [9] GASPARIN S, BERGER J, DUTYKH D, et al. On the solution of coupled heat and moisture transport in porous material[J]. Transport in Porous Media, 2018, 121: 665-702.
- [10] HUANG F, SHEN J. On a new class of BDF and IMEX schemes for parabolic type equations[J]. SIAM Journal on Numerical Analysis, 2024, 62(4): 1609-1637.
- [11] ISLAM M, HYE A, MAMUN A. Nonlinear effects on the convergence of Picard and Newton iteration methods in the numerical solution of one-dimensional variably saturated-unsaturated flow problems[J]. Hydrology, 2017, 4(4): 50.
- [12] ADROVER A, BRASIELLO A, PONSO G. A moving boundary model for food isothermal drying and shrinkage: general setting[J]. Journal of Food Engineering, 2019, 244: 178-191.
- [13] ADROVER A, BRASIELLO A, PONSO G. A moving boundary model for food isothermal drying and shrinkage: a shortcut numerical method for estimating the shrinkage factor[J]. Journal of Food Engineering, 2019, 244: 212-219.
- [14] ADROVER A, BRASIELLO A. A moving boundary model for food isothermal drying and shrinkage: one-dimensional versus two-dimensional approaches[J]. Journal of Food Process Engineering, 2019, 42(6): e13178.
- [15] SADOUN N, SI-AHMED E K, COLINET P, et al. On the boundary immobilization and variable space grid methods for transient heat conduction problems with phase change: discussion and refinement[J]. Comptes Rendus Mécanique, 2012, 340(7): 501-511.
- [16] CRANK J. Two methods for the numerical solution of moving-boundary problems in diffusion and heat flow[J]. The Quarterly Journal of Mechanics and Applied Mathematics, 1957, 10(2): 220-231.

[17] MURRAY W D, LANDIS F. Numerical and machine solutions of transient heat-conduction problems involving melting or freezing: Part I—method of analysis[J]. Journal of Heat Transfer, 1959, 81(2): 106-114.

[18] GIBSON R E. A heat conduction problem involving a specified moving boundary[J]. Quarterly of Applied Mathematics, 1959, 16(4).

[19] 胥文博. 热传导方程两类参数的统计反演方法 [J]. 应用数学进展, 2025, 14(3): 203-215.

[20] 王清艳. 扩散方程反问题的正则化方法 [J]. 理论数学, 2024, 14(4): 98-106.

[21] Optimal Adaptive Implicit Time Stepping[EB/OL]. arXiv:2506.18809, 2025.

附录按官方《论文格式规范》第五、十一条的要求给出两部分内容：支撑材料文件列表与源程序；另附关键中间结果的索引说明。源程序按” 关键程序全文+全部程序清单” 两段收录（附录 B：B.1-B.7 为关键程序，正文全文嵌入；B.8-B.13 为其余源程序，只列文件名清单），全部源程序的全文随支撑材料提交，并在表 A-1 中逐项登记。为免歧义，本附录表内所称含水率均指干基含水率。

A 支撑材料文件列表

支撑材料文件列表见表 A-1（按官方第十一条，随论文附录提交）。

表 A-1 类别 1.1 源程序（全部完整可运行、不裁剪）

#	文件	大小	说明
1	code/q1_core.py	16.4 KB	求解核：元体平衡（有限体积）加后向欧拉与三对角追赶法；中心两条独立装配路径；含退化与物性缩放模式供检验复用
2	code/q1_solver.py	11.4 KB	主力求解：附件核验、中心双路比对、单遍求解、写结果文件、程序化格式断言
3	code/q1_e1_gci.py	7.3 KB	检验（收敛性）：空间网格收敛指数（1 / 0.5 / 0.25 mm）与时间收敛（温度、含水率分别）
4	code/q1_e2_series.py	10.6 KB	检验（解析对拍）：圆柱非稳态级数解对拍（常边界与分段常边界）
5	code/q1_e3_explicit.py	19.5 KB	检验（独立实现互验）：显式格式第二套求解代码，全场逐点对比
6	code/q1_e3_figdata.py	2.3 KB	由互验结果生成对应图数据 CSV
7	code/q1_e4_watchdog.py	9.1 KB	检验（物理自检）：看门狗（方向 / 单调 / 极值）加三项退化自检、边界口径对照、人为注错演示
8	code/q1_e5_sensitivity.py	12.0 KB	检验（灵敏度）：单因素扰动、归一化灵敏度系数、龙卷风图数据
9	code/q1_e6_smooth.py	6.9 KB	检验（预处理对照）：边界移动平均平滑与不平滑对照

(续)

#	文件	大小	说明
10	code/q1_consistency_check.py	2.8 KB	一致性核验：单遍新版结果与历史两版结果逐值比对
11	code/q1_figdata_export.py	2.6 KB	由结果文件导出图表数据 CSV
12	code/q1_diag.py	4.8 KB	诊断脚本（保留以溯源）：常数扩散系数线性解与半无限估计对照、迭代残差序列
13	code/dq_check.py	5.5 KB	数据质量核查：附件数据点位数、单调性、缺失与异常检查
14	code/q1_conservation.py	5.4 KB	守恒性核算：区域积分闭合（质量与能量），报告相对残差并归因
15	requirements.txt	0.9 KB	运行依赖清单（数值库与表格库版本；绘图库不在本包范围）
16	code/q2_core_c.py	27.3 KB	第二问主力求解核（交付采用）：变物性配合交替推进与元体平衡全隐式，自实现追赶法
17	code/q2_core.py	25.2 KB	第二问求解核（备选版）：同上，三对角求解改用带状矩阵求解器，用于互校
18	code/q2_core_orig.py	19.4 KB	第二问求解核（优化前基线，供对照运行）
19	code/q2_solver.py	8.8 KB	第二问主力求解：附件核验、三小时全程求解、写结果文件、六项程序化断言
20	code/q2_solver_orig.py	6.7 KB	第二问主力求解（基线前台版，输出隔离，供对照运行）
21	code/q2_checks.py	10.7 KB	第二问检验：独立实现互验（显式格式、同步长、端点严格对齐）与守恒性核算
22	code/q2_e2_frozen.py	5.7 KB	第二问检验：常物性退化对拍（与圆柱非稳态导热级数解逐点比对）
23	code/q2_e5_sensitivity.py	5.3 KB	第二问检验：单因素灵敏度分析（并行执行 12 个算例）
24	code/q2_w2_foreground.py	5.6 KB	第二问步长与网格收敛预试验（空间二阶、时间一阶的判定依据）
25	code/q2_figdata_export.py	5.9 KB	第二问图表数据导出
26	code/q2_verify_BC.py	3.2 KB	双核一致性核对工具：两个求解核跑同一算例并输出逐点最大偏差
27	code/q2_bench_ab.py	3.6 KB	优化前后单迭代耗时对照基准
28	code/q2_opt_verify.py	6.3 KB	性能优化的数值等价性校验（组件级逐位比对与端到端回归）
29	code/q2_diag_picard.py	4.2 KB	诊断：内层迭代上限对结果的影响（对照定量确认迭代截断）
30	code/q2_quick_check.py	2.1 KB	早期快速自检脚本（已被检验脚本取代，保留以溯源）

(续)

#	文件	大小	说明
31	code/q2_w2_prestudy.py	4.8 KB	早期预研脚本（已被预试验脚本取代，保留以溯源）

表 A-2 （续） 类别 1.1b 问题一改进代码（全部完整可运行、不裁剪）

#	文件	大小	说明
15b	code/q1_exact.py	9.2 KB	问题一温度侧时间精确推进求解核：矩阵指数与分段线性推进系数；离线预分解一次、全程复用；被求解核以精确推进模式调用
i1	code/innov/q1_inv_lib.py	4.2 KB	改进模块公共库（附件读取、口径常量、结果写出）
i2	code/innov/q1_inv_par.py	2.8 KB	并行执行器（改进实验批量调度）
i3	code/innov/q1_inv_T0.py	9.0 KB	零成本理论化：传质判据分区定位、有效扩散系数反演、误差预算分配
i4	code/innov/q1_inv_T1_duhamel.py	6.5 KB	半解析参照轨：真实时变边界的积分推进与级数解，与主力逐时逐点对拍
i5	code/innov/q1_inv_T1_adaptive.py	4.2 KB	误差驱动自适应步长（后验估计与比例积分控制器），与固定步长对拍
i6	code/innov/q1_inv_T1_iface.py	3.3 KB	界面口径对照（调和型积分与现行沿含水率积分平均口径对照）
i7	code/innov/q1_inv_T1_interp.py	2.7 KB	边界插值口径对照（线性与单调保形插值；立场为不动原始数据）
i8	code/innov/q1_inv_T2_uq.py	6.6 KB	不确定性量化：边界噪声刻画、严格解采样、代理模型、分位区间
i9	code/innov/q1_inv_T2_da.py	5.6 KB	边界数据同化（状态空间滤波与平滑），作为第二条相互独立的量化路线

#	文件	大小	说明
i10	code/innov/q1_inv_T2_gs.py	5.2 KB	全局灵敏度（筛选法的均值与标准差、方差分解的一阶与总效应指数）
i11	code/innov/q1_inv_T2_adjoint.py	8.0 KB	伴随灵敏度（连续伴随，与单因素结果交叉验证）
i12	code/innov/q1_inv_T3.py	7.9 KB	主算法升级：温度精确积分的正式实现与四闸门验收核
L1-L10	code/innov/logs/ 下的十份日志（零成本理论化、自适应步长、积分推进、界面口径、插值口径、伴随灵敏度、数据同化、全局灵敏度、不确定性量化、主算法升级）	各改进实验的完整运行日志（含判据、对照数值与结论），供逐条复算；位于 logs/ 子目录	

表 A-3 （续） 类别 1.1c 问题一检验日志（11 份，位于代码与复现目录）

#	文件	大小	说明
J1	q1_run_log.txt	现行	主力求解日志：中心双路比对、单遍收集、程序化断言通过
J2	q1_e1_log.txt	现行	收敛性：空间网格与时间收敛（温度、含水率分别）
J3	q1_e2_log.txt	现行	解析对拍：圆柱级数解（常边界与分段常边界）
J4	q1_e3_log.txt	现行	独立实现互验：显式格式与主力实现
J5	q1_e4_log.txt	现行	物理看门狗：13 项全部通过并含注错演示
J6	q1_e5_log.txt	现行	灵敏度：扩散系数、换热系数、传质系数各 $\pm 20\%$ ，初始含水率 $\pm 5\%$ ，初始温度 $\pm 1\text{ }^{\circ}\text{C}$
J7	q1_e6_log.txt	现行	预处理对照（温度侧无内部子步）
J8	conservation_log.txt	现行	守恒性核算：质量与能量区域积分闭合
J9	q1_consistency_log.txt	现行	同口径幂等性核验：重跑主力结果与交付文件逐值一致，用于关闭“论文数字与程序输出不符”的风险
J12	q1_upgrade_diff_log.txt	对照轨（非缺陷）	升级前后对照（跨口径）：温度最大逐值差 $1.0 \times 10^{-4}\text{ }^{\circ}\text{C}$ （14189 格）、含水率逐位一致，证明升级只动温度侧
J10	dq_check_log.txt	与时间格式无关	数据质量核查（点位数、步长、缺失、离群）

#	文件	大小	说明
J11	q1_diag_log.txt	与交付口径无关	诊断（非真值）：保留旧口径以复现早期缺陷，故重跑后与旧日志一致，属自洽复现

表 A-4 （续） 类别 1.2 自主查阅数据资料

#	文件	大小	说明
—	本题无外部查阅数据	—	全部输入均来自题目附件（边界时序、半径序列、结果模板）。按官方规范，赛题原始数据不必列入支撑材料，故本类别为空

表 A-5 （续） 类别 1.3 较大篇幅中间结果图表

#	文件	大小	说明
16	results/result1.xlsx	367.4 KB	问题一完整结果文件（工作表为温度与水分浓度，1801 行 × 22 列，含表头；四位小数；A 列自 1 s 起逐秒输出、末行为 1800 s，t=0 为题面给定的初始条件；首行为到药材中心的距离 0-2.0 cm，间隔 0.1 cm）
17	figdata/fig_q1_field_T.csv	304.9 KB	温度场径向一时间全量数据
18	figdata/fig_q1_field_C.csv	268.0 KB	水分浓度场径向一时间全量数据
19	figdata/fig_q1_curves.csv	62.2 KB	中心与表面温度、含水率的关键点时程数据
20	figdata/fig_gci_data.csv	0.9 KB	网格与时间收敛检验数据
21	figdata/fig_series_check.csv	2.3 KB	解析级数解对拍数据
22	figdata/fig_e3_check.csv	0.7 KB	独立实现互验逐时点比对数据
23	figdata/fig_tornado_data.csv	1.7 KB	参数灵敏度龙卷风图数据
24	figdata/fig_smooth_check.csv	0.2 KB	数据预处理对照数据
24b	figdata/fig_q1_stepsize.csv	97.9 KB	误差驱动自适应步长的阶梯轨迹数据（已抽稀，阶梯形态不变）
24c	figdata/fig_q1_gs.csv	0.6 KB	问题一全局灵敏度数据

#	文件	大小	说明
25	results/ result2.xlsx	2.44 MB	问题二完整结果文件（10801 行 × 22 列，含表头；四位小数；六项程序化断言通过；逐秒结果覆盖题面表 5-5 / 表 5-6 的 3 h 区间，A 列自 1 s 起至 10800 s）
26	figdata/fig_ q2_field_T.csv	1807.3 KB	第二问温度场径向一时间全量数据
27	figdata/fig_ q2_field_C.csv	1561.2 KB	第二问水分浓度场径向一时间全量数据
28	figdata/fig_ q2_curves.csv	371.2 KB	第二问关键点时程数据
29	figdata/fig_ q2_props.csv	3.2 KB	第二问物性随含水率的演化数据
30	figdata/fig_ q1q2_overlap. csv	88.6 KB	前两问重叠段对照数据
31	figdata/fig_ q2_gci.csv	0.7 KB	第二问收敛性数据

表 A-6 （续） 类别 1.3b 第三问源程序与结果

#	文件	大小	说明
32	code/q3_core.py	—	第三问交付求解核（环境预查表与子步循环下沉）
33	code/q3_solver.py	—	第三问主力求解（事件驱动、60 s 输出、子步级二分定位、熔断 120 h、六项断言）
34	code/q3_pretest_bench.py	—	冒烟微基准（成本结构分解；不产交付物）
35	code/q3_w2_convergence.py	—	检验：长时程收敛性（空间四档与时间三档）
36	code/q3_grid_experiment.py	—	检验：网格裁决实验（四档加密）
37	code/q3_conservation.py	—	检验：守恒性核算（质量区域积分闭合）
38	code/q3_postcheck.py	—	后置检验：结果表生成、与第二问重叠段一致性、解析量级对照
39	code/q3_e5_sensitivity.py	—	检验：灵敏度（烘干时长对五个参数，10 算例并行）
40	code/q3_core_nu.py	—	复检：非均匀（表面加密）
41	code/q3_verify_nu.py / q3_ verify_grid.py / q3_verify_ opt.py	—	网格求解核；等距网格上与交付核逐点一致 复检：非均匀核验证、细网格收尾验证、加速等价性校验

#	文件	大小	说明
42	code/q3_probe_crit.py / q3_probe_dcut.py / q3_probe_grid.py	—	复检：判据敏感度、低含水率端扩散系数冻结对照、烘干时长网格裁决
43	code/q3_diag_mech.py / q3_diag_iface.py	—	界面口径诊断：常数扩散系数解析对拍、界面取法单变量对照（修正的直接依据）
44	code/q3_impact_check.py / q3_surface_theory.py / q3_diag_lowC.py / q3_diag_surface.py / q3_check_maxloc.py	—	只读分析与诊断（不参与交付数值）
45	code/q3_audit_consistency.py / q3_audit_tables.py	—	只读审计：口径一致性扫描、结果表与结果文件逐值核对
46	results/result3.xlsx	325.8 KB	第三问完整结果（单工作表，3453 行×22 列，含表头；四位小数；六项断言通过）
—	figdata/ 下第三问图数据 13 件（长时程含水率、收敛性、网格裁决各位置、干燥速率、判据敏感度、全局灵敏度三件、界面口径对照、自适应步长、不确定性量化两件）	合计约 428.3 KB	对应各图数据；源目录与交付包数据目录同名单件一致

表 A-7 （续） 类别 1.3c 第二问检验日志

#	文件	大小	说明
Q2-L1	q2_run_log.txt	现行	交付运行日志：交付核、交替推进、10800 步、六项断言全部通过；关键值与口径登记逐值一致
Q2-L2	q2_checks_log.txt	现行	独立实现互验与守恒核算的合并日志；以本文件为准
Q2-L3	q2_e3b_log.txt	现行	强制解耦退化检验（单向耦合下改换热系数，含水率场变化量级）
Q2-L4	q2_e5_log.txt	现行	单因素灵敏度（含初始含水率补检）
Q2-L5	q2_e5b_log.txt	现行	全局灵敏度（筛选法与方差分解法）
Q2-L6	q2_e7_log.txt	旧版留痕	守恒核算的旧口径输出（恒定边界），非交付口径；其价值在发现过程：首次暴露“变物性下含热容的温度积分不是守恒量”，现行结论以合并日志为准
Q2-L7	q2_e8_log.txt	现行	潜热对照（计入潜热后温度系统性偏低 0.074-0.077 °C）
Q2-L8	q2_e9_log.txt	现行	自适应步长（墙钟由 47.6 s 降至 9.9 s）
Q2-L9	q2_diag_picard_log.txt	现行（诊断）	内层迭代上限对照诊断：上限取 8 时每子步跑满；上限取 120 且容限收紧时平均 16.362 次每子步

#	文件	大小	说明
Q2-L10	q2_opt_verify_log.txt	现行	两个优化核的逐点一致性核对（差异不超过 1.8×10^{-11} ）
Q2-L11	w2_foreground_log.txt	现行（预试验）	空间与时间收敛预试验（确定空间步长与内部子步）
Q2-L12	q2_run_orig_log.txt	对照轨（明示）	优化前基线核的运行日志（输出与交付件隔离）
Q2-L13	q2_e1_log_废弃_早期未收敛.txt	废弃	早期外层迭代未收敛阶段留痕；不代表现行口径
Q2-L14	q2_e3_log_旧版对照.txt	旧版对照	旧编号体系下的对照文本
Q2-L15	q2_e5_log_旧版stdout.txt	旧版	串行版标准输出留痕
Q2-L16	q2_e5_log_旧编号残留.txt	旧编号	编号统一前的残留

表 A-8 （续） 类别 1.3d 第四问源程序与结果

#	文件	大小	说明
47	code/q4_core.py	22.2 KB	第四问交付求解核：物料坐标守恒离散（质量方程与守恒形式能量方程）、元体平衡、中心双路装配
48	code/q4_solver.py	9.7 KB	第四问主力求解：写结果文件（事件驱动、子步级二分、熔断 240 h）
49	code/q4_pretest.py	13.2 KB	第四问预试验批次（九例）：单元自检、冒烟、复用检验、绝热自检、时间与空间收敛、插值分解、口径移植对照；默认仅打印规模并退出，须显式开关才运行
50	code/q4_w6_lib.py	4.1 KB	第四问检验套件公共库
51	code/q4_w6_e15.py	3.4 KB	检验：24 h 三网格收敛指数与单因素灵敏度（11 算例）
52	code/q4_w6_e2.py	2.9 KB	检验：常物性加固定半径的退化对拍
53	code/q4_w6_e5.py	3.8 KB	检验：参数灵敏度
54	code/q4_w6_e78.py	2.5 KB	检验：守恒核算合并档
55	code/q4_w6_e78b.py	6.5 KB	检验：守恒核算增强档（含物性变化项的能量核算；潜热处理修复后重跑）
56	code/q4_table6.py	1.3 KB	由结果文件生成论文结果表（含机读形态）
57	code/q4_diag_adiab.py	2.9 KB	诊断：绝热自检（同时关断导热与对流条件下的不变量核算）
58	code/q4_diag_blow.py	4.5 KB	诊断：几何吹扫效应核查
59	code/q4_diag_e3g2.py	3.5 KB	诊断：零收缩率加附录 3 物性下逐位复现第二问的退化检验

#	文件	大小	说明
60	code/q4_core_fixed.py	22.2 KB	修复版对照核（用于与交付核做数值等价性核对）
61	code/q4_core_prebug.py	21.7 KB	缺陷前基线核（供溯源；不作交付）
62	code/run_chain.py	1.0 KB	第四问串联执行驱动（按序调用预试验与求解）
63	code/innov/q4_INV6_sobol.py	4.5 KB	全局灵敏度的方差分解档（六维含低含水率端截断因子；代理样本求解）
—	code/rerun_chain.ps1	0.7 KB	上述驱动的一键重跑脚本（辅助件，不占编号）；已改为路径自适应
64	code/innov/q4_core_INV.py	23.4 KB	改进项专用求解核（含低含水率端截断因子与界面模式开关；与交付核逐位一致）
65	code/innov/q4_INV_lib.py	4.5 KB	改进项公共库（数据装载与事件驱动求解封装）
66	code/innov/q4_INV0_regression.py	2.3 KB	改进项逐位一致回归验证（改进核与交付核）
67	code/innov/q4_INV1_gci_tend.py	4.3 KB	改进项：烘干时长的离散误差带（空间三档与时间三档）
68	code/innov/q4_INV2_uq_lowC.py	4.0 KB	改进项：低含水率端扩散系数外推的不确定性量化（截断族与分位抽样）
69	code/innov/q4_INV3_global.py	8.2 KB	改进项：全局灵敏度（筛选法与偏相关系数，六维含截断因子）
70	code/innov/q4_INV4_aux.py	5.4 KB	改进项：判据敏感度、固定半径效应分解、潜热中段曲线、取法与网格对照
71	code/innov/q4_INV5_theory.py	5.5 KB	改进项：解析两界夹逼与压缩温升机理（零偏微分方程求解）
72	results/result4.xlsx	226.2 KB	第四问完整结果（单工作表，3041 行 × 22 列，含表头；四位小数；A 列自 60 s 起逐分钟输出至达标时刻；末列为药材表面，半径收缩后越出边界的距离列留空）
73	results/result4_data.csv	348.5 KB	第四问结果的 CSV 形态（便于第三方逐值核对）
—	results/table6.csv	0.4 KB	第四问结果表的机读形态

表 A-9 （续） 类别 1.3e 第二问改进代码

#	文件	大小	说明
74	code/innov/q2_inv_tol_tradeoff.py	8.1 KB	内层迭代容限的成本—精度曲线（六档单变量隔离，五档并行）
75	code/innov/logs/q2_inv_tol_tradeoff.log	2.3 KB	上述实验的完整日志（逐档墙钟、四处输出、与最严档逐格差、裁决结论）
76	code/innov/q2_inv_tol_tradeoff.csv	0.5 KB	上述实验的机读数据

#	文件	大小	说明
77	code/innov/q2_inv_monolithic.py	9.6 KB	整体式联立求解实现（含两条耦合通道的解析耦合块），与分区式外层迭代及交替推进作三方同口径对照
78	code/innov/logs/q2_inv_monolithic.log	2.2 KB	上述三方对照的完整日志（终态对照、逐格差、墙钟与迭代成本、实现复杂度）
79	code/innov/q2_inv_tol_delivery.py	5.0 KB	放宽裁决：在交付口径下重做容限单变量对照
80	code/innov/logs/q2_inv_tol_delivery.log	1.9 KB	上述裁决的完整日志（整表逐格差、关键值对照、裁决结论）
81	code/innov/q2_inv_tol_converge.py	5.2 KB	收敛充分性裁决：容限四档扫描（以最严档为参考）
82	code/innov/logs/q2_inv_tol_converge.log	2.1 KB	上述扫描的完整日志（四档墙钟、内层迭代数、逐格差）

表 A-10 （续） 类别 1.3f 第二问改进链支持脚本

#	文件	大小	说明
83	code/innov/run_innov_chain.py	1.0 KB	第四问四支改进求解档的串行启动器
84	code/innov/run_innov_chain2.py	0.8 KB	第四问改进补跑链
85	code/q2_core_bdf2.py	28.8 KB	第二问求解核变体（二阶时间精度）
86	code/q2_core_latent.py	29.7 KB	第二问求解核变体（潜热对照，用于检验）
87	code/q2_e3b_decoupled.py	5.1 KB	第二问检验：强制解耦退化与耦合强度量化
88	code/q2_e5b_global.py	9.0 KB	第二问检验：全局灵敏度（筛选与方差分解）
89	code/q2_e8_latent.py	6.7 KB	第二问检验：蒸发潜热对照
90	code/q2_e9_adaptive.py	7.3 KB	第二问检验：自适应时间步
91	code/q2_f30_pareto.py	4.4 KB	精度—效率帕累托图数据导出
92	code/q2_fig_export.py	8.5 KB	第二问图数据导出
93	code/q2_verify_results.py	4.9 KB	第二问基线版与优化版结果的全表逐点核对

表 A-11 （续） 类别 1.3g 第三问改进链脚本（全部完整可运行、不裁剪）

#	文件	大小	说明
95	code/innov/q3_solver_INVbase.py	11.7 KB	第三问改进链基线求解档（事件驱动终止与子步级二分定位）
96	code/innov/q3_solver_INV1.py	10.4 KB	第三问改进档一：自适应时间步（与固定子步对拍）

#	文件	大小	说明
97	code/innov/q3_core_INVbase.py	8.6 KB	第三问改进链求解核（等距网格，供各改进档复用）
98	code/innov/q3_core_nu_INVbase.py	14.0 KB	第三问改进链求解核（表面加密的非均匀网格）
99	code/innov/q3_e3_INV2_cn.py	12.4 KB	第三问独立实现互验（另一时间格式轨，全场逐点比对）
100	code/innov/q3_e5_INVbase.py	6.4 KB	第三问单因素灵敏度（烘干时长对五个参数）
101	code/innov/q3_e5b_INV3_global.py	14.9 KB	第三问全局灵敏度（筛选法与方差分解两族方法）
102	code/innov/q3_gci_INV4_tend.py	10.2 KB	第三问烘干时长的离散误差带（空间三档与时间三档）
103	code/innov/q3_uq_INV4_lowC.py	8.9 KB	第三问低含水率端外推的不确定性量化（截断族与分位抽样）
104	code/innov/q3_w2_INVbase.py	5.6 KB	第三问长时程收敛性预试验（空间四档与时间三档）
105	code/innov/q3_diag_INV2_step.py	3.2 KB	第三问诊断：时间步进方式与结果的关系
106	code/innov/q3_diag_mech_T.py	8.8 KB	第三问诊断：温度侧机理与主导项核查
107	code/innov/q3_energy.py	9.6 KB	第三问能量核算（区域积分闭合与残差归因）
108	code/innov/q3_compare_INV.py	4.5 KB	第三问各改进档与基线的同口径对照
109	code/innov/q3_xchk_dt.py	6.8 KB	第三问交叉核验：时间步单向加密对照
110	code/innov/q3_xchk_gt02.py	12.5 KB	第三问交叉核验：干燥后期的含水率与时长关系
111	code/innov/q3_fig_export.py	10.1 KB	第三问图表数据导出（全量图数据CSV）
112	code/innov/_ref_e15.py	3.4 KB	第三问参照实现：24 h 三网格收敛指数与单因素灵敏度（11 算例）

表 A-12 （续） 类别 1.3h 复现与依赖（支撑材料根目录）

#	文件	大小	说明
113	run_all.py	5.3 KB	一键复现入口：按序调度全部求解与检验脚本（纯标准库；默认只打印计划，须显式开关才执行）
114	A_代码复现.md	26.7 KB	复现指南：论文每个关键数字的复算路径与对照方式
115	requirements.txt	1.7 KB	运行依赖与版本清单
116	README_跨平台一键复现.md	8.3 KB	跨平台一键复现说明（含第三方独立复跑记录）

#	文件	大小	说明
117	README_日志索引.md	21.7 KB	运行日志索引（日志与脚本、检验项的对应关系）

本节所列图件与论文正文所引图件分列、互不重复；随支撑材料提交，不在论文附录中呈现。

表 A-13 （续） 类别 1.3i 未进论文正文的中间结果图件（\texttt{figures\allowbreak{}}，40 张）

#	文件	大小
118	figures/图 02-四问递进关系图.pdf	51.1 KB
119	figures/图 03-烘房环境时序.pdf	19.8 KB
120	figures/图 04-药材半径收缩曲线.pdf	17.3 KB
121	figures/图 08-Q1 关键点时程曲线.pdf	15.3 KB
122	figures/图 10-独立实现互验.pdf	20.5 KB
123	figures/图 12-数据预处理对照.pdf	18.7 KB
124	figures/图 14-Q2 温度场时空分布.pdf	75.4 KB
125	figures/图 15-Q2 水分浓度场时空分布.pdf	203.4 KB
126	figures/图 16-Q2 关键点时程曲线.pdf	23.3 KB
127	figures/图 18-Q2 收敛性.pdf	19.3 KB
128	figures/图 19-IMEX 交替推进算法流程图.pdf	63.5 KB
129	figures/图 20-Q2 三维演化曲面.pdf	97.2 KB
130	figures/图 21-Q2 温度含水率相轨迹.pdf	19.1 KB
131	figures/图 22-Q2 全断面失水瀑布图.pdf	22.5 KB
132	figures/图 23-Q2 耦合强度分解图.pdf	19.1 KB
133	figures/图 25-Q2 潜热对照双联图.pdf	20.8 KB
134	figures/图 26-Q2 精度效率帕累托图.pdf	14.9 KB
135	figures/图 27-Q2 全局灵敏度热力图.pdf	23.2 KB
136	figures/图 28-Q2 自适应步长轨迹图.pdf	11.6 KB
137	figures/图 30a-Q3 收敛性与网格裁决-a.pdf	20.5 KB
138	figures/图 30b-Q3 收敛性与网格裁决-b.pdf	18.5 KB
139	figures/图 30c-Q3 收敛性与网格裁决-c.pdf	16.6 KB
140	figures/图 31-Q3 参数灵敏度.pdf	15.7 KB
141	figures/图 32-Q3 离散不确定度与误差带.pdf	17.6 KB
142	figures/图 33-Q3 全局灵敏度.pdf	19.0 KB
143	figures/图 36-Q3 干燥速率曲线.pdf	31.5 KB
144	figures/图 37-自适应步长轨迹.pdf	14.7 KB
145	figures/图 38-边界不确定度散布区间.pdf	15.7 KB
146	figures/图 39-全局灵敏度与 OAT 并置.pdf	17.6 KB
147	figures/图 40-Q4 离散误差带 GCI.pdf	12.9 KB
148	figures/图 41a-Q4 低 C 端外推 UQ-a.pdf	16.7 KB
149	figures/图 41b-Q4 低 C 端外推 UQ-b.pdf	16.8 KB
150	figures/图 42-Q4 全局灵敏度.pdf	21.6 KB
151	figures/图 43-Q4 收缩物性双效应分解.pdf	16.8 KB
152	figures/图 44-Q4 潜热中段温降曲线.pdf	16.9 KB
153	figures/图 46-Q4 界面取法与网格对照.pdf	13.8 KB

#	文件	大小
154	figures/图 47-F26_灵敏度雷达图.pdf	25.5 KB
155	figures/图 49-F07_Q1 求解原理示意图.pdf	46.3 KB
156	figures/图 50-F21_Q2 双向耦合关系图.pdf	41.5 KB
157	figures/图 51-F51_Q3 求解流程图.pdf	38.4 KB

表 A-14 （续） 类别 1.4 AI 工具使用详情（官方规定要求）

#	文件	大小	说明
158	AI 工具使用详情.pdf	PDF	依官方《人工智能工具使用规定》第 4 条随支撑材料提交，含四项内容：所用 AI 工具名称、版本或型号；具体使用目的和环节；主要提示方式与使用过程说明（附典型交互示例）；对 AI 输出的采纳、人工修改和核验的主要情况。该文件由机读登记源件（A_AI 工具使用登记.json）逐项整理导出；机读源件随包保留，便于逐条回查

说明：上表为论文附录中的列表，其随件实体见支撑材料包；电子版论文与纸质版的内容及格式须完全一致。若某项确未使用，则在附录中明确说明。

B 源程序

本附录收录建模所用的源程序：**关键程序**（B.1-B.7，公共模块与四问求解的交付核、主流程）**全文嵌入**；**其余源程序**（B.8-B.13，检验脚本与其余溯源脚本）**只列文件名清单**，其全文随**支撑材料**提交，并按官方第十一条在表 A-1 中逐项登记。全篇按”通用模块、分问求解、检验脚本”的顺序编排；附录中不描述运行环境与硬件信息，只保留程序本身及其接口说明。

表 B-1 附录 B 源程序模块清单

序	程序（模块）	作用	对应论文位置
B.1	公共模块：网格与物性	生成径向网格、按问题装配物性（附录 2 / 附录 3 / 附录 4）	5.0.1 / 假设 3
B.2	公共模块：元体平衡装配	组装三对角矩阵、界面系数（沿含水率的积分平均）、中心奇点半格系数	5.1.3 / 式 (5-13)
B.3	公共模块：时间推进	温度侧时间方向精确推进（矩阵指数与分段线性推进系数）；含水率侧后向欧拉与迭代	5.1.3 / 式 (5-8)(5-8)
B.4	问题一求解	常物性完全解耦情形	5.1
B.5	问题二求解	变物性双向耦合与交替推进	5.2
B.6	问题三求解	事件驱动终止与子步级二分定位	5.3
B.7	问题四求解	物料坐标、守恒形式、预估与校正、事件判定	5.4
B.8	检验脚本：收敛性与误差带	三档网格与三档时间步长、广义理查森外推	6.1

(续)

序	程序(模块)	作用	对应论文位置
B.9	检验脚本: 解析对拍	圆柱级数解(常边界与分段常边界)	6.2
B.10	检验脚本: 独立实验互验	第二套实现(显式格式与另一时间格式轨)	6.3
B.11	检验脚本: 守恒核算	质量与能量的区域积分闭合	6.5
B.12	检验脚本: 灵敏度分析	单因素与全局两族方法	6.6 / 6.7

代码一致性声明: 附录所列源程序与实际生成四个结果文件的程序同一版本; 结果文件与论文各结果表(共六张)逐格一致。按官方要求, 源程序缺失、不能运行或运行结果与论文不符者可能被取消评奖资格; 故本附录收录的关键程序**不做删节**(仅按上述顺序重组), 其余源程序**不裁剪、文件名列全**(全文随支撑材料提交)。

B.1 公共模块: 网格与物性

```

1  # -*- coding: utf-8 -*-
2  """
3  A 题 Q1 求解核(配置化)——主力求解与全部检验实验共用
4  =====
5  统一口径
6  =====
7  • 空间      : 元体平衡(有限体积) + 三对角追赶法
8  • 时间      : **温度=时间方向精确推进** (矩阵指数+分段线性 Duhamel, 见 `q1_exact.py`;
9                本核经 `tmethod='expm'` 调用, **无时间离散误差**)
10             其余(Q1 含水率、Q2-Q4 全部场量)=全隐式(后向欧拉)
11 • 中心      : 两条**真正独立**的装配路径, 经同一尺度归一化后逐系数比对
12               center='fv' —— 用中心控制体体积  $V_0=\Delta r^2/4$  与界面面积  $A_{12}=\Delta r$  显式装配
13               center='lh' —— 洛必达极限  $(1/r)\partial r(r\partial r T) \rightarrow 2\partial r^2 T$  的差分离散
14 • 表面      : surf='fvm' —— 元体平衡半格(主力)
15               surf='ghost' —— **一阶近似口径** (体积取  $R_0 \cdot \Delta r/2$ 、导热界面取  $R_0$ );
16                       **仅**用于边界口径对照实验。△命名沿用历史,
17                       **其实现并非"虚拟节点+中心差分"** (原虚拟节点装配符号有误、已改)
18 • 非线性    : D(C) 单因子; Picard 迭代+欠松弛; 时间导数右端**固定取上一时层解**
19 • 解耦      : 先全解 T, 再全解 C (T 方程不含 C, D 不含 T)
20 退化与灵敏度
21  =====
22 • kfac / Dfac: 物性缩放因子 (0.0 → 退化自检; 1±0.2 → 灵敏度 OAT)
23 • 环境条件以函数传入 (常数环境用于 E2 解析对拍与退化自检)
24
25 约定: 本核不引用、不记载任何资料中的日期。
26  """
27  import time
28  import numpy as np
29
30  DEF = dict(R0=0.02, rho=820.0, cp=2600.0, k=0.36, h=25.0, km=8.0e-7,
31            D0=7.0e-9, bD=0.89, T0=28.0, C0=2.55)
32
33  V0_OF = staticmethod(lambda dr: dr * dr / 4.0) # 中心控制体体积 (π 已约去)
34  VN_OF = staticmethod(lambda dr, R0: dr * (R0 - dr / 4.0) / 2.0) # 表面控制体体积
35
36
37  def thomas(a, b, c, d):
38      """三对角追赶法 (a/b/c 为下/主/上对角, a[0] 与 c[-1] 不使用)"""
39      n = len(b)
40      cp_ = np.empty(n - 1, dtype=float)
41      dp_ = np.empty(n, dtype=float)
42      cp_[0] = c[0] / b[0]
43      dp_[0] = d[0] / b[0]
44      for i in range(1, n):
45          m = b[i] - a[i] * cp_[i - 1]
46          dp_[i] = (d[i] - a[i] * dp_[i - 1]) / m
47          if i < n - 1:
48              cp_[i] = c[i] / m

```

```

49     x = np.empty(n, dtype=float)
50     x[-1] = dp_[-1]
51     for i in range(n - 2, -1, -1):
52         x[i] = dp_[i] - cp_[i] * x[i + 1]
53     return x
54
55
56 def thomas_prep(a, b, c):
57     """常数系数三对角矩阵的预消元（前向消元结果与 d 无关，可复用）"""
58     n = len(b)
59     cp_ = np.empty(n - 1, dtype=float)
60     inv = np.empty(n, dtype=float)
61     cp_[0] = c[0] / b[0]
62     inv[0] = 1.0 / b[0]
63     for i in range(1, n):
64         m = b[i] - a[i] * cp_[i - 1]
65         inv[i] = 1.0 / m
66         if i < n - 1:
67             cp_[i] = c[i] / m
68     return cp_, inv
69
70
71 def thomas_solve_prep(a, cp_, inv, d):
72     """配合 thomas_prep 使用的快速求解（与 thomas 同式，仅省去重复除法）"""
73     n = len(d)
74     dp_ = np.empty(n, dtype=float)
75     dp_[0] = d[0] * inv[0]
76     for i in range(1, n):
77         dp_[i] = (d[i] - a[i] * dp_[i - 1]) * inv[i]
78     x = np.empty(n, dtype=float)
79     x[-1] = dp_[-1]
80     for i in range(n - 2, -1, -1):
81         x[i] = dp_[i] - cp_[i] * x[i + 1]
82     return x
83
84
85 def D_of(C, D0=DEF['D0'], bD=DEF['bD']):
86     """有效水分扩散系数 D(C) = D0 * exp(-bD/C)"""
87     return D0 * np.exp(-bD / np.maximum(np.asarray(C, dtype=float), 1e-9))
88
89
90 def harm(dl, dr_):
91     """界面系数：距离加权调和平均（D=0 时退化为 0，避免除零）。
92
93     △ **本函数已不用于生产路径**：C 方程的界面扩散系数改由 `faceD_int`
94     （**沿 $C$ 的积分平均**，口径 **M6**）生成；本函数**仅保留作旧口径对照**。
95     """
96     if dl <= 0.0 or dr_ <= 0.0:
97         return 0.0
98     return 2.0 / (1.0 / dl + 1.0 / dr_)
99
100
101 # ----- T 方程
102 def assemble_T(N, dr, dt, p, center='fv', surf='fvm'):
103     """装配后向欧拉 T 方程系数 (a, b, c, meta)：右端另由 rhs_T 生成"""
104     rho, cp = p['rho'], p['cp']
105     k = p['k'] * p.get('kfacs', 1.0)
106     h, R0 = p['h'], p['R0']
107     r = np.arange(N + 1) * dr
108     a = np.zeros(N + 1); b = np.zeros(N + 1); c = np.zeros(N + 1)
109     meta = {}
110
111     # --- 中心节点：两条独立路径，均归一化到"单位体积"以便逐系数比对 ---
112     if center == 'fv':
113         V0 = dr * dr / 4.0
114         A12 = dr
115         b[0] = (rho * cp * V0 / dt + k * A12 / dr) / V0
116         c[0] = (-k * A12 / dr) / V0
117         meta['center_scale'] = V0
118     elif center == 'lh':
119         d2 = 2.0 / dr ** 2
120         b[0] = rho * cp / dt + 2.0 * k * d2
121         c[0] = -2.0 * k * d2

```



```

122     meta['center_scale'] = 1.0
123 else:
124     raise ValueError(center)
125 meta['center_row'] = (float(b[0]), float(c[0]))
126
127 # --- 内部节点: 标准元体平衡 (含  $r_{\{i\pm 1/2\}}$  几何权重) ---
128 for i in range(1, N):
129     rl, rr = r[i] - dr / 2.0, r[i] + dr / 2.0
130     b[i] = rho * cp * r[i] / dt + k * (rr + rl) / dr ** 2
131     a[i] = -k * rl / dr ** 2
132     c[i] = -k * rr / dr ** 2
133
134 # --- 表面节点 ---
135 if surf == 'fvm':
136     VN = dr * (R0 - dr / 4.0) / 2.0
137     b[N] = rho * cp * VN / dt + k * (R0 - dr / 2.0) / dr + h * R0
138     a[N] = -k * (R0 - dr / 2.0) / dr
139     meta['surf'] = 'fvm'
140 elif surf == 'ghost':
141     # 对照口径 (**仅用于边界离散敏感性对照**):
142     # 表面控制体体积取一阶近似  $R0 \cdot \Delta r / 2$  (忽略曲率), 导热界面半径取  $R0$ 
143     VNa = R0 * dr / 2.0
144     b[N] = rho * cp * VNa / dt + k * R0 / dr + h * R0
145     a[N] = -k * R0 / dr
146     meta['surf'] = 'approx'
147 else:
148     raise ValueError(surf)
149 return a, b, c, meta
150
151
152 def rhs_T(N, dr, dt, p, Told, Tinf, surf='fvm'):
153     """生成 T 方程右端向量 d"""
154     rho, cp = p['rho'], p['cp']
155     h, R0 = p['h'], p['R0']
156     r = np.arange(N + 1) * dr
157     d = np.zeros(N + 1)
158     d[0] = rho * cp * Told[0] / dt
159     for i in range(1, N):
160         d[i] = rho * cp * r[i] * Told[i] / dt
161     if surf == 'fvm':
162         VN = dr * (R0 - dr / 4.0) / 2.0
163         d[N] = rho * cp * VN * Told[N] / dt + h * R0 * Tinf
164     else:
165         VNa = R0 * dr / 2.0
166         d[N] = rho * cp * VNa * Told[N] / dt + h * R0 * Tinf
167     return d
168
169
170 # ----- C 方程
171 def assemble_C(N, dr, dt, Df, R0, km):
172     """装配 C 方程系数 (a, b, c, VN); Df 为界面扩散系数 (长度 N, **沿  $\$C\$$  的积分平均**, 由 `faceD_int`
    生成)"""
173     r = np.arange(N + 1) * dr
174     a = np.zeros(N + 1); b = np.zeros(N + 1); c = np.zeros(N + 1)
175     V0 = dr * dr / 4.0
176     A12 = dr
177     b[0] = (V0 / dt + Df[0] * A12 / dr) / V0
178     c[0] = (-Df[0] * A12 / dr) / V0
179     for i in range(1, N):
180         rl, rr = r[i] - dr / 2.0, r[i] + dr / 2.0
181         b[i] = r[i] / dt + (Df[i] * rr + Df[i - 1] * rl) / dr ** 2
182         a[i] = -Df[i - 1] * rl / dr ** 2
183         c[i] = -Df[i] * rr / dr ** 2
184     VN = dr * (R0 - dr / 4.0) / 2.0
185     b[N] = VN / dt + Df[N - 1] * (R0 - dr / 2.0) / dr + km * R0
186     a[N] = -Df[N - 1] * (R0 - dr / 2.0) / dr
187     return a, b, c, VN
188
189
190 def rhs_C(N, dr, dt, Cold, cinf, R0, km):
191     """生成 C 方程右端向量 d (**固定取上一时层解 Cold**)"""
192     r = np.arange(N + 1) * dr
193     d = np.zeros(N + 1)

```

```

194     d[0] = Cold[0] / dt
195     for i in range(1, N):
196         d[i] = r[i] * Cold[i] / dt
197     VN = dr * (R0 - dr / 4.0) / 2.0
198     d[N] = VN * Cold[N] / dt + km * R0 * cinf
199     return d
200
201
202 def faceD_int(Carr, D0, bD, Dfac=1.0, nsamp=8):
203     """*修正口径: 界面扩散系数取**沿 C 的积分平均** (D)=(1/ΔC)∫D dC。
204
205     (原 `harm` 的调和平均仅对"界面处系数间断"成立; 系数随 C 陡变时显著失真。)
206     """
207     C = np.asarray(Carr, dtype=float)
208     lo = np.minimum(C[:-1], C[1:])
209     hi = np.maximum(C[:-1], C[1:])
210     xs = lo[:, None] + (hi - lo[:, None] * (np.arange(nsamp) + 0.5)[None, :]) / nsamp
211     xs = np.maximum(xs, 1e-9)
212     fv = np.exp(-bD / xs).mean(axis=1)
213     flat = (hi - lo) < 1e-14
214     if flat.any():
215         fv[flat] = np.exp(-bD / np.maximum(lo[flat], 1e-9))
216     return D0 * Dfac * fv
217
218
219 def step_C(N, dr, dt, C, cinf, p, omega=0.7, tol=1e-9, maxit=30, Dfac=1.0,
220           dmode='func'):
221     """推进一个 C 子步: Picard 迭代+欠松弛。返回 (C_new, iters, res)
222
223     dmode='func': D = D(C) 单因子非线性 (主力)
224     dmode='const': D ≡ D(C0) (用于 E2 解析对拍; 此时单次迭代即收敛)
225     """
226     Cold = C.copy()
227     Cit = C.copy()
228     res = None
229     it = 0
230     Dconst = p['D0'] * np.exp(-p['bD'] / p['C0']) * Dfac
231     for it in range(1, maxit + 1):
232         if dmode == 'const':
233             Df = np.full(N, Dconst)
234         else:
235             Df = faceD_int(Cit, p['D0'], p['bD'], Dfac)
236             a, b, c, _ = assemble_C(N, dr, dt, Df, p['R0'], p['km'])
237             d = rhs_C(N, dr, dt, Cold, cinf, p['R0'], p['km'])
238             Ctry = thomas(a, b, c, d)
239             Cnew = omega * Ctry + (1.0 - omega) * Cit
240             res = float(np.max(np.abs(Cnew - Cit))) / max(1.0, float(np.max(np.abs(Cnew))))
241             Cit = Cnew
242             if res < tol:
243                 break
244     return Cit, it, res
245
246
247 # ----- 驱动
248 def run_sim(N, dr, dt_out, nsteps, p, Tenv_fn, Cenv_fn, sub=10, subT=1,
249            center='fv', surf='fvm', kfac=1.0, Dfac=1.0, dmode='func',
250            tol=1e-9, omega=0.7, maxit=30, snap_at=(), out_idx=None,
251            solve_T=True, solve_C=True, tmethod='expm'):
252     """统一求解驱动。
253
254     参数
255     ----
256     N, dr      : 空间区间数与步长 (节点数 N+1)
257     dt_out     : 输出时间步长 (s)
258     nsteps     : 输出步数 (总时长 = nsteps * dt_out)
259     p          : 物性/几何参数字典 (见 DEF)
260     Tenv_fn    : T∞(t) 函数; Cenv_fn: C∞(t) 函数
261     sub        : C 方程内部子步数 (内部步长 = dt_out/sub)
262     subT       : T 方程内部子步数 (内部步长 = dt_out/subT, 用于时间收敛实验)
263     out_idx    : 需要收集的列索引 (None → 不收集整表)
264     返回 dict
265     """
266     p = dict(p)

```

```

267 p['kfac'] = kfac
268 R0 = p['R0']
269 r = np.arange(N + 1) * dr
270 dt_int = dt_out / float(sub)
271 dtT = dt_out / float(subT)
272
273 res = dict(N=N, dr=dr, dt_out=dt_out, dt_int=dt_int, sub=sub,
274           center=center, surf=surf, kfac=kfac, Dfac=Dfac)
275
276 # ----- T -----
277 Tend = None
278 Tsnap = {}
279 res['tmethod'] = tmethod
280 if solve_T:
281     a, b, c, meta = assemble_T(N, dr, dtT, p, center, surf)
282     res['center_row'] = meta['center_row']
283     t0 = time.time()
284     if tmethod == 'expm':
285         # 时间方向**精确推进** (交付口径): 矩阵指数 + 分段线性 Duhamel
286         import ql_exact as _ex
287         rT = _ex.run_T_expm(N, dr, p, Tenv_fn, nsteps * dt_out,
288                             hstep=dt_out, center=center, surf=surf,
289                             t_snaps=[n * dt_out for n in snap_at])
290         Tsnap[0] = np.full(N + 1, p['T0'])
291         for n in snap_at:
292             Tsnap[n] = rT['T_snap'][round(float(n * dt_out), 9)].copy()
293         T = rT['T_end']
294     else:
295         cp_, inv = thomas_prep(a, b, c)
296         T = np.full(N + 1, p['T0'])
297         Tsnap[0] = T.copy()
298         for n in range(1, nsteps + 1):
299             for s in range(subT):
300                 ts = (n - 1) * dt_out + (s + 1) * dtT
301                 d = rhs_T(N, dr, dtT, p, T, float(Tenv_fn(ts)), surf)
302                 T = thomas_solve_prep(a, cp_, inv, d)
303             if n in snap_at:
304                 Tsnap[n] = T.copy()
305         res['T_time'] = time.time() - t0
306         Tend = T.copy()
307     res['T_end'] = Tend
308     res['T_snap'] = Tsnap
309
310 # ----- C -----
311 Cend = None
312 Csnap = {}
313 stats = dict(total_it=0, avg_it=0.0, max_it=0, max_res=0.0, nsub=0)
314 if solve_C:
315     C = np.full(N + 1, p['C0'])
316     Csnap[0] = C.copy()
317     tot, mx, mxr, nsub = 0, 0, 0.0, 0
318     t0 = time.time()
319     for n in range(1, nsteps + 1):
320         for s in range(sub):
321             tt = (n - 1) * dt_out + (s + 1) * dt_int
322             cinf = float(Cenv_fn(tt))
323             C, it, rr = step_C(N, dr, dt_int, C, cinf, p, omega, tol, maxit, Dfac, dmode)
324             tot += it; mx = max(mx, it); mxr = max(mxr, rr); nsub += 1
325         if n in snap_at:
326             Csnap[n] = C.copy()
327         res['C_time'] = time.time() - t0
328         Cend = C.copy()
329         stats.update(total_it=tot, avg_it=tot / max(1, nsub), max_it=mx,
330                     max_res=mxr, nsub=nsub)
331     res['C_end'] = Cend
332     res['C_snap'] = Csnap
333     res['stats'] = stats
334     return res
335
336
337 def run_sim_collect(N, dr, dt_out, nsteps, p, Tenv_fn, Cenv_fn, sub=10, subT=1,
338                    center='fv', surf='fvm', kfac=1.0, Dfac=1.0, dmode='func',
339                    tol=1e-9, omega=0.7, maxit=30, cols=None,

```

```

340         snap_at=(), solve_T=True, solve_C=True, tmethod='expm'):
341     """单遍运行并**同时**收集整表输出与快照（替代"两遍计算"）。
342
343     cols: 需要写出的列索引（如 np.arange(0, N+1, 4)），None 表示不收集
344     返回 (res, tableT, tableC); tableT/C 形状为 (nsteps, len(cols))
345     """
346     p = dict(p)
347     p['kfac'] = kfac
348     R0 = p['R0']
349     r = np.arange(N + 1) * dr
350     dt_int = dt_out / float(sub)
351     dtT = dt_out / float(subT)
352     ncol = 0 if cols is None else len(cols)
353
354     tableT = np.zeros((nsteps, ncol)) if (cols is not None and solve_T) else None
355     tableC = np.zeros((nsteps, ncol)) if (cols is not None and solve_C) else None
356     Tsnap, Csnap = {}, {}
357     res = dict(N=N, dr=dr, dt_out=dt_out, dt_int=dt_int, sub=sub,
358               center=center, surf=surf, kfac=kfac, Dfac=Dfac)
359
360     Tprep = None
361     res['tmethod'] = tmethod
362     use_expm_T = bool(solve_T and tmethod == 'expm')
363     if solve_T:
364         aT, bT, cT, meta = assemble_T(N, dr, dtT, p, center, surf)
365         res['center_row'] = meta['center_row']
366         if use_expm_T:
367             import ql_exact as _ex
368             rT = _ex.run_T_expm(N, dr, p, Tenv_fn, nsteps * dt_out, hstep=dt_out,
369                               center=center, surf=surf,
370                               t_snaps=[n * dt_out for n in snap_at],
371                               cols=cols, dt_out=dt_out)
372             tableT = rT['table']
373             T = rT['T_end']
374             Tsnap[0] = np.full(N + 1, p['T0'])
375             for n in snap_at:
376                 Tsnap[n] = rT['T_snap'][round(float(n * dt_out), 9)].copy()
377             tT = 0.0
378         else:
379             cpT, invT = thomas_prep(aT, bT, cT)
380             Tprep = (aT, cpT, invT)
381             T = np.full(N + 1, p['T0'])
382             Tsnap[0] = T.copy()
383
384     C = np.full(N + 1, p['C0']) if solve_C else None
385     Csnap[0] = C.copy() if C is not None else None
386
387     tot, mx, mxr, nsub = 0, 0, 0.0, 0
388     tT = tC = 0.0
389     for n in range(1, nsteps + 1):
390         if solve_T and not use_expm_T:
391             t0 = time.time()
392             a, cp_, inv = Tprep
393             for s in range(subT):
394                 ts = (n - 1) * dt_out + (s + 1) * dtT
395                 d = rhs_T(N, dr, dtT, p, T, float(Tenv_fn(ts)), surf)
396                 T = thomas_solve_prep(a, cp_, inv, d)
397             tT += time.time() - t0
398             if n in snap_at:
399                 Tsnap[n] = T.copy()
400             if cols is not None:
401                 tableT[n - 1, :] = T[cols]
402         if solve_C:
403             t0 = time.time()
404             for s in range(sub):
405                 tt = (n - 1) * dt_out + (s + 1) * dt_int
406                 C, it, rr = step_C(N, dr, dt_int, C, float(Cenv_fn(tt)), p,
407                                   omega, tol, maxit, Dfac, dmode)
408                 tot += it; mx = max(mx, it); mxr = max(mxr, rr); nsub += 1
409             tC += time.time() - t0
410             if n in snap_at:
411                 Csnap[n] = C.copy()
412             if cols is not None:

```

```

413         tableC[n - 1, :] = C[cols]
414
415     res['T_end'] = T.copy() if solve_T else None
416     res['C_end'] = C.copy() if solve_C else None
417     res['T_snap'] = Tsnap
418     res['C_snap'] = Csnap
419     res['T_time'] = tT
420     res['C_time'] = tC
421     res['stats'] = dict(total_it=tot, avg_it=tot / max(1, nsub), max_it=mx,
422                        max_res=mxr, nsub=nsub)
423     return res, tableT, tableC

```

```

1  # -*- coding: utf-8 -*-
2  """
3  A 题 Q4 求解核：物料坐标  $\xi=r/R(t)$  守恒形式 (MB-11 + L3-09)
4  =====
5  离散方案 (元体平衡, 全部推导见《A_Q4求解思路与 workflow》v2 §2.1):
6  质量方程 ( $\times R^2$  前的守恒形式):  $\xi \partial_t \tilde{C} = (1/R^2) \partial_\xi (\xi D \partial_\xi \tilde{C})$ 
7      内部:  $\xi_j \Delta \xi dC_j/dt = (1/R^2) [\xi_{j+1/2} D_{j+1/2} \Delta C - \xi_{j-1/2} D_{j-1/2} \Delta C] / \Delta \xi$ 
8      中心:  $dC_0/dt = 4 D_{1/2} (C_1 - C_0) / (R^2 \Delta \xi^2)$ 
9      表面:  $V_N \tilde{\xi} dC_N/dt = -(1/R^2) \xi_{N-1/2} D_{N-1/2} \Delta C / \Delta \xi - (k_m/R)(C_N - C_\infty)$ 
10 能量方程 (守恒形式):  $\xi \partial_t [\rho c_p T R^2] = \partial_\xi (k \xi \partial_\xi T)$ 
11      内部:  $\xi_j \Delta \xi [(\rho c_p T)^{n+1} R_{n+1}^2 - (\rho c_p T)^n R_n^2] / h = [k \xi \Delta T \text{ 差}] / \Delta \xi$ 
12      中心:  $[(\rho c_p T)^{n+1} R_{n+1}^2 - (\rho c_p T)^n R_n^2] / h = 4k_{1/2} (T_1 - T_0) / \Delta \xi^2$ 
13      (行已除以  $R_{n+1}^2$ , 避免大数标度)
14      表面:  $V_N \tilde{\xi} [ \dots ] / h = -k \xi_{N-1/2} \Delta T / \Delta \xi + h R_{n+1} (T_\infty - T_N)$ 
15 固定域极限 ( $R=R_0$ ,  $\Delta \xi=\Delta r/R_0$ ) 下逐项退化为 Q1/Q2 格式 (E3-g-② 复用检验的依据)。
16
17 时间推进: IMEX 交替 (物性逐内部子步更新); D6 取层口径由 t_corr 开关控制:
18     t_corr=0 : 纯滞后 ( $\rho c_p$  用  $C^n$ ) —— Q2 同款
19     t_corr=1 : 预估-校正 (C 解出后用  $\rho c_p(C^{n+1})$  重解一次 T) —— 交付解  $\rho c_p$  取  $n+1$  层
20 R(t) 取当前子步两端时刻的线性插值 (附件2, 端点钳位=H7 末段常值外推)。
21
22 性能: 全部热路径 njit (fastmath=False 保持可复现); 一个输出步的全部子步
23 一次 njit 调用完成 (q3_core 的"子步下沉"范式)。
24 """
25 import sys, os, time
26
27 try:
28     sys.stdout.reconfigure(encoding='utf-8')
29 except Exception:
30     pass
31
32 for _v in ('OMP_NUM_THREADS', 'MKL_NUM_THREADS', 'OPENBLAS_NUM_THREADS',
33           'NUMEXPR_NUM_THREADS', 'VECLIB_MAXIMUM_THREADS'):
34     os.environ.setdefault(_v, '1')
35
36 import numpy as np
37
38 try:
39     from numba import njit
40     _HAS_NUMBA = True
41 except Exception:
42     _HAS_NUMBA = False
43     def njit(*a, **k):
44         if len(a) == 1 and callable(a[0]):
45             return a[0]
46         def deco(f):
47             return f
48         return deco
49
50 DEF4 = dict(R0=0.02, h=25.0, km=8.0e-7, T0=28.0, C0=2.55)
51 D_FAC = 1.0 # D0 灵敏度缩放因子 (OAT 用)
52 CM = 273.15 # °C -> K
53
54 # =====
55 # 物性 (附录4: T 内部用 °C, D 公式内转 K)。app3=True 时切换附录3 (复用检验用)
56 # =====
57 @njit(cache=True, fastmath=False)
58 def _rho_nb(C, app3):
59     if app3:
60         return 650.0 + 128.0 * C

```

```

62     return 760.0 + 90.0 * C
63
64
65 @njit(cache=True, fastmath=False)
66 def _cp_nb(C, app3):
67     if app3:
68         return 1450.0 + 2736.0 * C / (C + 1.0)
69     return 1850.0 + 2150.0 * C / (C + 1.0)
70
71
72 @njit(cache=True, fastmath=False)
73 def _k_nb(C, app3):
74     if app3:
75         return 0.21 + 0.38 * C / (C + 1.0)
76     return 0.12 + 0.20 * C / (C + 1.0)
77
78
79 @njit(cache=True, fastmath=False)
80 def _D_nb(C, Tc, d_fac, app3):
81     c = C
82     if c < 1e-9:
83         c = 1e-9
84     if app3:
85         return d_fac * 2.4e-3 * np.exp(-0.45 / c) * np.exp(-3850.0 / (Tc + CM))
86     return d_fac * 4.2e-4 * np.exp(-0.30 / c) * np.exp(-3850.0 / (Tc + CM))
87
88
89 @njit(cache=True, fastmath=False)
90 def _iface_nb(f):
91     """界面值：距离加权调和平均（与 q2_core_c.iface 同式）。
92
93     △ 旧口径（仅保留作 S2 旧口径对照与新口径对照实验），主力已改用
94     _facek4_nb / _faceD4_nb（沿 C 的积分平均，2026-09-11 口径纠偏后）。
95     """
96     n = f.shape[0]
97     out = np.empty(n - 1)
98     for i in range(n - 1):
99         l = f[i]
100         r = f[i + 1]
101         s = l + r
102         if s > 0.0:
103             out[i] = 2.0 * l * r / s
104         else:
105             out[i] = 0.0
106     return out
107
108
109 @njit(cache=True, fastmath=False)
110 def _facek4_nb(Carr, app3):
111     """★修正口径（Q4 版）：界面导热系数取【沿 C 的积分平均】（解析式）。
112
113          $\langle k \rangle = k_0 + k_1 \cdot \Delta(C - \ln(1+C)) / \Delta C$  ,  $(k_0, k_1) = \text{附录4 } (0.12, 0.20) / \text{附录3 } (0.21, 0.38)$ 
114
115     依据：单元内稳态通量  $J = (1/\Delta r) \int k \, dC \Rightarrow$  界面系数=沿 C 积分平均。
116     调和平均只对"界面处系数间断"成立；Q4 尾部单胞内 D/k 为连续急变（同 Q3）。
117     模板：q2_core_c._facek_nb（只读复制，系数换附录4）。 $|\Delta C| \approx 0$  退化为节点值。
118     """
119     n = Carr.shape[0] - 1
120     out = np.empty(n)
121     for i in range(n):
122         a = Carr[i]
123         b = Carr[i + 1]
124         if abs(b - a) < 1e-14:
125             c = a
126             if app3:
127                 out[i] = 0.21 + 0.38 * c / (1.0 + c)
128             else:
129                 out[i] = 0.12 + 0.20 * c / (1.0 + c)
130         else:
131             ga = a - np.log(1.0 + a)
132             gb = b - np.log(1.0 + b)
133             if app3:
134                 out[i] = 0.21 + 0.38 * (gb - ga) / (b - a)

```

```

135         else:
136             out[i] = 0.12 + 0.20 * (gb - ga) / (b - a)
137     return out
138
139
140 @njit(cache=True, fastmath=False)
141 def _faceD4_nb(Carr, Tarr, d_fac, useT, nsamp, app3):
142     """★修正口径 (Q4 版): 界面扩散系数取【沿 C 的积分平均】(nsamp 点中矩采样)。
143
144         (D) = [D0 * {e^{-β/C}}_C] * [e^{-3850/T_face_K}] ,
145         (D0, β) = 附录4 (4.2e-4, 0.30) / 附录3 (2.4e-3, 0.45)
146
147     利用可分离性只需对 C 因子采样; T 取界面两侧均值的 T 因子 (T 内部 °C, 转 K)。
148     useT=False: 仅 C 因子 (decoupled, 不乘 d_fac)。|ΔC|≈0 退化为节点值。
149     模板: q2_core_c._faceD_nb (只读复制, 系数换附录4)。
150     """
151     n = Carr.shape[0] - 1
152     out = np.empty(n)
153     for i in range(n):
154         a = Carr[i]
155         b = Carr[i + 1]
156         lo = a if a < b else b
157         hi = b if a < b else a
158         if hi - lo < 1e-14:
159             c = lo if lo > 1e-9 else 1e-9
160             if app3:
161                 fv = np.exp(-0.45 / c)
162             else:
163                 fv = np.exp(-0.30 / c)
164         else:
165             s = 0.0
166             for q in range(nsamp):
167                 x = lo + (hi - lo) * (q + 0.5) / nsamp
168                 if x < 1e-9:
169                     x = 1e-9
170                 if app3:
171                     s += np.exp(-0.45 / x)
172                 else:
173                     s += np.exp(-0.30 / x)
174             fv = s / nsamp
175         if useT:
176             Tm = 0.5 * (Tarr[i] + Tarr[i + 1])
177             if app3:
178                 out[i] = d_fac * 2.4e-3 * fv * np.exp(-3850.0 / (Tm + 273.15))
179             else:
180                 out[i] = d_fac * 4.2e-4 * fv * np.exp(-3850.0 / (Tm + 273.15))
181         else:
182             if app3:
183                 out[i] = 2.4e-3 * fv
184             else:
185                 out[i] = 4.2e-4 * fv
186     return out
187
188
189 @njit(cache=True, fastmath=False)
190 def _thomas_nb(a, b, c, d):
191     n = b.shape[0]
192     cp = np.empty(n)
193     dp = np.empty(n)
194     cp[0] = c[0] / b[0]
195     dp[0] = d[0] / b[0]
196     for i in range(1, n):
197         m = b[i] - a[i] * cp[i - 1]
198         cp[i] = c[i] / m if i < n - 1 else 0.0
199         dp[i] = (d[i] - a[i] * dp[i - 1]) / m
200     x = np.empty(n)
201     x[n - 1] = dp[n - 1]
202     for i in range(n - 2, -1, -1):
203         x[i] = dp[i] - cp[i] * x[i + 1]
204     return x
205
206
207 @njit(cache=True, fastmath=False)

```

```

208 def _interpl_nb(t, ts, vs):
209     """分段线性插值, 端点钳位 (附件1/附件2 的常值延拓口径)。"""
210     n = ts.shape[0]
211     if t <= ts[0]:
212         return vs[0]
213     if t >= ts[n - 1]:
214         return vs[n - 1]
215     lo = 0
216     hi = n - 1
217     while hi - lo > 1:
218         mid = (lo + hi) // 2
219         if ts[mid] <= t:
220             lo = mid
221         else:
222             hi = mid
223     w = (t - ts[lo]) / (ts[hi] - ts[lo])
224     return vs[lo] + w * (vs[hi] - vs[lo])
225
226 # =====
227 # 一个输出步的全部子步 (下沉 njit)
228 # mode: 0=coupled 1=frozen(物性冻结于 C0) 2=decoupled(D 不含 T)
229 # flags: k_zero(绝热自检) fixR(R=R0) app3(附录3 物性)
230 # t_corr: 0=IMEX 滞后 1=预估-校正 (D6 方案 b)
231 # =====
232 @njit(cache=True, fastmath=False)
233 def _step_all_nb(N, dxi, hsub, n_sub, R0, hc, km, T, C, t0,
234                 ts_env, Tenv_v, Cenv_v, ts_R, Rv,
235                 d_fac, C0f, omega, tol, maxit,
236                 mode_flag, k_zero, fixR, app3, t_corr, hc_v, L):
237     NC = N + 1
238     Tcur = T.copy()
239     Ccur = C.copy()
240     if hc_v >= 0.0:
241         hc = hc_v # 绝热自检等场景的对流系数覆盖 (0 = 完全绝热)
242
243     aT = np.zeros(NC); bT = np.zeros(NC); cT = np.zeros(NC); dT = np.zeros(NC)
244     aC = np.zeros(NC); bC = np.zeros(NC); cC = np.zeros(NC); dC = np.zeros(NC)
245
246     xi = np.empty(NC)
247     xih = np.empty(N) #  $\xi_{j+1/2}$ , j=0..N-1
248     for j in range(NC):
249         xi[j] = j * dxi
250     for j in range(N):
251         xih[j] = (j + 0.5) * dxi
252     V0 = dxi * dxi / 4.0 # 中心控制体 ( $\xi$  域)
253     VN = dxi * (1.0 - dxi / 4.0) / 2.0 # 表面控制体 ( $\xi$  域)
254
255     inner_tot = 0
256     res_max = 0.0
257
258     for s in range(n_sub):
259         t0s = t0 + s * hsub
260         t1s = t0s + hsub
261         tinf = _interpl_nb(t1s, ts_env, Tenv_v)
262         cinf = _interpl_nb(t1s, ts_env, Cenv_v)
263         if fixR:
264             R1 = R0
265             R0m = R0
266         else:
267             R1 = _interpl_nb(t1s, ts_R, Rv)
268             R0m = _interpl_nb(t0s, ts_R, Rv)
269         R1sq = R1 * R1
270
271     # ---- ① 物性 (子步起点状态: frozen 用 C0 常值) ----
272     if mode_flag == 1:
273         rho = np.empty(NC); cp = np.empty(NC); kk = np.empty(NC)
274         for j in range(NC):
275             rho[j] = _rho_nb(C0f, app3)
276             cp[j] = _cp_nb(C0f, app3)
277             kk[j] = _k_nb(C0f, app3)
278     else:
279         rho = np.empty(NC); cp = np.empty(NC); kk = np.empty(NC)
280

```



```

281     for j in range(NC):
282         rho[j] = _rho_nb(Ccur[j], app3)
283         cp[j] = _cp_nb(Ccur[j], app3)
284         kk[j] = _k_nb(Ccur[j], app3)
285     if k_zero:
286         for j in range(NC):
287             kk[j] = 0.0
288
289     # ---- ② T (全隐式, 能量守恒形式) ----
290     # ★修正口径: 界面 k 取沿 C 积分平均 (k 只依赖 C, 直接由节点 C 求);
291     # k_zero (绝热自检) 在面系数上整体置零
292     kf = _facek4_nb(Ccur, app3)
293     if k_zero:
294         for j in range(N - 1 + 1):
295             kf[j] = 0.0
296     Told = Tcur.copy()      # D6 校正步需要旧状态 (防 Q2-α 重复推进)
297     rho_o = rho.copy()
298     cp_o = cp.copy()
299     # 内部 1..N-1 (★r 空间标定, q2 同款: 固定域极限逐字一致)
300     # ★存储项口径 (E7b 能量核算 0.645 暴露、绝热判决实验证实后修正):
301     # 整行除以 R1 后, 老态项必须为  $\xi_j \cdot R0m^2/R1$  (即  $rj0 = j \cdot drp \cdot (R0m/R1)^2$ ),
302     # 否则守恒形式  $d/dt[pcp \cdot T \cdot R^2]$  的几何压缩项被系统性减半 (fixR 时  $R0m=R1$  不可见)。
303     drp = R1 * dxi          #  $\Delta r' = R1 \cdot \Delta \xi$  (新时层 r 间距)
304     for j in range(1, N):
305         rl = (j - 0.5) * drp
306         rr = (j + 0.5) * drp
307         rj1 = j * drp        #  $r_j^{n+1}$ 
308         rj0 = j * drp * (R0m / R1) ** 2 #  $\xi_j \cdot R0m^2/R1$  (老态体积元, 含  $R^2$  口径)
309         aT[j] = -kf[j - 1] * rl / (drp * drp)
310         cT[j] = -kf[j] * rr / (drp * drp)
311         bT[j] = (rho[j] * cp[j] * rj1 / hsub
312                 + (kf[j] * rr + kf[j - 1] * rl) / (drp * drp))
313         dT[j] = rho[j] * cp[j] * Tcur[j] * rj0 / hsub
314     # 中心 0 (行除以 R1sq; 固定域极限下 = Q1 中心行)
315     bT[0] = rho[0] * cp[0] / hsub + 4.0 * kf[0] / (R1sq * dxi * dxi)
316     cT[0] = -4.0 * kf[0] / (R1sq * dxi * dxi)
317     dT[0] = rho[0] * cp[0] * Tcur[0] * (R0m * R0m) / (R1sq * hsub)
318     aT[0] = 0.0
319     # 表面 N
320     bT[N] = (VN * rho[N] * cp[N] * R1sq / hsub
321             + kf[N - 1] * xih[N - 1] / dxi + hc * R1)
322     aT[N] = -kf[N - 1] * xih[N - 1] / dxi
323     cT[N] = 0.0
324     dT[N] = (VN * rho[N] * cp[N] * Tcur[N] * R0m * R0m / hsub
325             + hc * R1 * tinf)
326     if L > 0.0:
327         # E8 潜热对照: 表面蒸发吸热源项 (显式, 用子步起点 C; q2_core_latent 同式)
328         dT[N] -= L * km * R1 * (Ccur[N] - cinf)
329     Tcur = _thomas_nb(aT, bT, cT, dT)
330
331     # ---- ③ C (Picard: 右端固定=存储项, Q1 P-1 纪律) ----
332     # 行标定: 内点 r 空间 (q2 同款), 中心/表面行保持守恒形式 (数学等价)
333     dC[0] = Ccur[0] / hsub
334     for j in range(1, N):
335         dC[j] = j * drp * Ccur[j] / hsub        #  $r_j^{n+1} \cdot C^n / \Delta t$ 
336     dC[N] = VN * R1sq * Ccur[N] / hsub + km * R1 * cinf
337
338     # 右端固定 (Q1 P-1 纪律): bse 在 Picard 中不变
339     bse = dC.copy()
340
341     Cit = Ccur.copy()
342     for it in range(1, maxit + 1):
343         # ★修正口径: 界面 D 取沿 C 的积分平均 (8 点中矩); T 因子用界面均值 T
344         if mode_flag == 1:
345             # frozen: D(C0, T0) 常值 (面值=节点值, 调和/积分平均同)
346             Df = np.empty(N - 1 + 1)
347             dv = _D_nb(C0f, 28.0, d_fac, app3)
348             for j in range(N - 1 + 1):
349                 Df[j] = dv
350         elif mode_flag == 2:
351             Df = _faceD4_nb(Cit, Tcur, 1.0, False, 8, app3) # decoupled: 仅 C 因子
352         else:
353             Df = _faceD4_nb(Cit, Tcur, d_fac, True, 8, app3) # coupled

```

```

354 # 中心行
355 bC[0] = 1.0 / hsub + 4.0 * Df[0] / (R1sq * dxi * dxi)
356 cC[0] = -4.0 * Df[0] / (R1sq * dxi * dxi)
357 # 内部行补 D (r 空间标定, q2 同款)
358 for j in range(1, N):
359     aC[j] = -Df[j - 1] * (j - 0.5) * drp / (drp * drp)
360     cC[j] = -Df[j] * (j + 0.5) * drp / (drp * drp)
361     bC[j] = j * drp / hsub + (Df[j] * (j + 0.5) * drp
362         + Df[j - 1] * (j - 0.5) * drp) / (drp * drp)
363
364 # 表面行补 D
365 bC[N] = VN * R1sq / hsub + Df[N - 1] * xih[N - 1] / dxi + km * R1
366 aC[N] = -Df[N - 1] * xih[N - 1] / dxi
367
368 Ctry = _thomas_nb(aC, bC, cC, bse)
369 Cnew = np.empty(NC)
370 num = 0.0
371 den = 0.0
372 for j in range(NC):
373     v = omega * Ctry[j] + (1.0 - omega) * Cit[j]
374     dv = v - Cit[j]
375     if dv < 0.0:
376         dv = -dv
377     if dv > num:
378         num = dv
379     av = v if v >= 0.0 else -v
380     if av > den:
381         den = av
382     Cnew[j] = v
383 if den < 1.0:
384     den = 1.0
385 rr = num / den
386 Cit = Cnew
387 inner_tot += 1
388 if rr > res_max:
389     res_max = rr
390 if rr < tol:
391     break
392 Ccur = Cit
393
394 # ---- ④ D6 方案 b: 用  $C^{n+1}$  的 pc_p 重解一次 T (交付解 pc_p 取 n+1 层) ----
395 # Δ 右端存储项必须用【旧状态】( $T^n$ ,  $pc_p(C^n)$ ,  $R_n$ )——否则即 Q2-α 重复推进
396 if t_corr == 1 and (not k_zero):
397     for j in range(NC):
398         if mode_flag == 1:
399             rho[j] = _rho_nb(C0f, app3) # frozen: pc_p 保持冻结常量
400             cp[j] = _cp_nb(C0f, app3) # (b/d 两侧口径一致, 防伪源项)
401         else:
402             rho[j] = _rho_nb(Ccur[j], app3)
403             cp[j] = _cp_nb(Ccur[j], app3)
404     kf = _facek4_nb(Ccur, app3)
405     if k_zero:
406         for j in range(N):
407             kf[j] = 0.0
408     for j in range(1, N):
409         rl = (j - 0.5) * drp
410         rr = (j + 0.5) * drp
411         rj1 = j * drp
412         rj0 = j * drp * (R0m / R1) ** 2 # ★同预测步:  $\xi_j \cdot R0m^2 / R1$  (老态体积元)
413         aT[j] = -kf[j - 1] * rl / (drp * drp)
414         cT[j] = -kf[j] * rr / (drp * drp)
415         bT[j] = (rho[j] * cp[j] * rj1 / hsub
416             + (kf[j] * rr + kf[j - 1] * rl) / (drp * drp))
417         dT[j] = rho_o[j] * cp_o[j] * Told[j] * rj0 / hsub
418     bT[0] = rho[0] * cp[0] / hsub + 4.0 * kf[0] / (R1sq * dxi * dxi)
419     cT[0] = -4.0 * kf[0] / (R1sq * dxi * dxi)
420     dT[0] = rho_o[0] * cp_o[0] * Told[0] * (R0m * R0m) / (R1sq * hsub)
421     bT[N] = (VN * rho[N] * cp[N] * R1sq / hsub
422         + kf[N - 1] * xih[N - 1] / dxi + hc * R1)
423     aT[N] = -kf[N - 1] * xih[N - 1] / dxi
424     dT[N] = (VN * rho_o[N] * cp_o[N] * Told[N] * R0m * R0m / hsub
425         + hc * R1 * tinf)
426 if L > 0.0:
427     # ★修正 (E8 缺陷): 校正步必须与预测步同样计入潜热源项,

```

```

427         # 否则 t_corr=1 下潜热效应被校正步整体抹除 (旧日志 max|dT|~2e-9 的根因)。
428         # 用 C^{n+1} (当前 Ccur) 评估蒸发通量, 与对流项同时层。
429         dT[N] -= L * km * R1 * (Ccur[N] - cinf)
430         Tcur = _thomas_nb(aT, bT, cT, dT)
431
432     return Tcur, Ccur, inner_tot, res_max
433
434
435 def load_R_table(xlsx_path):
436     """读附件2 → (ts_R 秒, Rv 米)。端点钳位由 _interp1_nb 承担 (H7 末段常值)。"""
437     import openpyxl
438     wb = openpyxl.load_workbook(xlsx_path, read_only=True, data_only=True)
439     ws = wb[wb.sheetnames[0]]
440     rows = list(ws.iter_rows(values_only=True))
441     data = [(float(r[0]), float(r[1])) for r in rows[1:] if r[0] is not None]
442     ts = np.ascontiguousarray([d[0] for d in data], dtype=np.float64)
443     Rv = np.ascontiguousarray([d[1] / 100.0 for d in data], dtype=np.float64) # cm→m
444     return ts, Rv
445
446
447 def env_tables_from_att1(t_arr, T_arr, C_arr,
448                          t_tail=14400.0, T_tail=50.00, C_tail=0.04999):
449     """附件1 → njit 查表数组 (恒温段常值延拓, 口径同 q3_core 同名函数)。"""
450     t0 = t_arr[-1]
451     ts = np.ascontiguousarray(
452         np.concatenate([t_arr, [t0 + 1e-6, t0 + 1.0]]), dtype=np.float64)
453     Tv = np.ascontiguousarray(
454         np.concatenate([T_arr, [T_tail, T_tail]]), dtype=np.float64)
455     Cv = np.ascontiguousarray(
456         np.concatenate([C_arr, [C_tail, C_tail]]), dtype=np.float64)
457     return ts, Tv, Cv
458
459
460 def step_imex4(N, dxi, hsub, n_sub, R0, hc, km, T, C, t0,
461               ts_env, Tenv_v, Cenv_v, ts_R, Rv,
462               omega=0.7, tol=1e-10, maxit=30, mode='coupled',
463               k_zero=False, fixR=False, app3=False, t_corr=0, d_fac=1.0,
464               hcv=-1.0, L=0.0):
465     """推进一个输出步。返回 (T, C, info)。hcv>=0 时覆盖对流系数; L>0 为潜热源项。"""
466     mf = {'coupled': 0, 'frozen': 1, 'decoupled': 2}[mode]
467     Tk, Ck, it, rm = _step_all_nb(
468         N, float(dxi), float(hsub), int(n_sub), float(R0), float(hc), float(km),
469         np.ascontiguousarray(T, dtype=np.float64),
470         np.ascontiguousarray(C, dtype=np.float64),
471         float(t0), ts_env, Tenv_v, Cenv_v, ts_R, Rv,
472         float(d_fac), float(DEF4['C0']), float(omega), float(tol), int(maxit),
473         int(mf), 1 if k_zero else 0, 1 if fixR else 0, 1 if app3 else 0,
474         int(t_corr), float(hcv), float(L))
475     return Tk, Ck, dict(inner_tot=it, nsub=n_sub, res_max=rm)
476
477
478 def run_q4(N=80, t_end=60.0, dt_out=1.0, n_sub=32, out_every=None,
479           ts_env=None, Tenv_v=None, Cenv_v=None, ts_R=None, Rv=None,
480           mode='coupled', k_zero=False, fixR=False, app3=False, t_corr=0,
481           collect_fields=(), verbose=False, hcv=-1.0, L=0.0):
482     """驱动: 从 t=0 推进到 t_end。collect_fields 给定采样时刻集合 (秒),
483     返回 (fields{t:(T,C)}, stats)。hcv>=0 时覆盖对流系数 (绝热自检用 0)。"""
484     dxi = 1.0 / N
485     hsub = dt_out / n_sub
486     T = np.full(N + 1, DEF4['T0'])
487     C = np.full(N + 1, DEF4['C0'])
488     fields = {}
489     tot_it = 0
490     rm_all = 0.0
491     t_all = time.perf_counter()
492     nsteps = int(round(t_end / dt_out))
493     for k in range(1, nsteps + 1):
494         t0 = (k - 1) * dt_out
495         T, C, info = step_imex4(N, dxi, hsub, n_sub, DEF4['R0'], DEF4['h'],
496                                DEF4['km'], T, C, t0, ts_env, Tenv_v, Cenv_v,
497                                ts_R, Rv, mode=mode, k_zero=k_zero,
498                                fixR=fixR, app3=app3, t_corr=t_corr, hcv=hcv,
499                                L=L)

```

```

500     tot_it += info['inner_tot']
501     rm_all = max(rm_all, info['res_max'])
502     tk = k * dt_out
503     for tc in collect_fields:
504         if abs(tk - tc) < 1e-9:
505             fields[float(tc)] = (T.copy(), C.copy())
506     # 终态必存（预试验/主力求解都需要末端场）
507     fields[float(nsteps * dt_out)] = (T.copy(), C.copy())
508     stats = dict(wall=time.perf_counter() - t_all, inner_tot=tot_it,
509                 res_max=rm_all, nsteps=nsteps)
510     return fields, stats
511
512
513 def interp_cubic(ksi_nodes, values, x):
514     """一维四点三次插值（非均匀节点；单点）。节点须单调递增。"""
515     return interp_lagrange(ksi_nodes, values, x, 4)
516
517
518 def interp_lagrange(ksi_nodes, values, x, npts=6):
519     """n 点 Lagrange 插值（npts=4 三次 / 6 五次；窗口贴边钳位）。
520
521     Q4 输出口径：默认 npts=6 —— S7 实测三次插值在 N=80 下误差 ~2.8e-5,
522     贴 L3-07 上限；升到五次后插值分量降一个量级以上（见预试验 S7）。
523     """
524     n = len(ksi_nodes)
525     if x <= ksi_nodes[0]:
526         return values[0]
527     if x >= ksi_nodes[-1]:
528         return values[-1]
529     hi = 1
530     while hi < n - 1 and ksi_nodes[hi] < x:
531         hi += 1
532     lo = hi - 1
533     i0 = lo - (npts // 2 - 1)
534     if i0 < 0:
535         i0 = 0
536     if i0 + npts > n:
537         i0 = n - npts
538     xs = ksi_nodes[i0:i0 + npts]
539     ys = values[i0:i0 + npts]
540     s = 0.0
541     for i in range(npts):
542         term = ys[i]
543         for j in range(npts):
544             if j != i:
545                 term *= (x - xs[j]) / (xs[i] - xs[j])
546     s += term
547     return s

```

B.2 公共模块：元体平衡装配

```

1  # -*- coding: utf-8 -*-
2  """
3  A 题 Q1 求解核（配置化）—— 主力求解与全部检验实验共用
4  =====
5  统一口径
6  -----
7  • 空间      : 元体平衡（有限体积）+ 三对角追赶法
8  • 时间      : **温度=时间方向精确推进**（矩阵指数+分段线性 Duhamel, 见 `q1_exact.py`；
9                本核经 `tmethod='expm'` 调用，**无时间离散误差**）
10             其余（Q1 含水率、Q2-Q4 全部场量）=全隐式（后向欧拉）
11 • 中心      : 两条**真正独立**的装配路径，经同一尺度归一化后逐系数比对
12             center='fv' —— 用中心控制体体积  $V_0=\Delta r^2/4$  与界面面积  $A_{12}=\Delta r$  显式装配
13             center='lh' —— 洛必达极限  $(1/r)\partial r(r\partial T) \rightarrow 2\partial r^2 T$  的差分离散
14 • 表面      : surf='fvm' —— 元体平衡半格（主力）
15             surf='ghost' —— **一阶近似口径**（体积取  $R_0 \cdot \Delta r/2$ 、导热界面取  $R_0$ ）；
16                     **仅**用于边界口径对照实验。△ 命名沿用历史，
17                     **其实现并非“虚拟节点+中心差分”**（原虚拟节点装配符号有误、已改）
18 • 非线性    : D(C) 单因子；Picard 迭代+欠松弛；时间导数右端**固定取上一时层解**
19 • 解耦      : 先全解 T，再全解 C（T 方程不含 C，D 不含 T）
20 退化与灵敏度
21  -----
22 • kfac / Dfac: 物性缩放因子（0.0 → 退化自检；1±0.2 → 灵敏度 OAT）

```

```

23 • 环境条件以函数传入（常数环境用于 E2 解析对拍与退化自检）
24
25 约定：本核不引用、不记载任何资料中的日期。
26 """
27 import time
28 import numpy as np
29
30 DEF = dict(R0=0.02, rho=820.0, cp=2600.0, k=0.36, h=25.0, km=8.0e-7,
31           D0=7.0e-9, bD=0.89, T0=28.0, C0=2.55)
32
33 V0_OF = staticmethod(lambda dr: dr * dr / 4.0) # 中心控制体体积（ $\pi$  已约去）
34 VN_OF = staticmethod(lambda dr, R0: dr * (R0 - dr / 4.0) / 2.0) # 表面控制体体积
35
36
37 def thomas(a, b, c, d):
38     """三对角追赶法（a/b/c 为下/主/上对角，a[0] 与 c[-1] 不使用）"""
39     n = len(b)
40     cp_ = np.empty(n - 1, dtype=float)
41     dp_ = np.empty(n, dtype=float)
42     cp_[0] = c[0] / b[0]
43     dp_[0] = d[0] / b[0]
44     for i in range(1, n):
45         m = b[i] - a[i] * cp_[i - 1]
46         dp_[i] = (d[i] - a[i] * dp_[i - 1]) / m
47         if i < n - 1:
48             cp_[i] = c[i] / m
49     x = np.empty(n, dtype=float)
50     x[-1] = dp_[-1]
51     for i in range(n - 2, -1, -1):
52         x[i] = dp_[i] - cp_[i] * x[i + 1]
53     return x
54
55
56 def thomas_prep(a, b, c):
57     """常数系数三对角矩阵的预消元（前向消元结果与 d 无关，可复用）"""
58     n = len(b)
59     cp_ = np.empty(n - 1, dtype=float)
60     inv = np.empty(n, dtype=float)
61     cp_[0] = c[0] / b[0]
62     inv[0] = 1.0 / b[0]
63     for i in range(1, n):
64         m = b[i] - a[i] * cp_[i - 1]
65         inv[i] = 1.0 / m
66         if i < n - 1:
67             cp_[i] = c[i] / m
68     return cp_, inv
69
70
71 def thomas_solve_prep(a, cp_, inv, d):
72     """配合 thomas_prep 使用的快速求解（与 thomas 同式，仅省去重复除法）"""
73     n = len(d)
74     dp_ = np.empty(n, dtype=float)
75     dp_[0] = d[0] * inv[0]
76     for i in range(1, n):
77         dp_[i] = (d[i] - a[i] * dp_[i - 1]) * inv[i]
78     x = np.empty(n, dtype=float)
79     x[-1] = dp_[-1]
80     for i in range(n - 2, -1, -1):
81         x[i] = dp_[i] - cp_[i] * x[i + 1]
82     return x
83
84
85 def D_of(C, D0=DEF['D0'], bD=DEF['bD']):
86     """有效水分扩散系数  $D(C) = D0 \cdot \exp(-bD/C)$ """
87     return D0 * np.exp(-bD / np.maximum(np.asarray(C, dtype=float), 1e-9))
88
89
90 def harm(dl, dr_):
91     """界面系数：距离加权调和平均（ $D=0$  时退化为 0，避免除零）。
92
93     △ **本函数已不用于生产路径**：C 方程的界面扩散系数改由 `faceD_int`
94     （**沿 $C$ 的积分平均**，口径 **M6**）生成；本函数**仅保留作旧口径对照**。
95     """

```

```

96     if dl <= 0.0 or dr_ <= 0.0:
97         return 0.0
98     return 2.0 / (1.0 / dl + 1.0 / dr_)
99
100
101 # ----- T 方程
102 def assemble_T(N, dr, dt, p, center='fv', surf='fvm'):
103     """装配后向欧拉 T 方程系数 (a, b, c, meta): 右端另由 rhs_T 生成"""
104     rho, cp = p['rho'], p['cp']
105     k = p['k'] * p.get('kfacs', 1.0)
106     h, R0 = p['h'], p['R0']
107     r = np.arange(N + 1) * dr
108     a = np.zeros(N + 1); b = np.zeros(N + 1); c = np.zeros(N + 1)
109     meta = {}
110
111     # --- 中心节点: 两条独立路径, 均归一化到"单位体积"以便逐系数比对 ---
112     if center == 'fv':
113         V0 = dr * dr / 4.0
114         A12 = dr
115         b[0] = (rho * cp * V0 / dt + k * A12 / dr) / V0
116         c[0] = (-k * A12 / dr) / V0
117         meta['center_scale'] = V0
118     elif center == 'lh':
119         d2 = 2.0 / dr ** 2
120         b[0] = rho * cp / dt + 2.0 * k * d2
121         c[0] = -2.0 * k * d2
122         meta['center_scale'] = 1.0
123     else:
124         raise ValueError(center)
125     meta['center_row'] = (float(b[0]), float(c[0]))
126
127     # --- 内部节点: 标准元体平衡 (含 r_{i±1/2} 几何权重) ---
128     for i in range(1, N):
129         rl, rr = r[i] - dr / 2.0, r[i] + dr / 2.0
130         b[i] = rho * cp * r[i] / dt + k * (rr + rl) / dr ** 2
131         a[i] = -k * rl / dr ** 2
132         c[i] = -k * rr / dr ** 2
133
134     # --- 表面节点 ---
135     if surf == 'fvm':
136         VN = dr * (R0 - dr / 4.0) / 2.0
137         b[N] = rho * cp * VN / dt + k * (R0 - dr / 2.0) / dr + h * R0
138         a[N] = -k * (R0 - dr / 2.0) / dr
139         meta['surf'] = 'fvm'
140     elif surf == 'ghost':
141         # 对照口径 (**仅用于边界离散敏感性对照**):
142         # 表面控制体体积取一阶近似  $R0 \cdot \Delta r / 2$  (忽略曲率), 导热界面半径取 R0
143         VNa = R0 * dr / 2.0
144         b[N] = rho * cp * VNa / dt + k * R0 / dr + h * R0
145         a[N] = -k * R0 / dr
146         meta['surf'] = 'approx'
147     else:
148         raise ValueError(surf)
149     return a, b, c, meta
150
151
152 def rhs_T(N, dr, dt, p, Told, Tinf, surf='fvm'):
153     """生成 T 方程右端向量 d"""
154     rho, cp = p['rho'], p['cp']
155     h, R0 = p['h'], p['R0']
156     r = np.arange(N + 1) * dr
157     d = np.zeros(N + 1)
158     d[0] = rho * cp * Told[0] / dt
159     for i in range(1, N):
160         d[i] = rho * cp * r[i] * Told[i] / dt
161     if surf == 'fvm':
162         VN = dr * (R0 - dr / 4.0) / 2.0
163         d[N] = rho * cp * VN * Told[N] / dt + h * R0 * Tinf
164     else:
165         VNa = R0 * dr / 2.0
166         d[N] = rho * cp * VNa * Told[N] / dt + h * R0 * Tinf
167     return d
168

```

```

169 # ----- C 方程
170 def assemble_C(N, dr, dt, Df, R0, km):
171     """装配 C 方程系数 (a, b, c, VN); Df 为界面扩散系数 (长度 N, **沿 $C$ 的积分平均**, 由 `faceD_int`
172     生成)"""
173     r = np.arange(N + 1) * dr
174     a = np.zeros(N + 1); b = np.zeros(N + 1); c = np.zeros(N + 1)
175     V0 = dr * dr / 4.0
176     A12 = dr
177     b[0] = (V0 / dt + Df[0] * A12 / dr) / V0
178     c[0] = (-Df[0] * A12 / dr) / V0
179     for i in range(1, N):
180         rl, rr = r[i] - dr / 2.0, r[i] + dr / 2.0
181         b[i] = r[i] / dt + (Df[i] * rr + Df[i - 1] * rl) / dr ** 2
182         a[i] = -Df[i - 1] * rl / dr ** 2
183         c[i] = -Df[i] * rr / dr ** 2
184     VN = dr * (R0 - dr / 4.0) / 2.0
185     b[N] = VN / dt + Df[N - 1] * (R0 - dr / 2.0) / dr + km * R0
186     a[N] = -Df[N - 1] * (R0 - dr / 2.0) / dr
187     return a, b, c, VN
188
189 def rhs_C(N, dr, dt, Cold, cinf, R0, km):
190     """生成 C 方程右端向量 d (**固定取上一时层解 Cold**)"""
191     r = np.arange(N + 1) * dr
192     d = np.zeros(N + 1)
193     d[0] = Cold[0] / dt
194     for i in range(1, N):
195         d[i] = r[i] * Cold[i] / dt
196     VN = dr * (R0 - dr / 4.0) / 2.0
197     d[N] = VN * Cold[N] / dt + km * R0 * cinf
198     return d
199
200
201 def faceD_int(Carr, D0, bD, Dfac=1.0, nsamp=8):
202     """*修正口径: 界面扩散系数取**沿 C 的积分平均**  $\langle D \rangle = (1/\Delta C) \int D \, dC$ 。
203
204     (原 `harm` 的调和平均仅对"界面处系数间断"成立; 系数随 C 陡变时显著失真。)
205     """
206     C = np.asarray(Carr, dtype=float)
207     lo = np.minimum(C[:-1], C[1:])
208     hi = np.maximum(C[:-1], C[1:])
209     xs = lo[:, None] + (hi - lo)[1:, None] * (np.arange(nsamp) + 0.5)[None, :] / nsamp
210     xs = np.maximum(xs, 1e-9)
211     fv = np.exp(-bD / xs).mean(axis=1)
212     flat = (hi - lo) < 1e-14
213     if flat.any():
214         fv[flat] = np.exp(-bD / np.maximum(lo[flat], 1e-9))
215     return D0 * Dfac * fv
216
217
218 def step_C(N, dr, dt, C, cinf, p, omega=0.7, tol=1e-9, maxit=30, Dfac=1.0,
219           dmode='func'):
220     """推进一个 C 子步: Picard 迭代+欠松弛。返回 (C_new, iters, res)
221
222     dmode='func' : D = D(C) 单因子非线性 (主力)
223     dmode='const': D ≡ D(C0) (用于 E2 解析对拍; 此时单次迭代即收敛)
224     """
225     Cold = C.copy()
226     Cit = C.copy()
227     res = None
228     it = 0
229     Dconst = p['D0'] * np.exp(-p['bD'] / p['C0']) * Dfac
230     for it in range(1, maxit + 1):
231         if dmode == 'const':
232             Df = np.full(N, Dconst)
233         else:
234             Df = faceD_int(Cit, p['D0'], p['bD'], Dfac)
235         a, b, c, _ = assemble_C(N, dr, dt, Df, p['R0'], p['km'])
236         d = rhs_C(N, dr, dt, Cold, cinf, p['R0'], p['km'])
237         Ctry = thomas(a, b, c, d)
238         Cnew = omega * Ctry + (1.0 - omega) * Cit
239         res = float(np.max(np.abs(Cnew - Cit))) / max(1.0, float(np.max(np.abs(Cnew))))
240

```

```

241         Cit = Cnew
242         if res < tol:
243             break
244     return Cit, it, res
245
246
247 # ----- 驱动
248 def run_sim(N, dr, dt_out, nsteps, p, Tenv_fn, Cenv_fn, sub=10, subT=1,
249             center='fv', surf='fvm', kfac=1.0, Dfac=1.0, dmode='func',
250             tol=1e-9, omega=0.7, maxit=30, snap_at=(), out_idx=None,
251             solve_T=True, solve_C=True, tmethod='expm'):
252     """统一求解驱动。
253
254     参数
255     ----
256     N, dr      : 空间区间数与步长 (节点数 N+1)
257     dt_out     : 输出时间步长 (s)
258     nsteps     : 输出步数 (总时长 = nsteps * dt_out)
259     p          : 物性/几何参数字典 (见 DEF)
260     Tenv_fn    :  $T_\infty(t)$  函数; Cenv_fn:  $C_\infty(t)$  函数
261     sub        : C 方程内部子步数 (内部步长 = dt_out/sub)
262     subT       : T 方程内部子步数 (内部步长 = dt_out/subT, 用于时间收敛实验)
263     out_idx    : 需要收集的列索引 (None → 不收集整表)
264     返回 dict
265     """
266     p = dict(p)
267     p['kfac'] = kfac
268     R0 = p['R0']
269     r = np.arange(N + 1) * dr
270     dt_int = dt_out / float(sub)
271     dtT = dt_out / float(subT)
272
273     res = dict(N=N, dr=dr, dt_out=dt_out, dt_int=dt_int, sub=sub,
274               center=center, surf=surf, kfac=kfac, Dfac=Dfac)
275
276     # ----- T -----
277     Tend = None
278     Tsnap = {}
279     res['tmethod'] = tmethod
280     if solve_T:
281         a, b, c, meta = assemble_T(N, dr, dtT, p, center, surf)
282         res['center_row'] = meta['center_row']
283         t0 = time.time()
284         if tmethod == 'expm':
285             # 时间方向**精确推进** (交付口径): 矩阵指数 + 分段线性 Duhamel
286             import q1_exact as _ex
287             rT = _ex.run_T_expm(N, dr, p, Tenv_fn, nsteps * dt_out,
288                                hstep=dt_out, center=center, surf=surf,
289                                t_snaps=[n * dt_out for n in snap_at])
290             Tsnap[0] = np.full(N + 1, p['T0'])
291             for n in snap_at:
292                 Tsnap[n] = rT['T_snap'][round(float(n * dt_out), 9)].copy()
293             T = rT['T_end']
294         else:
295             cp_, inv = thomas_prep(a, b, c)
296             T = np.full(N + 1, p['T0'])
297             Tsnap[0] = T.copy()
298             for n in range(1, nsteps + 1):
299                 for s in range(subT):
300                     ts = (n - 1) * dt_out + (s + 1) * dtT
301                     d = rhs_T(N, dr, dtT, p, T, float(Tenv_fn(ts)), surf)
302                     T = thomas_solve_prep(a, cp_, inv, d)
303                 if n in snap_at:
304                     Tsnap[n] = T.copy()
305             res['T_time'] = time.time() - t0
306             Tend = T.copy()
307     res['T_end'] = Tend
308     res['T_snap'] = Tsnap
309
310     # ----- C -----
311     Cend = None
312     Csnap = {}
313     stats = dict(total_it=0, avg_it=0.0, max_it=0, max_res=0.0, nsub=0)

```



```

314     if solve_C:
315         C = np.full(N + 1, p['C0'])
316         Csnap[0] = C.copy()
317         tot, mx, mxr, nsub = 0, 0, 0.0, 0
318         t0 = time.time()
319         for n in range(1, nsteps + 1):
320             for s in range(sub):
321                 tt = (n - 1) * dt_out + (s + 1) * dt_int
322                 cinf = float(Cenv_fn(tt))
323                 C, it, rr = step_C(N, dr, dt_int, C, cinf, p, omega, tol, maxit, Dfac, dmode)
324                 tot += it; mx = max(mx, it); mxr = max(mxr, rr); nsub += 1
325             if n in snap_at:
326                 Csnap[n] = C.copy()
327         res['C_time'] = time.time() - t0
328         Cend = C.copy()
329         stats.update(total_it=tot, avg_it=tot / max(1, nsub), max_it=mx,
330                     max_res=mxr, nsub=nsub)
331     res['C_end'] = Cend
332     res['C_snap'] = Csnap
333     res['stats'] = stats
334     return res
335
336
337 def run_sim_collect(N, dr, dt_out, nsteps, p, Tenv_fn, Cenv_fn, sub=10, subT=1,
338                   center='fv', surf='fvm', kfac=1.0, Dfac=1.0, dmode='func',
339                   tol=1e-9, omega=0.7, maxit=30, cols=None,
340                   snap_at=(), solve_T=True, solve_C=True, tmethod='expm'):
341     """单遍运行并**同时**收集整表输出与快照（替代"两遍计算"）。
342
343     cols: 需要写出的列索引（如 np.arange(0, N+1, 4)），None 表示不收集
344     返回 (res, tableT, tableC); tableT/C 形状为 (nsteps, len(cols))
345     """
346     p = dict(p)
347     p['kfac'] = kfac
348     R0 = p['R0']
349     r = np.arange(N + 1) * dr
350     dt_int = dt_out / float(sub)
351     dtT = dt_out / float(subT)
352     ncol = 0 if cols is None else len(cols)
353
354     tableT = np.zeros((nsteps, ncol)) if (cols is not None and solve_T) else None
355     tableC = np.zeros((nsteps, ncol)) if (cols is not None and solve_C) else None
356     Tsnap, Csnap = {}, {}
357     res = dict(N=N, dr=dr, dt_out=dt_out, dt_int=dt_int, sub=sub,
358               center=center, surf=surf, kfac=kfac, Dfac=Dfac)
359
360     Tprep = None
361     res['tmethod'] = tmethod
362     use_expm_T = bool(solve_T and tmethod == 'expm')
363     if solve_T:
364         aT, bT, cT, meta = assemble_T(N, dr, dtT, p, center, surf)
365         res['center_row'] = meta['center_row']
366         if use_expm_T:
367             import q1_exact as _ex
368             rT = _ex.run_T_expm(N, dr, p, Tenv_fn, nsteps * dt_out, hstep=dt_out,
369                                center=center, surf=surf,
370                                t_snaps=[n * dt_out for n in snap_at],
371                                cols=cols, dt_out=dt_out)
372             tableT = rT['table']
373             T = rT['T_end']
374             Tsnap[0] = np.full(N + 1, p['T0'])
375             for n in snap_at:
376                 Tsnap[n] = rT['T_snap'][round(float(n * dt_out), 9)].copy()
377             tT = 0.0
378         else:
379             cpT, invT = thomas_prep(aT, bT, cT)
380             Tprep = (aT, cpT, invT)
381             T = np.full(N + 1, p['T0'])
382             Tsnap[0] = T.copy()
383
384     C = np.full(N + 1, p['C0']) if solve_C else None
385     Csnap[0] = C.copy() if C is not None else None
386

```

```

387 tot, mx, mxr, nsub = 0, 0, 0.0, 0
388 tT = tC = 0.0
389 for n in range(1, nsteps + 1):
390     if solve_T and not use_expm_T:
391         t0 = time.time()
392         a, cp_, inv = Tprep
393         for s in range(subT):
394             ts = (n - 1) * dt_out + (s + 1) * dtT
395             d = rhs_T(N, dr, dtT, p, T, float(Tenv_fn(ts)), surf)
396             T = thomas_solve_prep(a, cp_, inv, d)
397             tT += time.time() - t0
398             if n in snap_at:
399                 Tsnap[n] = T.copy()
400             if cols is not None:
401                 tableT[n - 1, :] = T[cols]
402         if solve_C:
403             t0 = time.time()
404             for s in range(sub):
405                 tt = (n - 1) * dt_out + (s + 1) * dt_int
406                 C, it, rr = step_C(N, dr, dt_int, C, float(Cenv_fn(tt)), p,
407                                 omega, tol, maxit, Dfac, dmode)
408                 tot += it; mx = max(mx, it); mxr = max(mxr, rr); nsub += 1
409             tC += time.time() - t0
410             if n in snap_at:
411                 Csnap[n] = C.copy()
412             if cols is not None:
413                 tableC[n - 1, :] = C[cols]
414
415 res['T_end'] = T.copy() if solve_T else None
416 res['C_end'] = C.copy() if solve_C else None
417 res['T_snap'] = Tsnap
418 res['C_snap'] = Csnap
419 res['T_time'] = tT
420 res['C_time'] = tC
421 res['stats'] = dict(total_it=tot, avg_it=tot / max(1, nsub), max_it=mx,
422                   max_res=mxr, nsub=nsub)
423 return res, tableT, tableC

```

```

1  # -*- coding: utf-8 -*-
2  """
3  A 题 Q2 求解核（变物性 · 双向强耦合）· v4（★B + ★C 优化）
4  =====
5  ★C 优化（在 B 的基础上）：**以 numba 编译的手写追赶法（Thomas）替换
6  `scipy.linalg.solve_banded`**。
7
8  依据（用户指令解除“禁止替换 Thomas”限制后实施）：
9  • 本系统矩阵为**三对角且严格对角占优**（ $|b_i| > |a_i| + |c_i|$ ，由隐式格式的
10  正定性保证），故追赶法**无需选主元即可稳定**，与 LAPACK `dgbstv` 数值等价；
11  • `solve_banded` 对小规模带状矩阵存在固定调用开销（Python 校验 + 数据整理），
12  微基准实测 21.25  $\mu$ s（关检查后 14.48  $\mu$ s），而 njit 追赶法仅需数微秒；
13  • 追赶法**消元顺序与 LAPACK 不同**  $\Rightarrow$  结果存在  $\sim 1e-15$  量级的舍入差异
14  （远低于 4 位小数输出精度），**必须经 `q2_verify_BC.py` 核对**。
15
16  △ 其余一切（离散表达式、Picard 流程、tol/maxit/omega、物性公式、IMEX 子步）
17  与 B 版**完全相同**。
18
19  --- 以下为 B 版说明 ---
20
21  A 题 Q2 求解核（变物性 · 双向强耦合）· v3（★B 优化：numba JIT）
22  =====
23  与 Q1 核（q1_core.py）的关系
24  -----
25  Q2 = Q1 框架 + 变物性 + 强耦合。本核**照搬** Q1 的离散结构：
26  • 元体平衡（有限体积）+ 全隐式（后向欧拉）+ 三对角求解
27  • 中心用中心控制体（ $V0=\Delta r^2/4$ 、 $A_1^2=\Delta r$ ）；表面用半格元体平衡
28  • 界面系数一律**距离加权调和平均**
29  差异（两处）：
30  • 物性不再是常数： $p(C)$ 、 $c_p(C)$ 、 $k(C)$ 、 $D(C,T)$ （附录3，T 用 K）
31  • 两场**双向耦合**  $\Rightarrow$  外层耦合迭代 + 内层 Picard 迭代
32
33  v3 性能改造（★B：numba JIT；**不改任何数值语义**）
34  -----
35  • 用 `@njit(cache=True, fastmath=False)` 编译：物性  $p/c_p/k/D$ 、界面调和平均、

```

```

36  以及 T/C 的装配与右端函数（改为**接收预分配数组、原地填充**）。
37  • **`fastmath=False` 是硬性要求** —— 保持 IEEE 语义，禁止任何浮点重排/近似。
38  • **求解器仍是 `scipy.linalg.solve_banded`**（B 版不替换 Thomas；C 版另见 `q2_core_c.py`）。
39  • **Picard 的 `for it` 循环、残差计算、break 条件、欠松弛、外层逻辑全部留在
40  Python 层，一行未改**；离散公式与装配表达式**逐字保留**（见各函数注释对照）。
41  • numba 不可用时自动**降级**为原 numpy 实现（`_HAS_NUMBA=False`），行为不变。
42
43  口径（全部引自《A_数值口径总表》）
44  -----
45  N7   $\rho = 650 + 128C$  | N8   $c_p = 1450 + 2736 \cdot C / (C+1)$  | N9   $k = 0.21 + 0.38 \cdot C / (C+1)$ 
46  N10  $D = 2.4e-3 \cdot \exp(-0.45/C) \cdot \exp(-3850/T)$  [m2/s], T 用 K
47  N4   $h = 25$  | N5   $k_m = 8e-7$ （全程沿用附录2, H5）
48
49  模式
50  ----
51  'coupled'   主力：双向强耦合
52  'frozen'    E2 常物性退化对拍
53  'decoupled' E3 强制解耦（D 不含 T）
54
55  关键纪律
56  -----
57  • **每一轮外层迭代中，T 与 C 都从 n 层解出发**（右端固定），仅物性随迭代更新
58  • **C 方程每个子步的右端取本子步起点**（子步间串行推进）
59  • 欠松弛  $\omega=0.7$  必须保留；收敛判据一律用解的（绝对）增量
60
61  约定：本核不引用、不记载任何资料中的日期。
62  """
63  import time
64  import numpy as np
65  from scipy.linalg import solve_banded
66
67  # =====
68  # numba 可用性（不可用则自动降级为 numpy 实现，行为完全不变）
69  # =====
70  try:
71      from numba import njit
72      _HAS_NUMBA = True
73  except Exception:
74      njit = None
75      _HAS_NUMBA = False
76
77
78  DEF2 = dict(R0=0.02, h=25.0, km=8.0e-7, T0=28.0, C0=2.55)
79
80  # 全局扩散系数缩放因子（供灵敏度实验 OAT 使用；默认 1.0 不影响主力结果）
81  D_FAC = 1.0
82
83
84  # =====
85  # ★ JIT 内核（fastmath=False: 严格 IEC 559, 禁止浮点重排）
86  # 每个内核下方都注明其**原 numpy 表达式**，供逐字核对。
87  # =====
88  if _HAS_NUMBA:
89
90      @njit(cache=True, fastmath=False)
91      def _rho_nb(C):
92          """N7:  $\rho = 650 + 128C$  （原式  $650.0 + 128.0 * C$ ）"""
93          n = C.shape[0]
94          out = np.empty(n)
95          for i in range(n):
96              out[i] = 650.0 + 128.0 * C[i]
97          return out
98
99      @njit(cache=True, fastmath=False)
100      def _cp_nb(C):
101          """N8:  $c_p = 1450 + 2736 \cdot C / (C+1)$  （原式  $1450.0 + 2736.0 * C / (C + 1.0)$ ）"""
102          n = C.shape[0]
103          out = np.empty(n)
104          for i in range(n):
105              out[i] = 1450.0 + 2736.0 * C[i] / (C[i] + 1.0)
106          return out
107
108      @njit(cache=True, fastmath=False)

```

```

109 def _k_nb(C):
110     """N9:  $k = 0.21 + 0.38 \cdot C / (C+1)$  (原式  $0.21 + 0.38 \cdot C / (C + 1.0)$ ) """
111     n = C.shape[0]
112     out = np.empty(n)
113     for i in range(n):
114         out[i] = 0.21 + 0.38 * C[i] / (C[i] + 1.0)
115     return out
116
117 @njit(cache=True, fastmath=False)
118 def _D_nb(C, T_C, d_fac):
119     """N10:  $D = D\_FAC \cdot 2.4e-3 \cdot \exp(-0.45/C) \cdot \exp(-3850/T)$ , T 用 K
120
121     △ `D_FAC` 原为模块级全局量; 在 njit 内**必须显式传参**,
122     否则 numba 会将其在编译期冻结为常量 (OAT 扰动将失效)。
123     """
124     n = C.shape[0]
125     out = np.empty(n)
126     for i in range(n):
127         c = C[i]
128         if c < 1e-9:
129             c = 1e-9
130         out[i] = d_fac * 2.4e-3 * np.exp(-0.45 / c) * np.exp(-3850.0 / (T_C[i] + 273.15))
131     return out
132
133 @njit(cache=True, fastmath=False)
134 def _iface_nb(f):
135     """界面值: 距离加权调和平均; 任一侧 ≤0 则取 0
136     (原式  $out[m] = 2 \cdot l \cdot r / (l+r)$ , 其余位置保持 0)
137
138     △ **本式仅适用于**系数在界面处间断 (分段常数)**。
139     当系数随  $\$C\$$  平滑但陡峭变化时 (如 Q3 低含水率端  $\$D\$$  跨 2-3 个数量级),
140     应改用下方 `_faceD_nb` / `_facek_nb` 的**沿  $\$C\$$  积分平均**。
141     """
142     n = f.shape[0] - 1
143     out = np.zeros(n)
144     for i in range(n):
145         l = f[i]
146         r = f[i + 1]
147         if l > 0.0 and r > 0.0:
148             out[i] = 2.0 * l * r / (l + r)
149     return out
150
151 @njit(cache=True, fastmath=False)
152 def _faceD_nb(Carr, Tarr, d_fac, useT, nsamp):
153     """***修正口径**：界面扩散系数取**沿  $\$C\$$  的积分平均**
154
155     
$$\langle D \rangle = (1/\Delta C) \int_{C_{i+1}}^{C_i} D(C, T_{face}) dC$$

156
157     依据: 单元内稳态通量  $J = \frac{1}{\Delta r} \int D \, dC \Rightarrow$  界面系数 = 沿  $\$C\$$  的
158     积分平均。
159     利用可分离性  $D = [d_{fac} 2.4 \times 10^{-3} \exp\{-0.45/C\}] \exp\{-3850/T_K\}$ , 只需对
160      $\$C\$$  因子采样。
161     `useT=True`: 含  $\$T\$$  因子 (coupled); `False`: 仅  $\$C\$$  因子 (decoupled, 不乘  $d_{fac}$ )。
162      $\$C| \Delta C| \approx 0$  时退化为节点值。
163     """
164     n = Carr.shape[0] - 1
165     out = np.empty(n)
166     for i in range(n):
167         a = Carr[i]
168         b = Carr[i + 1]
169         lo = a if a < b else b
170         hi = b if a < b else a
171         if hi - lo < 1e-14:
172             c = lo if lo > 1e-9 else 1e-9
173             fv = np.exp(-0.45 / c)
174         else:
175             s = 0.0
176             for k in range(nsamp):
177                 x = lo + (hi - lo) * (k + 0.5) / nsamp
178                 if x < 1e-9:
179                     x = 1e-9
180                 s += np.exp(-0.45 / x)
181             fv = s / nsamp

```

```

180         if useT:
181             Tm = 0.5 * (Tarr[i] + Tarr[i + 1])
182             out[i] = d_fac * 2.4e-3 * fv * np.exp(-3850.0 / (Tm + 273.15))
183         else:
184             out[i] = 2.4e-3 * fv
185     return out
186
187 @njit(cache=True, fastmath=False)
188 def _facek_nb(Carr):
189     """***修正口径**：界面导热系数取沿 $C$ 的积分平均（解析式）
190
191     (k) = 0.21 + 0.38 * [Δ(C - ln(1+C))]/ΔC
192     """
193     n = Carr.shape[0] - 1
194     out = np.empty(n)
195     for i in range(n):
196         a = Carr[i]
197         b = Carr[i + 1]
198         if abs(b - a) < 1e-14:
199             c = a
200             out[i] = 0.21 + 0.38 * c / (1.0 + c)
201         else:
202             ga = a - np.log(1.0 + a)
203             gb = b - np.log(1.0 + b)
204             out[i] = 0.21 + 0.38 * (gb - ga) / (b - a)
205     return out
206
207 @njit(cache=True, fastmath=False)
208 def _assemble_T_nb(N, dr, dt, R0, h, rho, cp, k, a, b, c, kf):
209     """T 方程装配（原地填充 a/b/c）。表达式与原 numpy 版逐字对应。
210
211     `kf`（界面调和平均）由调用方预先算好传入 —— 与 v2 中
212     "函数内先 `kf = iface(k)`" 语义一致，只是把该步提到包装层以避免重复计算。
213     """
214     V0 = dr * dr / 4.0
215     b[0] = (rho[0] * cp[0] * V0 / dt + kf[0] * dr / dr) / V0
216     c[0] = (-kf[0] * dr / dr) / V0
217     a[0] = 0.0
218     for i in range(1, N):
219         r = i * dr
220         rl = r - dr / 2.0
221         rr = r + dr / 2.0
222         rc = rho[i] * cp[i]
223         b[i] = rc * r / dt + (kf[i] * rr + kf[i - 1] * rl) / dr ** 2
224         a[i] = -kf[i - 1] * rl / dr ** 2
225         c[i] = -kf[i] * rr / dr ** 2
226     VN = dr * (R0 - dr / 4.0) / 2.0
227     b[N] = rho[N] * cp[N] * VN / dt + kf[N - 1] * (R0 - dr / 2.0) / dr + h * R0
228     a[N] = -kf[N - 1] * (R0 - dr / 2.0) / dr
229     c[N] = 0.0
230
231 @njit(cache=True, fastmath=False)
232 def _rhs_T_nb(N, dr, dt, R0, h, rho, cp, Told, Tinf, d):
233     """T 方程右端（原地填充 d）：固定取上一时层解 Told"""
234     rcT = rho * cp * Told
235     d[0] = rcT[0] / dt
236     for i in range(1, N):
237         d[i] = (i * dr) * rcT[i] / dt
238     VN = dr * (R0 - dr / 4.0) / 2.0
239     d[N] = rcT[N] * VN / dt + h * R0 * Tinf
240     return d
241
242 @njit(cache=True, fastmath=False)
243 def _assemble_C_nb(N, dr, dt, Df, R0, km, a, b, c):
244     """C 方程装配（原地填充 a/b/c）。表达式与原 numpy 版逐字对应。"""
245     V0 = dr * dr / 4.0
246     b[0] = (V0 / dt + Df[0] * dr / dr) / V0
247     c[0] = (-Df[0] * dr / dr) / V0
248     a[0] = 0.0
249     for i in range(1, N):
250         r = i * dr
251         rl = r - dr / 2.0
252         rr = r + dr / 2.0

```

```

253         b[i] = r / dt + (Df[i] * rr + Df[i - 1] * rl) / dr ** 2
254         a[i] = -Df[i - 1] * rl / dr ** 2
255         c[i] = -Df[i] * rr / dr ** 2
256     VN = dr * (R0 - dr / 4.0) / 2.0
257     b[N] = VN / dt + Df[N - 1] * (R0 - dr / 2.0) / dr + km * R0
258     a[N] = -Df[N - 1] * (R0 - dr / 2.0) / dr
259     c[N] = 0.0
260
261 @njit(cache=True, fastmath=False)
262 def _rhs_C_nb(N, dr, dt, R0, km, Cold, cinf, d):
263     """C 方程右端 (原地填充 d)"""
264     d[0] = Cold[0] / dt
265     for i in range(1, N):
266         d[i] = (i * dr) * Cold[i] / dt
267     VN = dr * (R0 - dr / 4.0) / 2.0
268     d[N] = VN * Cold[N] / dt + km * R0 * cinf
269     return d
270
271 @njit(cache=True, fastmath=False)
272 def _thomas_nb(a, b, c, d):
273     """*C: 手写追赶法 (Thomas 消元, 无选主元)
274
275     矩阵严格对角占优  $\Rightarrow$  无需主元选择即数值稳定。
276
277     算法 (标准前向消元 + 后向回代):
278         前向: cp[0] = c[0]/b[0]; dp[0] = d[0]/b[0]
279             cp[i] = c[i] / (b[i] - a[i] * cp[i-1])
280             dp[i] = (d[i] - a[i] * dp[i-1]) / (b[i] - a[i] * cp[i-1])
281         后向: x[n-1] = dp[n-1]; x[i] = dp[i] - cp[i] * x[i+1]
282
283     `a[0]` 与 `c[n-1]` 在装配时已置零, 故首末行自动退化为正确形式。
284     """
285     n = b.shape[0]
286     cp_ = np.empty(n)
287     dp_ = np.empty(n)
288     cp_[0] = c[0] / b[0]
289     dp_[0] = d[0] / b[0]
290     for i in range(1, n):
291         m = b[i] - a[i] * cp_[i - 1]
292         cp_[i] = c[i] / m
293         dp_[i] = (d[i] - a[i] * dp_[i - 1]) / m
294     x = np.empty(n)
295     x[n - 1] = dp_[n - 1]
296     for i in range(n - 2, -1, -1):
297         x[i] = dp_[i] - cp_[i] * x[i + 1]
298     return x
299
300
301 # =====
302 # 公共接口 (签名与 v2 完全一致  $\Rightarrow$  既有脚本零改动)
303 # =====
304 def rho_of(C):
305     if _HAS_NUMBA:
306         arr = np.asarray(C, dtype=np.float64)
307         if arr.ndim == 0: # 标量输入  $\Rightarrow$  返回标量 (与原版 float() 兼容)
308             return float(_rho_nb(arr.reshape(1))[0])
309             return _rho_nb(np.ascontiguousarray(arr))
310     return 650.0 + 128.0 * np.asarray(C, dtype=float)
311
312
313 def cp_of(C):
314     if _HAS_NUMBA:
315         arr = np.asarray(C, dtype=np.float64)
316         if arr.ndim == 0:
317             return float(_cp_nb(arr.reshape(1))[0])
318             return _cp_nb(np.ascontiguousarray(arr))
319     C = np.asarray(C, dtype=float)
320     return 1450.0 + 2736.0 * C / (C + 1.0)
321
322
323 def k_of(C):
324     if _HAS_NUMBA:
325         arr = np.asarray(C, dtype=np.float64)

```

```

326         if arr.ndim == 0:
327             return float(_k_nb(arr.reshape(1))[0])
328         return _k_nb(np.ascontiguousarray(arr))
329     C = np.asarray(C, dtype=float)
330     return 0.21 + 0.38 * C / (C + 1.0)
331
332
333 def D_of(C, T_C):
334     """N10:  $D = 2.4e-3 \cdot \exp(-0.45/C) \cdot \exp(-3850/T)$ , T 用 K"""
335     if _HAS_NUMBA:
336         Ca = np.asarray(C, dtype=np.float64)
337         Ta = np.asarray(T_C, dtype=np.float64)
338         if Ca.ndim == 0:
339             v = _D_nb(Ca.reshape(1), Ta.reshape(1), float(D_FAC))
340             return float(v[0])
341         return _D_nb(np.ascontiguousarray(Ca), np.ascontiguousarray(Ta), float(D_FAC))
342     C = np.maximum(np.asarray(C, dtype=float), 1e-9)
343     TK = np.asarray(T_C, dtype=float) + 273.15
344     return D_FAC * 2.4e-3 * np.exp(-0.45 / C) * np.exp(-3850.0 / TK)
345
346
347 def harm(dl, dr_):
348     """**标量版**界面系数: 距离加权调和平均 (与 `iface` 等价)"""
349     if dl <= 0.0 or dr_ <= 0.0:
350         return 0.0
351     return 2.0 / (1.0 / dl + 1.0 / dr_)
352
353
354 def iface(f):
355     """界面值: 距离加权调和平均 (矢量化)。任一侧  $\leq 0$  则取 0."""
356     if _HAS_NUMBA:
357         arr = np.asarray(f, dtype=np.float64)
358         if arr.ndim == 0:
359             return np.zeros(0)
360         return _iface_nb(np.ascontiguousarray(arr))
361     f = np.asarray(f, dtype=float)
362     l, r = f[:-1], f[1:]
363     s = l + r
364     out = np.zeros_like(l)
365     m = (l > 0.0) & (r > 0.0)
366     out[m] = 2.0 * l[m] * r[m] / s[m]
367     return out
368
369
370 # =====
371 # ***修正口径**: 界面系数取"沿 C 的积分平均"
372 # (原 `iface` 的调和平均仅对"界面处系数间断"成立; 本项在系数随 C 陡变时失真)
373 # =====
374 NSAMP_FACE = 8          # 沿 C 的 Midpoint 采样点数 (已验 4/8/16 收敛)
375
376
377 def faceD_of(C, T, d_fac=1.0, useT=True, nsamp=NSAMP_FACE):
378     """界面扩散系数 (沿  $C$  的积分平均) —— **修正口径**。
379
380     
$$\langle D \rangle = \frac{1}{\Delta C} \int D \, dC, \quad D \text{ 对 } C \text{ 因子采样, } T \text{ 取界面均值。}$$

381     """
382     Ca = np.ascontiguousarray(np.asarray(C, dtype=np.float64))
383     Ta = np.ascontiguousarray(np.asarray(T, dtype=np.float64))
384     if _HAS_NUMBA:
385         return _faceD_nb(Ca, Ta, float(d_fac), bool(useT), int(nsamp))
386     lo, hi = np.minimum(Ca[:-1], Ca[1:]), np.maximum(Ca[:-1], Ca[1:])
387     xs = lo[:, None] + (hi - lo)[:, None] * (np.arange(nsamp) + 0.5)[None, :] / nsamp
388     xs = np.maximum(xs, 1e-9)
389     fv = np.exp(-0.45 / xs).mean(axis=1)
390     flat = (hi - lo) < 1e-14
391     if flat.any():
392         fv[flat] = np.exp(-0.45 / np.maximum(lo[flat], 1e-9))
393     if useT:
394         Tm = 0.5 * (Ta[:-1] + Ta[1:])
395         return d_fac * 2.4e-3 * fv * np.exp(-3850.0 / (Tm + 273.15))
396     return 2.4e-3 * fv
397

```

```

398
399 def facek_of(C):
400     """界面导热系数 (沿 $C$ 的积分平均, 解析式) ——**修正口径**。"""
401     Ca = np.ascontiguousarray(np.asarray(C, dtype=np.float64))
402     if _HAS_NUMBA:
403         return _facek_nb(Ca)
404     a, b = Ca[:-1], Ca[1:]
405     db = np.abs(b - a)
406     safe = np.where(db < 1e-14, 1.0, b - a)
407     gb = b - np.log(1.0 + b)
408     ga = a - np.log(1.0 + a)
409     out = 0.21 + 0.38 * (gb - ga) / safe
410     flat = db < 1e-14
411     if flat.any():
412         out[flat] = 0.21 + 0.38 * a[flat] / (1.0 + a[flat])
413     return out
414
415
416 # =====
417 # 三对角求解 (B 版: **保留 scipy.linalg.solve_banded**)
418 # =====
419 def thomas(a, b, c, d):
420     """解三对角系统 (a/b/c 为下/主/上对角, a[0] 与 c[-1] 不使用)
421
422     *C 版: 改为 **numba 手写追赶法** (原 `scipy.linalg.solve_banded` 仅作降级回退)。
423     调用签名**完全不变**  $\Rightarrow$  上层装配/Picard 逻辑一行未改。
424     """
425     if _HAS_NUMBA:
426         return _thomas_nb(a, b, c, d)
427     n = len(b)
428     ab = np.zeros((3, n), dtype=float)
429     ab[0, 1:] = c[:-1]
430     ab[1, :] = b
431     ab[2, :-1] = a[1:]
432     return solve_banded((1, 1), ab, d, check_finite=False, overwrite_ab=True)
433
434
435 # =====
436 # T 方程装配 (Java 侧包装: 负责缓冲分配, 内核原地填充)
437 # =====
438 def assemble_T_var(N, dr, dt, R0, h, rho, cp, k, out=None, kf=None):
439     """返回 (a, b, c)。rho/cp/k 为长度 N+1 的节点数组。
440
441     `out=(a,b,c)` 时**原地覆写**预分配缓冲; 否则新建。
442     `kf`: 可选的**预计算界面导热系数** (修正口径 `facek_of(C)`); None 时回退旧调和平均。
443     """
444     if out is None:
445         a = np.zeros(N + 1); b = np.zeros(N + 1); c = np.zeros(N + 1)
446     else:
447         a, b, c = out
448     if _HAS_NUMBA:
449         if kf is None:
450             kf = _iface_nb(np.ascontiguousarray(k, dtype=np.float64))
451             _assemble_T_nb(N, dr, dt, R0, h,
452                             np.ascontiguousarray(rho, dtype=np.float64),
453                             np.ascontiguousarray(cp, dtype=np.float64),
454                             np.ascontiguousarray(k, dtype=np.float64),
455                             a, b, c, kf)
456         return a, b, c
457     r = np.arange(N + 1) * dr
458     if kf is None:
459         kf = iface(k)
460     V0 = dr * dr / 4.0
461     b[0] = (rho[0] * cp[0] * V0 / dt + kf[0] * dr / dr) / V0
462     c[0] = (-kf[0] * dr / dr) / V0
463     a[0] = 0.0
464     ii = np.arange(1, N)
465     rl, rr = r[ii] - dr / 2.0, r[ii] + dr / 2.0
466     rc = rho[ii] * cp[ii]
467     b[ii] = rc * r[ii] / dt + (kf[ii] * rr + kf[ii - 1] * rl) / dr ** 2
468     a[ii] = -kf[ii - 1] * rl / dr ** 2
469     c[ii] = -kf[ii] * rr / dr ** 2
470     VN = dr * (R0 - dr / 4.0) / 2.0

```



```

471     b[N] = rho[N] * cp[N] * VN / dt + kf[N - 1] * (R0 - dr / 2.0) / dr + h * R0
472     a[N] = -kf[N - 1] * (R0 - dr / 2.0) / dr
473     c[N] = 0.0
474     return a, b, c
475
476
477 def rhs_T_var(N, dr, dt, R0, h, rho, cp, Told, Tinf, out=None):
478     """右端: 固定取上一时层解 Told"""
479     d = np.zeros(N + 1) if out is None else out
480     if _HAS_NUMBA:
481         _rhs_T_nb(N, dr, dt, R0, h,
482                 np.ascontiguousarray(rho, dtype=np.float64),
483                 np.ascontiguousarray(cp, dtype=np.float64),
484                 np.ascontiguousarray(Told, dtype=np.float64),
485                 float(Tinf), d)
486     return d
487     r = np.arange(N + 1) * dr
488     rcT = rho * cp * Told
489     d[0] = rcT[0] / dt
490     d[1:N] = r[1:N] * rcT[1:N] / dt
491     VN = dr * (R0 - dr / 4.0) / 2.0
492     d[N] = rcT[N] * VN / dt + h * R0 * Tinf
493     return d
494
495
496 # =====
497 # C 方程装配
498 # =====
499 def assemble_C_var(N, dr, dt, Df, R0, km, out=None):
500     """同 `assemble_T_var`: `out=(a,b,c)` 时原地覆写。"""
501     if out is None:
502         a = np.zeros(N + 1); b = np.zeros(N + 1); c = np.zeros(N + 1)
503     else:
504         a, b, c = out
505     if _HAS_NUMBA:
506         _assemble_C_nb(N, dr, dt,
507                        np.ascontiguousarray(Df, dtype=np.float64),
508                        R0, km, a, b, c)
509     return a, b, c
510     r = np.arange(N + 1) * dr
511     V0 = dr * dr / 4.0
512     b[0] = (V0 / dt + Df[0] * dr / dr) / V0
513     c[0] = (-Df[0] * dr / dr) / V0
514     a[0] = 0.0
515     ii = np.arange(1, N)
516     rl, rr = r[ii] - dr / 2.0, r[ii] + dr / 2.0
517     b[ii] = r[ii] / dt + (Df[ii] * rr + Df[ii - 1] * rl) / dr ** 2
518     a[ii] = -Df[ii - 1] * rl / dr ** 2
519     c[ii] = -Df[ii] * rr / dr ** 2
520     VN = dr * (R0 - dr / 4.0) / 2.0
521     b[N] = VN / dt + Df[N - 1] * (R0 - dr / 2.0) / dr + km * R0
522     a[N] = -Df[N - 1] * (R0 - dr / 2.0) / dr
523     c[N] = 0.0
524     return a, b, c
525
526
527 def rhs_C_var(N, dr, dt, R0, km, Cold, cinf, out=None):
528     d = np.zeros(N + 1) if out is None else out
529     if _HAS_NUMBA:
530         _rhs_C_nb(N, dr, dt, R0, km,
531                 np.ascontiguousarray(Cold, dtype=np.float64),
532                 float(cinf), d)
533     return d
534     r = np.arange(N + 1) * dr
535     d[0] = Cold[0] / dt
536     d[1:N] = r[1:N] * Cold[1:N] / dt
537     VN = dr * (R0 - dr / 4.0) / 2.0
538     d[N] = VN * Cold[N] / dt + km * R0 * cinf
539     return d
540
541
542 # =====
543 # 单时步: 双层迭代 (逻辑与 v2 **一行未改**)

```

```

544 # =====
545 def step_coupled(N, dr, dtT, dtC, subT, subC, R0, h, km,
546                 T, C, Tenv_fn, Cenv_fn, t0,
547                 omega=0.7, tol=1e-9, maxit=30, max_outer=20, tol_outer=1e-8,
548                 mode='coupled', lag_props=True, verbose=False):
549     """推进一个输出步。返回 (T_new, C_new, info)。
550
551     右端纪律：每轮外层的 T、C 求解**都从 n 层解出发**；C 的每个子步从**本子步起点**出发。
552     """
553     Told, Cold = T.copy(), C.copy()
554     Tk, Ck = T.copy(), C.copy()
555     res_hist = []
556     inner_tot = 0
557     res_T = res_C = None
558     flg = 'ok'
559     outer_it = 0
560     n_outer = 1 if lag_props else max_outer
561
562     if lag_props:
563         if mode == 'frozen':
564             rho0 = np.full(N + 1, float(rho_of(DEF2['C0'])))
565             cp0 = np.full(N + 1, float(cp_of(DEF2['C0'])))
566             k0 = np.full(N + 1, float(k_of(DEF2['C0'])))
567             D0 = np.full(N + 1, float(D_of(DEF2['C0'], DEF2['T0'])))
568         elif mode == 'decoupled':
569             rho0, cp0, k0 = rho_of(Cold), cp_of(Cold), k_of(Cold)
570             D0 = 2.4e-3 * np.exp(-0.45 / np.maximum(Cold, 1e-9))
571         else:
572             rho0, cp0, k0 = rho_of(Cold), cp_of(Cold), k_of(Cold)
573             D0 = D_of(Cold, Told)
574
575     for k in range(1, n_outer + 1):
576         outer_it = k
577         if lag_props:
578             rho, cp, kk = rho0, cp0, k0
579         elif mode == 'frozen':
580             rho = np.full(N + 1, float(rho_of(DEF2['C0'])))
581             cp = np.full(N + 1, float(cp_of(DEF2['C0'])))
582             kk = np.full(N + 1, float(k_of(DEF2['C0'])))
583         else:
584             rho, cp, kk = rho_of(Ck), cp_of(Ck), k_of(Ck)
585
586         Tprev = Tk.copy()
587         Tcur = Told.copy()
588         for s in range(subT):
589             ts = t0 + (s + 1) * dtT
590             kf = np.full(N, float(k_of(DEF2['C0']))) if mode == 'frozen' else facek_of(Ck)
591             a, b, c = assemble_T_var(N, dr, dtT, R0, h, rho, cp, kk, kf=kf)
592             d = rhs_T_var(N, dr, dtT, R0, h, rho, cp, Tcur, float(Tenv_fn(ts)))
593             Tcur = thomas(a, b, c, d)
594         Tk = Tcur
595
596         if lag_props:
597             Dnode = D0
598         elif mode == 'frozen':
599             Dnode = np.full(N + 1, float(D_of(DEF2['C0'], DEF2['T0'])))
600         elif mode == 'decoupled':
601             Dnode = 2.4e-3 * np.exp(-0.45 / np.maximum(Ck, 1e-9))
602         else:
603             Dnode = D_of(Ck, Tk)
604
605         Cit = Ck.copy()
606         for s in range(subC):
607             tt = t0 + (s + 1) * dtC
608             cinf = float(Cenv_fn(tt))
609             Cbase = Cit.copy()
610             base = rhs_C_var(N, dr, dtC, R0, km, Cbase, cinf)
611             for it in range(1, maxit + 1):
612                 if lag_props:
613                     if mode == 'frozen':
614                         Df = np.full(N, float(D_of(DEF2['C0'], DEF2['T0'])))
615                     elif mode == 'decoupled':
616                         Df = faceD_of(Cbase, Tk, 1.0, useT=False)

```

```

617         else:
618             Df = faceD_of(Cbase, Told, float(D_FAC), useT=True)
619         else:
620             if mode == 'frozen':
621                 Df = np.full(N, float(D_of(DEF2['C0'], DEF2['T0'])))
622             elif mode == 'decoupled':
623                 Df = faceD_of(Cit, Tk, 1.0, useT=False)
624             else:
625                 Df = faceD_of(Cit, Tk, float(D_FAC), useT=True)
626             a, b, c = assemble_C_var(N, dr, dtC, Df, R0, km)
627             Ctry = thomas(a, b, c, base)
628             Cnew = omega * Ctry + (1.0 - omega) * Cit
629             rr = float(np.max(np.abs(Cnew - Cit))) / max(1.0, float(np.max(np.abs(Cnew))))
630             Cit = Cnew
631             inner_tot += 1
632             if rr < tol:
633                 break
634         Ck = Cit
635
636         res_T = float(np.max(np.abs(Tk - Tprev)))
637         res_hist.append(res_T)
638         if k > 1:
639             res_C = float(np.max(np.abs(Ck - Cprev)))
640             if res_T < tol_outer and res_C < tol_outer:
641                 break
642         Cprev = Ck.copy()
643
644     if outer_it >= max_outer:
645         flg = 'outer_max'
646     if verbose:
647         print("    outer res_T hist:", " ".join(f"{v:.2e}" for v in res_hist[:8]),
648               "...", f"{res_hist[-1]:.2e}" if len(res_hist) > 8 else "")
649     return Tk, Ck, dict(outer_it=outer_it, inner_tot=inner_tot, res_T=res_T,
650                        res_C=res_C, flag=flg, res_hist=res_hist)
651
652 # =====
653 # 单时步: IMEX 交替推进 (主力格式) ★B 优化: 装配缓冲预分配并原地复用
654 # =====
655 def step_imex(N, dr, h, n_sub, R0, hc, km, T, C, Tenv_fn, Cenv_fn, t0,
656              omega=0.7, tol=1e-10, maxit=30, mode='coupled'):
657     """把**一个输出步**细分为 n_sub 个内部步 (内部步长 h = dt_out/n_sub),
658     每个内部步内**更新物性**并交替推进 T、C:
659
660     ① 物性 ← 当前 T, C (本内部子步起点)
661     ② 解 T (1 步 h, 物性冻结 ⇒ 线性三对角)
662     ③ D ← 新的 T 与当前 C
663     ④ 解 C (Picard 线性化, 1 步 h)
664
665     ★B 优化仅涉及**缓冲复用** (a/b/c/d 每输出步建一次),
666     迭代次数、判据、松弛、物性更新时机**全部不变**."""
667     Tcur, Ccur = T.copy(), C.copy()
668     inner_tot = 0
669     res_max = 0.0
670     # 预分配缓冲 (每输出步一次)
671     bufT = (np.zeros(N + 1), np.zeros(N + 1), np.zeros(N + 1))
672     bufC = (np.zeros(N + 1), np.zeros(N + 1), np.zeros(N + 1))
673     dT = np.zeros(N + 1)
674     dC = np.zeros(N + 1)
675     for s in range(n_sub):
676         ts = t0 + (s + 1) * h
677         tinf = float(Tenv_fn(ts))
678         cinf = float(Cenv_fn(ts))
679
680         # ① 物性
681         if mode == 'frozen':
682             rho = np.full(N + 1, float(rho_of(DEF2['C0'])))
683             cp = np.full(N + 1, float(cp_of(DEF2['C0'])))
684             kk = np.full(N + 1, float(k_of(DEF2['C0'])))
685         else:
686             rho, cp, kk = rho_of(Ccur), cp_of(Ccur), k_of(Ccur)
687

```

```

690 # ② T (全隐式, 物性冻结): 界面 k 取沿 C 的积分平均 (修正口径)
691 kf = np.full(N, float(k_of(DEF2['C0']))) if mode == 'frozen' else facek_of(Ccur)
692 a, b, c = assemble_T_var(N, dr, h, R0, hc, rho, cp, kk, out=bufT, kf=kf)
693 d = rhs_T_var(N, dr, h, R0, hc, rho, cp, Tcur, tinf, out=dT)
694 Tcur = thomas(a, b, c, d)
695
696 # ③ D 基准 (仅 frozen 需常数基准)
697 if mode == 'frozen':
698     Dbase = np.full(N + 1, float(D_of(DEF2['C0'], DEF2['T0'])))
699
700 # ④ C (Picard 线性化: D 随迭代中的 C 更新, T 固定): 界面 D 取沿 C 的积分平均 (修正口径)
701 base = rhs_C_var(N, dr, h, R0, km, Ccur, cinf) # 每子步更新  $\Rightarrow$  不复用缓冲
702 Cit = Ccur.copy()
703 for it in range(1, maxit + 1):
704     if mode == 'frozen':
705         Df = np.full(N, Dbase[0])
706     elif mode == 'decoupled':
707         Df = faceD_of(Cit, Tcur, 1.0, useT=False)
708     else:
709         Df = faceD_of(Cit, Tcur, float(D_FAC), useT=True)
710     a, b, c = assemble_C_var(N, dr, h, Df, R0, km, out=bufC)
711     Ctry = thomas(a, b, c, base)
712     Cnew = omega * Ctry + (1.0 - omega) * Cit
713     rr = float(np.max(np.abs(Cnew - Cit))) / max(1.0, float(np.max(np.abs(Cnew))))
714     Cit = Cnew
715     inner_tot += 1
716     res_max = max(res_max, rr)
717     if rr < tol:
718         break
719 Ccur = Cit
720
721 return Tcur, Ccur, dict(inner_tot=inner_tot, nsub=n_sub,
722                          res_max=res_max, outer_it=1, res_T=None, res_C=None,
723                          flag='ok', res_hist=[])
724
725 # =====
726 # 驱动 (逻辑与 v2 **一行未改**)
727 # =====
728 def run_q2(N=80, dr=2.5e-4, dt_out=1.0, nsteps=10800,
729           Tenv_fn=None, Cenv_fn=None, subT=32, subC=10, n_sub=None,
730           omega=0.7, tol=1e-10, maxit=30, max_outer=20, tol_outer=1e-8,
731           mode='coupled', scheme='imex', lag_props=True,
732           cols=None, snap_at=(), progress=0, verbose=False):
733     """Q2 主力求解 (单遍运行, 同时收集整表与快照)。返回 (res, tableT, tableC)。"""
734     R0, h, km = DEF2['R0'], DEF2['h'], DEF2['km']
735     dtT, dtC = dt_out / float(subT), dt_out / float(subC)
736     nsub = n_sub if n_sub is not None else subT
737     hh = dt_out / float(nsub)
738     ncol = 0 if cols is None else len(cols)
739     tableT = np.zeros((nsteps, ncol)) if (cols is not None) else None
740     tableC = np.zeros((nsteps, ncol)) if (cols is not None) else None
741
742     T = np.full(N + 1, DEF2['T0'])
743     C = np.full(N + 1, DEF2['C0'])
744     Tsnap, Csnap = {0: T.copy()}, {0: C.copy()}
745
746     stat = dict(outer_tot=0, outer_max=0, inner_tot=0, nsub=0,
747                resT_max=0.0, resC_max=0.0, flags={}, res_hist_last=None)
748     t0 = time.time()
749     for n in range(1, nsteps + 1):
750         if scheme == 'imex':
751             T, C, info = step_imex(N, dr, hh, nsub, R0, h, km, T, C,
752                                   Tenv_fn, Cenv_fn, (n - 1) * dt_out,
753                                   omega, tol, maxit, mode)
754             stat['resC_max'] = max(stat['resC_max'], info.get('res_max', 0.0))
755         else:
756             T, C, info = step_coupled(N, dr, dtT, dtC, subT, subC, R0, h, km,
757                                       T, C, Tenv_fn, Cenv_fn, (n - 1) * dt_out,
758                                       omega, tol, maxit, max_outer, tol_outer,
759                                       mode, lag_props=lag_props,
760                                       verbose=(verbose and n <= 2))
761             stat['resT_max'] = max(stat['resT_max'], info['res_T'] or 0.0)

```

```

763     stat['resC_max'] = max(stat['resC_max'], info['res_C'] or 0.0)
764     stat['outer_tot'] += info.get('outer_it', 1)
765     stat['outer_max'] = max(stat['outer_max'], info.get('outer_it', 1))
766     stat['inner_tot'] += info['inner_tot']
767     stat['nsub'] += 1
768     stat['res_hist_last'] = info.get('res_hist')
769     if info.get('flag', 'ok') != 'ok':
770         stat['flags'][info['flag']] = stat['flags'].get(info['flag'], 0) + 1
771     if cols is not None:
772         tableT[n - 1, :] = T[cols]
773         tableC[n - 1, :] = C[cols]
774     if n in snap_at:
775         Tsnap[n], Csnap[n] = T.copy(), C.copy()
776     if progress and n % progress == 0:
777         el = time.time() - t0
778         print(f"    ... t={n*dt_out:.0f}s 已用 {el:.1f}s "
779               f"预计剩余 {el/n*(nsteps-n):.1f}s", flush=True)
780
781     res = dict(N=N, dr=dr, dt_out=dt_out, dtT=hh, dtC=hh, subT=subT, subC=subC,
782               n_sub=nsub, scheme=scheme, mode=mode,
783               T_end=T.copy(), C_end=C.copy(),
784               T_snap=Tsnap, C_snap=Csnap, stats=stat, wall=time.time() - t0)
785     return res, tableT, tableC

```

B.3 公共模块：时间推进

```

1  # -*- coding: utf-8 -*-
2  """A 题 Q1 · 时间方向**精确推进**模块（创新化路径 T3）
3  =====
4  思路（Q1 独有红利：T 方程为**线性常系数**、C 方程非线性**仅**来自 D(C)）
5
6  半离散后：  $d\psi/dt = A\psi + \kappa \cdot g(t) \cdot e_N$  ,  $g(t)$ =环境量（附件1 分段线性）
7  解析解：  $\psi_{n+1} = E\psi_n + \int_0^h e^{A(h-s)} b(t_n+s) ds$  ,  $E = e^{Ah}$ 
8  b 分段线性  $\Rightarrow$  积分**闭式**：  $\int = g_n \cdot w_0 + (g_{n+1} - g_n) \cdot w_1$ 
9   $w_0 = \kappa \cdot G_0 e_N$  ,  $w_1 = \kappa \cdot G_1 e_N$ 
10  $G_0 = A^{-1}\{E-I\}$  ,  $G_1 = A^{-2}\{E-I-Ah\}$ 
11
12 实现用**增广矩阵**一次算齐（避免求逆）：
13  $\expm([A, e_N, e_N], [0, 0, 0], [0, 0, 0]) \cdot h$  的右上两块  $\Rightarrow G_0 e_N$  与  $h \cdot G_1 e_N$ 
14
15  $\Rightarrow$  时间方向**零离散误差**（仅剩空间离散）；h 可在数据节点步长内任意取（1 s 或 60 s）。
16 性质：A 为非奇异 M 矩阵、非对角元  $\geq 0$ 、 $A \cdot 1 = -\kappa e_N \leq 0 \Rightarrow E \geq 0$  且  $E \cdot 1 \leq 1$ （次随机）
17  $\Rightarrow$  当边界值非负时，解为初值与边界值的凸组合（离散极值原理在**连续时间层面**成立）。
18
19 C 方程（T3-2）：每个子步内**冻结 D**（界面仍取口径 M6「沿 C 的积分平均」），
20 则该子步同为零性常系数问题，用同一机制精确推进，再做少量 Picard 校正。
21
22 约定：本文件不引用、不记载任何资料中的日期。
23 """
24 import numpy as np
25 from scipy.linalg import expm
26
27 import q1_core as core
28
29
30 # ----- 装配（ODE 形式）
31 def assemble_T_ode(N, dr, p, center='fv', surf='fvm'):
32     """半离散 T 方程  $dT/dt = A T + \kappa \cdot T_\infty(t) \cdot e_N$ ：返回 (A,  $\kappa$ )。
33     与 `q1_core.assemble_T` 的后向欧拉格式**同源**（同一元体平衡算子）。"""
34     rho, cp = p['rho'], p['cp']
35     k = p['k'] * p.get('kfacs', 1.0)
36     h, R0 = p['h'], p['R0']
37     r = np.arange(N + 1) * dr
38     A = np.zeros((N + 1, N + 1))
39     # 中心：元体平衡几何式与洛必达极限式给出**同一**算子  $4k/(pc_p \Delta r^2)$ 
40     A[0, 0] = -4.0 * k / (rho * cp * dr ** 2)
41     A[0, 1] = +4.0 * k / (rho * cp * dr ** 2)
42     for i in range(1, N):
43         rl, rr = r[i] - dr / 2.0, r[i] + dr / 2.0
44         den = rho * cp * r[i] * dr ** 2
45         A[i, i - 1] = k * rl / den
46         A[i, i] = -k * (rl + rr) / den
47         A[i, i + 1] = k * rr / den

```

```

48     if surf == 'fvm':
49         VN = dr * (R0 - dr / 4.0) / 2.0
50         rad = R0 - dr / 2.0
51     else:
52         VN = R0 * dr / 2.0
53         rad = R0
54     A[N, N - 1] = k * rad / (rho * cp * VN * dr)
55     A[N, N] = -(k * rad / dr + h * R0) / (rho * cp * VN)
56     return A, h * R0 / (rho * cp * VN)
57
58
59 def assemble_C_ode(N, dr, Df, R0, km):
60     """半离散 C 方程  $dC/dt = A_C C + \eta \cdot C\infty(t) \cdot e_N$  (冻结界面系数 Df)；返回 (A,  $\eta$ )。"""
61     r = np.arange(N + 1) * dr
62     A = np.zeros((N + 1, N + 1))
63     V0, A12 = dr * dr / 4.0, dr
64     A[0, 0] = -(Df[0] * A12 / dr) / V0
65     A[0, 1] = +(Df[0] * A12 / dr) / V0
66     for i in range(1, N):
67         rl, rr = r[i] - dr / 2.0, r[i] + dr / 2.0
68         den = r[i] * dr ** 2
69         A[i, i - 1] = Df[i - 1] * rl / den
70         A[i, i] = -(Df[i] * rr + Df[i - 1] * rl) / den
71         A[i, i + 1] = Df[i] * rr / den
72     VN = dr * (R0 - dr / 4.0) / 2.0
73     rad = R0 - dr / 2.0
74     A[N, N - 1] = Df[N - 1] * rad / (VN * dr)
75     A[N, N] = -(Df[N - 1] * rad / dr + km * R0) / VN
76     return A, km * R0 / VN
77
78
79 def etd_prop(A, kappa, h):
80     """返回 (E, w0, w1) 使  $\psi_{n+1} = E \psi_n + g_n \cdot w0 + (g_{n+1} - g_n) \cdot w1$  ( $b = k \cdot g \cdot e_N$ )。
81
82     增广矩阵须带**线性项的 Jordan 链** (右下  $[[0, 1], [0, 0]]$ ) :
83      $M = \begin{bmatrix} A & e_N & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$  ( $n = N+1$  个节点,  $e_N$  位于第  $n-1$  行)
84      $\Rightarrow \expm(M \cdot h)$  的右上两块给出
85      $X[:n, n] = \int_0^h e^{A(h-s)} ds \cdot e_N = G0 \cdot e_N$ 
86      $X[:n, n+1] = \int_0^h e^{A(h-s)} s ds \cdot e_N = h \cdot G1 \cdot e_N$ 
87     (若把第三列也填  $e_N$ , 则两块**都**退化为  $G0 \cdot e_N \Rightarrow$  格式不再是精确积分——已修正。)
88     """
89     n = A.shape[0]
90     M = np.zeros((n + 2, n + 2))
91     M[:n, :n] = A
92     M[n - 1, n] = 1.0 # u =  $e_N$  (最后一个节点)
93     M[n, n + 1] = 1.0 # Jordan 链  $\Rightarrow$  线性项
94     X = expm(M * h)
95     return X[:n, :n], kappa * X[:n, n], kappa * X[:n, n + 1] / h
96
97
98 # ----- T 精确推进
99 def run_T_expnm(N, dr, p, Tenv_fn, t_end, hstep=1.0, center='fv', surf='fvm',
100                t_snaps=(), knot_dt=60.0, cols=None, dt_out=None):
101     """T 方程时间精确推进。T $\infty$  为附件1 分段线性  $\Rightarrow$  **无时间离散误差**。
102
103     **关键**：步长必须在「数据节点 (knot_dt 的倍数)」与「快照时刻」处**切分**——
104     否则一步跨越节点会横跨两段线性区 (函数有折点)  $\Rightarrow$  闭式积分不再成立、h 相关性出现。
105     cols 非 None 时按 dt_out 逐行收集整表 (形状 (n, len(cols)))。
106     返回 dict: T_snap (按 t_snaps)、table、E、hstep、nstep、nprop。
107     """
108     n = N + 1
109     A, kappa = assemble_T_ode(N, dr, p, center, surf)
110     E, w0, w1 = etd_prop(A, kappa, hstep)
111     T = np.full(n, p['T0'])
112     snaps = {0.0: T.copy()}
113     table = None
114     if cols is not None:
115         dt_out = 1.0 if dt_out is None else float(dt_out)
116         nrow = int(round(t_end / dt_out))
117         table = np.zeros((nrow, len(cols)))
118     cuts = set()
119     for step in (knot_dt, hstep, dt_out):
120         if not step:

```

```

121         continue
122     k = 0
123     while k * step < t_end - 1e-12:
124         cuts.add(round(k * step, 9))
125         k += 1
126     for x in t_snaps:
127         cuts.add(round(float(x), 9))
128     cuts.add(round(float(t_end), 9))
129     grid = sorted(x for x in cuts if x <= t_end + 1e-12)
130     snapset = set(round(float(x), 9) for x in t_snaps) | {round(float(t_end), 9)}
131     # 传播子 (E, w0, w1) **依赖步长** ==> 按实际步长取, 并对同长度步长缓存复用
132     pcache = {round(hstep, 9): (E, w0, w1)}
133     for a, b in zip(grid[:-1], grid[1:]):
134         hh = round(b - a, 9)
135         if hh not in pcache:
136             pcache[hh] = etd_prop(A, kappa, hh)
137         Ee, ww0, ww1 = pcache[hh]
138         ga, gb = float(Tenv_fn(a)), float(Tenv_fn(b))
139         T = Ee @ T + ga * ww0 + (gb - ga) * ww1
140         if round(b, 9) in snapset:
141             snaps[round(b, 9)] = T.copy()
142         if table is not None:
143             nr = int(round(b / dt_out))
144             if abs(b - nr * dt_out) < 1e-9 and 1 <= nr <= table.shape[0]:
145                 table[nr - 1, :] = T[cols]
146     return dict(T_end=T, T_snap=snaps, table=table, A=A, kappa=kappa,
147               E=pcache[round(hstep, 9)][0], nstep=len(grid) - 1,
148               nprop=len(pcache), hstep=hstep)
149
150
151 # ----- C 精确推进
152 def run_C_etd(N, dr, p, Cenv_fn, t_end, h_step=1.0, dt_col=1.0, maxit=30,
153              Dfac=1.0, tol=1e-9, snap_at=(), cols=None, T0_C=None, knot_dt=60.0):
154     """C 方程 ETD: 每子步冻结 D (界面口径 M6) -> 精确推进 -> Picard 迭代至收敛。
155
156     子步同样在**数据节点**与**快照时刻**处切分 (保证每步内  $C_\infty$  线性)。
157     收敛判据与主力**同口径** (相对残差 < tol, 上限 maxit) ——但残差按**相邻两次迭代**之差计算。
158     """
159     n = N + 1
160     C = np.full(n, p['C0'] if T0_C is None else T0_C)
161     snaps = {0.0: C.copy()}
162     rows = [] if cols is not None else None
163     tot_it, mx_res, nsub = 0, 0.0, 0
164     cuts = set()
165     k = 0
166     while k * knot_dt < t_end - 1e-12:
167         cuts.add(round(k * knot_dt, 9)); k += 1
168     k = 0
169     while k * h_step < t_end - 1e-12:
170         cuts.add(round(k * h_step, 9)); k += 1
171     for x in snap_at:
172         cuts.add(round(float(x), 9))
173     grid = sorted(x for x in cuts if x <= t_end + 1e-12)
174     if grid[0] != 0.0:
175         grid.insert(0, 0.0)
176     snapset = set(round(float(x), 9) for x in snap_at)
177     for gi in range(len(grid) - 1):
178         ts, nxt = grid[gi], grid[gi + 1]
179         ca, cb = float(Cenv_fn(ts)), float(Cenv_fn(nxt))
180         hh = nxt - ts
181         Ccur = C.copy()
182         for it in range(1, maxit + 1):
183             Df = core.faceD_int(Ccur, p['D0'], p['bD'], Dfac)
184             A, eta = assemble_C_code(N, dr, Df, p['R0'], p['km'])
185             E, w0, w1 = etd_prop(A, eta, hh)
186             Cnew = E @ C + ca * w0 + (cb - ca) * w1
187             # 真正的 Picard 残差: **相邻两次迭代**之差 (非"本步变化量")
188             res = float(np.max(np.abs(Cnew - Ccur))) / max(1.0, float(np.max(np.abs(Cnew))))
189             Ccur = Cnew
190             tot_it += 1
191             mx_res = max(mx_res, res)
192             if res < tol:
193                 break

```

```

194     C = Ccur
195     nsub += 1
196     ts = nxt
197     if snap_at and any(abs(ts - s) < 1e-9 for s in snap_at):
198         snaps[round(ts, 9)] = C.copy()
199     if rows is not None:
200         nc = int(round(ts / dt_col))
201         if abs(ts - nc * dt_col) < 1e-9:
202             rows.append((nc, C[cols].copy()))
203     return dict(C_end=C, C_snap=snaps, rows=rows,
204               stats=dict(total_it=tot_it, avg_it=tot_it / max(1, nsub),
205                        max_res=mx_res, nsub=nsub))

```

```

1  # -*- coding: utf-8 -*-
2  """
3  A 题 Q1 求解核（配置化）—— 主力求解与全部检验实验共用
4  =====
5  统一口径
6  -----
7  • 空间      : 元体平衡（有限体积）+ 三对角追赶法
8  • 时间      : **温度=时间方向精确推进**（矩阵指数+分段线性 Duhamel, 见 `q1_exact.py`；
9              本核经 `tmethod='expm'` 调用, **无时间离散误差**）
10             其余（Q1 含水率、Q2-Q4 全部场量）=全隐式（后向欧拉）
11  • 中心      : 两条**真正独立**的装配路径，经同一尺度归一化后逐系数比对
12               center='fv' —— 用中心控制体体积  $V_0=\Delta r^2/4$  与界面面积  $A_{12}=\Delta r$  显式装配
13               center='lh' —— 洛必达极限  $(1/r)\partial r(r\partial rT) \rightarrow 2\partial r^2T$  的差分离散
14  • 表面      : surf='fvm' —— 元体平衡半格（主力）
15               surf='ghost' —— **一阶近似口径**（体积取  $R_0 \cdot \Delta r/2$ 、导热界面取  $R_0$ ）；
16                       **仅**用于边界口径对照实验。 $\Delta$  命名沿用历史，
17                       **其实现并非“虚拟节点+中心差分”**（原虚拟节点装配符号有误、已改）
18  • 非线性    : D(C) 单因子；Picard 迭代+欠松弛；时间导数右端**固定取上一时层解**
19  • 解耦      : 先全程解 T，再全程解 C（T 方程不含 C，D 不含 T）
20 退化与灵敏度
21  -----
22  • kfac / Dfac: 物性缩放因子（0.0 → 退化自检；1±0.2 → 灵敏度 OAT）
23  • 环境条件以函数传入（常数环境用于 E2 解析对拍与退化自检）
24
25  约定：本核不引用、不记载任何资料中的日期。
26  """
27  import time
28  import numpy as np
29
30  DEF = dict(R0=0.02, rho=820.0, cp=2600.0, k=0.36, h=25.0, km=8.0e-7,
31            D0=7.0e-9, bD=0.89, T0=28.0, C0=2.55)
32
33  V0_OF = staticmethod(lambda dr: dr * dr / 4.0) # 中心控制体体积（ $\pi$  已约去）
34  VN_OF = staticmethod(lambda dr, R0: dr * (R0 - dr / 4.0) / 2.0) # 表面控制体体积
35
36
37  def thomas(a, b, c, d):
38      """三对角追赶法（a/b/c 为下/主/上对角，a[0] 与 c[-1] 不使用）"""
39      n = len(b)
40      cp_ = np.empty(n - 1, dtype=float)
41      dp_ = np.empty(n, dtype=float)
42      cp_[0] = c[0] / b[0]
43      dp_[0] = d[0] / b[0]
44      for i in range(1, n):
45          m = b[i] - a[i] * cp_[i - 1]
46          dp_[i] = (d[i] - a[i] * dp_[i - 1]) / m
47          if i < n - 1:
48              cp_[i] = c[i] / m
49      x = np.empty(n, dtype=float)
50      x[-1] = dp_[-1]
51      for i in range(n - 2, -1, -1):
52          x[i] = dp_[i] - cp_[i] * x[i + 1]
53      return x
54
55
56  def thomas_prep(a, b, c):
57      """常数系数三对角矩阵的预消元（前向消元结果与 d 无关，可复用）"""
58      n = len(b)
59      cp_ = np.empty(n - 1, dtype=float)
60      inv = np.empty(n, dtype=float)

```



```

61     cp_[0] = c[0] / b[0]
62     inv[0] = 1.0 / b[0]
63     for i in range(1, n):
64         m = b[i] - a[i] * cp_[i - 1]
65         inv[i] = 1.0 / m
66         if i < n - 1:
67             cp_[i] = c[i] / m
68     return cp_, inv
69
70
71 def thomas_solve_prep(a, cp_, inv, d):
72     """配合 thomas_prep 使用的快速求解（与 thomas 同式，仅省去重复除法）"""
73     n = len(d)
74     dp_ = np.empty(n, dtype=float)
75     dp_[0] = d[0] * inv[0]
76     for i in range(1, n):
77         dp_[i] = (d[i] - a[i] * dp_[i - 1]) * inv[i]
78     x = np.empty(n, dtype=float)
79     x[-1] = dp_[-1]
80     for i in range(n - 2, -1, -1):
81         x[i] = dp_[i] - cp_[i] * x[i + 1]
82     return x
83
84
85 def D_of(C, D0=DEF['D0'], bD=DEF['bD']):
86     """有效水分扩散系数  $D(C) = D0 \cdot \exp(-bD/C)$ """
87     return D0 * np.exp(-bD / np.maximum(np.asarray(C, dtype=float), 1e-9))
88
89
90 def harm(dl, dr_):
91     """界面系数：距离加权调和平均（ $D=0$  时退化为 0，避免除零）。
92
93     △ **本函数已不用于生产路径**：C 方程的界面扩散系数改由 `faceD_int`
94     （**沿  $\$C\$$  的积分平均**，口径 **M6**）生成；本函数**仅保留作旧口径对照**。
95     """
96     if dl <= 0.0 or dr_ <= 0.0:
97         return 0.0
98     return 2.0 / (1.0 / dl + 1.0 / dr_)
99
100
101 # ----- T 方程
102 def assemble_T(N, dr, dt, p, center='fv', surf='fvm'):
103     """装配后向欧拉 T 方程系数 (a, b, c, meta)；右端另由 rhs_T 生成"""
104     rho, cp = p['rho'], p['cp']
105     k = p['k'] * p.get('kfacs', 1.0)
106     h, R0 = p['h'], p['R0']
107     r = np.arange(N + 1) * dr
108     a = np.zeros(N + 1); b = np.zeros(N + 1); c = np.zeros(N + 1)
109     meta = {}
110
111     # --- 中心节点：两条独立路径，均归一化到"单位体积"以便逐系数比对 ---
112     if center == 'fv':
113         V0 = dr * dr / 4.0
114         A12 = dr
115         b[0] = (rho * cp * V0 / dt + k * A12 / dr) / V0
116         c[0] = (-k * A12 / dr) / V0
117         meta['center_scale'] = V0
118     elif center == 'lh':
119         d2 = 2.0 / dr ** 2
120         b[0] = rho * cp / dt + 2.0 * k * d2
121         c[0] = -2.0 * k * d2
122         meta['center_scale'] = 1.0
123     else:
124         raise ValueError(center)
125     meta['center_row'] = (float(b[0]), float(c[0]))
126
127     # --- 内部节点：标准元体平衡（含  $r_{i\pm 1/2}$  几何权重） ---
128     for i in range(1, N):
129         rl, rr = r[i] - dr / 2.0, r[i] + dr / 2.0
130         b[i] = rho * cp * r[i] / dt + k * (rr + rl) / dr ** 2
131         a[i] = -k * rl / dr ** 2
132         c[i] = -k * rr / dr ** 2
133

```

```

134 # --- 表面节点 ---
135 if surf == 'fvm':
136     VN = dr * (R0 - dr / 4.0) / 2.0
137     b[N] = rho * cp * VN / dt + k * (R0 - dr / 2.0) / dr + h * R0
138     a[N] = -k * (R0 - dr / 2.0) / dr
139     meta['surf'] = 'fvm'
140 elif surf == 'ghost':
141     # 对照口径 (**仅用于边界离散敏感性对照**):
142     # 表面控制体体积取一阶近似  $R0 \cdot \Delta r / 2$  (忽略曲率), 导热界面半径取 R0
143     VNa = R0 * dr / 2.0
144     b[N] = rho * cp * VNa / dt + k * R0 / dr + h * R0
145     a[N] = -k * R0 / dr
146     meta['surf'] = 'approx'
147 else:
148     raise ValueError(surf)
149 return a, b, c, meta
150
151
152 def rhs_T(N, dr, dt, p, Told, Tinf, surf='fvm'):
153     """生成 T 方程右端向量 d"""
154     rho, cp = p['rho'], p['cp']
155     h, R0 = p['h'], p['R0']
156     r = np.arange(N + 1) * dr
157     d = np.zeros(N + 1)
158     d[0] = rho * cp * Told[0] / dt
159     for i in range(1, N):
160         d[i] = rho * cp * r[i] * Told[i] / dt
161     if surf == 'fvm':
162         VN = dr * (R0 - dr / 4.0) / 2.0
163         d[N] = rho * cp * VN * Told[N] / dt + h * R0 * Tinf
164     else:
165         VNa = R0 * dr / 2.0
166         d[N] = rho * cp * VNa * Told[N] / dt + h * R0 * Tinf
167     return d
168
169
170 # ----- C 方程
171 def assemble_C(N, dr, dt, Df, R0, km):
172     """装配 C 方程系数 (a, b, c, VN); Df 为界面扩散系数 (长度 N, **沿 $C$ 的积分平均**, 由 `faceD_int`
    生成)"""
173     r = np.arange(N + 1) * dr
174     a = np.zeros(N + 1); b = np.zeros(N + 1); c = np.zeros(N + 1)
175     V0 = dr * dr / 4.0
176     A12 = dr
177     b[0] = (V0 / dt + Df[0] * A12 / dr) / V0
178     c[0] = (-Df[0] * A12 / dr) / V0
179     for i in range(1, N):
180         rl, rr = r[i] - dr / 2.0, r[i] + dr / 2.0
181         b[i] = r[i] / dt + (Df[i] * rr + Df[i - 1] * rl) / dr ** 2
182         a[i] = -Df[i - 1] * rl / dr ** 2
183         c[i] = -Df[i] * rr / dr ** 2
184     VN = dr * (R0 - dr / 4.0) / 2.0
185     b[N] = VN / dt + Df[N - 1] * (R0 - dr / 2.0) / dr + km * R0
186     a[N] = -Df[N - 1] * (R0 - dr / 2.0) / dr
187     return a, b, c, VN
188
189
190 def rhs_C(N, dr, dt, Cold, cinf, R0, km):
191     """生成 C 方程右端向量 d (**固定取上一时层解 Cold**)"""
192     r = np.arange(N + 1) * dr
193     d = np.zeros(N + 1)
194     d[0] = Cold[0] / dt
195     for i in range(1, N):
196         d[i] = r[i] * Cold[i] / dt
197     VN = dr * (R0 - dr / 4.0) / 2.0
198     d[N] = VN * Cold[N] / dt + km * R0 * cinf
199     return d
200
201
202 def faceD_int(Carr, D0, bD, Dfac=1.0, nsamp=8):
203     """*修正口径: 界面扩散系数取**沿 C 的积分平均**  $\langle D \rangle = (1/\Delta C) \int D \, dC$ 。
204     (原 `harm` 的调和平均仅对"界面处系数间断"成立; 系数随 C 陡变时显著失真。)
205

```

```

206     """
207     C = np.asarray(Carr, dtype=float)
208     lo = np.minimum(C[:-1], C[1:])
209     hi = np.maximum(C[:-1], C[1:])
210     xs = lo[:, None] + (hi - lo)[:, None] * (np.arange(nsamp) + 0.5)[None, :] / nsamp
211     xs = np.maximum(xs, 1e-9)
212     fv = np.exp(-bD / xs).mean(axis=1)
213     flat = (hi - lo) < 1e-14
214     if flat.any():
215         fv[flat] = np.exp(-bD / np.maximum(lo[flat], 1e-9))
216     return D0 * Dfac * fv
217
218
219 def step_C(N, dr, dt, C, cinf, p, omega=0.7, tol=1e-9, maxit=30, Dfac=1.0,
220           dmode='func'):
221     """推进一个 C 子步: Picard 迭代+欠松弛。返回 (C_new, iters, res)
222
223     dmode='func': D = D(C) 单因子非线性 (主力)
224     dmode='const': D = D(C0) (用于 E2 解析对拍; 此时单次迭代即收敛)
225     """
226     Cold = C.copy()
227     Cit = C.copy()
228     res = None
229     it = 0
230     Dconst = p['D0'] * np.exp(-p['bD'] / p['C0']) * Dfac
231     for it in range(1, maxit + 1):
232         if dmode == 'const':
233             Df = np.full(N, Dconst)
234         else:
235             Df = faceD_int(Cit, p['D0'], p['bD'], Dfac)
236         a, b, c, _ = assemble_C(N, dr, dt, Df, p['R0'], p['km'])
237         d = rhs_C(N, dr, dt, Cold, cinf, p['R0'], p['km'])
238         Ctry = thomas(a, b, c, d)
239         Cnew = omega * Ctry + (1.0 - omega) * Cit
240         res = float(np.max(np.abs(Cnew - Cit))) / max(1.0, float(np.max(np.abs(Cnew))))
241         Cit = Cnew
242         if res < tol:
243             break
244     return Cit, it, res
245
246
247 # ----- 驱动
248 def run_sim(N, dr, dt_out, nsteps, p, Tenv_fn, Cenv_fn, sub=10, subT=1,
249            center='fv', surf='fvm', kfac=1.0, Dfac=1.0, dmode='func',
250            tol=1e-9, omega=0.7, maxit=30, snap_at=(), out_idx=None,
251            solve_T=True, solve_C=True, tmethod='expm'):
252     """统一求解驱动。
253
254     参数
255     ----
256     N, dr      : 空间区间数与步长 (节点数 N+1)
257     dt_out     : 输出时间步长 (s)
258     nsteps     : 输出步数 (总时长 = nsteps * dt_out)
259     p          : 物性/几何参数字典 (见 DEF)
260     Tenv_fn    :  $T_\infty(t)$  函数; Cenv_fn:  $C_\infty(t)$  函数
261     sub        : C 方程内部子步数 (内部步长 = dt_out/sub)
262     subT       : T 方程内部子步数 (内部步长 = dt_out/subT, 用于时间收敛实验)
263     out_idx    : 需要收集的列索引 (None → 不收集整表)
264     返回 dict
265     """
266     p = dict(p)
267     p['kfac'] = kfac
268     R0 = p['R0']
269     r = np.arange(N + 1) * dr
270     dt_int = dt_out / float(sub)
271     dtT = dt_out / float(subT)
272
273     res = dict(N=N, dr=dr, dt_out=dt_out, dt_int=dt_int, sub=sub,
274               center=center, surf=surf, kfac=kfac, Dfac=Dfac)
275
276     # ----- T -----
277     Tend = None
278     Tsnap = {}

```

```

279 res['tmethod'] = tmethod
280 if solve_T:
281     a, b, c, meta = assemble_T(N, dr, dtT, p, center, surf)
282     res['center_row'] = meta['center_row']
283     t0 = time.time()
284     if tmethod == 'expm':
285         # 时间方向**精确推进** (交付口径): 矩阵指数 + 分段线性 Duhamel
286         import q1_exact as _ex
287         rT = _ex.run_T_expm(N, dr, p, Tenv_fn, nsteps * dt_out,
288                             hstep=dt_out, center=center, surf=surf,
289                             t_snaps=[n * dt_out for n in snap_at])
290         Tsnap[0] = np.full(N + 1, p['T0'])
291         for n in snap_at:
292             Tsnap[n] = rT['T_snap'][round(float(n * dt_out), 9)].copy()
293         T = rT['T_end']
294     else:
295         cp_, inv = thomas_prep(a, b, c)
296         T = np.full(N + 1, p['T0'])
297         Tsnap[0] = T.copy()
298         for n in range(1, nsteps + 1):
299             for s in range(subT):
300                 ts = (n - 1) * dt_out + (s + 1) * dtT
301                 d = rhs_T(N, dr, dtT, p, T, float(Tenv_fn(ts)), surf)
302                 T = thomas_solve_prep(a, cp_, inv, d)
303             if n in snap_at:
304                 Tsnap[n] = T.copy()
305         res['T_time'] = time.time() - t0
306         Tend = T.copy()
307         res['T_end'] = Tend
308         res['T_snap'] = Tsnap
309
310 # ----- C -----
311 Cend = None
312 Csnap = {}
313 stats = dict(total_it=0, avg_it=0.0, max_it=0, max_res=0.0, nsub=0)
314 if solve_C:
315     C = np.full(N + 1, p['C0'])
316     Csnap[0] = C.copy()
317     tot, mx, mxr, nsub = 0, 0, 0.0, 0
318     t0 = time.time()
319     for n in range(1, nsteps + 1):
320         for s in range(sub):
321             tt = (n - 1) * dt_out + (s + 1) * dt_int
322             cinf = float(Cenv_fn(tt))
323             C, it, rr = step_C(N, dr, dt_int, C, cinf, p, omega, tol, maxit, Dfac, dmode)
324             tot += it; mx = max(mx, it); mxr = max(mxr, rr); nsub += 1
325         if n in snap_at:
326             Csnap[n] = C.copy()
327         res['C_time'] = time.time() - t0
328         Cend = C.copy()
329         stats.update(total_it=tot, avg_it=tot / max(1, nsub), max_it=mx,
330                     max_res=mxr, nsub=nsub)
331     res['C_end'] = Cend
332     res['C_snap'] = Csnap
333     res['stats'] = stats
334     return res
335
336
337 def run_sim_collect(N, dr, dt_out, nsteps, p, Tenv_fn, Cenv_fn, sub=10, subT=1,
338                   center='fv', surf='fvm', kfac=1.0, Dfac=1.0, dmode='func',
339                   tol=1e-9, omega=0.7, maxit=30, cols=None,
340                   snap_at=(), solve_T=True, solve_C=True, tmethod='expm'):
341     """单遍运行并**同时**收集整表输出与快照 (替代"两遍计算")。
342
343     cols: 需要写出的列索引 (如 np.arange(0, N+1, 4)), None 表示不收集
344     返回 (res, tableT, tableC); tableT/C 形状为 (nsteps, len(cols))
345     """
346     p = dict(p)
347     p['kfac'] = kfac
348     R0 = p['R0']
349     r = np.arange(N + 1) * dr
350     dt_int = dt_out / float(sub)
351     dtT = dt_out / float(subT)

```

```

352     ncol = 0 if cols is None else len(cols)
353
354     tableT = np.zeros((nsteps, ncol)) if (cols is not None and solve_T) else None
355     tableC = np.zeros((nsteps, ncol)) if (cols is not None and solve_C) else None
356     Tsnap, Csnap = {}, {}
357     res = dict(N=N, dr=dr, dt_out=dt_out, dt_int=dt_int, sub=sub,
358               center=center, surf=surf, kfac=kfac, Dfac=Dfac)
359
360     Tprep = None
361     res['tmethod'] = tmethod
362     use_expm_T = bool(solve_T and tmethod == 'expm')
363     if solve_T:
364         aT, bT, cT, meta = assemble_T(N, dr, dtT, p, center, surf)
365         res['center_row'] = meta['center_row']
366         if use_expm_T:
367             import q1_exact as _ex
368             rT = _ex.run_T_expm(N, dr, p, Tenv_fn, nsteps * dt_out, hstep=dt_out,
369                               center=center, surf=surf,
370                               t_snaps=[n * dt_out for n in snap_at],
371                               cols=cols, dt_out=dt_out)
372             tableT = rT['table']
373             T = rT['T_end']
374             Tsnap[0] = np.full(N + 1, p['T0'])
375             for n in snap_at:
376                 Tsnap[n] = rT['T_snap'][round(float(n * dt_out), 9)].copy()
377             tT = 0.0
378         else:
379             cpT, invT = thomas_prep(aT, bT, cT)
380             Tprep = (aT, cpT, invT)
381             T = np.full(N + 1, p['T0'])
382             Tsnap[0] = T.copy()
383
384     C = np.full(N + 1, p['C0']) if solve_C else None
385     Csnap[0] = C.copy() if C is not None else None
386
387     tot, mx, mxr, nsub = 0, 0, 0.0, 0
388     tT = tC = 0.0
389     for n in range(1, nsteps + 1):
390         if solve_T and not use_expm_T:
391             t0 = time.time()
392             a, cp_, inv = Tprep
393             for s in range(subT):
394                 ts = (n - 1) * dt_out + (s + 1) * dtT
395                 d = rhs_T(N, dr, dtT, p, T, float(Tenv_fn(ts)), surf)
396                 T = thomas_solve_prep(a, cp_, inv, d)
397             tT += time.time() - t0
398             if n in snap_at:
399                 Tsnap[n] = T.copy()
400             if cols is not None:
401                 tableT[n - 1, :] = T[cols]
402         if solve_C:
403             t0 = time.time()
404             for s in range(sub):
405                 tt = (n - 1) * dt_out + (s + 1) * dt_int
406                 C, it, rr = step_C(N, dr, dt_int, C, float(Cenv_fn(tt)), p,
407                                   omega, tol, maxit, Dfac, dmode)
408                 tot += it; mx = max(mx, it); mxr = max(mxr, rr); nsub += 1
409             tC += time.time() - t0
410             if n in snap_at:
411                 Csnap[n] = C.copy()
412             if cols is not None:
413                 tableC[n - 1, :] = C[cols]
414
415     res['T_end'] = T.copy() if solve_T else None
416     res['C_end'] = C.copy() if solve_C else None
417     res['T_snap'] = Tsnap
418     res['C_snap'] = Csnap
419     res['T_time'] = tT
420     res['C_time'] = tC
421     res['stats'] = dict(total_it=tot, avg_it=tot / max(1, nsub), max_it=mx,
422                       max_res=mxr, nsub=nsub)
423     return res, tableT, tableC

```

B.4 问题一求解

```

1  # -*- coding: utf-8 -*-
2  """
3  A 题 Q1 (预热平衡阶段 0-1800 s) 主求解器 v4
4  =====
5  方法: 空间=元体平衡 (有限体积) + 三对角追赶法;
6        时间=**温度侧时间方向精确推进** (`q1_exact.run_T_expm`, h=1 s、无时间离散误差)
7        + 含水率侧全隐式 (后向欧拉, 子步 0.1 s)
8  物性: 附录2 (常物性 + D(C) 单因子非线性)
9  中心: 两条**独立推导**的装配路径 (元体平衡几何式 / 洛必达极限式), 做解级比对
10 表面: 元体平衡半格 (Robin)
11
12 相对 v3 的变更 (据《A_Q1求解记录》§九 待办)
13  =====
14  1. **单遍运行**: 同一次求解同时产出整表输出与论文快照,
15     消除"表1/表2 与 result1.xlsx 来自两遍计算"的异源风险;
16  2. **中心双路真装配**: v3 中两条分支写成了同一表达式的两遍 (断言自证),
17     本版改为两条独立推导 (几何式 vs 极限式) 并做**解级**比对;
18  3. 求解核抽离至 `q1_core.py` (配置化), 供 E1-E5 复用;
19  4. 清理死代码 `write_and_assert`; 日志补运行环境与耗时分解。
20
21 运行 (**须显式 `--go`, 见《工作约束》§12.1 运行闸门):
22  python q1_solver.py --go
23 不加 --go 时**只打印 [SCALE] 规模声明并退出**, 不执行求解。
24
25 约定: 本文件不引用、不记载任何资料中的日期。
26  """
27  import os
28  import io
29  import sys
30  import time
31  import platform
32  import openpyxl
33  import numpy as np
34  from openpyxl import load_workbook, Workbook
35
36  import q1_core as core
37  import q1_exact as ex
38  from q1_core import DEF
39
40  HERE = os.path.dirname(os.path.abspath(__file__))
41  # 自适应定位工作区根: 向上查找含 10_赛题 的目录
42  # (兼容两种目录深度: 工作区 11_建模/11-3_算法与管线/Q1/code/ 与交付包 09 的 code/)
43  def _find_root(p, _marker='10_赛题', _max=6):
44      cur = p
45      for _ in range(_max):
46          if os.path.isdir(os.path.join(cur, _marker)):
47              return cur
48          parent = os.path.dirname(cur)
49          if parent == cur:
50              break
51          cur = parent
52      return os.path.abspath(os.path.join(p, '..', '..', '..'))
53
54  ROOT = _find_root(HERE)
55  ATT1 = os.path.join(ROOT, '10_赛题', 'A题', '附件', '附件1.xlsx')
56  LOG = os.path.join(HERE, 'q1_run_log.txt')
57  RESDIR = os.path.join(ROOT, '20_交付包', '09_代码与复现', 'results')
58  RES1 = os.path.join(RESDIR, 'result1.xlsx')
59
60  DR = 0.00025      # 空间步长 (m)
61  DT = 1.0          # 输出步长 (s)
62  SUB = 10          # C 方程内部子步数 (内部步长 = 0.1 s)
63  SUBT = 32         # **后向欧拉对照路径**下 T 方程的内部子步数 (内部步长 = 1/32 s)
64  # ↑ Δ **交付路径不使用本参数**: 温度侧已改为「时间方向精确推进」(hstep = DT = 1 s, 无内部子步);
65  # 本参数仅供"后向欧拉对照口径" (E1-b 时间收敛实验等) 使用。
66  # 该旧口径下 dtT = 1 s 存在 ~1.2e-3 °C 一阶误差、细化到 1/32 s 仍残留 ~4e-5 °C,
67  # 这正是改用精确推进的原因 (步长无关性: h=1 s 与 h=60 s 差 3.1e-11 °C)。
68  OMEGA = 0.7       # Picard 欠松弛
69  TOL = 1e-9         # 相对残差判据
70  MAXIT = 30
71  NSTEP = 1800

```

```

73 SNAP = (100, 300, 600, 900, 1200, 1500, 1800)
74 LINES = []
75
76
77 def say(s):
78     LINES.append(str(s))
79     print(s)
80
81
82 def load_att1():
83     wb = load_workbook(ATT1, data_only=True, read_only=True)
84     ws = wb[wb.sheetnames[0]]
85     data = list(ws.iter_rows(values_only=True))[1:]
86     return (np.array([float(r[0]) for r in data]),
87             np.array([float(r[1]) for r in data]),
88             np.array([float(r[2]) for r in data]))
89
90
91 def assert_result1():
92     """程序化格式断言: 工作表名 / 维度 / 表头 / 时间列 / 4 位小数"""
93     wb = load_workbook(RES1, read_only=True)
94     msgs = []
95     ok = True
96     if wb.sheetnames != ['温度', '水分浓度']:
97         ok = False
98         msgs.append('FAIL sheetnames=%s' % wb.sheetnames)
99     else:
100         msgs.append('OK sheetnames=%s' % wb.sheetnames)
101     for sn in wb.sheetnames:
102         ws = wb[sn]
103         rows = list(ws.iter_rows(values_only=True))
104         msgs.append('%s rows=%d cols=%d' % (sn, len(rows), ws.max_column))
105         if len(rows) != 1801 or ws.max_column != 22:
106             ok = False
107             msgs.append('FAIL dims (expect 1801x22)')
108         if rows[0][0] != '时间\到药材中心的距离':
109             ok = False
110             msgs.append('FAIL header A1')
111         if [round(float(x), 4) for x in rows[0][1:]] != [round(0.1 * k, 4) for k in range(21)]:
112             ok = False
113             msgs.append('FAIL header cols')
114         else:
115             msgs.append('OK header cols 0..2.0 (21)')
116         if [r[0] for r in rows[1:]] != list(range(1, 1801)):
117             ok = False
118             msgs.append('FAIL time col')
119         else:
120             msgs.append('OK time col 1..1800')
121         bad = sum(1 for r in rows[1:] for v in r[1:])
122             if v is None or abs(round(float(v), 4) - float(v)) > 1e-12)
123         if bad:
124             ok = False
125             msgs.append('FAIL not 4-decimal: %d' % bad)
126         else:
127             msgs.append('OK all cells 4-decimal')
128     return ok, msgs
129
130
131 def main():
132     t_all = time.time()
133     t, Tinf, Cinf = load_att1()
134     p = dict(DEF)
135     N = int(round(p['R0'] / DR))
136     cols = np.arange(0, N + 1, 4) # 0.1 cm 步长 → 21 列
137     Tenv = lambda s: float(np.interp(s, t, Tinf))
138     Cenv = lambda s: float(np.interp(s, t, Cinf))
139
140     # ----- W0
141     say('=== W0 附件核验与平台统计 ===')
142     say('points=%d t=[%g,%g] dt=%g' % (len(t), t[0], t[-1], t[1] - t[0]))
143     say('Tinf first/last=%.4f/%.4f Cinf first/last=%.5f/%.5f'
144         % (Tinf[0], Tinf[-1], Cinf[0], Cinf[-1]))

```

```

145 seg = (t >= 10000) & (t <= 14400)
146 say('plateau[10000,14400]: Tinf mean=%.4f range=%.4f ; Cinf mean=%.6f range=%.6f'
147     % (Tinf[seg].mean(), Tinf[seg].max() - Tinf[seg].min(),
148     Cinf[seg].mean(), Cinf[seg].max() - Cinf[seg].min()))
149 say('env: python=%s numpy=%s openpyxl=%s %s %s'
150     % (platform.python_version(), np.__version__, openpyxl.__version__,
151     platform.system(), platform.release()))
152
153 # ----- W2 中心双路（真装配）
154 say('')
155 say('=== W2 中心节点双路独立装配与解级比对 ===')
156 a1, b1, c1, m1 = core.assemble_T(N, DR, DT, p, 'fv', 'fvm')
157 a2, b2, c2, m2 = core.assemble_T(N, DR, DT, p, 'lh', 'fvm')
158 row_ok = bool(np.allclose([b1[0], c1[0]], [b2[0], c2[0]], rtol=0.0, atol=1e-12))
159 say('归一化中心行 fv: b0=%.12e c0=%.12e' % (b1[0], c1[0]))
160 say('归一化中心行 lh: b0=%.12e c0=%.12e' % (b2[0], c2[0]))
161 say('中心行逐系数一致 = %s' % row_ok)
162
163 rT_fv = core.run_sim(N, DR, DT, NSTEP, p, Tenv, Cenv, sub=SUB, subT=SUBT,
164     center='fv', surf='fvm', solve_C=False)
165 rT_lh = core.run_sim(N, DR, DT, NSTEP, p, Tenv, Cenv, sub=SUB, subT=SUBT,
166     center='lh', surf='fvm', solve_C=False)
167 dmax = float(np.max(np.abs(rT_fv['T_end'] - rT_lh['T_end'])))
168 bitwise = bool(np.array_equal(rT_fv['T_end'], rT_lh['T_end']))
169 say('解级比对: 两条中心装配的 T_end 最大逐点差 = %.3e K (逐位相同=%s)' % (dmax, bitwise))
170
171 # ----- 主力求解
172 say('')
173 say('=== 主力求解 (T: 时间精确推进; C: 隐式+Picard; 整表与快照同源) ===')
174 t0 = time.time()
175 rT = ex.run_T_expm(N, DR, p, Tenv, NSTEP * DT, hstep=DT, center='fv', surf='fvm',
176     t_snaps=set(SNAP), cols=cols, dt_out=DT)
177 tT = time.time() - t0
178 tabT = rT['table']
179 resC, __, tabC = core.run_sim_collect(
180     N=N, dr=DR, dt_out=DT, nsteps=NSTEP, p=p,
181     Tenv_fn=Tenv, Cenv_fn=Cenv, sub=SUB, subT=SUBT,
182     center='fv', surf='fvm', tol=TOL, omega=OMEGA, maxit=MAXIT,
183     cols=cols, snap_at=set(SNAP), solve_T=False, solve_C=True)
184 st = resC['stats']
185 say('N=%d dr=%.3g m 节点数=%d 输出列=%d' % (N, DR, N + 1, len(cols)))
186 say('T 精确推进 %.2f s (%d 步 / 传播子 %d 个, h=%.3g s); C 隐式 %.1f s (%d 子步, dt_int=%.3g s)'
187     % (tT, rT['nstep'], rT['nprop'], rT['hstep'], resC['C_time'], st['nsub'], resC['dt_int']
188     '))')
189 say('C Picard: avg_iters=%.2f max_iters=%d max_res=%.2e'
190     % (st['avg_it'], st['max_it'], st['max_res']))
191 Tend, Cend = rT['T_end'], resC['C_end']
192 say('T center/surface=%.6f/%.6f C center/surface=%.6f/%.6f'
193     % (Tend[0], Tend[-1], Cend[0], Cend[-1]))
194
195 # ----- W5 输出
196 say('')
197 say('=== W5 输出与程序化断言 ===')
198 os.makedirs(RESDIR, exist_ok=True)
199 wb = Workbook()
200 wsT = wb.active
201 wsT.title = '温度'
202 wsC = wb.create_sheet('水分浓度')
203 for ws in (wsT, wsC):
204     ws.cell(row=1, column=1, value='时间\\到药材中心的距离')
205     for j in range(21):
206         ws.cell(row=1, column=2 + j, value=round(0.1 * j, 4))
207 for n in range(1, NSTEP + 1):
208     wsT.cell(row=1 + n, column=1, value=n)
209     wsC.cell(row=1 + n, column=1, value=n)
210     for j in range(21):
211         wsT.cell(row=1 + n, column=2 + j, value=round(float(tabT[n - 1, j]), 4))
212         wsC.cell(row=1 + n, column=2 + j, value=round(float(tabC[n - 1, j]), 4))
213 wb.save(RES1)
214 say('result1.xlsx written')
215 ok, msgs = assert_result1()
216 for m in msgs:
217     say(' ' + m)

```



```

217 say('ASSERTION %s' % ('PASS' if ok else 'FAIL'))
218
219 # ----- 论文表格
220 say('')
221 say('=== 表1 温度 (°C) / 表2 水分浓度 (kg/kg), 列=到中心距离 0-2.0 cm ===')
222 idx5 = [int(round(x / DR)) for x in (0.0, 0.005, 0.01, 0.015, 0.02)]
223 say('Table 1 (T degC)')
224 for ts in SNAP:
225     say('%5d ' % ts + ' '.join('%4f' % rT['T_snap'][float(ts)][i] for i in idx5))
226 say('Table 2 (C kg/kg)')
227 for ts in SNAP:
228     say('%5d ' % ts + ' '.join('%4f' % resC['C_snap'][ts][i] for i in idx5))
229
230 say('')
231 say('total elapsed=%1fs' % (time.time() - t_all))
232 with io.open(LOG, 'w', encoding='utf-8') as f:
233     f.write('\n'.join(LINES) + '\n')
234 print('DONE assert=%s' % ok)
235 return 0 if ok else 1
236
237
238 # -----
239 # 运行闸门 (《工作约束》§12.1): 默认只"声明规模"并退出; 须显式 `--go`
240 # (或环境变量 ALLOW_RUN=1) 且规模不超批准值, 才真正执行。
241 # 首行输出 [SCALE] 声明行, 便于事后审计。
242 # -----
243 def _gate(name, scale, approved=None, argv=None, env=None):
244     args = sys.argv if argv is None else argv
245     envv = os.environ if env is None else env
246     print('[SCALE] %s | ' % name + ' '.join('%s=%s' % (k, v) for k, v in scale.items()),
247           flush=True)
248     if not (('--go' in args) or (envv.get('ALLOW_RUN', '') == '1')):
249         print('[GATE] 未传入 --go (或 ALLOW_RUN=1) ==> 仅声明规模, 不执行。', flush=True)
250         print('[GATE] 真正执行: python %s --go' % os.path.basename(__file__), flush=True)
251         return False
252     if approved:
253         bad = ['%s %s>%s' % (k, scale[k], approved[k])
254               for k in approved if k in scale and scale[k] > approved[k]]
255         if bad:
256             print('[GATE] 规模超批准值 (%s) ==> 拒绝执行。' % '; '.join(bad), flush=True)
257             return False
258     print('[GATE] 规模在校验范围内 ==> 执行。', flush=True)
259     return True
260
261
262 if __name__ == '__main__':
263     if not _gate('Q1 主力求解 v4',
264                 dict(网格='Δr=0.25 mm (81 节点)', 输出='0-1800 s @1 s (1800 行)',
265                     内部子步='T 无 (时间精确推进, h=1 s); C 0.1 s', 预计耗时='约 55 s')):
266         sys.exit(0)
267     sys.exit(main())

```

```

1  # -*- coding: utf-8 -*-
2  """
3  A 题 Q1 求解核 (配置化) —— 主力求解与全部检验实验共用
4  =====
5  统一口径
6  =====
7
8  • 空间      : 元体平衡 (有限体积) + 三对角追赶法
9  • 时间      : **温度=时间方向精确推进** (矩阵指数+分段线性 Duhamel, 见 `q1_exact.py`;
10               本核经 `tmethod='expm'` 调用, **无时间离散误差**)
11               其余 (Q1 含水率、Q2-Q4 全部场量) = 全隐式 (后向欧拉)
12  • 中心      : 两条**真正独立**的装配路径, 经同一尺度归一化后逐系数比对
13               center='fv' —— 用中心控制体体积  $V_0 = \Delta r^2 / 4$  与界面面积  $A_{12} = \Delta r$  显式装配
14               center='lh' —— 洛必达极限  $(1/r) \partial r (r \partial r T) \rightarrow 2 \partial r^2 T$  的差分离散
15  • 表面      : surf='fvm' —— 元体平衡半格 (主力)
16               surf='ghost' —— **一阶近似口径** (体积取  $R_0 \cdot \Delta r / 2$ 、导热界面取  $R_0$ );
17               **仅**用于边界口径对照实验。△ 命名沿用历史,
18               **其实现并非"虚拟节点+中心差分" (原虚拟节点装配符号有误、已改)
19  • 非线性    : D(C) 单因子; Picard 迭代+欠松弛; 时间导数右端**固定取上一时层解**
20  • 解耦      : 先全程解 T, 再全程解 C (T 方程不含 C, D 不含 T)
21 退化与灵敏度
22  =====

```

```

22 • kfac / Dfac: 物性缩放因子 (0.0 → 退化自检; 1±0.2 → 灵敏度 OAT)
23 • 环境条件以函数传入 (常数环境用于 E2 解析对拍与退化自检)
24
25 约定: 本核不引用、不记载任何资料中的日期。
26 """
27 import time
28 import numpy as np
29
30 DEF = dict(R0=0.02, rho=820.0, cp=2600.0, k=0.36, h=25.0, km=8.0e-7,
31           D0=7.0e-9, bD=0.89, T0=28.0, C0=2.55)
32
33 V0_OF = staticmethod(lambda dr: dr * dr / 4.0) # 中心控制体体积 (π 已约去)
34 VN_OF = staticmethod(lambda dr, R0: dr * (R0 - dr / 4.0) / 2.0) # 表面控制体体积
35
36
37 def thomas(a, b, c, d):
38     """三对角追赶法 (a/b/c 为下/主/上对角, a[0] 与 c[-1] 不使用)"""
39     n = len(b)
40     cp_ = np.empty(n - 1, dtype=float)
41     dp_ = np.empty(n, dtype=float)
42     cp_[0] = c[0] / b[0]
43     dp_[0] = d[0] / b[0]
44     for i in range(1, n):
45         m = b[i] - a[i] * cp_[i - 1]
46         dp_[i] = (d[i] - a[i] * dp_[i - 1]) / m
47         if i < n - 1:
48             cp_[i] = c[i] / m
49     x = np.empty(n, dtype=float)
50     x[-1] = dp_[-1]
51     for i in range(n - 2, -1, -1):
52         x[i] = dp_[i] - cp_[i] * x[i + 1]
53     return x
54
55
56 def thomas_prep(a, b, c):
57     """常数系数三对角矩阵的预消元 (前向消元结果与 d 无关, 可复用)"""
58     n = len(b)
59     cp_ = np.empty(n - 1, dtype=float)
60     inv = np.empty(n, dtype=float)
61     cp_[0] = c[0] / b[0]
62     inv[0] = 1.0 / b[0]
63     for i in range(1, n):
64         m = b[i] - a[i] * cp_[i - 1]
65         inv[i] = 1.0 / m
66         if i < n - 1:
67             cp_[i] = c[i] / m
68     return cp_, inv
69
70
71 def thomas_solve_prep(a, cp_, inv, d):
72     """配合 thomas_prep 使用的快速求解 (与 thomas 同式, 仅省去重复除法)"""
73     n = len(d)
74     dp_ = np.empty(n, dtype=float)
75     dp_[0] = d[0] * inv[0]
76     for i in range(1, n):
77         dp_[i] = (d[i] - a[i] * dp_[i - 1]) * inv[i]
78     x = np.empty(n, dtype=float)
79     x[-1] = dp_[-1]
80     for i in range(n - 2, -1, -1):
81         x[i] = dp_[i] - cp_[i] * x[i + 1]
82     return x
83
84
85 def D_of(C, D0=DEF['D0'], bD=DEF['bD']):
86     """有效水分扩散系数 D(C) = D0 * exp(-bD/C)"""
87     return D0 * np.exp(-bD / np.maximum(np.asarray(C, dtype=float), 1e-9))
88
89
90 def harm(dl, dr_):
91     """界面系数: 距离加权调和平均 (D=0 时退化为 0, 避免除零)。
92
93     △ **本函数已不用于生产路径**: C 方程的界面扩散系数改由 `faceD_int`
94     (**沿 $C$ 的积分平均**, 口径 **M6**) 生成; 本函数**仅保留作旧口径对照**。
```

```

95     """
96     if dl <= 0.0 or dr_ <= 0.0:
97         return 0.0
98     return 2.0 / (1.0 / dl + 1.0 / dr_)
99
100
101 # ----- T 方程
102 def assemble_T(N, dr, dt, p, center='fv', surf='fvm'):
103     """装配后向欧拉 T 方程系数 (a, b, c, meta); 右端另由 rhs_T 生成"""
104     rho, cp = p['rho'], p['cp']
105     k = p['k'] * p.get('kfac', 1.0)
106     h, R0 = p['h'], p['R0']
107     r = np.arange(N + 1) * dr
108     a = np.zeros(N + 1); b = np.zeros(N + 1); c = np.zeros(N + 1)
109     meta = {}
110
111     # --- 中心节点: 两条独立路径, 均归一化到"单位体积"以便逐系数比对 ---
112     if center == 'fv':
113         V0 = dr * dr / 4.0
114         A12 = dr
115         b[0] = (rho * cp * V0 / dt + k * A12 / dr) / V0
116         c[0] = (-k * A12 / dr) / V0
117         meta['center_scale'] = V0
118     elif center == 'lh':
119         d2 = 2.0 / dr ** 2
120         b[0] = rho * cp / dt + 2.0 * k * d2
121         c[0] = -2.0 * k * d2
122         meta['center_scale'] = 1.0
123     else:
124         raise ValueError(center)
125     meta['center_row'] = (float(b[0]), float(c[0]))
126
127     # --- 内部节点: 标准元体平衡 (含  $r_{i\pm 1/2}$  几何权重) ---
128     for i in range(1, N):
129         rl, rr = r[i] - dr / 2.0, r[i] + dr / 2.0
130         b[i] = rho * cp * r[i] / dt + k * (rr + rl) / dr ** 2
131         a[i] = -k * rl / dr ** 2
132         c[i] = -k * rr / dr ** 2
133
134     # --- 表面节点 ---
135     if surf == 'fvm':
136         VN = dr * (R0 - dr / 4.0) / 2.0
137         b[N] = rho * cp * VN / dt + k * (R0 - dr / 2.0) / dr + h * R0
138         a[N] = -k * (R0 - dr / 2.0) / dr
139         meta['surf'] = 'fvm'
140     elif surf == 'ghost':
141         # 对照口径 (**仅用于边界离散敏感性对照**):
142         # 表面控制体体积取一阶近似  $R0 \cdot \Delta r / 2$  (忽略曲率), 导热界面半径取  $R0$ 
143         VNa = R0 * dr / 2.0
144         b[N] = rho * cp * VNa / dt + k * R0 / dr + h * R0
145         a[N] = -k * R0 / dr
146         meta['surf'] = 'approx'
147     else:
148         raise ValueError(surf)
149     return a, b, c, meta
150
151
152 def rhs_T(N, dr, dt, p, Told, Tinf, surf='fvm'):
153     """生成 T 方程右端向量 d"""
154     rho, cp = p['rho'], p['cp']
155     h, R0 = p['h'], p['R0']
156     r = np.arange(N + 1) * dr
157     d = np.zeros(N + 1)
158     d[0] = rho * cp * Told[0] / dt
159     for i in range(1, N):
160         d[i] = rho * cp * r[i] * Told[i] / dt
161     if surf == 'fvm':
162         VN = dr * (R0 - dr / 4.0) / 2.0
163         d[N] = rho * cp * VN * Told[N] / dt + h * R0 * Tinf
164     else:
165         VNa = R0 * dr / 2.0
166         d[N] = rho * cp * VNa * Told[N] / dt + h * R0 * Tinf
167     return d

```

```

168
169
170 # ----- C 方程
171 def assemble_C(N, dr, dt, Df, R0, km):
172     """装配 C 方程系数 (a, b, c, VN); Df 为界面扩散系数 (长度 N, **沿 $C$ 的积分平均**, 由 `faceD_int`
    生成) """
173     r = np.arange(N + 1) * dr
174     a = np.zeros(N + 1); b = np.zeros(N + 1); c = np.zeros(N + 1)
175     V0 = dr * dr / 4.0
176     A12 = dr
177     b[0] = (V0 / dt + Df[0] * A12 / dr) / V0
178     c[0] = (-Df[0] * A12 / dr) / V0
179     for i in range(1, N):
180         rl, rr = r[i] - dr / 2.0, r[i] + dr / 2.0
181         b[i] = r[i] / dt + (Df[i] * rr + Df[i - 1] * rl) / dr ** 2
182         a[i] = -Df[i - 1] * rl / dr ** 2
183         c[i] = -Df[i] * rr / dr ** 2
184     VN = dr * (R0 - dr / 4.0) / 2.0
185     b[N] = VN / dt + Df[N - 1] * (R0 - dr / 2.0) / dr + km * R0
186     a[N] = -Df[N - 1] * (R0 - dr / 2.0) / dr
187     return a, b, c, VN
188
189
190 def rhs_C(N, dr, dt, Cold, cinf, R0, km):
191     """生成 C 方程右端向量 d (**固定取上一时层解 Cold**) """
192     r = np.arange(N + 1) * dr
193     d = np.zeros(N + 1)
194     d[0] = Cold[0] / dt
195     for i in range(1, N):
196         d[i] = r[i] * Cold[i] / dt
197     VN = dr * (R0 - dr / 4.0) / 2.0
198     d[N] = VN * Cold[N] / dt + km * R0 * cinf
199     return d
200
201
202 def faceD_int(Carr, D0, bD, Dfac=1.0, nsamp=8):
203     """*修正口径: 界面扩散系数取**沿 C 的积分平均**  $\langle D \rangle = (1/\Delta C) \int D \, dC$ 。
204
205     (原 `harm` 的调和平均仅对"界面处系数间断"成立; 系数随 C 陡变时显著失真。)
206     """
207     C = np.asarray(Carr, dtype=float)
208     lo = np.minimum(C[:-1], C[1:])
209     hi = np.maximum(C[:-1], C[1:])
210     xs = lo[:, None] + (hi - lo)[:, None] * (np.arange(nsamp) + 0.5)[None, :] / nsamp
211     xs = np.maximum(xs, 1e-9)
212     fv = np.exp(-bD / xs).mean(axis=1)
213     flat = (hi - lo) < 1e-14
214     if flat.any():
215         fv[flat] = np.exp(-bD / np.maximum(lo[flat], 1e-9))
216     return D0 * Dfac * fv
217
218
219 def step_C(N, dr, dt, C, cinf, p, omega=0.7, tol=1e-9, maxit=30, Dfac=1.0,
220           dmode='func'):
221     """推进一个 C 子步: Picard 迭代+欠松弛。返回 (C_new, iters, res)
222
223     dmode='func': D = D(C) 单因子非线性 (主力)
224     dmode='const': D ≡ D(C0) (用于 E2 解析对拍; 此时单次迭代即收敛)
225     """
226     Cold = C.copy()
227     Cit = C.copy()
228     res = None
229     it = 0
230     Dconst = p['D0'] * np.exp(-p['bD'] / p['C0']) * Dfac
231     for it in range(1, maxit + 1):
232         if dmode == 'const':
233             Df = np.full(N, Dconst)
234         else:
235             Df = faceD_int(Cit, p['D0'], p['bD'], Dfac)
236         a, b, c, _ = assemble_C(N, dr, dt, Df, p['R0'], p['km'])
237         d = rhs_C(N, dr, dt, Cold, cinf, p['R0'], p['km'])
238         Ctry = thomas(a, b, c, d)
239         Cnew = omega * Ctry + (1.0 - omega) * Cit

```

```

240         res = float(np.max(np.abs(Cnew - Cit))) / max(1.0, float(np.max(np.abs(Cnew))))
241         Cit = Cnew
242         if res < tol:
243             break
244     return Cit, it, res
245
246 # ----- 驱动
247 def run_sim(N, dr, dt_out, nsteps, p, Tenv_fn, Cenv_fn, sub=10, subT=1,
248            center='fv', surf='fvm', kfac=1.0, Dfac=1.0, dmode='func',
249            tol=1e-9, omega=0.7, maxit=30, snap_at=(), out_idx=None,
250            solve_T=True, solve_C=True, tmethod='expm'):
251     """统一求解驱动。
252
253     参数
254     ----
255     N, dr      : 空间区间数与步长 (节点数 N+1)
256     dt_out     : 输出时间步长 (s)
257     nsteps     : 输出步数 (总时长 = nsteps * dt_out)
258     p          : 物性/几何参数字典 (见 DEF)
259     Tenv_fn    :  $T_\infty(t)$  函数; Cenv_fn:  $C_\infty(t)$  函数
260     sub        : C 方程内部子步数 (内部步长 = dt_out/sub)
261     subT       : T 方程内部子步数 (内部步长 = dt_out/subT, 用于时间收敛实验)
262     out_idx    : 需要收集的列索引 (None → 不收集整表)
263     返回 dict
264     """
265     p = dict(p)
266     p['kfac'] = kfac
267     R0 = p['R0']
268     r = np.arange(N + 1) * dr
269     dt_int = dt_out / float(sub)
270     dtT = dt_out / float(subT)
271
272     res = dict(N=N, dr=dr, dt_out=dt_out, dt_int=dt_int, sub=sub,
273              center=center, surf=surf, kfac=kfac, Dfac=Dfac)
274
275     # ----- T -----
276     Tend = None
277     Tsnap = {}
278     res['tmethod'] = tmethod
279     if solve_T:
280         a, b, c, meta = assemble_T(N, dr, dtT, p, center, surf)
281         res['center_row'] = meta['center_row']
282         t0 = time.time()
283         if tmethod == 'expm':
284             # 时间方向**精确推进** (交付口径): 矩阵指数 + 分段线性 Duhamel
285             import ql_exact as _ex
286             rT = _ex.run_T_expm(N, dr, p, Tenv_fn, nsteps * dt_out,
287                               hstep=dt_out, center=center, surf=surf,
288                               t_snaps=[n * dt_out for n in snap_at])
289             Tsnap[0] = np.full(N + 1, p['T0'])
290             for n in snap_at:
291                 Tsnap[n] = rT['T_snap'][round(float(n * dt_out), 9)].copy()
292             T = rT['T_end']
293         else:
294             cp_, inv = thomas_prep(a, b, c)
295             T = np.full(N + 1, p['T0'])
296             Tsnap[0] = T.copy()
297             for n in range(1, nsteps + 1):
298                 for s in range(subT):
299                     ts = (n - 1) * dt_out + (s + 1) * dtT
300                     d = rhs_T(N, dr, dtT, p, T, float(Tenv_fn(ts)), surf)
301                     T = thomas_solve_prep(a, cp_, inv, d)
302                 if n in snap_at:
303                     Tsnap[n] = T.copy()
304             res['T_time'] = time.time() - t0
305             Tend = T.copy()
306     res['T_end'] = Tend
307     res['T_snap'] = Tsnap
308
309     # ----- C -----
310     Cend = None
311     Csnap = {}
312

```

```

313 stats = dict(total_it=0, avg_it=0.0, max_it=0, max_res=0.0, nsub=0)
314 if solve_C:
315     C = np.full(N + 1, p['C0'])
316     Csnap[0] = C.copy()
317     tot, mx, mxr, nsub = 0, 0, 0.0, 0
318     t0 = time.time()
319     for n in range(1, nsteps + 1):
320         for s in range(sub):
321             tt = (n - 1) * dt_out + (s + 1) * dt_int
322             cinf = float(Cenv_fn(tt))
323             C, it, rr = step_C(N, dr, dt_int, C, cinf, p, omega, tol, maxit, Dfac, dmode)
324             tot += it; mx = max(mx, it); mxr = max(mxr, rr); nsub += 1
325         if n in snap_at:
326             Csnap[n] = C.copy()
327     res['C_time'] = time.time() - t0
328     Cend = C.copy()
329     stats.update(total_it=tot, avg_it=tot / max(1, nsub), max_it=mx,
330                 max_res=mxr, nsub=nsub)
331 res['C_end'] = Cend
332 res['C_snap'] = Csnap
333 res['stats'] = stats
334 return res
335
336
337 def run_sim_collect(N, dr, dt_out, nsteps, p, Tenv_fn, Cenv_fn, sub=10, subT=1,
338                   center='fv', surf='fvm', kfac=1.0, Dfac=1.0, dmode='func',
339                   tol=1e-9, omega=0.7, maxit=30, cols=None,
340                   snap_at=(), solve_T=True, solve_C=True, tmethod='expm'):
341     """单遍运行并**同时**收集整表输出与快照（替代“两遍计算”）。
342
343     cols: 需要写出的列索引（如 np.arange(0, N+1, 4)），None 表示不收集
344     返回 (res, tableT, tableC); tableT/C 形状为 (nsteps, len(cols))
345     """
346     p = dict(p)
347     p['kfac'] = kfac
348     R0 = p['R0']
349     r = np.arange(N + 1) * dr
350     dt_int = dt_out / float(sub)
351     dtT = dt_out / float(subT)
352     ncol = 0 if cols is None else len(cols)
353
354     tableT = np.zeros((nsteps, ncol)) if (cols is not None and solve_T) else None
355     tableC = np.zeros((nsteps, ncol)) if (cols is not None and solve_C) else None
356     Tsnap, Csnap = {}, {}
357     res = dict(N=N, dr=dr, dt_out=dt_out, dt_int=dt_int, sub=sub,
358               center=center, surf=surf, kfac=kfac, Dfac=Dfac)
359
360     Tprep = None
361     res['tmethod'] = tmethod
362     use_expm_T = bool(solve_T and tmethod == 'expm')
363     if solve_T:
364         aT, bT, cT, meta = assemble_T(N, dr, dtT, p, center, surf)
365         res['center_row'] = meta['center_row']
366         if use_expm_T:
367             import q1_exact as _ex
368             rT = _ex.run_T_expm(N, dr, p, Tenv_fn, nsteps * dt_out, hstep=dt_out,
369                                center=center, surf=surf,
370                                t_snaps=[n * dt_out for n in snap_at],
371                                cols=cols, dt_out=dt_out)
372             tableT = rT['table']
373             T = rT['T_end']
374             Tsnap[0] = np.full(N + 1, p['T0'])
375             for n in snap_at:
376                 Tsnap[n] = rT['T_snap'][round(float(n * dt_out), 9)].copy()
377             tT = 0.0
378         else:
379             cpT, invT = thomas_prep(aT, bT, cT)
380             Tprep = (aT, cpT, invT)
381             T = np.full(N + 1, p['T0'])
382             Tsnap[0] = T.copy()
383
384     C = np.full(N + 1, p['C0']) if solve_C else None
385     Csnap[0] = C.copy() if C is not None else None

```

```

386 tot, mx, mxr, nsub = 0, 0, 0.0, 0
387 tT = tC = 0.0
388
389 for n in range(1, nsteps + 1):
390     if solve_T and not use_expm_T:
391         t0 = time.time()
392         a, cp_, inv = Tprep
393         for s in range(subT):
394             ts = (n - 1) * dt_out + (s + 1) * dtT
395             d = rhs_T(N, dr, dtT, p, T, float(Tenv_fn(ts)), surf)
396             T = thomas_solve_prep(a, cp_, inv, d)
397             tT += time.time() - t0
398             if n in snap_at:
399                 Tsnap[n] = T.copy()
400             if cols is not None:
401                 tableT[n - 1, :] = T[cols]
402         if solve_C:
403             t0 = time.time()
404             for s in range(sub):
405                 tt = (n - 1) * dt_out + (s + 1) * dt_int
406                 C, it, rr = step_C(N, dr, dt_int, C, float(Cenv_fn(tt)), p,
407                                 omega, tol, maxit, Dfac, dmode)
408                 tot += it; mx = max(mx, it); mxr = max(mxr, rr); nsub += 1
409             tC += time.time() - t0
410             if n in snap_at:
411                 Csnap[n] = C.copy()
412             if cols is not None:
413                 tableC[n - 1, :] = C[cols]
414
415 res['T_end'] = T.copy() if solve_T else None
416 res['C_end'] = C.copy() if solve_C else None
417 res['T_snap'] = Tsnap
418 res['C_snap'] = Csnap
419 res['T_time'] = tT
420 res['C_time'] = tC
421 res['stats'] = dict(total_it=tot, avg_it=tot / max(1, nsub), max_it=mx,
422                  max_res=mxr, nsub=nsub)
423 return res, tableT, tableC

```

B.5 问题二求解

```

1  # -*- coding: utf-8 -*-
2  """
3  A 题 Q2 主力求解: 变物性双向耦合 · 0-3 h 全程 → result2.xlsx
4  =====
5  口径 (全部引自《A_数值口径总表》, 禁止手抄)
6  -----
7  K10 Q2 全程用附录3、无物性阶段切换
8  N7-N10 变物性公式 (ρ、c_p、k、D)
9  N4/N5 h=25、k_m=8e-7 (H5 沿用附录2)
10 H6/N19/N20 t>14400 s 外推: T∞=50.00 °C、C∞=0.04999 kg/kg
11 K12/03 result2 = 0-3 h @1 s (10800 行 × 22 列), 4 位小数 (06)
12 05 内部 SI, 输出回填 s / cm / °C
13
14 数值设置 (W2 预试验定值)
15  -----
16 Δr = 0.25 mm (N=80 区间 / 81 节点) —— 输出点=计算点
17 内部步长 h = 1/32 s (n_sub=32, T 与 C 统一)
18 格式: IMEX 交替推进 (物性每内部子步更新; 滞后误差 α h ≈ 1e-8 K 量级)
19
20 运行 (**须显式 `--go`**, 见《工作约束》§12.1 运行闸门; 前台带进度, 结束后按 Enter 退出):
21 Linux/macOS : Q2_CORE=c python q2_solver.py --go
22 Windows PS : $env:Q2_CORE='c'; python q2_solver.py --go
23 不加 --go 时**只打印 [SCALE] 规模声明并退出**, 不执行求解。
24 """
25 import sys
26 import os
27 import time
28
29 try:
30     sys.stdout.reconfigure(encoding='utf-8')
31 except Exception:
32     pass

```

```

33
34 # 线程环境变量须早于 numpy 导入: 小规模三对角求解无需 BLAS/OMP 多线程,
35 # 关闭可避免每次调用的线程调度开销 (零数值影响, 纯环境级优化)
36 for _v in ('OMP_NUM_THREADS', 'MKL_NUM_THREADS', 'OPENBLAS_NUM_THREADS',
37           'NUMEXPR_NUM_THREADS', 'VECLIB_MAXIMUM_THREADS'):
38     os.environ.setdefault(_v, '1')
39
40 import numpy as np # noqa: E402
41 from openpyxl import Workbook # noqa: E402
42
43 HERE = os.path.dirname(os.path.abspath(__file__))
44 sys.path.insert(0, HERE)
45 # 求解核可切换 (环境变量 Q2_CORE):
46 # *默认 'c' —— q2_core_c *B+C 版 (numba JIT + 手写追赶法)
47 # **唯一符合 M6 口径 (界面系数沿 C 的积分平均) 的 Q2 核;
48 # 交付文件 result2.xlsx 必须由本核产出** (见《A_复核差异台账》D-22)
49 # 'b' —— q2_core B 版 (numba JIT + solve_banded)
50 # △ 旧口径: 界面系数仍为"距离加权调和平均",
51 # **不得用于重出交付文件** (仅保留作口径对照)
52 # 'orig' —— q2_core_orig 优化前基线 (仅供对照, 不建议在主力中使用)
53 _CORE = os.environ.get('Q2_CORE', 'c').strip().lower()
54 if _CORE == 'c':
55     from q2_core_c import run_q2, DEF2 # noqa: E402
56 elif _CORE == 'orig':
57     from q2_core_orig import run_q2, DEF2 # noqa: E402
58 else:
59     from q2_core import run_q2, DEF2 # noqa: E402
60
61
62 def _find_root(p, marker='10_赛题', _max=6):
63     """自适应定位工作区根 (兼容工作区与交付包两种深度)"""
64     cur = p
65     for _ in range(_max):
66         if os.path.isdir(os.path.join(cur, marker)):
67             return cur
68         par = os.path.dirname(cur)
69         if par == cur:
70             break
71         cur = par
72     return os.path.abspath(os.path.join(p, '..', '..', '..'))
73
74
75 ROOT = _find_root(HERE)
76 ATT1 = os.path.join(ROOT, '10_赛题', 'A题', '附件', '附件1.xlsx')
77 RESDIR = os.path.join(ROOT, '20_交付包', '09_代码与复现', 'results')
78 RES2 = os.path.join(RESDIR, 'result2.xlsx')
79 LOG = os.path.join(HERE, 'q2_run_log.txt')
80
81 # ----- 数值设置 -----
82 DR = 2.5e-4 # 空间步长 (m)
83 N = 80 # 区间数 (节点 81)
84 DT_OUT = 1.0 # 输出步长 (s)
85 NSTEP = 10800 # 3 h (K12)
86 NSUB = 32 # 内部子步数 (内部步长 1/32 s, W2 定值)
87 COLS = np.arange(0, N + 1, 4) # 0, 0.1, ..., 2.0 cm (21 列)
88 SNAP = (1800, 3600, 5400, 7200, 9000, 10800) # 表3/表4 取用时刻 (0.5 - 3 h)
89 T_EXT, C_EXT = 50.00, 0.04999 # H6/N19/N20 外推值
90
91 BUF = []
92
93
94 def say(s=''):
95     print(s, flush=True)
96     BUF.append(s)
97     with open(LOG, 'a', encoding='utf-8') as f:
98         f.write(s + '\n')
99
100
101 def load_att1():
102     """读附件1 (烘房温度与水分浓度时序), 返回 (t, T, C)"""
103     from openpyxl import load_workbook
104     wb = load_workbook(ATT1, data_only=True)
105     ws = wb.active

```



```

106 rows = [[c for c in r] for r in ws.iter_rows(values_only=True)]
107 hdr = rows[0]
108 data = [r for r in rows[1:] if r[0] is not None]
109 t = np.array([float(r[0]) for r in data], dtype=float)
110 T = np.array([float(r[1]) for r in data], dtype=float)
111 C = np.array([float(r[2]) for r in data], dtype=float)
112 return t, T, C, hdr
113
114
115 def main():
116     open(LOG, 'w', encoding='utf-8').close()
117     t0 = time.time()
118     say('=' * 78)
119     say('A 题 · 第二问 (Q2) 主力求解: 变物性双向耦合 0-3 h')
120     say(f'求解核: {_CORE.upper()} 版 (Q2_CORE 环境变量; b=numba+solve_banded, '
121         f'c=numba+手写追赶法)')
122     say('=' * 78)
123
124     # ----- 附件核验 -----
125     t1, T1, C1, hdr = load_att1()
126     say(f'[附件1] 表头={hdr} 点数={len(t1)} 时间 {t1[0]:.0f} - {t1[-1]:.0f} s '
127         f'步长={t1[1]-t1[0]:.0f} s')
128     say(f' 温度 {T1.min():.4f} - {T1.max():.4f} °C '
129         f'水分浓度 {C1.min():.5f} - {C1.max():.5f} kg/kg')
130     say(f' 外推 (H6, t>{t1[-1]:.0f} s): T∞={T_EXT} °C, C∞={C_EXT} kg/kg')
131
132     def Tenv(t):
133         return float(np.interp(t, t1, T1)) if t <= t1[-1] else T_EXT
134
135     def Cenv(t):
136         return float(np.interp(t, t1, C1)) if t <= t1[-1] else C_EXT
137
138     # ----- 求解 -----
139     say(f'\n[求解] Δr={DR*1e3:.2f} mm (N={N}区间/{N+1}节点) 输出步长={DT_OUT:g} s '
140         f'内部步长=1/{NSUB} s 格式=IMEX')
141     say(f' 共 {NSTEP} 个输出步, 预计约 {NSTEP*NSUB*0.0006/60:.1f} 分钟...')
142     res, tableT, tableC = run_q2(N=N, dr=DR, dt_out=DT_OUT, nsteps=NSTEP,
143                                 Tenv_fn=Tenv, Cenv_fn=Cenv, n_sub=NSUB,
144                                 mode='coupled', cols=COLS, snap_at=SNAP,
145                                 progress=1800)
146     wall = time.time() - t0
147     st = res['stats']
148     say(f'[完成] 耗时 {wall:.1f} s 内层均 {st["inner_tot"]/max(1,st["nsub"]):.1f} 次/步 '
149         f'内层最大残差 {st["resC_max"]:.2e} 异常标记 {st["flags"]}')
150
151     T_end, C_end = res['T_end'], res['C_end']
152     say(f'[终态] T(0)={T_end[0]:.4f} °C T(R)={T_end[-1]:.4f} °C '
153         f'C(0)={C_end[0]:.6f} C(R)={C_end[-1]:.6f} max_r C={C_end.max():.6f}')
154
155     # ----- 写 result2.xlsx -----
156     os.makedirs(RES2DIR, exist_ok=True)
157     wb = Workbook(write_only=True)
158     for name, tab in (('温度', tableT), ('水分浓度', tableC)):
159         ws = wb.create_sheet(name)
160         # 表头与附件3 模板一致 (首列名 + 数值型距离列头)
161         ws.append(['时间\\到药材中心的距离'] + [round(c * DR * 100, 1) for c in COLS])
162         t = time.time()
163         for i in range(NSTEP):
164             ws.append([i + 1] + [round(float(v), 4) for v in tab[i]])
165         say(f' 写入表「{name}」: {NSTEP} 行 × {len(COLS)+1} 列 {time.time()-t:.1f}s')
166     wb.save(RES2)
167     say(f'[输出] {os.path.relpath(RES2, R00T)} '
168         f'({os.path.getsize(RES2)/1048576:.2f} MB)')
169
170     # ----- 程序化断言 -----
171     # 判据上界一律由**输入数据**推出, 不得硬编码 (旧版写死 50.01, 而附件1 实测最高
172     # 50.246 °C —— 只是"碰巧"未触发, 属判据缺陷, 本轮修正)
173     T_ENV_MAX = float(np.max(T1)) # 附件1 环境温度最大值
174     T_ENV_MIN = float(np.min(T1))
175     T_UB = T_ENV_MAX + 0.01 # 容差 0.01 °C
176     T_LB = min(T_ENV_MIN, DEF2['T0']) - 0.01
177     say('\n[断言]')
178     chk = []

```

```

179     chk.append((f'形状 {NSTEP}×{len(COLS)+1}', tableT.shape == (NSTEP, len(COLS))
180                and tableC.shape == (NSTEP, len(COLS))))
181     chk.append((f'温度不超环境极值 {T_UB:.4f} °C', bool(np.nanmax(tableT) <= T_UB)))
182     chk.append((f'温度不低于下界 {T_LB:.4f} °C', bool(np.nanmin(tableT) >= T_LB)))
183     chk.append((f'含水率非负', bool(np.nanmin(tableC) >= 0.0)))
184     dC = np.diff(tableC, axis=0)
185     chk.append((f'含水率单调不增', bool(np.all(dC <= 1e-9))))
186     chk.append((f'含水率不超初值 {DEF2["C0"]}', bool(np.nanmax(tableC) <= DEF2["C0"] + 1e-4)))
187     for name, ok in chk:
188         say(f'    {"PASS" if ok else "FAIL"} {name}')
189     if not all(ok for _, ok in chk):
190         say('\n△ 有断言未通过, 请勿采信本结果! ')
191     else:
192         say('\n✓ 全部断言通过。')
193
194     # ----- 表3/表4 关键值 -----
195     say('\n[表3/表4 取用时刻的关键值] (0.5~3 h) ')
196     say(f'{"t/h":>6} {"T(θ)":>10} {"T(R)":>10} {"C(θ)":>12} {"C(R)":>12} {"maxC":>12} ')
197     for n in SNAP:
198         Tv, Cv = res['T_snap'][n], res['C_snap'][n]
199         say(f'{"n/3600:6.1f"} {"Tv[0]:10.4f"} {"Tv[-1]:10.4f"} '
200            f'{"Cv[0]:12.6f"} {"Cv[-1]:12.6f"} {"Cv.max():12.6f} ')
201
202     say(f'\n总耗时 {time.time()-t0:.1f} s    日志: {LOG} ')
203
204
205 # -----
206 # 运行闸门 (《工作约束》§12.1): 默认只"声明规模"并退出; 须显式 `--go`
207 # (或环境变量 ALLOW_RUN=1) 且规模不超批准值, 才真正执行。
208 # 首行输出 [SCALE] 声明行, 便于事后审计。
209 # -----
210 def _gate(name, scale, approved=None, argv=None, env=None):
211     args = sys.argv if argv is None else argv
212     envv = os.environ if env is None else env
213     print('[SCALE] %s | ' % name + ' | '.join('%s=%s' % (k, v) for k, v in scale.items()),
214           flush=True)
215     if not (('--go' in args) or (envv.get('ALLOW_RUN', '') == '1')):
216         print('[GATE] 未传入 --go (或 ALLOW_RUN=1) ⇒ 仅声明规模, 不执行.', flush=True)
217         print('[GATE] 真正执行: Q2_CORE=c python %s --go'
218               % os.path.basename(__file__), flush=True)
219         return False
220     if approved:
221         bad = ['%s %s>%s' % (k, scale[k], approved[k])
222                for k in approved if k in scale and scale[k] > approved[k]]
223         if bad:
224             print('[GATE] 规模超批准值 (%s) ⇒ 拒绝执行.' % '; '.join(bad), flush=True)
225             return False
226     print('[GATE] 规模在校验范围内 ⇒ 执行.', flush=True)
227     return True
228
229
230 if __name__ == '__main__':
231     if not _gate('Q2 主力求解',
232                 dict(核=_CORE, 网格='Δr=0.25 mm (81 节点)',
233                     输出='0~10800 s @1 s (10800 行)', 内部子步='1/32 s',
234                     预计耗时='约 85~130 s')):
235         sys.exit(0)
236     main()
237     try:
238         if sys.stdin.isatty():
239             input('\n按 Enter 键退出...')
240     except Exception:
241         pass

```

```

1  # -*- coding: utf-8 -*-
2  """
3  A 题 Q2 求解核 (变物性 · 双向强耦合) · v4 (★B + ★C 优化)
4  =====
5  ★C 优化 (在 B 的基础上): **以 numba 编译的手写追赶法 (Thomas) 替换
6  `scipy.linalg.solve_banded`**.
7
8  依据 (用户指令解除"禁止替换 Thomas"限制后实施):
9  • 本系统矩阵为**三对角且严格对角占优** ( $|b_i| > |a_i| + |c_i|$ ), 由隐式格式的

```

```

10 正定性保证)，故追赶法**无需选主元即可稳定**，与 LAPACK `dgbv` 数值等价；
11 • `solve_banded` 对小规模带状矩阵存在固定调用开销（Python 校验 + 数据整理），
12 微基准实测 21.25  $\mu$ s（关检查后 14.48  $\mu$ s），而 njit 追赶法仅需数微秒；
13 • 追赶法**消元顺序与 LAPACK 不同**  $\Rightarrow$  结果存在  $\sim 1e-15$  量级的舍入差异
14 （远低于 4 位小数输出精度），**必须经 `q2_verify_BC.py` 核对**。
```

△ 其余一切（离散表达式、Picard 流程、tol/maxit/omega、物性公式、IMEX 子步）与 B 版**完全相同**。

--- 以下为 B 版说明 ---

A 题 Q2 求解核（变物性 • 双向强耦合）• v3（★B 优化：numba JIT）

=====

与 Q1 核（q1_core.py）的关系

Q2 = Q1 框架 + 变物性 + 强耦合。本核**照搬** Q1 的离散结构：

- 元体平衡（有限体积）+ 全隐式（后向欧拉）+ 三对角求解
- 中心用中心控制体（ $V_0 = \Delta r^2/4$ 、 $A_1 = \Delta r$ ）；表面用半格元体平衡
- 界面系数一律**距离加权调和平均**

差异（两处）：

- 物性不再是常数： $\rho(C)$ 、 $c_p(C)$ 、 $k(C)$ 、 $D(C, T)$ （附录3，T 用 K）
- 两场**双向耦合** $\Rightarrow$ 外层耦合迭代 + 内层 Picard 迭代

v3 性能改造（★B：numba JIT；**不改任何数值语义**）

- 用 `@njit(cache=True, fastmath=False)` 编译：物性 $\rho/c_p/k/D$ 、界面调和平均、以及 T/C 的装配与右端函数（改为**接收预分配数组、原地填充**）。
- **`fastmath=False` 是硬性要求** —— 保持 IEEE 语义，禁止任何浮点重排/近似。
- **求解器仍是 `scipy.linalg.solve\_banded`**（B 版不替换 Thomas；C 版另见 `q2\_core\_c.py`）。
- **Picard 的 `for it` 循环、残差计算、break 条件、欠松弛、外层逻辑全部留在 Python 层，一行未改**：离散公式与装配表达式**逐字保留**（见各函数注释对照）。
- numba 不可用时自动**降级**为原 numpy 实现（`\_HAS\_NUMBA=False`），行为不变。

口径（全部引自《A_数值口径总表》）

N7 $\rho = 650 + 128C$ | N8 $c_p = 1450 + 2736 \cdot C/(C+1)$ | N9 $k = 0.21 + 0.38 \cdot C/(C+1)$
N10 $D = 2.4e-3 \cdot \exp(-0.45/C) \cdot \exp(-3850/T)$ [m^2/s]，T 用 K
N4 $h = 25$ | N5 $k_m = 8e-7$ （全程沿用附录2，H5）

模式

'coupled' 主力：双向强耦合
'frozen' E2 常物性退化对拍
'decoupled' E3 强制解耦（D 不含 T）

关键纪律

- **每一轮外层迭代中，T 与 C 都从 n 层解出发**（右端固定），仅物性随迭代更新
- **C 方程每个子步的右端取本子步起点**（子步间串行推进）
- 欠松弛 $\omega=0.7$ 必须保留；收敛判据一律用解的（绝对）增量

约定：本核不引用、不记载任何资料中的日期。

```

62 """
63 import time
64 import numpy as np
65 from scipy.linalg import solve_banded
66
67 # =====
68 # numba 可用性（不可用则自动降级为 numpy 实现，行为完全不变）
69 # =====
70 try:
71     from numba import njit
72     _HAS_NUMBA = True
73 except Exception:
74     njit = None
75     _HAS_NUMBA = False
76
77
78 DEF2 = dict(R0=0.02, h=25.0, km=8.0e-7, T0=28.0, C0=2.55)
79
80 # 全局扩散系数缩放因子（供灵敏度实验 OAT 使用；默认 1.0 不影响主力结果）
81 D_FAC = 1.0
82

```

```

83
84 # =====
85 # * JIT 内核 (fastmath=False: 严格 IEC 559, 禁止浮点重排)
86 #   每个内核下方都注明其**原 numpy 表达式**, 供逐字核对。
87 # =====
88 if _HAS_NUMBA:
89
90     @jit(cache=True, fastmath=False)
91     def _rho_nb(C):
92         """N7:  $\rho = 650 + 128C$  (原式  $650.0 + 128.0 * C$ ) """
93         n = C.shape[0]
94         out = np.empty(n)
95         for i in range(n):
96             out[i] = 650.0 + 128.0 * C[i]
97         return out
98
99     @jit(cache=True, fastmath=False)
100     def _cp_nb(C):
101         """N8:  $c_p = 1450 + 2736 \cdot C / (C+1)$  (原式  $1450.0 + 2736.0 * C / (C + 1.0)$ ) """
102         n = C.shape[0]
103         out = np.empty(n)
104         for i in range(n):
105             out[i] = 1450.0 + 2736.0 * C[i] / (C[i] + 1.0)
106         return out
107
108     @jit(cache=True, fastmath=False)
109     def _k_nb(C):
110         """N9:  $k = 0.21 + 0.38 \cdot C / (C+1)$  (原式  $0.21 + 0.38 * C / (C + 1.0)$ ) """
111         n = C.shape[0]
112         out = np.empty(n)
113         for i in range(n):
114             out[i] = 0.21 + 0.38 * C[i] / (C[i] + 1.0)
115         return out
116
117     @jit(cache=True, fastmath=False)
118     def _D_nb(C, T_C, d_fac):
119         """N10:  $D = D\_FAC \cdot 2.4e-3 \cdot \exp(-0.45/C) \cdot \exp(-3850/T)$ , T 用 K
120
121          $\Delta$  `D_FAC` 原为模块级全局量; 在 njit 内**必须显式传参**,
122         否则 numba 会将其在编译期冻结为常量 (OAT 扰动将失效)。
123         """
124         n = C.shape[0]
125         out = np.empty(n)
126         for i in range(n):
127             c = C[i]
128             if c < 1e-9:
129                 c = 1e-9
130             out[i] = d_fac * 2.4e-3 * np.exp(-0.45 / c) * np.exp(-3850.0 / (T_C[i] + 273.15))
131         return out
132
133     @jit(cache=True, fastmath=False)
134     def _iface_nb(f):
135         """界面值: 距离加权调和平均; 任一侧  $\leq 0$  则取 0
136         (原式  $out[m] = 2 \cdot l \cdot r / (l+r)$ , 其余位置保持 0)
137
138          $\Delta$  **本式仅适用于**系数在界面处间断 (分段常数) **。
139         当系数随  $\$C\$$  平滑但陡峭变化时 (如 Q3 低含水率端  $\$D\$$  跨 2-3 个数量级),
140         应改用下方 `_faceD_nb` / `_facek_nb` 的**沿  $\$C\$$  积分平均**。
141         """
142         n = f.shape[0] - 1
143         out = np.zeros(n)
144         for i in range(n):
145             l = f[i]
146             r = f[i + 1]
147             if l > 0.0 and r > 0.0:
148                 out[i] = 2.0 * l * r / (l + r)
149         return out
150
151     @jit(cache=True, fastmath=False)
152     def _faceD_nb(Carr, Tarr, d_fac, useT, nsamp):
153         """**修正口径**: 界面扩散系数取**沿  $\$C\$$  的积分平均**
154
155          $\langle D \rangle = (1/\Delta C) \int_{C_{i+1}}^{C_i} D(C, T_{face}) dC$ 

```

依据: 单元内稳态通量 $J = \frac{1}{\Delta r} \int D \, \mathrm{d}C \Rightarrow$ 界面系数 = 沿 C 的积分平均。

利用可分离性 $D = [d_{\text{fac}} 2.4 \times 10^{-3} e^{-0.45/C}] e^{-3850/T_K}$, 只需对 C 因子采样。

`useT=True`: 含 T 因子 (coupled); `False`: 仅 C 因子 (decoupled, 不乘 d_{fac})。 $\Delta C \approx 0$ 时退化为节点值。

```

"""
n = Carr.shape[0] - 1
out = np.empty(n)
for i in range(n):
    a = Carr[i]
    b = Carr[i + 1]
    lo = a if a < b else b
    hi = b if a < b else a
    if hi - lo < 1e-14:
        c = lo if lo > 1e-9 else 1e-9
        fv = np.exp(-0.45 / c)
    else:
        s = 0.0
        for k in range(nsamp):
            x = lo + (hi - lo) * (k + 0.5) / nsamp
            if x < 1e-9:
                x = 1e-9
            s += np.exp(-0.45 / x)
        fv = s / nsamp
    if useT:
        Tm = 0.5 * (Tarr[i] + Tarr[i + 1])
        out[i] = d_fac * 2.4e-3 * fv * np.exp(-3850.0 / (Tm + 273.15))
    else:
        out[i] = 2.4e-3 * fv
return out

```

@njit(cache=True, fastmath=False)

def _facek_nb(Carr):

"""***修正口径**：界面导热系数取沿 C 的积分平均 (解析式)

```

"""
(k) = 0.21 + 0.38 * [Δ(C - ln(1+C))]/ΔC
"""
n = Carr.shape[0] - 1
out = np.empty(n)
for i in range(n):
    a = Carr[i]
    b = Carr[i + 1]
    if abs(b - a) < 1e-14:
        c = a
        out[i] = 0.21 + 0.38 * c / (1.0 + c)
    else:
        ga = a - np.log(1.0 + a)
        gb = b - np.log(1.0 + b)
        out[i] = 0.21 + 0.38 * (gb - ga) / (b - a)
return out

```

@njit(cache=True, fastmath=False)

def _assemble_T_nb(N, dr, dt, R0, h, rho, cp, k, a, b, c, kf):

"""T 方程装配 (原地填充 a/b/c)。表达式与原 numpy 版逐字对应。

`kf` (界面调和平均) 由调用方预先算好传入 —— 与 v2 中
"函数内先 `kf = iface(k)`" 语义一致, 只是把该步提到包装层以避免重复计算。

```

"""
V0 = dr * dr / 4.0
b[0] = (rho[0] * cp[0] * V0 / dt + kf[0] * dr / dr) / V0
c[0] = (-kf[0] * dr / dr) / V0
a[0] = 0.0
for i in range(1, N):
    r = i * dr
    rl = r - dr / 2.0
    rr = r + dr / 2.0
    rc = rho[i] * cp[i]
    b[i] = rc * r / dt + (kf[i] * rr + kf[i - 1] * rl) / dr ** 2
    a[i] = -kf[i - 1] * rl / dr ** 2
    c[i] = -kf[i] * rr / dr ** 2
VN = dr * (R0 - dr / 4.0) / 2.0

```

```

227     b[N] = rho[N] * cp[N] * VN / dt + kf[N - 1] * (R0 - dr / 2.0) / dr + h * R0
228     a[N] = -kf[N - 1] * (R0 - dr / 2.0) / dr
229     c[N] = 0.0
230
231 @njit(cache=True, fastmath=False)
232 def _rhs_T_nb(N, dr, dt, R0, h, rho, cp, Told, Tinf, d):
233     """T 方程右端 (原地填充 d): 固定取上一时层解 Told"""
234     rcT = rho * cp * Told
235     d[0] = rcT[0] / dt
236     for i in range(1, N):
237         d[i] = (i * dr) * rcT[i] / dt
238     VN = dr * (R0 - dr / 4.0) / 2.0
239     d[N] = rcT[N] * VN / dt + h * R0 * Tinf
240     return d
241
242 @njit(cache=True, fastmath=False)
243 def _assemble_C_nb(N, dr, dt, Df, R0, km, a, b, c):
244     """C 方程装配 (原地填充 a/b/c)。表达式与原 numpy 版逐字对应。"""
245     V0 = dr * dr / 4.0
246     b[0] = (V0 / dt + Df[0] * dr / dr) / V0
247     c[0] = (-Df[0] * dr / dr) / V0
248     a[0] = 0.0
249     for i in range(1, N):
250         r = i * dr
251         rl = r - dr / 2.0
252         rr = r + dr / 2.0
253         b[i] = r / dt + (Df[i] * rr + Df[i - 1] * rl) / dr ** 2
254         a[i] = -Df[i - 1] * rl / dr ** 2
255         c[i] = -Df[i] * rr / dr ** 2
256     VN = dr * (R0 - dr / 4.0) / 2.0
257     b[N] = VN / dt + Df[N - 1] * (R0 - dr / 2.0) / dr + km * R0
258     a[N] = -Df[N - 1] * (R0 - dr / 2.0) / dr
259     c[N] = 0.0
260
261 @njit(cache=True, fastmath=False)
262 def _rhs_C_nb(N, dr, dt, R0, km, Cold, cinf, d):
263     """C 方程右端 (原地填充 d)"""
264     d[0] = Cold[0] / dt
265     for i in range(1, N):
266         d[i] = (i * dr) * Cold[i] / dt
267     VN = dr * (R0 - dr / 4.0) / 2.0
268     d[N] = VN * Cold[N] / dt + km * R0 * cinf
269     return d
270
271 @njit(cache=True, fastmath=False)
272 def _thomas_nb(a, b, c, d):
273     """*C: 手写追赶法 (Thomas 消元, 无选主元)
274
275     矩阵严格对角占优  $\implies$  无需主元选择即数值稳定。
276
277     算法 (标准前向消元 + 后向回代):
278         前向:  $cp[0] = c[0]/b[0]$ ;  $dp[0] = d[0]/b[0]$ 
279              $cp[i] = c[i] / (b[i] - a[i] \cdot cp[i-1])$ 
280              $dp[i] = (d[i] - a[i] \cdot dp[i-1]) / (b[i] - a[i] \cdot cp[i-1])$ 
281         后向:  $x[n-1] = dp[n-1]$ ;  $x[i] = dp[i] - cp[i] \cdot x[i+1]$ 
282
283     `a[0]` 与 `c[n-1]` 在装配时已置零, 故首末行自动退化为正确形式。
284     """
285     n = b.shape[0]
286     cp_ = np.empty(n)
287     dp_ = np.empty(n)
288     cp_[0] = c[0] / b[0]
289     dp_[0] = d[0] / b[0]
290     for i in range(1, n):
291         m = b[i] - a[i] * cp_[i - 1]
292         cp_[i] = c[i] / m
293         dp_[i] = (d[i] - a[i] * dp_[i - 1]) / m
294     x = np.empty(n)
295     x[n - 1] = dp_[n - 1]
296     for i in range(n - 2, -1, -1):
297         x[i] = dp_[i] - cp_[i] * x[i + 1]
298     return x
299

```

```

300
301 # =====
302 # 公共接口（签名与 v2 完全一致  $\Rightarrow$  既有脚本零改动）
303 # =====
304 def rho_of(C):
305     if _HAS_NUMBA:
306         arr = np.asarray(C, dtype=np.float64)
307         if arr.ndim == 0: # 标量输入  $\Rightarrow$  返回标量（与原版 float() 兼容）
308             return float(_rho_nb(arr.reshape(1))[0])
309         return _rho_nb(np.ascontiguousarray(arr))
310     return 650.0 + 128.0 * np.asarray(C, dtype=float)
311
312
313 def cp_of(C):
314     if _HAS_NUMBA:
315         arr = np.asarray(C, dtype=np.float64)
316         if arr.ndim == 0:
317             return float(_cp_nb(arr.reshape(1))[0])
318         return _cp_nb(np.ascontiguousarray(arr))
319     C = np.asarray(C, dtype=float)
320     return 1450.0 + 2736.0 * C / (C + 1.0)
321
322
323 def k_of(C):
324     if _HAS_NUMBA:
325         arr = np.asarray(C, dtype=np.float64)
326         if arr.ndim == 0:
327             return float(_k_nb(arr.reshape(1))[0])
328         return _k_nb(np.ascontiguousarray(arr))
329     C = np.asarray(C, dtype=float)
330     return 0.21 + 0.38 * C / (C + 1.0)
331
332
333 def D_of(C, T_C):
334     """N10:  $D = 2.4e-3 \cdot \exp(-0.45/C) \cdot \exp(-3850/T)$ , T 用 K"""
335     if _HAS_NUMBA:
336         Ca = np.asarray(C, dtype=np.float64)
337         Ta = np.asarray(T_C, dtype=np.float64)
338         if Ca.ndim == 0:
339             v = _D_nb(Ca.reshape(1), Ta.reshape(1), float(D_FAC))
340             return float(v[0])
341         return _D_nb(np.ascontiguousarray(Ca), np.ascontiguousarray(Ta), float(D_FAC))
342     C = np.maximum(np.asarray(C, dtype=float), 1e-9)
343     TK = np.asarray(T_C, dtype=float) + 273.15
344     return D_FAC * 2.4e-3 * np.exp(-0.45 / C) * np.exp(-3850.0 / TK)
345
346
347 def harm(dl, dr_):
348     """**标量版**界面系数: 距离加权调和平均（与 `iface` 等价）"""
349     if dl <= 0.0 or dr_ <= 0.0:
350         return 0.0
351     return 2.0 / (1.0 / dl + 1.0 / dr_)
352
353
354 def iface(f):
355     """界面值: 距离加权调和平均（矢量化）。任一侧  $\leq 0$  则取 0."""
356     if _HAS_NUMBA:
357         arr = np.asarray(f, dtype=np.float64)
358         if arr.ndim == 0:
359             return np.zeros(0)
360         return _iface_nb(np.ascontiguousarray(arr))
361     f = np.asarray(f, dtype=float)
362     l, r = f[:-1], f[1:]
363     s = l + r
364     out = np.zeros_like(l)
365     m = (l > 0.0) & (r > 0.0)
366     out[m] = 2.0 * l[m] * r[m] / s[m]
367     return out
368
369
370 # =====
371 # ***修正口径*: 界面系数取"沿 C 的积分平均"
372 # （原 `iface` 的调和平均仅对"界面处系数间断"成立；本项在系数随 C 陡变时失真）

```

```

373 # =====
374 NSAMP_FACE = 8          # 沿 C 的 Midpoint 采样点数 (已验 4/8/16 收敛)
375
376
377 def faceD_of(C, T, d_fac=1.0, useT=True, nsamp=NSAMP_FACE):
378     """界面扩散系数 (沿 $C$ 的积分平均) ——**修正口径**。
379
380     
$$\langle D \rangle = \frac{1}{\Delta C} \int D \, dC$$

381     对 $C$ 因子采样、$T$ 取界面
382     均值。
383     """
384     Ca = np.ascontiguousarray(np.asarray(C, dtype=np.float64))
385     Ta = np.ascontiguousarray(np.asarray(T, dtype=np.float64))
386     if _HAS_NUMBA:
387         return _faceD_nb(Ca, Ta, float(d_fac), bool(useT), int(nsamp))
388     lo, hi = np.minimum(Ca[:-1], Ca[1:]), np.maximum(Ca[:-1], Ca[1:])
389     xs = lo[:, None] + (hi - lo)[:, None] * (np.arange(nsamp) + 0.5)[None, :] / nsamp
390     xs = np.maximum(xs, 1e-9)
391     fv = np.exp(-0.45 / xs).mean(axis=1)
392     flat = (hi - lo) < 1e-14
393     if flat.any():
394         fv[flat] = np.exp(-0.45 / np.maximum(lo[flat], 1e-9))
395     if useT:
396         Tm = 0.5 * (Ta[:-1] + Ta[1:])
397         return d_fac * 2.4e-3 * fv * np.exp(-3850.0 / (Tm + 273.15))
398     return 2.4e-3 * fv
399
400 def facek_of(C):
401     """界面导热系数 (沿 $C$ 的积分平均, 解析式) ——**修正口径**。"""
402     Ca = np.ascontiguousarray(np.asarray(C, dtype=np.float64))
403     if _HAS_NUMBA:
404         return _facek_nb(Ca)
405     a, b = Ca[:-1], Ca[1:]
406     db = np.abs(b - a)
407     safe = np.where(db < 1e-14, 1.0, b - a)
408     gb = b - np.log(1.0 + b)
409     ga = a - np.log(1.0 + a)
410     out = 0.21 + 0.38 * (gb - ga) / safe
411     flat = db < 1e-14
412     if flat.any():
413         out[flat] = 0.21 + 0.38 * a[flat] / (1.0 + a[flat])
414     return out
415
416 # =====
417 # 三对角求解 (B 版: **保留 scipy.linalg.solve_banded**)
418 # =====
419 def thomas(a, b, c, d):
420     """解三对角系统 (a/b/c 为下/主/上对角, a[0] 与 c[-1] 不使用)
421
422     *C 版: 改为 **numba 手写追赶法** (原 `scipy.linalg.solve_banded` 仅作降级回退)。
423     调用签名**完全不变**  $\Rightarrow$  上层装配/Picard 逻辑一行未改。
424     """
425     if _HAS_NUMBA:
426         return _thomas_nb(a, b, c, d)
427     n = len(b)
428     ab = np.zeros((3, n), dtype=float)
429     ab[0, 1:] = c[:-1]
430     ab[1, :] = b
431     ab[2, :-1] = a[1:]
432     return solve_banded((1, 1), ab, d, check_finite=False, overwrite_ab=True)
433
434 # =====
435 # T 方程装配 (Java 侧包装: 负责缓冲分配, 内核原地填充)
436 # =====
437 def assemble_T_var(N, dr, dt, R0, h, rho, cp, k, out=None, kf=None):
438     """返回 (a, b, c)。rho/cp/k 为长度 N+1 的节点数组。
439
440     `out=(a,b,c)` 时**原地覆写**预分配缓冲; 否则新建。
441     `kf`: 可选的**预计算界面导热系数** (修正口径 `facek_of(C)`); None 时回退旧调和平均。
442     """
443     if out is None:

```



```

445     a = np.zeros(N + 1); b = np.zeros(N + 1); c = np.zeros(N + 1)
446 else:
447     a, b, c = out
448 if _HAS_NUMBA:
449     if kf is None:
450         kf = _iface_nb(np.ascontiguousarray(k, dtype=np.float64))
451         _assemble_T_nb(N, dr, dt, R0, h,
452             np.ascontiguousarray(rho, dtype=np.float64),
453             np.ascontiguousarray(cp, dtype=np.float64),
454             np.ascontiguousarray(k, dtype=np.float64),
455             a, b, c, kf)
456     return a, b, c
457 r = np.arange(N + 1) * dr
458 if kf is None:
459     kf = iface(k)
460 V0 = dr * dr / 4.0
461 b[0] = (rho[0] * cp[0] * V0 / dt + kf[0] * dr / dr) / V0
462 c[0] = (-kf[0] * dr / dr) / V0
463 a[0] = 0.0
464 ii = np.arange(1, N)
465 rl, rr = r[ii] - dr / 2.0, r[ii] + dr / 2.0
466 rc = rho[ii] * cp[ii]
467 b[ii] = rc * r[ii] / dt + (kf[ii] * rr + kf[ii - 1] * rl) / dr ** 2
468 a[ii] = -kf[ii - 1] * rl / dr ** 2
469 c[ii] = -kf[ii] * rr / dr ** 2
470 VN = dr * (R0 - dr / 4.0) / 2.0
471 b[N] = rho[N] * cp[N] * VN / dt + kf[N - 1] * (R0 - dr / 2.0) / dr + h * R0
472 a[N] = -kf[N - 1] * (R0 - dr / 2.0) / dr
473 c[N] = 0.0
474 return a, b, c
475
476
477 def rhs_T_var(N, dr, dt, R0, h, rho, cp, Told, Tinf, out=None):
478     """右端: 固定取上一时层解 Told"""
479     d = np.zeros(N + 1) if out is None else out
480     if _HAS_NUMBA:
481         _rhs_T_nb(N, dr, dt, R0, h,
482             np.ascontiguousarray(rho, dtype=np.float64),
483             np.ascontiguousarray(cp, dtype=np.float64),
484             np.ascontiguousarray(Told, dtype=np.float64),
485             float(Tinf), d)
486     return d
487 r = np.arange(N + 1) * dr
488 rcT = rho * cp * Told
489 d[0] = rcT[0] / dt
490 d[1:N] = r[1:N] * rcT[1:N] / dt
491 VN = dr * (R0 - dr / 4.0) / 2.0
492 d[N] = rcT[N] * VN / dt + h * R0 * Tinf
493 return d
494
495
496 # =====
497 # C 方程装配
498 # =====
499 def assemble_C_var(N, dr, dt, Df, R0, km, out=None):
500     """同 `assemble_T_var`: `out=(a,b,c)` 时原地覆写。"""
501     if out is None:
502         a = np.zeros(N + 1); b = np.zeros(N + 1); c = np.zeros(N + 1)
503     else:
504         a, b, c = out
505     if _HAS_NUMBA:
506         _assemble_C_nb(N, dr, dt,
507             np.ascontiguousarray(Df, dtype=np.float64),
508             R0, km, a, b, c)
509     return a, b, c
510 r = np.arange(N + 1) * dr
511 V0 = dr * dr / 4.0
512 b[0] = (V0 / dt + Df[0] * dr / dr) / V0
513 c[0] = (-Df[0] * dr / dr) / V0
514 a[0] = 0.0
515 ii = np.arange(1, N)
516 rl, rr = r[ii] - dr / 2.0, r[ii] + dr / 2.0
517 b[ii] = r[ii] / dt + (Df[ii] * rr + Df[ii - 1] * rl) / dr ** 2

```

```

518     a[ii] = -Df[ii - 1] * rl / dr ** 2
519     c[ii] = -Df[ii] * rr / dr ** 2
520     VN = dr * (R0 - dr / 4.0) / 2.0
521     b[N] = VN / dt + Df[N - 1] * (R0 - dr / 2.0) / dr + km * R0
522     a[N] = -Df[N - 1] * (R0 - dr / 2.0) / dr
523     c[N] = 0.0
524     return a, b, c
525
526
527 def rhs_C_var(N, dr, dt, R0, km, Cold, cinf, out=None):
528     d = np.zeros(N + 1) if out is None else out
529     if _HAS_NUMBA:
530         _rhs_C_nb(N, dr, dt, R0, km,
531                 np.ascontiguousarray(Cold, dtype=np.float64),
532                 float(cinf), d)
533     return d
534     r = np.arange(N + 1) * dr
535     d[0] = Cold[0] / dt
536     d[1:N] = r[1:N] * Cold[1:N] / dt
537     VN = dr * (R0 - dr / 4.0) / 2.0
538     d[N] = VN * Cold[N] / dt + km * R0 * cinf
539     return d
540
541
542 # =====
543 # 单步: 双层迭代 (逻辑与 v2 **一行未改**)
544 # =====
545 def step_coupled(N, dr, dtT, dtC, subT, subC, R0, h, km,
546                 T, C, Tenv_fn, Cenv_fn, t0,
547                 omega=0.7, tol=1e-9, maxit=30, max_outer=20, tol_outer=1e-8,
548                 mode='coupled', lag_props=True, verbose=False):
549     """推进一个输出步。返回 (T_new, C_new, info)。
550
551     右端纪律: 每轮外层的 T、C 求解**都从 n 层解出发**；C 的每个子步从**本子步起点**出发。
552     """
553     Told, Cold = T.copy(), C.copy()
554     Tk, Ck = T.copy(), C.copy()
555     res_hist = []
556     inner_tot = 0
557     res_T = res_C = None
558     flg = 'ok'
559     outer_it = 0
560     n_outer = 1 if lag_props else max_outer
561
562     if lag_props:
563         if mode == 'frozen':
564             rho0 = np.full(N + 1, float(rho_of(DEF2['C0'])))
565             cp0 = np.full(N + 1, float(cp_of(DEF2['C0'])))
566             k0 = np.full(N + 1, float(k_of(DEF2['C0'])))
567             D0 = np.full(N + 1, float(D_of(DEF2['C0'], DEF2['T0'])))
568         elif mode == 'decoupled':
569             rho0, cp0, k0 = rho_of(Cold), cp_of(Cold), k_of(Cold)
570             D0 = 2.4e-3 * np.exp(-0.45 / np.maximum(Cold, 1e-9))
571         else:
572             rho0, cp0, k0 = rho_of(Cold), cp_of(Cold), k_of(Cold)
573             D0 = D_of(Cold, Told)
574
575     for k in range(1, n_outer + 1):
576         outer_it = k
577         if lag_props:
578             rho, cp, kk = rho0, cp0, k0
579         elif mode == 'frozen':
580             rho = np.full(N + 1, float(rho_of(DEF2['C0'])))
581             cp = np.full(N + 1, float(cp_of(DEF2['C0'])))
582             kk = np.full(N + 1, float(k_of(DEF2['C0'])))
583         else:
584             rho, cp, kk = rho_of(Ck), cp_of(Ck), k_of(Ck)
585
586         Tprev = Tk.copy()
587         Tcur = Told.copy()
588         for s in range(subT):
589             ts = t0 + (s + 1) * dtT
590             kf = np.full(N, float(k_of(DEF2['C0']))) if mode == 'frozen' else facek_of(Ck)

```

```

591     a, b, c = assemble_T_var(N, dr, dtT, R0, h, rho, cp, kk, kf=kf)
592     d = rhs_T_var(N, dr, dtT, R0, h, rho, cp, Tcur, float(Tenv_fn(ts)))
593     Tcur = thomas(a, b, c, d)
594     Tk = Tcur
595
596     if lag_props:
597         Dnode = D0
598     elif mode == 'frozen':
599         Dnode = np.full(N + 1, float(D_of(DEF2['C0'], DEF2['T0'])))
600     elif mode == 'decoupled':
601         Dnode = 2.4e-3 * np.exp(-0.45 / np.maximum(Ck, 1e-9))
602     else:
603         Dnode = D_of(Ck, Tk)
604
605     Cit = Ck.copy()
606     for s in range(subC):
607         tt = t0 + (s + 1) * dtC
608         cinf = float(Cenv_fn(tt))
609         Cbase = Cit.copy()
610         base = rhs_C_var(N, dr, dtC, R0, km, Cbase, cinf)
611         for it in range(1, maxit + 1):
612             if lag_props:
613                 if mode == 'frozen':
614                     Df = np.full(N, float(D_of(DEF2['C0'], DEF2['T0'])))
615                 elif mode == 'decoupled':
616                     Df = faceD_of(Cbase, Tk, 1.0, useT=False)
617                 else:
618                     Df = faceD_of(Cbase, Told, float(D_FAC), useT=True)
619             else:
620                 if mode == 'frozen':
621                     Df = np.full(N, float(D_of(DEF2['C0'], DEF2['T0'])))
622                 elif mode == 'decoupled':
623                     Df = faceD_of(Cit, Tk, 1.0, useT=False)
624                 else:
625                     Df = faceD_of(Cit, Tk, float(D_FAC), useT=True)
626             a, b, c = assemble_C_var(N, dr, dtC, Df, R0, km)
627             Ctry = thomas(a, b, c, base)
628             Cnew = omega * Ctry + (1.0 - omega) * Cit
629             rr = float(np.max(np.abs(Cnew - Cit))) / max(1.0, float(np.max(np.abs(Cnew))))
630             Cit = Cnew
631             inner_tot += 1
632             if rr < tol:
633                 break
634     Ck = Cit
635
636     res_T = float(np.max(np.abs(Tk - Tprev)))
637     res_hist.append(res_T)
638     if k > 1:
639         res_C = float(np.max(np.abs(Ck - Cprev)))
640         if res_T < tol_outer and res_C < tol_outer:
641             break
642     Cprev = Ck.copy()
643
644     if outer_it >= max_outer:
645         flg = 'outer_max'
646     if verbose:
647         print("      outer res_T hist:", " ".join(f"{v:.2e}" for v in res_hist[:8]),
648               "...", f"{res_hist[-1]:.2e}" if len(res_hist) > 8 else "")
649     return Tk, Ck, dict(outer_it=outer_it, inner_tot=inner_tot, res_T=res_T,
650                        res_C=res_C, flag=flg, res_hist=res_hist)
651
652
653 # =====
654 # 单时步: IMEX 交替推进 (主力格式) *B 优化: 装配缓冲预分配并原地复用
655 # =====
656 def step_imex(N, dr, h, n_sub, R0, hc, km, T, C, Tenv_fn, Cenv_fn, t0,
657              omega=0.7, tol=1e-10, maxit=30, mode='coupled'):
658     """把**一个输出步**细分为 n_sub 个内部步 (内部步长 h = dt_out/n_sub),
659     每个内部步内**更新物性**并交替推进 T、C:
660
661     ① 物性 ← 当前 T, C (本内部子步起点)
662     ② 解 T (1 步 h, 物性冻结 ⇒ 线性三对角)
663     ③ D ← 新的 T 与当前 C

```

```

664         ④ 解 C (Picard 线性化, 1 步 h)
665
666     ★B 优化仅涉及**缓冲复用** (a/b/c/d 每输出步建一次),
667     迭代次数、判据、松弛、物性更新时机**全部不变**。
668     """
669     Tcur, Ccur = T.copy(), C.copy()
670     inner_tot = 0
671     res_max = 0.0
672     # 预分配缓冲 (每输出步一次)
673     bufT = (np.zeros(N + 1), np.zeros(N + 1), np.zeros(N + 1))
674     bufC = (np.zeros(N + 1), np.zeros(N + 1), np.zeros(N + 1))
675     dT = np.zeros(N + 1)
676     dC = np.zeros(N + 1)
677     for s in range(n_sub):
678         ts = t0 + (s + 1) * h
679         tinf = float(Tenv_fn(ts))
680         cinf = float(Cenv_fn(ts))
681
682         # ① 物性
683         if mode == 'frozen':
684             rho = np.full(N + 1, float(rho_of(DEF2['C0'])))
685             cp = np.full(N + 1, float(cp_of(DEF2['C0'])))
686             kk = np.full(N + 1, float(k_of(DEF2['C0'])))
687         else:
688             rho, cp, kk = rho_of(Ccur), cp_of(Ccur), k_of(Ccur)
689
690         # ② T (全隐式, 物性冻结): 界面 k 取沿 C 的积分平均 (修正口径)
691         kf = np.full(N, float(k_of(DEF2['C0']))) if mode == 'frozen' else facek_of(Ccur)
692         a, b, c = assemble_T_var(N, dr, h, R0, hc, rho, cp, kk, out=bufT, kf=kf)
693         d = rhs_T_var(N, dr, h, R0, hc, rho, cp, Tcur, tinf, out=dT)
694         Tcur = thomas(a, b, c, d)
695
696         # ③ D 基准 (仅 frozen 需常数基准)
697         if mode == 'frozen':
698             Dbase = np.full(N + 1, float(D_of(DEF2['C0'], DEF2['T0'])))
699
700         # ④ C (Picard 线性化: D 随迭代中的 C 更新, T 固定): 界面 D 取沿 C 的积分平均 (修正口径)
701         base = rhs_C_var(N, dr, h, R0, km, Ccur, cinf) # 每子步更新 ==> 不复用缓冲
702         Cit = Ccur.copy()
703         for it in range(1, maxit + 1):
704             if mode == 'frozen':
705                 Df = np.full(N, Dbase[0])
706             elif mode == 'decoupled':
707                 Df = faceD_of(Cit, Tcur, 1.0, useT=False)
708             else:
709                 Df = faceD_of(Cit, Tcur, float(D_FAC), useT=True)
710             a, b, c = assemble_C_var(N, dr, h, Df, R0, km, out=bufC)
711             Ctry = thomas(a, b, c, base)
712             Cnew = omega * Ctry + (1.0 - omega) * Cit
713             rr = float(np.max(np.abs(Cnew - Cit))) / max(1.0, float(np.max(np.abs(Cnew))))
714             Cit = Cnew
715             inner_tot += 1
716             res_max = max(res_max, rr)
717             if rr < tol:
718                 break
719         Ccur = Cit
720
721     return Tcur, Ccur, dict(inner_tot=inner_tot, nsub=n_sub,
722                             res_max=res_max, outer_it=1, res_T=None, res_C=None,
723                             flag='ok', res_hist=[])
724
725
726     # =====
727     # 驱动 (逻辑与 v2 **一行未改**)
728     # =====
729     def run_q2(N=80, dr=2.5e-4, dt_out=1.0, nsteps=10800,
730               Tenv_fn=None, Cenv_fn=None, subT=32, subC=10, n_sub=None,
731               omega=0.7, tol=1e-10, maxit=30, max_outer=20, tol_outer=1e-8,
732               mode='coupled', scheme='imex', lag_props=True,
733               cols=None, snap_at=(), progress=0, verbose=False):
734         """Q2 主力求解 (单遍运行, 同时收集整表与快照)。返回 (res, tableT, tableC)。"""
735         R0, h, km = DEF2['R0'], DEF2['h'], DEF2['km']
736         dtT, dtC = dt_out / float(subT), dt_out / float(subC)

```

```

737 nsub = n_sub if n_sub is not None else subT
738 hh = dt_out / float(nsub)
739 ncol = 0 if cols is None else len(cols)
740 tableT = np.zeros((nsteps, ncol)) if (cols is not None) else None
741 tableC = np.zeros((nsteps, ncol)) if (cols is not None) else None
742
743 T = np.full(N + 1, DEF2['T0'])
744 C = np.full(N + 1, DEF2['C0'])
745 Tsnap, Csnap = {0: T.copy()}, {0: C.copy()}
746
747 stat = dict(outer_tot=0, outer_max=0, inner_tot=0, nsub=0,
748             resT_max=0.0, resC_max=0.0, flags={}, res_hist_last=None)
749 t0 = time.time()
750 for n in range(1, nsteps + 1):
751     if scheme == 'imex':
752         T, C, info = step_imex(N, dr, hh, nsub, R0, h, km, T, C,
753                               Tenv_fn, Cenv_fn, (n - 1) * dt_out,
754                               omega, tol, maxit, mode)
755         stat['resC_max'] = max(stat['resC_max'], info.get('res_max', 0.0))
756     else:
757         T, C, info = step_coupled(N, dr, dtT, dtC, subT, subC, R0, h, km,
758                                   T, C, Tenv_fn, Cenv_fn, (n - 1) * dt_out,
759                                   omega, tol, maxit, max_outer, tol_outer,
760                                   mode, lag_props=lag_props,
761                                   verbose=(verbose and n <= 2))
762         stat['resT_max'] = max(stat['resT_max'], info['res_T'] or 0.0)
763         stat['resC_max'] = max(stat['resC_max'], info['res_C'] or 0.0)
764         stat['outer_tot'] += info.get('outer_it', 1)
765         stat['outer_max'] = max(stat['outer_max'], info.get('outer_it', 1))
766         stat['inner_tot'] += info['inner_tot']
767         stat['nsub'] += 1
768         stat['res_hist_last'] = info.get('res_hist')
769         if info.get('flag', 'ok') != 'ok':
770             stat['flags'][info['flag']] = stat['flags'].get(info['flag'], 0) + 1
771         if cols is not None:
772             tableT[n - 1, :] = T[cols]
773             tableC[n - 1, :] = C[cols]
774         if n in snap_at:
775             Tsnap[n], Csnap[n] = T.copy(), C.copy()
776         if progress and n % progress == 0:
777             el = time.time() - t0
778             print(f"    ... t={n*dt_out:.0f}s 已用 {el:.1f}s "
779                   f"预计剩余 {el/n*(nsteps-n):.1f}s", flush=True)
780
781 res = dict(N=N, dr=dr, dt_out=dt_out, dtT=hh, dtC=hh, subT=subT, subC=subC,
782            n_sub=nsub, scheme=scheme, mode=mode,
783            T_end=T.copy(), C_end=C.copy(),
784            T_snap=TSnap, C_snap=Csnap, stats=stat, wall=time.time() - t0)
785 return res, tableT, tableC

```

B.6 问题三求解

```

1  # -*- coding: utf-8 -*-
2  """
3  A 题 Q3 主力求解: 事件驱动终止 + 60 s 输出 + 达标定位 + 上界熔断
4  =====
5  口径 (严格依 Q3 计划文档《A_Q3求解思路与 workflow》v3):
6  • 物性: 附录 3 (N7 - N10), 全程无阶段切换
7  • 空间:  $\Delta r = 0.25$  mm (N=80 区间 / 81 节点)
8  • 内部步: 1/32 s (先按基线, 与 Q2 一致)
9  • 格式: IMEX 交替推进 (物性每内部子步更新)
10 • 边界:  $t \leq 14400$  s 附件1 线性插值;  $t > 14400$  s 取  $T_\infty = 50.00$  °C、 $C_\infty = 0.04999$ 
11 • 判据: **全局各处  $C < 0.15$ ** (严格小于; 最慢点=中心)
12 • 输出: 每 60 s, 21 个距离点 (0~2 cm, 每 0.1 cm)
13 • 核: **q3_core (A/B 优化的 B+C 核)**; 禁止 `__orig`
14
15 终止条件 (P1):
16 • 达标: 全域  $\max_r C < 0.15 \Rightarrow$  记录并**停止**
17 • 熔断: t 超过 --max-hours (默认 120 h) 仍未达标  $\Rightarrow$  停止并报警
18
19 达标定位 (P3):
20 • 以 60 s 输出网格记录; 达标判定在**每个内部步**检查 (比输出更密)
21 • 记录"首次达标的内部时刻" t_end_inner 与"首个达标输出点" t_out

```

```

22 用法:
23  python q3_solver.py                                # 正式: 事件驱动, 上限 120 h
24  python q3_solver.py --steps 10800                  # 回归: 固定 10800 步 (用于与 result2 比对)
25  python q3_solver.py --out ../tmp_test.xlsx          # 指定输出
26  python q3_solver.py --hours 1 --no-assert           # 快速冒烟
27  """
28
29  import sys
30  import os
31  import time
32  import argparse
33
34  try:
35      sys.stdout.reconfigure(encoding='utf-8')
36  except Exception:
37      pass
38
39  for _v in ('OMP_NUM_THREADS', 'MKL_NUM_THREADS', 'OPENBLAS_NUM_THREADS',
40            'NUMEXPR_NUM_THREADS', 'VECLIB_MAXIMUM_THREADS'):
41      os.environ.setdefault(_v, '1')
42
43  import numpy as np
44
45  HERE = os.path.dirname(os.path.abspath(__file__))
46  sys.path.insert(0, HERE)
47  sys.path.insert(0, os.path.join(os.path.dirname(HERE), 'Q2', 'code'))
48
49  import openpyxl                                # noqa: E402
50  from openpyxl import Workbook                  # noqa: E402
51  import q2_core_c as base                        # noqa: E402
52  import q3_core as q3                          # noqa: E402
53
54  WS = os.path.dirname(os.path.dirname(os.path.dirname(HERE))) # Work_Space 根
55  ATT1 = os.path.join(WS, '10_赛题', 'A题', '附件', '附件1.xlsx')
56
57  # ----- 口径常量 -----
58  N = 80
59  DR = 2.5e-4
60  DT_OUT = 60.0                                # Q3 输出步长 (题面: 每隔 60 s)
61  # ★ 内部步长必须与 Q2 已验证值一致 (1/32 s) —— 不得因输出间隔变大而放大内部步!
62  H_INNER = 1.0 / 32.0
63  NSUB = int(round(DT_OUT / H_INNER))           # = 1920 个子步 / 输出步
64  HH = DT_OUT / NSUB                          # 实际内部步长 (≈1/32 s, 由整除保证)
65  R0 = base.DEF2['R0']
66  HC = base.DEF2['h']
67  KM = base.DEF2['km']
68  C_CRIT = 0.15                                # 达标判据 (N18: 严格小于)
69  C_WATCH = 0.16                              # 进入"细粒度达标定位"的预警阈值
70  C_ENV_TAIL = 0.04999                        # H6 / N20
71  T_ENV_TAIL = 50.00                          # H6 / N19
72  COLS = np.arange(0, N + 1, 4)               # 0, 0.1, ..., 2.0 cm ⇒ 21 列
73  NCOL = len(COLS)
74  NC = N + 1
75
76  LOG = os.path.join(HERE, 'q3_run_log.txt')
77  BUF = []
78
79
80  def say(s=''):
81      print(s)
82      BUF.append(s)
83
84
85  def load_att1():
86      wb = openpyxl.load_workbook(ATT1, data_only=True)
87      ws = wb.active
88      rows = [r for r in ws.iter_rows(values_only=True)][1:]
89      d = [r for r in rows if r[0] is not None]
90      t = np.array([float(r[0]) for r in d])
91      T = np.array([float(r[1]) for r in d])
92      C = np.array([float(r[2]) for r in d])
93      wb.close()
94      return t, T, C

```

```

95
96
97 def main():
98     ap = argparse.ArgumentParser()
99     ap.add_argument('--steps', type=int, default=0,
100                    help='固定输出步数 (>0 时忽略事件驱动, 用于回归/冒烟)')
101     ap.add_argument('--hours', type=float, default=0.0,
102                    help='固定时程上限 (h); 与 --steps 二选一')
103     ap.add_argument('--max-hours', type=float, default=120.0,
104                    help='事件驱动模式的熔断上限 (h), 默认 120')
105     ap.add_argument('--out', type=str, default='')
106     ap.add_argument('--no-assert', action='store_true')
107     args = ap.parse_args()
108
109     t_a1, T_a1, C_a1 = load_att1()
110     ts_env, Tenv_v, Cenv_v = q3.env_tables_from_att1(t_a1, T_a1, C_a1)
111
112     # 决定运行模式
113     if args.steps > 0:
114         nsteps_max = args.steps
115         mode_txt = '固定 %d 步 (%.2f h)' % (args.steps, args.steps * DT_OUT / 3600.0)
116     elif args.hours > 0:
117         nsteps_max = int(round(args.hours * 3600.0 / DT_OUT))
118         mode_txt = '固定 %.2f h' % args.hours
119     else:
120         nsteps_max = int(round(args.max_hours * 3600.0 / DT_OUT))
121         mode_txt = '事件驱动 (熔断 %.0f h)' % args.max_hours
122
123     out = args.out if args.out else os.path.join(HERE, 'result3_raw.xlsx')
124
125     say('=' * 70)
126     say('Q3 主力求解')
127     say('=' * 70)
128     say(' 物性          附录3 全程 (N7-N10)')
129     say(' 空间          Δr=%.2f mm (N=%d 区间 / %d 节点)' % (DR * 1e3, N, NC))
130     say(' 输出步          %.1f s 内部子步 %d (内部步长 %.6f s)' % (DT_OUT, NSUB, HH))
131     say(' 判据          全域 C < %.2f kg/kg (严格小于, 中心为最慢点)' % C_CRIT)
132     say(' 运行模式        %s' % mode_txt)
133     say(' 使用核          q3_core (A/B 优化: 环境查表 + 子步下沉 njit)')
134     say(' 输出          21 列 (0~2 cm, 每 0.1 cm) ⇒ 文件 %s' % os.path.basename(out))
135     say('-' * 70)
136
137     T = np.full(NC, base.DEF2['T0'])
138     C = np.full(NC, base.DEF2['C0'])
139
140     rows = [] # 每 60 s 一行 (仅存 21 个输出点)
141     inner_tot_all = 0
142     res_max_all = 0.0
143     t_end_inner = None # 首个"全域达标"的内部时刻 (s)
144     t_end_i_step = -1
145     reached = False
146     t0 = time.time()
147
148     for n in range(1, nsteps_max + 1):
149         t_start = (n - 1) * DT_OUT
150         T_prev, C_prev = T.copy(), C.copy()
151         # ★ 批量推进**一整输出步** (1920 个子步), 一次 njit 调用
152         # —— 这是 A 优化 (子步循环下沉) 的收益所在
153         T, C, info = q3.step_imex_fast(
154             N, DR, HH, NSUB, R0, HC, KM, T, C,
155             ts_env, Tenv_v, Cenv_v, t_start)
156         inner_tot_all += info['inner_tot']
157         res_max_all = max(res_max_all, info['res_max'])
158         rows.append(C[COLS].copy())
159
160     cmax = float(np.max(C))
161     if cmax < C_CRIT:
162         # ★ 达标: 在本输出步内**二分细定位** (P3)
163         reached = True
164         lo, hi = 1, NSUB
165         while lo < hi:
166             mid = (lo + hi) // 2
167             _, Ca, _ = q3.step_imex_fast(

```

```

168         N, DR, HH, mid, R0, HC, KM, T_prev, C_prev,
169         ts_env, Tenv_v, Cenv_v, t_start)
170     if float(np.max(Ca)) < C_CRIT:
171         hi = mid
172     else:
173         lo = mid + 1
174     t_end_inner = t_start + lo * HH
175     t_end_i_step = lo - 1
176     # 用精确定位所得状态覆盖末行 (保证末行与 t_end 严格对应)
177     _, Cx, _ = q3.step_imex_fast(
178         N, DR, HH, lo, R0, HC, KM, T_prev, C_prev,
179         ts_env, Tenv_v, Cenv_v, t_start)
180     rows[-1] = Cx[COLS].copy()
181     C = Cx
182     break
183 if cmax < C_WATCH:
184     say(' [预警] t=%1f h, 全域最大 C=%6f 已接近判据 %2f'
185         % (n * DT_OUT / 3600.0, cmax, C_CRIT))
186 if n % max(1, nsteps_max // 20) == 0:
187     frac = 100.0 * n / nsteps_max
188     say(' [%5.1f%%] t=%8.1f s (%6.2f h) C(0)=%6f T(0)=%4f wall=%1fs'
189         % (frac, n * DT_OUT, n * DT_OUT / 3600.0,
190            C[0], T[0], time.time() - t0))
191
192 wall = time.time() - t0
193 say('-' * 70)
194 say(' 完成: %d 个输出步, 总墙钟 %1f s (%1f min)' % (len(rows), wall, wall / 60))
195 say(' 内部迭代累计 %d 次; 内层最大残差 %.3e' % (inner_tot_all, res_max_all))
196 say(' 末端 T(0)=%4f °C T(R)=%4f °C C(0)=%6f C(R)=%6f'
197     % (T[0], T[-1], C[0], C[-1]))
198 if reached:
199     say(' ★ 达标: 全域 C<%.2f 首次出现于 t = %.3f s = %.4f h (第 %d 个内部子步)'
200         % (C_CRIT, t_end_inner, t_end_inner / 3600.0, t_end_i_step + 1))
201 else:
202     say(' △ 未达标: 运行至 %d 步 (%2f h) 仍未全域 C<%.2f'
203         % (len(rows), len(rows) * DT_OUT / 3600.0, C_CRIT))
204
205 # ----- 写盘 -----
206 # ★ 010 口径修正:
207 # ① 距离列头改为**数值** (与 result1 / result4 / 附件3 模板一致);
208 # ② 达标时**末行时间列写精确 t_end** (与 result4 同款), 而非输出步序号
209 # —— 末行数值本已由二分所得 t_end 状态覆盖, 此前仅时间标签未同步。
210 wb = Workbook(write_only=True)
211 ws = wb.create_sheet('Sheet1')
212 ws.append(['时间\到药材中心的距离'] + [round(c * DR * 100, 1) for c in COLS])
213 n_rows = len(rows)
214 for i, r in enumerate(rows):
215     t_row = (i + 1) * DT_OUT
216     if reached and i == n_rows - 1:
217         t_row = round(t_end_inner, 3)
218     ws.append([t_row] + [round(float(v), 4) for v in r])
219 wb.save(out)
220 say(' 已写出: %s' % out)
221
222 # ----- 断言 -----
223 # △ 断言口径说明 (Q3 相对 Q2 的必要修正):
224 # Q2 仅 3 h, 全域含水率单调不增成立;
225 # Q3 长时程下**表面会出现"局部增湿"式回升** (内部水分扩散补充超过表面脱除),
226 # 属**真实物理现象** (与文献所述"局部增湿"同机理)。
227 # 故断言改为: ①**中心点** (=最慢点, 决定达标) 严格单调不增;
228 # ②全场上界 = 初值; ③非负; ④达标后末行全域 < 判据。
229 if not args.no_assert:
230     arr = np.array(rows)
231     dC = np.diff(arr, axis=0) if len(arr) > 1 else np.zeros((1, NCOL))
232     # "最大值在中心"改用**容差判据**:
233     # 中心与邻点可能存在 1~数个 ulp 的差异 (写盘 round 到 4 位后即消失),
234     # 属数值可分辨范围内的小抖动; 差异超过 1e-6 才算真实违反。
235     TOL_MAXLOC = 1e-6
236     gap = arr.max(axis=1) - arr[:, 0]
237     n_beyond = int(np.sum(gap > TOL_MAXLOC))
238     chk = [
239         ('形状 (行数, %d)' % (NCOL + 1), arr.shape[1] == NCOL),
240         ('含水率非负 (全域)', bool(np.min(arr) >= 0.0)),

```



```

241         ('中心点单调不增', bool(np.all(dC[:, 0] <= 1e-9))),
242         ('含水率不超初值', bool(np.max(arr) <= base.DEF2['C0'] + 1e-4)),
243         ('全域最大值在中心 (容差 %.0e)' % TOL_MAXLOC, n_beyond == 0),
244     ]
245     if reached:
246         chk.append(('末行全域 < %.2f' % C_CRIT, bool(np.max(arr[-1]) < C_CRIT)))
247     say('-' * 70)
248     say(' [断言]')
249     for nm, ok in chk:
250         say('      %s %s' % ('PASS' if ok else 'FAIL', nm))
251     if not all(ok for _, ok in chk):
252         say(' *** 存在 FAIL ***')
253     say(' [留痕] 中心低于全场最大值的最大偏差 = %.3e (阈值 %.0e): '
254         '其中写盘 round 到 4 位后并列的行数 = %d / %d'
255         % (float(gap.max()), TOL_MAXLOC,
256           int(np.sum(np.argmax(arr, axis=1) == 0)),
257           len(arr)))
258     # 登记表面回升 (不判 FAIL, 仅留痕)
259     nup = int(np.sum(dC[:, -1] > 1e-9))
260     say(' [留痕] 表面 (r=%.1f cm) 出现回升的输出步: %d 个 (占 %.1f%%), 最大单次 %+.3e'
261         % (COLS[-1] * DR * 100, nup, 100.0 * nup / max(1, len(dC)),
262           float(dC[:, -1].max()) if len(dC) else 0.0))
263     say('      ==> 属"局部增湿"物理现象, 不影响达标判定 (最大值恒在中心)')
264
265     with open(LOG, 'w', encoding='utf-8') as f:
266         f.write('\n'.join(BUF) + '\n')
267
268
269 if __name__ == '__main__':
270     main()

```

```

1  # -*- coding: utf-8 -*-
2  """
3  A 题 Q3 求解核: 在 Q2 优化核 (q2_core_c) 之上实施 **A/B 优化**
4  =====
5  设计原则 (**零数值风险**):
6  * **复用** `q2_core_c` 的全部物性 / 界面 / 装配 / 右端 / 追赶法实现 (一行不改);
7  * 仅在**驱动层**做两项工程优化:
8  * **B | 环境边界改"预插值查表"**: 把 Python lambda 调用改为 njit 内的
9    线性插值查表 (附件1 仅 241 点, 二分查找 + 线性插值);
10 * **A | "子步循环下沉 JIT"**: 把 `for s in range(n_sub)` 整段循环下沉为
11   单个 njit 函数, 使 Python 层调用次数由 O(n_sub) 降为 O(1)。
12 * **数学表达式、迭代流程、tol/maxit/omega、物性公式、离散格式全部不变**；
13 * `fastmath=False` 保持不变 (保证与 B/C 版**逐位一致**)。
14
15 验证方式: `q3_verify_opt.py` —— 同输入下与 `q2_core_c.step_imex` 逐位比对。
16
17  △ 本文件仅新增"组装层", 不改变任何物理或离散口径。
18  """
19  import sys
20  import os
21  import time
22
23  try:
24      sys.stdout.reconfigure(encoding='utf-8')
25  except Exception:
26      pass
27
28  for _v in ('OMP_NUM_THREADS', 'MKL_NUM_THREADS', 'OPENBLAS_NUM_THREADS',
29            'NUMEXPR_NUM_THREADS', 'VECLIB_MAXIMUM_THREADS'):
30      os.environ.setdefault(_v, '1')
31
32  import numpy as np
33
34  sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
35  _Q2CODE = os.path.join(
36      os.path.dirname(os.path.dirname(os.path.abspath(__file__))), 'Q2', 'code')
37  sys.path.insert(0, _Q2CODE)
38
39  import q2_core_c as base # noqa: E402 (B+C 优化核, 逐字复用)
40  from q2_core_c import (DEF2, D_FAC, njit, _HAS_NUMBA, # noqa: E402
41                        _rho_nb, _cp_nb, _k_nb, _D_nb, _iface_nb,
42                        _faceD_nb, _facek_nb,

```

```

43         _assemble_T_nb, _rhs_T_nb, _assemble_C_nb, _rhs_C_nb,
44         _thomas_nb)
45
46 MODE_MAP = {'coupled': 0, 'frozen': 1, 'decoupled': 2}
47
48
49 # =====
50 # B | 环境边界的 njit 线性插值查表 (替代 Python lambda 调用)
51 # =====
52 @njit(cache=True, fastmath=False)
53 def _interpl_nb(t, ts, vs):
54     """分段线性插值 (njit): t<=ts[0] 取 vs[0]; t>=ts[-1] 取 vs[-1]。
55
56     与原 Python 版 `np.interp` / lambda 的语义一致;
57     端点外推取常值, 对应"恒温段延拓"口径 (H6 / N19 / N20)。
58     """
59     n = ts.shape[0]
60     if t <= ts[0]:
61         return vs[0]
62     if t >= ts[n - 1]:
63         return vs[n - 1]
64     lo = 0
65     hi = n - 1
66     while hi - lo > 1:
67         mid = (lo + hi) // 2
68         if ts[mid] <= t:
69             lo = mid
70         else:
71             hi = mid
72     w = (t - ts[lo]) / (ts[hi] - ts[lo])
73     return vs[lo] + w * (vs[hi] - vs[lo])
74
75
76 # =====
77 # A | 子步循环下沉: 一个输出步的全部内部子步 + Picard 迭代, 一次 njit 调用完成
78 # (数学流程与 q2_core_c.step_imex **逐行等价**)
79 # =====
80 @njit(cache=True, fastmath=False)
81 def _step_all_nb(N, dr, h, n_sub, R0, hc, km, T, C, t0,
82                 ts_env, Tenv_v, Cenv_v, d_fac, C0f, T0f,
83                 omega, tol, maxit, mode_flag):
84     NC = N + 1
85     Tcur = T.copy()
86     Ccur = C.copy()
87
88     aT = np.zeros(NC); bT = np.zeros(NC); cT = np.zeros(NC)
89     aC = np.zeros(NC); bC = np.zeros(NC); cC = np.zeros(NC)
90     dT = np.zeros(NC)
91     dC = np.zeros(NC)
92
93     inner_tot = 0
94     res_max = 0.0
95
96     for s in range(n_sub):
97         ts = t0 + (s + 1) * h
98         tinf = _interpl_nb(ts, ts_env, Tenv_v)      # ← B 优化
99         cinf = _interpl_nb(ts, ts_env, Cenv_v)      # ← B 优化
100
101         # ① 物性 (逐子步按局部状态更新; frozen 模式用 C0 常值)
102         if mode_flag == 1:
103             rho = np.full(NC, 650.0 + 128.0 * C0f)
104             cp = np.full(NC, 1450.0 + 2736.0 * C0f / (C0f + 1.0))
105             kk = np.full(NC, 0.21 + 0.38 * C0f / (C0f + 1.0))
106         else:
107             rho = _rho_nb(Ccur)
108             cp = _cp_nb(Ccur)
109             kk = _k_nb(Ccur)
110
111         # ② T (全隐式, 物性冻结); 界面 k 取沿 C 的积分平均 (修正口径)
112         if mode_flag == 1:
113             kf = np.full(N, 0.21 + 0.38 * C0f / (C0f + 1.0))
114         else:
115             kf = _facek_nb(Ccur)

```

```

116 _assemble_T_nb(N, dr, h, R0, hc, rho, cp, kk, aT, bT, cT, kf)
117 _rhs_T_nb(N, dr, h, R0, hc, rho, cp, Tcur, tinf, dT)
118 Tcur = _thomas_nb(aT, bT, cT, dT)
119
120 # ③④ C (Picard 线性化; D 随迭代中的 C 更新, T 固定)
121 # 界面 D 取**沿 C 的积分平均** (修正口径; 原调和平均仅对系数间断成立)
122 # △ base 在 numpy 版是**新数组** (不复用缓冲) ⇒ 此处同样 copy
123 _rhs_C_nb(N, dr, h, R0, km, Ccur, cinf, dC)
124 bse = dC.copy()
125 Cit = Ccur.copy()
126 for it in range(1, maxit + 1):
127     if mode_flag == 1:
128         Df = np.full(N, d_fac * 2.4e-3 * np.exp(-0.45 / C0f)
129                     * np.exp(-3850.0 / (T0f + 273.15)))
130     elif mode_flag == 2:
131         Df = _faceD_nb(Cit, Tcur, 1.0, False, 8)
132     else:
133         Df = _faceD_nb(Cit, Tcur, d_fac, True, 8)
134     _assemble_C_nb(N, dr, h, Df, R0, km, aC, bC, cC)
135     Ctry = _thomas_nb(aC, bC, cC, bse)
136     Cnew = np.empty(NC)
137     for i in range(NC):
138         Cnew[i] = omega * Ctry[i] + (1.0 - omega) * Cit[i]
139
140     # 残差: 与原式  $\max|Cnew-Cit| / \max(1, \max|Cnew|)$  等价
141     num = 0.0
142     den = 0.0
143     for i in range(NC):
144         v = abs(Cnew[i] - Cit[i])
145         if v > num:
146             num = v
147         w = abs(Cnew[i])
148         if w > den:
149             den = w
150     if den < 1.0:
151         den = 1.0
152     rr = num / den
153
154     Cit = Cnew
155     inner_tot += 1
156     if rr > res_max:
157         res_max = rr
158     if rr < tol:
159         break
160     Ccur = Cit
161
162 return Tcur, Ccur, inner_tot, res_max
163
164
165 def env_tables_from_att1(t_arr, T_arr, C_arr,
166                          t_tail=14400.0, T_tail=50.00, C_tail=0.04999):
167     """把附件1 时序转为 njit 可用的等长数组, 并实现**恒温段常值延拓**。
168
169     △ **延拓口径 (H6 / N19 / N20, 必须严格遵守)**:
170         t ≤ 14400 s : 附件1 原数据 (线性插值)
171         t > 14400 s : **常值** T∞=50.00 °C、C∞=0.04999
172
173     实现: 把 (t_tail, T_tail) 与 (t_tail+1, T_tail) 两点**追加在末尾**,
174     使 t>t_tail 时插值恒为常值 T_tail (‘_interp1_nb’ 对 t>=ts[-1] 返回末值)。
175
176     △ 注意: 附件1 末点本身是 (14400, 50.165)。本函数**不改动**该点,
177     故 t=14400 处取 50.165、t>14400 处取 50.00——与 Q2 主力口径
178     (‘np.interp(t, ...) if t <= t_arr[-1] else 50.00’) **完全一致**。
179     """
180     # △ 追加点的时间必须**严格大于** t_arr[-1], 否则产生重复节点,
181     # 二分插值会出现 0/0。取 +1e-6 s 的微小偏移: 对物理量无影响
182     # (内部步长 1/32 s, 远大于 1e-6 s)。
183     t0 = t_arr[-1]
184     ts = np.ascontiguousarray(
185         np.concatenate([t_arr, [t0 + 1e-6, t0 + 1.0]]), dtype=np.float64)
186     Tv = np.ascontiguousarray(
187         np.concatenate([T_arr, [T_tail, T_tail]]), dtype=np.float64)
188     Cv = np.ascontiguousarray(

```

```

189         np.concatenate([C_arr, [C_tail, C_tail]]), dtype=np.float64)
190     return ts, Tv, Cv
191
192
193 def step_imex_fast(N, dr, h, n_sub, R0, hc, km, T, C,
194                   ts_env, Tenv_v, Cenv_v, t0,
195                   omega=0.7, tol=1e-10, maxit=30, mode='coupled'):
196     """A/B 优化版的一输出步推进（接口与 q2_core_c.step_imex 兼容，
197     但环境边界以**查表数组**传入而非 lambda）。返回 (T, C, info)。"""
198     mf = MODE_MAP[mode]
199     Tk, Ck, inner_tot, res_max = _step_all_nb(
200         N, dr, h, n_sub, R0, hc, km,
201         np.ascontiguousarray(T, dtype=np.float64),
202         np.ascontiguousarray(C, dtype=np.float64),
203         float(t0), ts_env, Tenv_v, Cenv_v, float(D_FAC),
204         float(DEF2['C0']), float(DEF2['T0']),
205         float(omega), float(tol), int(maxit), int(mf))
206     return Tk, Ck, dict(inner_tot=inner_tot, nsub=n_sub, res_max=res_max,
207                        outer_it=1, res_T=None, res_C=None, flag='ok', res_hist=[])

```

B.7 问题四求解

```

1  # -*- coding: utf-8 -*-
2  """
3  A 题 Q4 主力求解（事件驱动 + 子步级二分 + 熔断）
4  =====
5  [SCALE] 正式求解（T3）：时程 0 -> t_dry（事件终止，熔断 240 h）；
6  N=160（Δξ=1/160）、n_sub=32（内部步 1/32 s）、t_corr=1（D6 方案 b）、
7  IMEX + Picard(ω=0.7, tol=1e-10, maxit=30)；预计墙钟 30-70 min。
8  授权：用户指令（2026-09-11）"不再有明确的正式放行门禁，可以直接跑主求解程序"；
9  申请单见《A_Q4求解思路与 workflow》§5.6（已按本轮指令更新为已批准）。
10  用法：python q4_solver.py          -> 只打印 [SCALE] 并退出
11       python q4_solver.py --go      -> 实际执行
12  输出：results/result4.xlsx + results/result4_data.csv；日志 logs/q4_solve_log.txt；
13       进度 logs/q4_solve_progress.txt（供后台轮询）。
14  口径：判据 max_r C < 0.15（严格）；result4 = 60 s x (0..1.9 cm + 药材表面)，
15       r>R(t) 留空（04）；t_dry 非 60 s 整数倍时追加精确行（MB-21/010）。
16  """
17  import sys, os, time
18
19  try:
20      sys.stdout.reconfigure(encoding='utf-8')
21  except Exception:
22      pass
23
24  HERE = os.path.dirname(os.path.abspath(__file__))
25  sys.path.insert(0, HERE)
26  WS = os.path.dirname(os.path.dirname(os.path.dirname(HERE)))
27
28  import numpy as np
29  import openpyxl
30  import q4_core as qc
31
32  RES = os.path.join(HERE, '..', 'results')
33  LOG = os.path.join(HERE, '..', 'logs')
34  os.makedirs(RES, exist_ok=True)
35  os.makedirs(LOG, exist_ok=True)
36  LOGF = os.path.join(LOG, 'q4_solve_log.txt')
37  PROG = os.path.join(LOG, 'q4_solve_progress.txt')
38
39  N = 160
40  NSUB = 32
41  DT_OUT = 1.0
42  CC = 0.15          # 判据（严格小于）
43  BREAK_H = 240.0    # 熔断（h）
44  R_TOL = 1e-12
45
46  rep = []
47
48
49  def log(s=''):
50      rep.append(s)
51  try:

```

```

52     print(s.encode('ascii', 'replace').decode('ascii'))
53 except Exception:
54     pass
55
56
57 def flush():
58     open(LOGF, 'w', encoding='utf-8').write('\n'.join(rep))
59
60
61 def prog(s):
62     open(PROG, 'w', encoding='utf-8').write(s)
63
64
65 log('[SCALE] Q4 formal solve (T3): horizon 0->t_dry (event, breaker %.0f h)' % BREAK_H)
66 log('[SCALE] N=%d, n_sub=%d, t_corr=1, IMEX+Picard(0.7,1e-10,30)' % (N, NSUB))
67 log('[SCALE] est wall 30-70 min; output result4.xlsx (60s x 21col+surface)')
68 if '--go' not in sys.argv:
69     log('[SCALE] --go not given -> exit (gate closed)')
70     flush()
71     sys.exit(0)
72
73 t_start = time.perf_counter()
74 prog('running: init')
75
76 # ---- 数据装载 ----
77 ts_env, Tv, Cv = None, None, None
78 att1 = os.path.join(WS, '10_赛题', 'A题', '附件', '附件1.xlsx')
79 wb = openpyxl.load_workbook(att1, read_only=True, data_only=True)
80 ws = wb[wb.sheetnames[0]]
81 # *修复: isinstance 过滤已剔除表头, 原 [1:] 误删 t=0 行 (首分钟环境钳位偏移)
82 rr = [(float(a), float(b), float(c)) for a, b, c in ws.iter_rows(values_only=True)
83        if a is not None and not isinstance(a, str)]
84 t1 = np.array([x[0] for x in rr])
85 T1v = np.array([x[1] for x in rr])
86 C1v = np.array([x[2] for x in rr])
87 ts_env, Tv, Cv = qc.env_tables_from_att1(t1, T1v, C1v)
88 log('att1: %d pts, T %.3f-%.3f C, Cinf %.5f-%.5f' %
89     (len(t1), T1v.min(), T1v.max(), C1v.min(), C1v.max()))
90 TENVM = float(T1v.max())
91
92 ts_R, Rv = qc.load_R_table(os.path.join(WS, '10_赛题', 'A题', '附件', '附件2.xlsx'))
93
94
95 def Rof(t):
96     return float(np.interp(t, ts_R, Rv))
97
98
99 # 模板表头
100 att3 = os.path.join(WS, '10_赛题', 'A题', '附件', '附件3', 'result4.xlsx')
101 wbt = openpyxl.load_workbook(att3, read_only=True)
102 hrow = list(wbt[wbt.sheetnames[0]].iter_rows(min_row=1, max_row=1, values_only=True))[0]
103 HEADER = ['时间\到药材中心的距离'] + [round(0.1 * i, 1) for i in range(20)] + ['药材表面']
104 log('template header ok (last col = %r)' % (hrow[-1],))
105
106 # ---- 主循环 ----
107 dxi = 1.0 / N
108 hsub = DT_OUT / NSUB
109 T = np.full(N + 1, qc.DEF4['T0'])
110 C = np.full(N + 1, qc.DEF4['C0'])
111 rows60 = [] # (t, T.copy(), C.copy())
112 t_dry = None
113 dry_field = None
114 prev_T, prev_C, prev_t = T.copy(), C.copy(), 0.0
115 max_steps = int(BREAK_H * 3600 / DT_OUT)
116 tot_it = 0
117 rm_all = 0.0
118 t_all = time.perf_counter()
119
120 k = 0
121 while k < max_steps:
122     k += 1
123     t0 = (k - 1) * DT_OUT
124     T, C, info = qc.step_imex4(N, dxi, hsub, NSUB, qc.DEF4['R0'], qc.DEF4['h'],

```

```

125                 qc.DEF4['km'], T, C, t0, ts_env, Tv, Cv,
126                 ts_R, Rv, t_corr=1)
127     tot_it += info['inner_tot']
128     if info['res_max'] > rm_all:
129         rm_all = info['res_max']
130     tk = k * DT_OUT
131     if k % 60 == 0:
132         rows60.append((tk, T.copy(), C.copy()))
133     if C.max() < CC:
134         t_dry = tk
135         dry_field = (T.copy(), C.copy())
136         log('event: max C < 0.15 at t=%d s (step %d)' % (tk, k))
137         break
138     prev_T, prev_C, prev_t = T.copy(), C.copy(), tk
139     if k % 600 == 0:
140         el = time.perf_counter() - t_all
141         msg = 'running: step %d/%d? t=%.1f h C(0)=%.4f wall=%.0f s it/step=%.1f' % (
142             k, max_steps, tk / 3600.0, C[0], el, tot_it / k)
143         prog(msg)
144         log(msg)
145         flush()
146
147 if t_dry is None:
148     log('BREAKER: no drying event within %.0f h -- abort' % BREAK_H)
149     flush()
150     sys.exit(2)
151
152 # ---- 子步级二分精化 (区间 [prev_t, t_dry]) ----
153 prog('running: bisection')
154 t_bad, T_bad, C_bad = prev_t, prev_T, prev_C
155 t_ok, T_ok, C_ok = float(t_dry), dry_field[0], dry_field[1]
156 nit = 0
157 while t_ok - t_bad > 1e-3:
158     mid = 0.5 * (t_ok + t_bad)
159     w = mid - t_bad
160     ns = max(1, int(round(w * NSUB / DT_OUT)))
161     Tm, Cm, info = qc.step_imex4(N, dxi, w / ns, ns, qc.DEF4['R0'], qc.DEF4['h'],
162                                 qc.DEF4['km'], T_bad, C_bad, t_bad,
163                                 ts_env, Tv, Cv, ts_R, Rv, t_corr=1)
164     tot_it += info['inner_tot']
165     if Cm.max() < CC:
166         t_ok, T_ok, C_ok = mid, Tm, Cm
167     else:
168         t_bad, T_bad, C_bad = mid, Tm, Cm
169     nit += 1
170 log('bisection: %d iters -> t_dry = %.3f s (%.4f h)' % (nit, t_ok, t_ok / 3600.0))
171
172 # ---- 组装 result4 行 ----
173 xis = np.array([j / N for j in range(N + 1)])
174 R60 = [Rof(r[0]) for r in rows60]
175 data_rows = [] # (t, [21 values or None])
176 for (tk, Tk, Ck) in rows60:
177     Rk = Rof(tk)
178     vals = []
179     for ci in range(20):
180         rst = 0.1 * ci / 100.0
181         if rst <= Rk + R_TOL:
182             vals.append(round(float(qc.interp_lagrange(xis, Ck, rst / Rk, 6)), 4))
183         else:
184             vals.append(None)
185     vals.append(round(float(Ck[-1]), 4))
186     data_rows.append((tk, vals))
187 # 精确 t_dry 行 (若非 60 s 整数倍)
188 Rd = Rof(t_ok)
189 if abs(t_ok - round(t_ok / 60.0) * 60.0) > 1e-6:
190     vals = []
191     for ci in range(20):
192         rst = 0.1 * ci / 100.0
193         if rst <= Rd + R_TOL:
194             vals.append(round(float(qc.interp_lagrange(xis, C_ok, rst / Rd, 6)), 4))
195         else:
196             vals.append(None)
197     vals.append(round(float(C_ok[-1]), 4))

```

```

198     data_rows.append((round(t_ok, 3), vals))
199     log('extra exact t_dry row appended (t=%.3f s)' % t_ok)
200 else:
201     log('t_dry is multiple of 60 s -> no extra row (MB-21 dedup)')
202
203 # ---- 断言 ----
204 sec = lambda: None
205 log('--- assertions ---')
206 ok_all = True
207 n_all = len(data_rows)
208 a1 = n_all == len(rows60) + (1 if abs(t_ok - round(t_ok / 60.0) * 60.0) > 1e-6 else 0)
209 log('1 shape: %d rows (expect %d) -> %s' % (n_all, len(rows60), 'PASS' if a1 else 'FAIL'))
210 ok_all &= a1
211 allv = [v for _, vv in data_rows for v in vv if v is not None]
212 a2 = min(allv) >= 0.0
213 log('2 C nonneg (min=%.4f) -> %s' % (min(allv), 'PASS' if a2 else 'FAIL'))
214 ok_all &= a2
215 a3 = max(allv) <= qc.DEF4['C0'] + 1e-9
216 log('3 C <= C0 (max=%.4f) -> %s' % (max(allv), 'PASS' if a3 else 'FAIL'))
217 ok_all &= a3
218 c0s = [vv[0] for _, vv in data_rows]
219 a4 = all(c0s[i] >= c0s[i + 1] - 1e-9 for i in range(len(c0s) - 1))
220 log('4 center monotone non-increasing -> %s' % ('PASS' if a4 else 'FAIL'))
221 ok_all &= a4
222 a5 = all(max([v for v in vv if v is not None]) - vv[0] <= 1e-6 for _, vv in data_rows)
223 log('5 max at center (tol 1e-6) -> %s' % ('PASS' if a5 else 'FAIL'))
224 ok_all &= a5
225 last = [v for v in data_rows[-1][1] if v is not None]
226 a6_raw = float(dry_field[1].max()) < CC # 原始全精度值 (二分构造保证严格小于)
227 a6_disp = max(last) <= CC + 5e-5 # 4 位小数显示可容许 0.1500 (Q3 表5 同款先例)
228 a6 = a6_raw and a6_disp
229 log('6 final row: raw max=%.8f (<0.15 strictly: %s); displayed max=%.6f -> %s'
230     % (float(dry_field[1].max()), a6_raw, max(last), 'PASS' if a6 else 'FAIL'))
231 ok_all &= a6
232 # T 压缩升温报告 (F-Q4-2: 不作断言, 如实报告)
233 Tmax = max(r[1].max() for r in rows60)
234 n_above = sum(int((r[1] > TENVM + 1e-9).sum()) for r in rows60)
235 log('T report: Tmax=%.4f C (env max %.3f); steps with T>env: %d (info only, F-Q4-2)'
236     % (Tmax, TENVM, n_above))
237 log('assertions overall: %s' % ('PASS' if ok_all else 'FAIL'))
238
239 # ---- 写 result4.xlsx + CSV ----
240 prog('running: writing result4')
241 out = os.path.join(RES, 'result4.xlsx')
242 wb = openpyxl.Workbook()
243 w = wb.active
244 w.title = 'Sheet1'
245 w.append(HEADER)
246 for (tk, vv) in data_rows:
247     w.append([tk] + vv)
248 wb.save(out)
249 csvp = os.path.join(RES, 'result4_data.csv')
250 with open(csvp, 'w', encoding='utf-8') as f:
251     f.write(','.join(str(h) for h in HEADER) + '\n')
252     for (tk, vv) in data_rows:
253         f.write(str(tk) + ',' + ','.join(' ' if v is None else ('%.4f' % v) for v in vv) + '\n')
254
255 wall = time.perf_counter() - t_all
256 log('--- summary ---')
257 log('t_dry = %.3f s = %.4f h' % (t_ok, t_ok / 3600.0))
258 log('rows=%d ; wall=%.0f s ; Picard iters=%d ; res_max=%.2e'
259     % (n_all, wall, tot_it, rm_all))
260 log('output: %s' % out)
261 log('R final=%.4f cm ; R(t_dry)=%.4f cm' % (Rv[-1] * 100, Rd * 100))
262 prog('done: t_dry=%.4f h' % (t_ok / 3600.0))
263 flush()
264 print('LOG-WRITTEN')

```

```

1 # -*- coding: utf-8 -*-
2 """
3 A 题 Q4 求解核: 物料坐标  $\xi=r/R(t)$  守恒形式 (MB-11 + L3-09)
4 =====

```

```

5 离散方案（元体平衡，全部推导见《A_Q4求解思路与 workflow》v2 §2.1）：
6 质量方程（ $\times R^2$  前的守恒形式）： $\xi \partial_t \tilde{C} = (1/R^2) \partial \xi (\xi D \partial \xi \tilde{C})$ 
7 内部： $\xi_j \Delta \xi dC_j/dt = (1/R^2) [\xi_{j+1/2} D_{j+1/2} \Delta C - \xi_{j-1/2} D_{j-1/2} \Delta C] / \Delta \xi$ 
8 中心： $dC_0/dt = 4 D_{1/2} (C_1 - C_0) / (R^2 \Delta \xi^2)$ 
9 表面： $V_N \xi dC_N/dt = -(1/R^2) \xi_{N-1/2} D_{N-1/2} \Delta C / \Delta \xi - (k_m/R)(C_N - C_\infty)$ 
10 能量方程（守恒形式）： $\xi \partial_t [\rho c_p T R^2] = \partial \xi (k \xi \partial \xi T)$ 
11 内部： $\xi_j \Delta \xi [(\rho c_p T)^{n+1} R_{n+1}^2 - (\rho c_p T)^n R_n^2] / h = [k \xi \Delta T \text{ 差}] / \Delta \xi$ 
12 中心： $[(\rho c_p T)^{n+1} R_{n+1}^2 - (\rho c_p T)^n R_n^2] / h = 4k_{1/2} (T_1 - T_0) / \Delta \xi^2$ 
13 （行已除以  $R_{n+1}^2$ ，避免大数标度）
14 表面： $V_N \xi [...] / h = -k \xi_{N-1/2} \Delta T / \Delta \xi + h R_{N+1} (T_\infty - T_N)$ 
15 固定域极限（ $R=R_0$ ,  $\Delta \xi = \Delta r / R_0$ ）下逐项退化为 Q1/Q2 格式（E3-g-② 复用检验的依据）。
16
17 时间推进：IMEX 交替（物性逐内部子步更新）；D6 取层口径由 t_corr 开关控制：
18 t_corr=0：纯滞后（pc_p 用  $C^n$ ）——Q2 同款
19 t_corr=1：预估-校正（C 解出后用 pc_p( $C^{n+1}$ ) 重解一次 T）——交付解 pc_p 取 n+1 层
20 R(t) 取当前子步两端时刻的线性插值（附件2，端点钳位=H7 末段常值外推）。
21
22 性能：全部热路径 njit (fastmath=False 保持可复现)；一个输出步的全部子步
23 一次 njit 调用完成（q3_core 的“子步下沉”范式）。
24 """
25 import sys, os, time
26
27 try:
28     sys.stdout.reconfigure(encoding='utf-8')
29 except Exception:
30     pass
31
32 for _v in ('OMP_NUM_THREADS', 'MKL_NUM_THREADS', 'OPENBLAS_NUM_THREADS',
33           'NUMEXPR_NUM_THREADS', 'VECLIB_MAXIMUM_THREADS'):
34     os.environ.setdefault(_v, '1')
35
36 import numpy as np
37
38 try:
39     from numba import njit
40     _HAS_NUMBA = True
41 except Exception:
42     _HAS_NUMBA = False
43     def njit(*a, **k):
44         if len(a) == 1 and callable(a[0]):
45             return a[0]
46         def deco(f):
47             return f
48         return deco
49
50 DEF4 = dict(R0=0.02, h=25.0, km=8.0e-7, T0=28.0, C0=2.55)
51 D_FAC = 1.0 # D0 灵敏度缩放因子（OAT 用）
52 CM = 273.15 # °C -> K
53
54
55 # =====
56 # 物性（附录4；T 内部用 °C，D 公式内转 K）。app3=True 时切换附录3（复用检验用）
57 # =====
58 @njit(cache=True, fastmath=False)
59 def _rho_nb(C, app3):
60     if app3:
61         return 650.0 + 128.0 * C
62     return 760.0 + 90.0 * C
63
64
65 @njit(cache=True, fastmath=False)
66 def _cp_nb(C, app3):
67     if app3:
68         return 1450.0 + 2736.0 * C / (C + 1.0)
69     return 1850.0 + 2150.0 * C / (C + 1.0)
70
71
72 @njit(cache=True, fastmath=False)
73 def _k_nb(C, app3):
74     if app3:
75         return 0.21 + 0.38 * C / (C + 1.0)
76     return 0.12 + 0.20 * C / (C + 1.0)
77

```



```

78
79 @njit(cache=True, fastmath=False)
80 def _D_nb(C, Tc, d_fac, app3):
81     c = C
82     if c < 1e-9:
83         c = 1e-9
84     if app3:
85         return d_fac * 2.4e-3 * np.exp(-0.45 / c) * np.exp(-3850.0 / (Tc + CM))
86     return d_fac * 4.2e-4 * np.exp(-0.30 / c) * np.exp(-3850.0 / (Tc + CM))
87
88
89 @njit(cache=True, fastmath=False)
90 def _iface_nb(f):
91     """界面值：距离加权调和平均（与 q2_core_c.iface 同式）。
92
93     △ 旧口径（仅保留作 S2 旧口径对照与新口径对照实验），主力已改用
94     _facek4_nb / _faceD4_nb（沿 C 的积分平均，2026-09-11 口径纠偏后）。
95     """
96     n = f.shape[0]
97     out = np.empty(n - 1)
98     for i in range(n - 1):
99         l = f[i]
100        r = f[i + 1]
101        s = l + r
102        if s > 0.0:
103            out[i] = 2.0 * l * r / s
104        else:
105            out[i] = 0.0
106    return out
107
108
109 @njit(cache=True, fastmath=False)
110 def _facek4_nb(Carr, app3):
111     """★修正口径（Q4 版）：界面导热系数取【沿 C 的积分平均】（解析式）。
112
113      $(k) = k_0 + k_1 \cdot \Delta(C - \ln(1+C)) / \Delta C$ ， $(k_0, k_1) = \text{附录4 } (0.12, 0.20) / \text{附录3 } (0.21, 0.38)$ 
114
115     依据：单元内稳态通量  $J = (1/\Delta r) \int k \, dC \Rightarrow$  界面系数=沿 C 积分平均。
116     调和平均只对“界面处系数间断”成立；Q4 尾部单胞内 D/k 为连续急变（同 Q3）。
117     模板：q2_core_c._facek_nb（只读复制，系数换附录4）。 $|\Delta C| \approx 0$  退化为节点值。
118     """
119     n = Carr.shape[0] - 1
120     out = np.empty(n)
121     for i in range(n):
122         a = Carr[i]
123         b = Carr[i + 1]
124         if abs(b - a) < 1e-14:
125             c = a
126             if app3:
127                 out[i] = 0.21 + 0.38 * c / (1.0 + c)
128             else:
129                 out[i] = 0.12 + 0.20 * c / (1.0 + c)
130         else:
131             ga = a - np.log(1.0 + a)
132             gb = b - np.log(1.0 + b)
133             if app3:
134                 out[i] = 0.21 + 0.38 * (gb - ga) / (b - a)
135             else:
136                 out[i] = 0.12 + 0.20 * (gb - ga) / (b - a)
137    return out
138
139
140 @njit(cache=True, fastmath=False)
141 def _faceD4_nb(Carr, Tarr, d_fac, useT, nsamp, app3):
142     """★修正口径（Q4 版）：界面扩散系数取【沿 C 的积分平均】（nsamp 点中矩采样）。
143
144      $(D) = [D_0 \cdot (e^{\{-\beta/C\}})_C] \cdot [e^{\{-3850/T\_face\_K\}}]$ ，
145      $(D_0, \beta) = \text{附录4 } (4.2e-4, 0.30) / \text{附录3 } (2.4e-3, 0.45)$ 
146
147     利用可分离性只需对 C 因子采样；T 取界面两侧均值的 T 因子（T 内部 °C，转 K）。
148     useT=False：仅 C 因子（decoupled，不乘 d_fac）。 $|\Delta C| \approx 0$  退化为节点值。
149     模板：q2_core_c._faceD_nb（只读复制，系数换附录4）。
150     """

```

```

151 n = Carr.shape[0] - 1
152 out = np.empty(n)
153 for i in range(n):
154     a = Carr[i]
155     b = Carr[i + 1]
156     lo = a if a < b else b
157     hi = b if a < b else a
158     if hi - lo < 1e-14:
159         c = lo if lo > 1e-9 else 1e-9
160         if app3:
161             fv = np.exp(-0.45 / c)
162         else:
163             fv = np.exp(-0.30 / c)
164     else:
165         s = 0.0
166         for q in range(nsamp):
167             x = lo + (hi - lo) * (q + 0.5) / nsamp
168             if x < 1e-9:
169                 x = 1e-9
170             if app3:
171                 s += np.exp(-0.45 / x)
172             else:
173                 s += np.exp(-0.30 / x)
174         fv = s / nsamp
175     if useT:
176         Tm = 0.5 * (Tarr[i] + Tarr[i + 1])
177         if app3:
178             out[i] = d_fac * 2.4e-3 * fv * np.exp(-3850.0 / (Tm + 273.15))
179         else:
180             out[i] = d_fac * 4.2e-4 * fv * np.exp(-3850.0 / (Tm + 273.15))
181     else:
182         if app3:
183             out[i] = 2.4e-3 * fv
184         else:
185             out[i] = 4.2e-4 * fv
186 return out
187
188 @njit(cache=True, fastmath=False)
189 def _thomas_nb(a, b, c, d):
190     n = b.shape[0]
191     cp = np.empty(n)
192     dp = np.empty(n)
193     cp[0] = c[0] / b[0]
194     dp[0] = d[0] / b[0]
195     for i in range(1, n):
196         m = b[i] - a[i] * cp[i - 1]
197         cp[i] = c[i] / m if i < n - 1 else 0.0
198         dp[i] = (d[i] - a[i] * dp[i - 1]) / m
199     x = np.empty(n)
200     x[n - 1] = dp[n - 1]
201     for i in range(n - 2, -1, -1):
202         x[i] = dp[i] - cp[i] * x[i + 1]
203     return x
204
205
206 @njit(cache=True, fastmath=False)
207 def _interp1_nb(t, ts, vs):
208     """分段线性插值，端点钳位（附件1/附件2 的常值延拓口径）。"""
209     n = ts.shape[0]
210     if t <= ts[0]:
211         return vs[0]
212     if t >= ts[n - 1]:
213         return vs[n - 1]
214     lo = 0
215     hi = n - 1
216     while hi - lo > 1:
217         mid = (lo + hi) // 2
218         if ts[mid] <= t:
219             lo = mid
220         else:
221             hi = mid
222     w = (t - ts[lo]) / (ts[hi] - ts[lo])
223

```

```

224     return vs[lo] + w * (vs[hi] - vs[lo])
225
226
227 # =====
228 # 一个输出步的全部子步 (下沉 njit)
229 # mode: 0=coupled 1=frozen(物性冻结于 C0) 2=decoupled(D 不含 T)
230 # flags: k_zero(绝热自检) fixR(R=R0) app3(附录3 物性)
231 # t_corr: 0=IMEX 滞后 1=预估-校正 (D6 方案 b)
232 # =====
233 @njit(cache=True, fastmath=False)
234 def _step_all_nb(N, dxi, hsub, n_sub, R0, hc, km, T, C, t0,
235                 ts_env, Tenv_v, Cenv_v, ts_R, Rv,
236                 d_fac, C0f, omega, tol, maxit,
237                 mode_flag, k_zero, fixR, app3, t_corr, hcv, L):
238     NC = N + 1
239     Tcur = T.copy()
240     Ccur = C.copy()
241     if hcv >= 0.0:
242         hc = hcv          # 绝热自检等场景的对流系数覆盖 (0 = 完全绝热)
243
244     aT = np.zeros(NC); bT = np.zeros(NC); cT = np.zeros(NC); dT = np.zeros(NC)
245     aC = np.zeros(NC); bC = np.zeros(NC); cC = np.zeros(NC); dC = np.zeros(NC)
246
247     xi = np.empty(NC)
248     xih = np.empty(N)          #  $\xi_{j+1/2}$ , j=0..N-1
249     for j in range(NC):
250         xi[j] = j * dxi
251     for j in range(N):
252         xih[j] = (j + 0.5) * dxi
253     V0 = dxi * dxi / 4.0      # 中心控制体 ( $\xi$  域)
254     VN = dxi * (1.0 - dxi / 4.0) / 2.0  # 表面控制体 ( $\xi$  域)
255
256     inner_tot = 0
257     res_max = 0.0
258
259     for s in range(n_sub):
260         t0s = t0 + s * hsub
261         t1s = t0s + hsub
262         tinf = _interpl_nb(t1s, ts_env, Tenv_v)
263         cinf = _interpl_nb(t1s, ts_env, Cenv_v)
264         if fixR:
265             R1 = R0
266             R0m = R0
267         else:
268             R1 = _interpl_nb(t1s, ts_R, Rv)
269             R0m = _interpl_nb(t0s, ts_R, Rv)
270         Rlsq = R1 * R1
271
272     # ---- ① 物性 (子步起点状态: frozen 用 C0 常值) ----
273     if mode_flag == 1:
274         rho = np.empty(NC); cp = np.empty(NC); kk = np.empty(NC)
275         for j in range(NC):
276             rho[j] = _rho_nb(C0f, app3)
277             cp[j] = _cp_nb(C0f, app3)
278             kk[j] = _k_nb(C0f, app3)
279     else:
280         rho = np.empty(NC); cp = np.empty(NC); kk = np.empty(NC)
281         for j in range(NC):
282             rho[j] = _rho_nb(Ccur[j], app3)
283             cp[j] = _cp_nb(Ccur[j], app3)
284             kk[j] = _k_nb(Ccur[j], app3)
285     if k_zero:
286         for j in range(NC):
287             kk[j] = 0.0
288
289     # ---- ② T (全隐式, 能量守恒形式) ----
290     # ★修正口径: 界面 k 取沿 C 积分平均 (k 只依赖 C, 直接由节点 C 求):
291     # k_zero (绝热自检) 在面系数上整体置零
292     kf = _facek4_nb(Ccur, app3)
293     if k_zero:
294         for j in range(N - 1 + 1):
295             kf[j] = 0.0
296     Told = Tcur.copy()      # D6 校正步需要旧状态 (防 Q2- $\alpha$  重复推进)

```

```

297 rho_o = rho.copy()
298 cp_o = cp.copy()
299 # 内部 1..N-1 (★r 空间标定, q2 同款; 固定域极限逐字一致)
300 # ★存储项口径 (E7b 能量核算 0.645 暴露、绝热判决实验证实后修正):
301 # 整行除以 R1 后, 老态项必须为  $\xi_j \cdot R0m^2 / R1$  (即  $rj0 = j \cdot drp \cdot (R0m/R1)^2$ ),
302 # 否则守恒形式  $d/dt[pcp \cdot T \cdot R^2]$  的几何压缩项被系统性减半 (fixR 时  $R0m=R1$  不可见)。
303 drp = R1 * dxi #  $\Delta r' = R1 \cdot \Delta \xi$  (新时层 r 间距)
304 for j in range(1, N):
305     rl = (j - 0.5) * drp
306     rr = (j + 0.5) * drp
307     rj1 = j * drp #  $r_j^{n+1}$ 
308     rj0 = j * drp * (R0m / R1) ** 2 #  $\xi_j \cdot R0m^2 / R1$  (老态体积元, 含  $R^2$  口径)
309     aT[j] = -kf[j - 1] * rl / (drp * drp)
310     cT[j] = -kf[j] * rr / (drp * drp)
311     bT[j] = (rho[j] * cp[j] * rj1 / hsub
312             + (kf[j] * rr + kf[j - 1] * rl) / (drp * drp))
313     dT[j] = rho[j] * cp[j] * Tcur[j] * rj0 / hsub
314 # 中心 0 (行除以 R1sq; 固定域极限下 = Q1 中心行)
315 bT[0] = rho[0] * cp[0] / hsub + 4.0 * kf[0] / (R1sq * dxi * dxi)
316 cT[0] = -4.0 * kf[0] / (R1sq * dxi * dxi)
317 dT[0] = rho[0] * cp[0] * Tcur[0] * (R0m * R0m) / (R1sq * hsub)
318 aT[0] = 0.0
319 # 表面 N
320 bT[N] = (VN * rho[N] * cp[N] * R1sq / hsub
321         + kf[N - 1] * xih[N - 1] / dxi + hc * R1)
322 aT[N] = -kf[N - 1] * xih[N - 1] / dxi
323 cT[N] = 0.0
324 dT[N] = (VN * rho[N] * cp[N] * Tcur[N] * R0m * R0m / hsub
325         + hc * R1 * tinf)
326 if L > 0.0:
327     # E8 潜热对照: 表面蒸发吸热源项 (显式, 用子步起点 C; q2_core_latent 同式)
328     dT[N] -= L * km * R1 * (Ccur[N] - cinf)
329     Tcur = _thomas_nb(aT, bT, cT, dT)
330
331 # ---- ③ C (Picard; 右端固定=存储项, Q1 P-1 纪律) ----
332 # 行标定: 内点 r 空间 (q2 同款), 中心/表面行保持守恒形式 (数学等价)
333 dC[0] = Ccur[0] / hsub
334 for j in range(1, N):
335     dC[j] = j * drp * Ccur[j] / hsub #  $r_j^{n+1} \cdot C^n / \Delta t$ 
336 dC[N] = VN * R1sq * Ccur[N] / hsub + km * R1 * cinf
337
338 # 右端固定 (Q1 P-1 纪律): bse 在 Picard 中不变
339 bse = dC.copy()
340
341 Cit = Ccur.copy()
342 for it in range(1, maxit + 1):
343     # ★修正口径: 界面 D 取沿 C 的积分平均 (8 点中矩); T 因子用界面均值 T
344     if mode_flag == 1:
345         # frozen: D(C0,T0) 常值 (面值=节点值, 调和/积分平均同)
346         Df = np.empty(N - 1 + 1)
347         dv = _D_nb(C0f, 28.0, d_fac, app3)
348         for j in range(N - 1 + 1):
349             Df[j] = dv
350     elif mode_flag == 2:
351         Df = _faceD4_nb(Cit, Tcur, 1.0, False, 8, app3) # decoupled: 仅 C 因子
352     else:
353         Df = _faceD4_nb(Cit, Tcur, d_fac, True, 8, app3) # coupled
354     # 中心行
355     bC[0] = 1.0 / hsub + 4.0 * Df[0] / (R1sq * dxi * dxi)
356     cC[0] = -4.0 * Df[0] / (R1sq * dxi * dxi)
357     # 内部行补 D (r 空间标定, q2 同款)
358     for j in range(1, N):
359         aC[j] = -Df[j - 1] * (j - 0.5) * drp / (drp * drp)
360         cC[j] = -Df[j] * (j + 0.5) * drp / (drp * drp)
361         bC[j] = j * drp / hsub + (Df[j] * (j + 0.5) * drp
362                                 + Df[j - 1] * (j - 0.5) * drp) / (drp * drp)
363     # 表面行补 D
364     bC[N] = VN * R1sq / hsub + Df[N - 1] * xih[N - 1] / dxi + km * R1
365     aC[N] = -Df[N - 1] * xih[N - 1] / dxi
366
367     Ctry = _thomas_nb(aC, bC, cC, bse)
368     Cnew = np.empty(NC)
369     num = 0.0

```

```

370     den = 0.0
371     for j in range(NC):
372         v = omega * Ctry[j] + (1.0 - omega) * Cit[j]
373         dv = v - Cit[j]
374         if dv < 0.0:
375             dv = -dv
376         if dv > num:
377             num = dv
378         av = v if v >= 0.0 else -v
379         if av > den:
380             den = av
381         Cnew[j] = v
382     if den < 1.0:
383         den = 1.0
384     rr = num / den
385     Cit = Cnew
386     inner_tot += 1
387     if rr > res_max:
388         res_max = rr
389     if rr < tol:
390         break
391     Ccur = Cit
392
393     # ---- ④ D6 方案 b: 用  $C^{n+1}$  的 pc_p 重解一次 T (交付解 pc_p 取 n+1 层) ----
394     # △ 右端存储项必须用【旧状态】( $T^n$ ,  $pc_p(C^n)$ ,  $R_n$ )——否则即 Q2- $\alpha$  重复推进
395     if t_corr == 1 and (not k_zero):
396         for j in range(NC):
397             if mode_flag == 1:
398                 rho[j] = _rho_nb(C0f, app3) # frozen: pc_p 保持冻结常量
399                 cp[j] = _cp_nb(C0f, app3) # (b/d 两侧口径一致, 防伪源项)
400             else:
401                 rho[j] = _rho_nb(Ccur[j], app3)
402                 cp[j] = _cp_nb(Ccur[j], app3)
403         kf = _facek4_nb(Ccur, app3)
404         if k_zero:
405             for j in range(N):
406                 kf[j] = 0.0
407         for j in range(1, N):
408             rl = (j - 0.5) * drp
409             rr = (j + 0.5) * drp
410             rj1 = j * drp
411             rj0 = j * drp * (R0m / R1) ** 2 # ★同预测步:  $\xi_j \cdot R0m^2 / R1$  (老态体积元)
412             aT[j] = -kf[j - 1] * rl / (drp * drp)
413             cT[j] = -kf[j] * rr / (drp * drp)
414             bT[j] = (rho[j] * cp[j] * rj1 / hsub
415                     + (kf[j] * rr + kf[j - 1] * rl) / (drp * drp))
416             dT[j] = rho_o[j] * cp_o[j] * Told[j] * rj0 / hsub
417         bT[0] = rho[0] * cp[0] / hsub + 4.0 * kf[0] / (R1sq * dxi * dxi)
418         cT[0] = -4.0 * kf[0] / (R1sq * dxi * dxi)
419         dT[0] = rho_o[0] * cp_o[0] * Told[0] * (R0m * R0m) / (R1sq * hsub)
420         bT[N] = (VN * rho[N] * cp[N] * R1sq / hsub
421                 + kf[N - 1] * xih[N - 1] / dxi + hc * R1)
422         aT[N] = -kf[N - 1] * xih[N - 1] / dxi
423         dT[N] = (VN * rho_o[N] * cp_o[N] * Told[N] * R0m * R0m / hsub
424                 + hc * R1 * tinf)
425         if L > 0.0:
426             # ★修正 (E8 缺陷): 校正步必须与预测步同样计入潜热源项,
427             # 否则 t_corr=1 下潜热效应被校正步整体抹除 (旧日志 max|dT|~2e-9 的根因)。
428             # 用  $C^{n+1}$  (当前 Ccur) 评估蒸发通量, 与对流项同时层。
429             dT[N] -= L * km * R1 * (Ccur[N] - cinf)
430             Tcur = _thomas_nb(aT, bT, cT, dT)
431
432     return Tcur, Ccur, inner_tot, res_max
433
434
435 def load_R_table(xlsx_path):
436     """读附件2 → (ts_R 秒, Rv 米)。端点钳位由 _interp1_nb 承担 (H7 末段常值)。"""
437     import openpyxl
438     wb = openpyxl.load_workbook(xlsx_path, read_only=True, data_only=True)
439     ws = wb[wb.sheetnames[0]]
440     rows = list(ws.iter_rows(values_only=True))
441     data = [(float(r[0]), float(r[1])) for r in rows[1:] if r[0] is not None]
442     ts = np.ascontiguousarray([d[0] for d in data], dtype=np.float64)

```

```

443     Rv = np.ascontiguousarray([d[1] / 100.0 for d in data], dtype=np.float64) # cm→m
444     return ts, Rv
445
446
447 def env_tables_from_att1(t_arr, T_arr, C_arr,
448                          t_tail=14400.0, T_tail=50.00, C_tail=0.04999):
449     """附件1 → njit 查表数组（恒温段常值延拓，口径同 q3_core 同名函数）。"""
450     t0 = t_arr[-1]
451     ts = np.ascontiguousarray(
452         np.concatenate([t_arr, [t0 + 1e-6, t0 + 1.0]]), dtype=np.float64)
453     Tv = np.ascontiguousarray(
454         np.concatenate([T_arr, [T_tail, T_tail]]), dtype=np.float64)
455     Cv = np.ascontiguousarray(
456         np.concatenate([C_arr, [C_tail, C_tail]]), dtype=np.float64)
457     return ts, Tv, Cv
458
459
460 def step_imex4(N, dxi, hsub, n_sub, R0, hc, km, T, C, t0,
461               ts_env, Tenv_v, Cenv_v, ts_R, Rv,
462               omega=0.7, tol=1e-10, maxit=30, mode='coupled',
463               k_zero=False, fixR=False, app3=False, t_corr=0, d_fac=1.0,
464               hcv=-1.0, L=0.0):
465     """推进一个输出步。返回 (T, C, info)。hcv>=0 时覆盖对流系数；L>0 为潜热源项。"""
466     mf = {'coupled': 0, 'frozen': 1, 'decoupled': 2}[mode]
467     Tk, Ck, it, rm = _step_all_nb(
468         N, float(dxi), float(hsub), int(n_sub), float(R0), float(hc), float(km),
469         np.ascontiguousarray(T, dtype=np.float64),
470         np.ascontiguousarray(C, dtype=np.float64),
471         float(t0), ts_env, Tenv_v, Cenv_v, ts_R, Rv,
472         float(d_fac), float(DEF4['C0']), float(omega), float(tol), int(maxit),
473         int(mf), 1 if k_zero else 0, 1 if fixR else 0, 1 if app3 else 0,
474         int(t_corr), float(hcv), float(L))
475     return Tk, Ck, dict(inner_tot=it, nsub=n_sub, res_max=rm)
476
477
478 def run_q4(N=80, t_end=60.0, dt_out=1.0, n_sub=32, out_every=None,
479           ts_env=None, Tenv_v=None, Cenv_v=None, ts_R=None, Rv=None,
480           mode='coupled', k_zero=False, fixR=False, app3=False, t_corr=0,
481           collect_fields=(), verbose=False, hcv=-1.0, L=0.0):
482     """驱动：从 t=0 推进到 t_end。collect_fields 给定采样时刻集合（秒），
483     返回 (fields{t:(T,C)}, stats)。hcv>=0 时覆盖对流系数（绝热自检用 0）。"""
484     dxi = 1.0 / N
485     hsub = dt_out / n_sub
486     T = np.full(N + 1, DEF4['T0'])
487     C = np.full(N + 1, DEF4['C0'])
488     fields = {}
489     tot_it = 0
490     rm_all = 0.0
491     t_all = time.perf_counter()
492     nsteps = int(round(t_end / dt_out))
493     for k in range(1, nsteps + 1):
494         t0 = (k - 1) * dt_out
495         T, C, info = step_imex4(N, dxi, hsub, n_sub, DEF4['R0'], DEF4['h'],
496                                DEF4['km'], T, C, t0, ts_env, Tenv_v, Cenv_v,
497                                ts_R, Rv, mode=mode, k_zero=k_zero,
498                                fixR=fixR, app3=app3, t_corr=t_corr, hcv=hcv,
499                                L=L)
500         tot_it += info['inner_tot']
501         rm_all = max(rm_all, info['res_max'])
502         tk = k * dt_out
503         for tc in collect_fields:
504             if abs(tk - tc) < 1e-9:
505                 fields[float(tc)] = (T.copy(), C.copy())
506     # 终态必存（预试验/主力求解都需要末端场）
507     fields[float(nsteps * dt_out)] = (T.copy(), C.copy())
508     stats = dict(wall=time.perf_counter() - t_all, inner_tot=tot_it,
509                 res_max=rm_all, nsteps=nsteps)
510     return fields, stats
511
512
513 def interp_cubic(ksi_nodes, values, x):
514     """一维四点三次插值（非均匀节点；单点）。节点须单调递增。"""
515     return interp_lagrange(ksi_nodes, values, x, 4)

```

```

516
517
518 def interp_lagrange(ksi_nodes, values, x, npts=6):
519     """n 点 Lagrange 插值 (npts=4 三次 / 6 五次; 窗口贴边钳位)。
520
521     Q4 输出口径: 默认 npts=6 —— S7 实测三次插值在 N=80 下误差 ~2.8e-5,
522     贴 L3-07 上限; 升到五次后插值分量降一个量级以上 (见预试验 S7)。
523     """
524     n = len(ksi_nodes)
525     if x <= ksi_nodes[0]:
526         return values[0]
527     if x >= ksi_nodes[-1]:
528         return values[-1]
529     hi = 1
530     while hi < n - 1 and ksi_nodes[hi] < x:
531         hi += 1
532     lo = hi - 1
533     i0 = lo - (npts // 2 - 1)
534     if i0 < 0:
535         i0 = 0
536     if i0 + npts > n:
537         i0 = n - npts
538     xs = ksi_nodes[i0:i0 + npts]
539     ys = values[i0:i0 + npts]
540     s = 0.0
541     for i in range(npts):
542         term = ys[i]
543         for j in range(npts):
544             if j != i:
545                 term *= (x - xs[j]) / (xs[i] - xs[j])
546     s += term
547     return s

```

B.8 检验脚本：收敛性与误差带本块为检验脚本，正文不随附录印出；源文件全文随支撑材料提交，文件名与附录 A 表 A-1 逐项对应：q1_e1_gci.py q2_w2_foreground.py q3_w2_convergence.py q3_grid_experiment.py q4_w6_e15.py

B.9 检验脚本：解析对拍本块为检验脚本，正文不随附录印出；源文件全文随支撑材料提交，文件名与附录 A 表 A-1 逐项对应：q1_e2_series.py q2_e2_frozen.py q4_w6_e2.py

B.10 检验脚本：独立实现互验本块为检验脚本，正文不随附录印出；源文件全文随支撑材料提交，文件名与附录 A 表 A-1 逐项对应：q1_e3_explicit.py q2_checks.py q3_verify_nu.py q3_verify_grid.py q3_verify_opt.py q4_core_fixed.py

B.11 检验脚本：守恒核算本块为检验脚本，正文不随附录印出；源文件全文随支撑材料提交，文件名与附录 A 表 A-1 逐项对应：q1_conservation.py q2_checks.py q3_conservation.py q4_w6_e78b.py

B.12 检验脚本：灵敏度分析本块为检验脚本，正文不随附录印出；源文件全文随支撑材料提交，文件名与附录 A 表 A-1 逐项对应：q1_e5_sensitivity.py q2_e5_sensitivity.py q2_e5b_global.py q3_e5_sensitivity.py q4_w6_e5.py innov/q4_INV3_global.py innov/q4_INV6_sobol.py

B.13 其余源程序（清单，正文不嵌入）下列 93 个文件与 B.1-B.12 不重复（含改进链脚本 innov/）；全部源程序随支撑材料提交，并在附录 A 表 A-1 逐项登记；检验日志与结果数据同属支撑材料。dq_check.py innov/_ref_e15.py innov/q1_inv_T0.py innov/q1_inv_T1_adaptive.py innov/q1_inv_T1_duhamel.py innov/q1_inv_T1_iface.py innov/q1_inv_T1_interp.py innov/q1_inv_T2_adjoint.py innov/q1_inv_T2_da.py innov/q1_inv_T2_gs.py innov/q1_inv_T2_uq.py innov/q1_inv_T3.py innov/q1_inv_lib.py innov/q1_inv_par.py innov/q2_inv_monolithic.py innov/q2_inv_tol_converge.py innov/q2_inv_tol_delivery.py innov/q2_inv_tol_tradeoff.py innov/q3_compare_INV.py innov/q3_core_INVbase.py innov/q3_core_nu_INVbase.py innov/q3_diag_INV2_step.py innov/q3_diag_mech_T.py innov/q3_e3_INV2_cn.py innov/q3_e5_INVbase.py innov/q3_e5b_INV3_global.py innov/q3_energy.py innov/q3_fig_export.py innov/q3_gci_INV4_tend.py innov/q3_solver_INV1.py innov/q3_solver_INVbase.py innov/q3_uq_INV4_lowC.py innov/q3_w2_INVbase.py innov/q3_xchk_dt.py innov/q3_xchk_gt02.py innov/q4_INV0_regression.py innov/q4_INV1_gci_tend.py innov/q4_INV2_uq_lowC.py innov/q4_INV4_aux.py innov/q4_INV5_theory.py innov/q4_INV_lib.py innov/q4_core_INV.py innov/run_innov_chain.py innov/run_innov_chain2.py q1_consistency_check.py q1_diag.py q1_e3_figdata.py q1_e4_watchdog.py q1_e6_smooth.py q1_figdata_export.py q2_bench_ab.py q2_core.py q2_core_bdf2.py q2_core_latent.py q2_core_orig.py q2_diag_picard.py q2_e3b_decoupled.py q2_e8_latent.py q2_e9_adaptive.py q2_f30_pareto.py q2_fig_export.py q2_figdata_export.py q2_opt_verify.py q2_quick_check.py q2_solver_orig.py q2_verify_

BC.py q2_verify_results.py q2_w2_prestudy.py q3_audit_consistency.py q3_audit_tables.py q3_check_maxloc.py q3_core_nu.py q3_diag_iface.py q3_diag_lowC.py q3_diag_mech.py q3_diag_surface.py q3_impact_check.py q3_postcheck.py q3_pretest_bench.py q3_probe_crit.py q3_probe_dcut.py q3_probe_grid.py q3_surface_theory.py q4_core_prebug.py q4_diag_adiab.py q4_diag_blow.py q4_diag_e3g2.py q4_pretest.py q4_table6.py q4_w6_e78.py q4_w6_lib.py rerun_chain.ps1 run_chain.py

C 关键中间结果与图示说明

表 C-1 附录 C 关键中间结果索引

项	内容	对应论文位置
C.1	空间与时间收敛性的加密序列明细	6.1
C.2	解析对拍与独立互验的逐时刻偏差表	6.2 / 6.3
C.3	守恒核算的分量明细	6.5
C.4	灵敏度算例的完整结果表	6.6 / 6.7
C.5	低含水率端外推的多档扫描结果与概率区间	6.10
C.6	各问不宜入正文的检验图表	第六章
C.7	各问的场分布与关键点曲线（补充图）	5.1-5.4 的补充

说明：附录 C 只作中间结果的陈列与索引，不重复正文结论；凡正文已给出的结论，附录不再复述。